



- Expert Verified, Online, **Free**.



CERTIFICATION TEST

- CertificationTest.net - Cheap & Quality Resources With Best Support

An upstream system has been configured to pass the date for a given batch of data to the Databricks Jobs API as a parameter. The notebook to be scheduled will use this parameter to load data with the following code: `df = spark.read.format("parquet").load(f"/mnt/source/{date}")`
Which code block should be used to create the date Python variable used in the above code block?

- A. `date = spark.conf.get("date")`
- B. `input_dict = input()`
`date= input_dict["date"]`
- C. `import sys`
`date = sys.argv[1]`
- D. `date = dbutils.notebooks.getParam("date")`
- E. `dbutils.widgets.text("date", "null")`
`date = dbutils.widgets.get("date")`

Suggested Answer: E

Community vote distribution

E (100%)

  **hal2401me** Highly Voted 1 year, 4 months ago

Selected Answer: E

`dbutils.widget.`

Just passed the exam with score >80%.

examtopics covers about 90% of questions. there were 5 questions I didn't see here in examtopics.

But friends, you need to look at the discussions, and do test yourself.

many answers provided here, even most voted answer, does NOT exists anymore in the exam - not the question, but the answer.

Wish you all good luck, friends!

upvoted 12 times

  **ernestohp** Most Recent 1 month, 2 weeks ago

I just take the exam today, and not any of the 238 questions in this bundle or something similar come in the exam. The new version is completely different since november 30.

upvoted 1 times

  **ankitpandey** 1 month, 3 weeks ago

Selected Answer: E

None of the questions come from this so don't waste your time and money.

upvoted 3 times

  **nyoike** 2 months, 2 weeks ago

Selected Answer: D

`dbutils.widgets.get("date")` is used for interactive notebook widgets (like a dropdown or a text box) that are visible on the notebook UI. While widgets can be used to retrieve parameters when running a notebook manually or as a job, the `dbutils.notebooks.getParam()` (Option D) is the more robust and direct method for job execution parameters, especially when the goal is to read the parameter passed by the upstream system, not define an interactive widget.

upvoted 1 times

  **_Lukas_** 3 months ago

Selected Answer: E

Correct:

E - recommended pattern for accessing job parameters; create a widget and retrieve the parameter value

Incorrect:

A - `spark.conf.get()` - is for accessing Spark configuration settings, not job parameters.

B - is for standard Python input, which doesn't apply to Databricks job

C - is for Python script tasks that receive command-line arguments.

D - is not valid method

upvoted 3 times

sd_dp200 1 month, 2 weeks ago

Excellent answer

upvoted 1 times

yangmei 7 months ago

Selected Answer: E

I'm confused, shall the f-string use curly brackets instead of parenthesis?

upvoted 1 times

Gowtham02 7 months, 4 weeks ago

Selected Answer: E

widget is correct

upvoted 1 times

KadELbied 9 months, 1 week ago

Selected Answer: E

Surely E

upvoted 2 times

prasioso 7 months, 3 weeks ago

Agree with E. Did you clear the exam? Are the questions still relevant?

upvoted 1 times

ultimomassimo 10 months, 2 weeks ago

Selected Answer: E

E 100%

There is no such thing as `dbutils.notebooks.getParam`, so no idea why some halfwits suggest D...

upvoted 1 times

shoaibmohammed3 11 months, 2 weeks ago

Selected Answer: E

`dbutils.widgets` is where we store all the param and use `.get` to fetch the params

upvoted 1 times

johnserafim 11 months, 3 weeks ago

Selected Answer: D

D is correct!

The question states that the upstream system passes the date as a parameter to the Databricks Jobs API. In Databricks, when a parameter is passed to a notebook via the Jobs API, it can be retrieved using the `dbutils.notebooks.getParam()` method.

Option D directly retrieves the parameter value using this method, which is the correct approach for this scenario.

upvoted 1 times

EelkeV 1 year ago

Selected Answer: E

It is the way to fill in a parameter in a notebook

upvoted 1 times

HairyTorso 1 year, 1 month ago

Selected Answer: E

Around question #130 they just repeat themselves. So it's not 226 but around 130... Shame

upvoted 2 times

akashdesarda 1 year, 4 months ago

Selected Answer: E

Jobs API allows to sending parameters via jobs parameter. This parameter must have the same notebook params. Eventually, it can be read using `dbutils.widgets.get`

upvoted 1 times

HorskKos 1 year, 5 months ago

E is correct because:

A - gets configuration of a spark session

B - gets a value from a manual input - non relevant for the job run

C - `sys.argv` - gets parameters, which were used to run a Python script from CMD - completely non-related

D - haven't found this function on the web at all, assume that it doesn't exist

Therefore E is correct, though it's a bad practice to type a date as a parameter, it's better to get it with datetime library and then use it in the code
upvoted 4 times

  **Shailly** 1 year, 6 months ago

Answer is E.

Even though the value is passed from a upstream system, you can create parameters using widgets inside notebook and use the value as an input from the databricks jobs API.

upvoted 1 times

  **Isio05** 1 year, 8 months ago

Selected Answer: E

Widgets are used to create parameters in notebook that can be then utilized by e.g. jobs

upvoted 1 times

The Databricks workspace administrator has configured interactive clusters for each of the data engineering groups. To control costs, clusters are set to terminate after 30 minutes of inactivity. Each user should be able to execute workloads against their assigned clusters at any time of the day.

Assuming users have been added to a workspace but not granted any permissions, which of the following describes the minimal permissions a user would need to start and attach to an already configured cluster.

- A. "Can Manage" privileges on the required cluster
- B. Workspace Admin privileges, cluster creation allowed, "Can Attach To" privileges on the required cluster
- C. Cluster creation allowed, "Can Attach To" privileges on the required cluster
- D. "Can Restart" privileges on the required cluster
- E. Cluster creation allowed, "Can Restart" privileges on the required cluster

Suggested Answer: A

Community vote distribution

D (100%)

 **ultimomassimo** Highly Voted 10 months, 2 weeks ago

Selected Answer: D

100% D

User has to be able to start the cluster as it shuts down after 30 min of inactivity.

Everything is explain clearly in the table here - Compute ACLs: <https://docs.databricks.com/aws/en/security/auth/access-control/#clusters>

upvoted 6 times

 **nyoike** Most Recent 2 months, 2 weeks ago

Selected Answer: D

Why Option D is the specific correct answer:

The Requirement: The user needs to start (because it terminates after 30 minutes) and attach (to execute workloads).

Why D works: The "Can Restart" permission explicitly grants the ability to start a terminated cluster. Because Databricks permissions are hierarchical, "Can Restart" automatically includes "Can Attach To" privileges. Therefore, this single permission covers both requirements.

Why it is "Minimal": While "Can Manage" (Option A) would also work, it grants the user the ability to edit the cluster settings (e.g., change the instance type or auto-termination window), which violates the principle of least privilege. "Can Restart" restricts them to only operating the power state and using the cluster.

upvoted 1 times

 **_Lukas_** 3 months ago

Selected Answer: D

Correct:

D - provides minimal permission to start and attach to the cluster

Incorrect:

A - grants excessive permissions including the ability to edit cluster configurations, which is not needed

B, C, E - allow cluster creation, which is not needed

Hierarchy of compute permission levels:

NO PERMISSIONS -> CAN ATTACH TO -> CAN RESTART -> CAN MANAGE

upvoted 1 times

 **8c1d06f** 4 months, 2 weeks ago

Selected Answer: C

C. Cluster creation allowed, "Can Attach To" privileges on the required cluster

Clusters are already configured by the admin.

Users just need to start a terminated cluster and attach to it.

We want minimal permissions (not over-privileged).

"Cluster creation allowed" is irrelevant, since they're not creating new clusters, only using existing ones.

"Can Manage" is too much (gives full control, including edit/delete).

"Can Restart" is not enough (lets you restart, but not attach).

The correct minimal set is "Can Attach To", which allows users to attach notebooks/jobs to the cluster and start it if it's terminated.

upvoted 1 times

  **kingzmole** 5 months, 2 weeks ago

Selected Answer: D

Can Restart → Allows starting/restarting/terminating the cluster.

User can then attach notebooks because attaching is implicit once the cluster is running and he owns the notebook.

upvoted 1 times

  **coolank03** 6 months, 1 week ago

Selected Answer: D

Answer is D

upvoted 1 times

  **79f0e18** 7 months, 1 week ago

Selected Answer: C

To allow users to start a terminated cluster (due to 30 minutes of inactivity) and attach to it to run workloads, they need:

Cluster creation entitlement:

This is a workspace-level entitlement that enables users to start clusters, including pre-existing ones that are in a terminated state.

Without this, users cannot start any clusters – even existing interactive ones.

"Can Attach To" cluster permission:

This is a per-cluster permission that lets users attach notebooks, jobs, or run commands on the cluster.

It does not allow users to restart or manage the cluster without the cluster creation entitlement.

upvoted 1 times

  **dkhodyriev** 7 months, 2 weeks ago

Selected Answer: D

The answer is D. "Can Restart" privileges on the required cluster.

This is the minimal permission needed because:

It allows starting terminated clusters (necessary due to 30-minute auto-termination)

It includes the ability to attach to and execute workloads on clusters

It doesn't require cluster creation permissions since clusters already exist

It's less privileged than "Can Manage" which would include unnecessary configuration editing rights

upvoted 1 times

  **SP_07** 9 months, 1 week ago

Selected Answer: D

The minimal permission a user needs to start (i.e., restart a terminated cluster) and attach to an already configured Databricks cluster is "Can Restart" privileges on the required cluster. This allows users to restart and attach to clusters but not manage or modify them.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: D

suretly D

upvoted 1 times

  **codebender** 10 months, 1 week ago

Selected Answer: D

Restart is necessary to start the cluster

upvoted 2 times

  **Ashok_Choudhary_CT** 10 months, 1 week ago

Selected Answer: A

There are three types of permissions:

1. Can Restart
2. Can Attach To

3. Can Manage

Can manage provides both the permissions. Can restart doesn't provide "Can Attach To" permission, So the correct answer is "A"
upvoted 2 times

🗨️ 👤 **Tedet** 11 months, 2 weeks ago

Selected Answer: C

To execute workloads against an already configured cluster in Databricks, users need the following minimum permissions:

1. Cluster Creation Allowed: Users need the ability to create clusters, which ensures they can start a cluster if it's not already running or if the cluster has been terminated after the inactivity period.
2. "Can Attach To" Privileges on the Required Cluster: This permission allows users to attach their notebooks or jobs to the existing cluster. The "Can Attach To" permission is the key to allowing users to interact with the cluster for running jobs or notebooks.

upvoted 1 times

🗨️ 👤 **johnserafim** 11 months, 3 weeks ago

Selected Answer: C

I should choose the C answer because the "Can Attach To" permission is the minimal requirement for a user to attach to an interactive cluster and execute workloads.

upvoted 1 times

🗨️ 👤 **shaswat1404** 12 months ago

Selected Answer: D

can manage all permission

workspace admin irrelevant

can attach to cannot start the cluster

can restart correct can do both

cluster creation + restart (cluster is already created, dont need to give permission)

upvoted 1 times

🗨️ 👤 **EelkeV** 1 year ago

Selected Answer: C

You do not need all the permissions, just the lowest

upvoted 1 times

🗨️ 👤 **Dhusanth** 1 year ago

Selected Answer: D

<https://docs.azure.cn/en-us/databricks/security/auth-authz/access-control/cluster-acl#cluster-permissions>

upvoted 4 times

When scheduling Structured Streaming jobs for production, which configuration automatically recovers from query failures and keeps costs low?

- A. Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: Unlimited
- B. Cluster: New Job Cluster;
Retries: None;
Maximum Concurrent Runs: 1
- C. Cluster: Existing All-Purpose Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
- D. Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
- E. Cluster: Existing All-Purpose Cluster;
Retries: None;
Maximum Concurrent Runs: 1

Suggested Answer: D

Community vote distribution

D (100%)

8605246 Highly Voted 2 years, 6 months ago

the answer given is correct:

Maximum concurrent runs: Set to 1. There must be only one instance of each query concurrently active.

Retries: Set to Unlimited.

<https://docs.databricks.com/en/structured-streaming/query-recovery.html>

upvoted 11 times

nyoike Most Recent 2 months, 2 weeks ago

Selected Answer: D

1. Cluster: New Job Cluster (Low Cost)

New Job Clusters are designed for running automated, non-interactive workloads. They are created when the job starts and terminate immediately when the job finishes, or in the case of a continuous streaming job, when it fails or is paused.

2. Retries: Unlimited (Automatic Recovery)

Structured Streaming jobs must be configured for automatic restarts to ensure high availability. Failures in a stream—due to transient network issues, cloud resource instability, or an unexpected data-related bug—are common.

3. Maximum Concurrent Runs: 1 (Data Integrity & Cost Control)

A key requirement for Structured Streaming (especially when using checkpointing) is that only one instance of a query should be running at a time against a single checkpoint location.

upvoted 2 times

Lukas 3 months ago

Selected Answer: D

Correct:

D - Job Clusters are cheaper than all-purpose compute, and retries recover from query failures. Setting one Maximum Concurrent Run keeps costs low.

Incorrect:

A - Unlimited Maximum Concurrent Runs increase costs.

B - Without retries, query failures are not recovered.

C, E - All-Purpose clusters do not keep costs low.

upvoted 1 times

🗳️ 👤 **Kundankg** 5 months ago

Selected Answer: D

The best configuration for scheduling Structured Streaming jobs in production that automatically recovers from query failures while keeping costs low is

upvoted 1 times

🗳️ 👤 **363c4c5** 7 months, 1 week ago

Selected Answer: D

New Job Cluster: Using a new job cluster ensures that the compute resources are appropriately sized and dedicated to the job, which can help in managing costs and performance more effectively than using an existing all-purpose cluster.

Retries: Unlimited: Setting retries to unlimited ensures that the job will automatically recover from failures by retrying until it succeeds.

Maximum Concurrent Runs: 1: Limiting the maximum concurrent runs to 1 prevents multiple instances of the job from running simultaneously, which can help in controlling costs and avoiding resource contention.

Databricks recommends using jobs compute instead of all-purpose compute when scheduling workflows, as it helps in managing resources more efficiently and reduces costs.

<https://learn.microsoft.com/en-us/azure/databricks/structured-streaming/production>

<https://learn.microsoft.com/en-us/azure/databricks/jobs/continuous>

upvoted 1 times

🗳️ 👤 **79f0e18** 7 months, 1 week ago

Selected Answer: D

When running Structured Streaming jobs in production, you want:

Automatic failure recovery → Requires setting Retries: Unlimited

Efficient cost control → Use a New Job Cluster, which auto-terminates after job completion

Concurrency control → Maximum Concurrent Runs: 1 prevents overlapping runs, which can corrupt streaming state or double-process data

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

Surely d

upvoted 1 times

🗳️ 👤 **codebender** 10 months, 1 week ago

Selected Answer: D

Cant be all purpose general compute

upvoted 1 times

🗳️ 👤 **EelkeV** 1 year ago

Selected Answer: D

Job cluster autoterminates, and you want retries for recover

upvoted 1 times

🗳️ 👤 **akashdesarda** 1 year, 4 months ago

Selected Answer: D

Use databricks jobs as it as native integration with Streaming use case. See the example Job here <https://docs.databricks.com/en/structured-streaming/query-recovery.html#configure-structured-streaming-jobs-to-restart-streaming-queries-on-failure>

upvoted 2 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

D. Cluster: New Job Cluster;

Retries: Unlimited;

Maximum Concurrent Runs: 1

upvoted 1 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

D. Cluster: New Job Cluster;

Retries: Unlimited;

Maximum Concurrent Runs: 1

upvoted 1 times

🗳️ 👤 **juliom6** 1 year, 10 months ago

D is correct

<https://docs.databricks.com/en/structured-streaming/query-recovery.html>

upvoted 1 times

  **AziLa** 2 years ago

Correct Ans is D

upvoted 1 times

  **Jay_98_11** 2 years ago

Selected Answer: D

D is correct

upvoted 1 times

  **kz_data** 2 years, 1 month ago

Selected Answer: D

D is correct

upvoted 1 times

  **sturcu** 2 years, 3 months ago

Selected Answer: D

D is correct

upvoted 1 times

The data engineering team has configured a Databricks SQL query and alert to monitor the values in a Delta Lake table. The recent_sensor_recordings table contains an identifying sensor_id alongside the timestamp and temperature for the most recent 5 minutes of recordings.

The below query is used to create the alert:

```
SELECT MEAN(temperature), MAX(temperature), MIN(temperature)
FROM recent_sensor_recordings
GROUP BY sensor_id
```

The query is set to refresh each minute and always completes in less than 10 seconds. The alert is set to trigger when mean (temperature) > 120. Notifications are triggered to be sent at most every 1 minute.

If this alert raises notifications for 3 consecutive minutes and then stops, which statement must be true?

- A. The total average temperature across all sensors exceeded 120 on three consecutive executions of the query
- B. The recent_sensor_recordings table was unresponsive for three consecutive runs of the query
- C. The source query failed to update properly for three consecutive minutes and then restarted
- D. The maximum temperature recording for at least one sensor exceeded 120 on three consecutive executions of the query
- E. The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query

Suggested Answer: E

Community vote distribution

E (100%)

  **_Lukas_** 3 months ago

Selected Answer: E

Correct:

E - MEAN is the same as AVG. The trigger was fired when one of the sensors exceeded 120.

Incorrect:

A - Total Average is incorrect because the SQL there uses GROUP BY sensor_id, which means all values are aggregated by sensor_id and do not present a total value for all sensors.

B - It's clearly stated that the trigger is fired when mean(temperature) > 120. However, table unresponsiveness contradicts the successful alert triggers.

C - Query failure doesn't align with the query completing and successfully raising notifications.

D - The trigger is fired only when mean(temperature) > 120, not when max(temperature) > 120.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: E

suretly E

upvoted 1 times

  **EelkeV** 1 year ago

Selected Answer: E

Because the mean is calculated for each sensor, and on that the alert is raised. It happened for three times, unknown for which sensor. Could be any

upvoted 3 times

  **AndreFR** 1 year, 5 months ago

A excluded because there is a group by clause

B & C excluded table needs to be updated to mean value to change

D excluded, because alert is set on average not max temperature

Correct answer is E by elimination

upvoted 2 times

  **panya** 1 year, 7 months ago

Correct

upvoted 1 times

🗨️ 👤 **imatheushenrique** 1 year, 8 months ago

E. The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query
upvoted 1 times

🗨️ 👤 **Jay_98_11** 2 years ago

Selected Answer: E

E is correct
upvoted 1 times

🗨️ 👤 **sturcu** 2 years, 3 months ago

Selected Answer: E

correct
upvoted 2 times

🗨️ 👤 **saikot** 2 years, 4 months ago

The correct answer is E

<https://www.myexamcollection.com/databricks-certified-professional-data-engineer-databricks-certified-professional-data-engineer-exam-question-answers.htm>

upvoted 1 times

A junior developer complains that the code in their notebook isn't producing the correct results in the development environment. A shared screenshot reveals that while they're using a notebook versioned with Databricks Repos, they're using a personal branch that contains old logic. The desired branch named dev-2.3.9 is not available from the branch selection dropdown. Which approach will allow this developer to review the current logic for this notebook?

- A. Use Repos to make a pull request use the Databricks REST API to update the current branch to dev-2.3.9
- B. Use Repos to pull changes from the remote Git repository and select the dev-2.3.9 branch.
- C. Use Repos to checkout the dev-2.3.9 branch and auto-resolve conflicts with the current branch
- D. Merge all changes back to the main branch in the remote Git repository and clone the repo again
- E. Use Repos to merge the current branch and the dev-2.3.9 branch, then make a pull request to sync with the remote repository

Suggested Answer: B

Community vote distribution

B (100%)

🗳️ 👤 **_Lukas_** 3 months ago

Selected Answer: B

Correct:

B - Pulling changes allows you to fetch the branch from the remote into your local repository and then review the current logic.

Incorrect:

A - Updating the branch is unnecessary. It is clearly stated that the developer only needs to review the current logic.

C - This option is overly complicated for simply reviewing existing code. We do not need to update our branch just to review the logic.

D - The description mentions about dev-2.3.9 branch, not the main branch. This option does not allow you to see the logic implemented there.
upvoted 1 times

🗳️ 👤 **Gowtham02** 7 months, 4 weeks ago

Selected Answer: B

B is correct

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

🗳️ 👤 **codebender** 10 months, 1 week ago

Selected Answer: B

First step is to pull from the latest commits

upvoted 2 times

🗳️ 👤 **benni_ale** 1 year, 4 months ago

Selected Answer: B

I would also say B but could anyone explain how to pick that branch if it is not available from dropdown?

upvoted 1 times

🗳️ 👤 **NBurman** 1 year, 2 months ago

The first step is to perform a pull. This fetches all the changes from the remote branch and only after that will you see the dev branch.

upvoted 6 times

🗳️ 👤 **benni_ale** 1 year, 4 months ago

Selected Answer: B

I would say B

upvoted 1 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

B. Use Repos to pull changes from the remote Git repository and select the dev-2.3.9 branch.

upvoted 2 times

  **AziLa** 2 years ago

correct ans is B

upvoted 1 times

  **Jay_98_11** 2 years ago

Selected Answer: B

vote for B also

upvoted 2 times

  **sturcu** 2 years, 3 months ago

Selected Answer: B

B is correct

upvoted 1 times

The security team is exploring whether or not the Databricks secrets module can be leveraged for connecting to an external database. After testing the code with all Python variables being defined with strings, they upload the password to the secrets module and configure the correct permissions for the currently active user. They then modify their code to the following (leaving all other variables unchanged).

```
password = dbutils.secrets.get(scope="db_creds", key="jdbc_password")

print(password)

df = (spark
      .read
      .format("jdbc")
      .option("url", connection)
      .option("dbtable", tablename)
      .option("user", username)
      .option("password", password)
      )
```

Which statement describes what will happen when the above code is executed?

- A. The connection to the external table will fail; the string "REDACTED" will be printed.
- B. An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the encoded password will be saved to DBFS.
- C. An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the password will be printed in plain text.
- D. The connection to the external table will succeed; the string value of password will be printed in plain text.
- E. The connection to the external table will succeed; the string "REDACTED" will be printed.

Suggested Answer: E

Community vote distribution

E (100%)

 **akashdesarda** Highly Voted 1 year, 4 months ago

Selected Answer: E

Whatever we read using dbutils.secret module is always printed as '[REDACTED]', but when consumed in code, underlying vales are passed.
upvoted 6 times

 **_Lukas_** Most Recent 2 months, 1 week ago

Selected Answer: E

Correct:

E - dbutils.secrets.get() retrieves the secret value for the 'jdbc_password' secret. The connection succeed because the 'password' variable recieves the real password even then when we try to display it, it automtically redacts.

Incorrect:

- A - The actual secret value is available programmatically even though it's redacted in display output. The connection will not fail.
- B - No interactive input box appears when using 'dbutils.secrets.get()'. Secrets are retrieved directly from the configured secret scope.
- C - No interactive input box appears.
- D - The password will not be printed in plain text.

upvoted 1 times

 **JohnXavier** 4 months, 2 weeks ago

Selected Answer: E

REDACTED is a correct answer
upvoted 1 times

 **KadELbied** 9 months, 1 week ago

Selected Answer: E

Suretly E

upvoted 2 times

 **benni_ale** 1 year, 4 months ago

Selected Answer: E

Shave like a bomber
upvoted 1 times

🗉 **imatheushenrique** 1 year, 8 months ago

E. The connection to the external table will succeed; the string "REDACTED" will be printed.
upvoted 1 times

🗉 **PrashantTiwari** 2 years ago

E is correct
upvoted 1 times

🗉 **AziLa** 2 years ago

correct ans is E
upvoted 1 times

🗉 **Jay_98_11** 2 years ago

Selected Answer: E

E is correct
upvoted 2 times

🗉 **ATLTennis** 2 years, 1 month ago

Selected Answer: E

E is correct
upvoted 2 times

🗉 **kz_data** 2 years, 1 month ago

Selected Answer: E

Correct answer E
upvoted 2 times

🗉 **sturcu** 2 years, 3 months ago

Selected Answer: E

Correct: <https://docs.databricks.com/en/external-data/jdbc.html>
upvoted 4 times

🗉 **Brian9** 2 years, 5 months ago

<https://learn.microsoft.com/en-us/azure/databricks/security/secrets/redaction>

Option E - which is selected answer seems correct.

upvoted 3 times

🗉 **8605246** 2 years, 6 months ago

This option is correct, although the password won't be printed out, the connection will still succeed.
upvoted 1 times

The data science team has created and logged a production model using MLflow. The following code correctly imports and applies the production model to output the predictions as a new DataFrame named preds with the schema "customer_id LONG, predictions DOUBLE, date DATE".

```
from pyspark.sql.functions import current_date

model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
preds = (df.select(
    "customer_id",
    model(*columns).alias("predictions"),
    current_date().alias("date")
))
```

The data science team would like predictions saved to a Delta Lake table with the ability to compare all predictions across time. Churn predictions will be made at most once per day.

Which code block accomplishes this task while minimizing potential compute costs?

A. `preds.write.mode("append").saveAsTable("churn_preds")`

B. `preds.write.format("delta").save("/preds/churn_preds")`

C. `(preds.writeStream
 .outputMode("overwrite")
 .option("checkpointPath", "_checkpoints/churn_preds")
 .start("/preds/churn_preds")
)`

D. `(preds.write
 .format("delta")
 .mode("overwrite")
 .saveAsTable("churn_preds")
)`

E. `(preds.writeStream
 .outputMode("append")
 .option("checkpointPath", "_checkpoints/churn_preds")
 .table("churn_preds")
)`

Suggested Answer: A

Community vote distribution

A (100%)

thxsgod Highly Voted 2 years, 5 months ago

Selected Answer: A

You need:

- Batch operation since it is at most once a day
- Append, since you need to keep track of past predictions

A is the correct answer. You don't need to specify "format" when you use `saveAsTable`.

upvoted 13 times

Lukas Most Recent 2 months, 1 week ago

Selected Answer: A

Correct:

A - 'at most once per day' indicates a batch workload. Mode 'append' satisfies the requirement for historical comparison.

Incorrect:

B - Without specifying a mode, it defaults to "error" which will fail on subsequent writes. This doesn't provide the historical tracking.

C - Streaming load is inappropriate here because predictions are generated 'at most once per day'.

D - The mode("overwrite") will replace the entire table each time, erasing historical data.

E - Using a streaming load for once-daily batch predictions is significant over-engineering.

upvoted 2 times

🗄️ 👤 **JohnXavier** 4 months, 2 weeks ago

Selected Answer: A

A is a correct answer because of Batch and Append operations
upvoted 1 times

🗄️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: A

Suretly A
upvoted 1 times

🗄️ 👤 **benni_ale** 1 year, 4 months ago

Selected Answer: A

Batch, Append
upvoted 1 times

🗄️ 👤 **coercion** 1 year, 8 months ago

Selected Answer: A

default table format is delta so no need to specify the format.
As per the requirement, "append" mode is required to maintain the history. Default mode is "ErrorIfExists"
upvoted 1 times

🗄️ 👤 **Jay_98_11** 2 years ago

Selected Answer: A

A is correct
upvoted 1 times

🗄️ 👤 **kz_data** 2 years, 1 month ago

Selected Answer: A

A is correct
upvoted 1 times

🗄️ 👤 **sturcu** 2 years, 3 months ago

Selected Answer: A

Correct
upvoted 1 times

🗄️ 👤 **sturcu** 2 years, 3 months ago

Correct
upvoted 1 times

🗄️ 👤 **Eertyy** 2 years, 4 months ago

answer is B
upvoted 2 times

🗄️ 👤 **Eertyy** 2 years, 4 months ago

Here's why:

A. saves the data as a managed table, which may not be efficient for large-scale data or frequent updates. It doesn't utilize Delta Lake capabilities.
C. is used for streaming operations, not batch processing. Also, using "overwrite" as output mode will replace the existing data each time, which is not suitable for keeping historical predictions.
D. is similar to option A but with "overwrite" mode. It will replace the entire table each time, which is not suitable for maintaining a historical record of predictions.

E. is also for streaming operations and not for batch processing. Additionally, it uses the "table" method, which is not typically used for writing batch data into Delta Lake tables.

Option B is suitable for batch processing, writes data in Delta Lake format, and allows you to efficiently maintain a historical record of predictions while minimizing compute costs.

upvoted 3 times

🗄️ 👤 **pradyumn9999** 2 years, 3 months ago

Its also said they want to compare past values as well, so mode needs to be append. By default is error mode.
upvoted 4 times

🗄️ 👤 **Starvosxant** 2 years, 4 months ago

First: the default node Databricks saves tables IS Delta Format. So no reason why you say it wouldn't benefit from Lakehouse features.
Second: the default write mode is Error, means that if you try to write to a location and that already exists there, it will prone a Error. And the

question specify that you gonna write once a day.

You better revisit basic topics before continue to the professional level certification, or buy the dump entirely.

upvoted 4 times

 **buggumaster** 2 years, 5 months ago

Selected answer is wrong, not write Format is specified in A.

upvoted 1 times

 **buggumaster** 2 years, 5 months ago

Selected answer is wrong, not writeMode is specified in A.

upvoted 1 times

An upstream source writes Parquet data as hourly batches to directories named with the current date. A nightly batch job runs the following code to ingest all data from the previous day as indicated by the date variable:

```
(spark.read
  .format("parquet")
  .load(f"/mnt/raw_orders/{date}")
  .dropDuplicates(["customer_id", "order_id"])
  .write
  .mode("append")
  .saveAsTable("orders")
)
```

Assume that the fields `customer_id` and `order_id` serve as a composite key to uniquely identify each order.

If the upstream system is known to occasionally produce duplicate entries for a single order hours apart, which statement is correct?

- A. Each write to the orders table will only contain unique records, and only those records without duplicates in the target table will be written.
- B. Each write to the orders table will only contain unique records, but newly written records may have duplicates already present in the target table.
- C. Each write to the orders table will only contain unique records; if existing records with the same key are present in the target table, these records will be overwritten.
- D. Each write to the orders table will only contain unique records; if existing records with the same key are present in the target table, the operation will fail.
- E. Each write to the orders table will run deduplication over the union of new and existing records, ensuring no duplicate records are present.

Suggested Answer: B

Community vote distribution

B (100%)

 **Eertyy**  2 years, 4 months ago

B. Each write to the orders table will only contain unique records, but newly written records may have duplicates already present in the target table.

Explanation:

In the provided code, the `.dropDuplicates(["customer_id", "order_id"])` operation is performed on the data loaded from the Parquet files. This operation ensures that only unique records, based on the composite key of "customer_id" and "order_id," are retained in the DataFrame before writing to the "orders" table.

However, this operation does not consider duplicates that may already exist in the "orders" table. It only filters duplicates from the current batch of data. If there are duplicates in the "orders" table from previous batches, they will remain in the table.

So, newly written records will not have duplicates within the batch being written, but duplicates from previous batches may still exist in the target table.

upvoted 23 times

 **meatpoof** 1 year ago

The question doesn't say orders already exists. Arekm's answer is more correct

upvoted 1 times

 **_Lukas_**  3 months ago

Selected Answer: B

Correct:

B - `'dropDuplicates(["customer_id", "order_id"])` guarantees that each write to the orders table will only contain unique records.

`'mode("append")` - incurs the risk that newly written records may have duplicates already present in the target table. This mode does not perform validation or comparison with the destination.

Incorrect:

A - Append mode doesn't filter out records that match existing table data.

C, D - Append mode neither overwrites nor fails when duplicate keys exist; it simply adds all records.

E - dropDuplicates() operates only on the current DataFrame (incoming new records), not on the union of new and existing table records.

To prevent duplicates in the target table, you would need to use Delta Lake's 'MERGE' operation.

upvoted 1 times

🗳️ 👤 **JohnXavier** 4 months, 2 weeks ago

Selected Answer: B

B is a correct answer . Eertty explanation correct

upvoted 1 times

🗳️ 👤 **Gowtham02** 4 months, 4 weeks ago

Selected Answer: B

B is correct

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: B

No duplicates in the current batch - that is obvious. The duplicates may happen since the source occasionally produces duplicates hours apart. This means that one record can be generated by the source and processed on day 1, the duplicate on day 2. Since there is no logic checking if the corresponding record exists in the target - you get the duplicates there given we use append mode.

upvoted 2 times

🗳️ 👤 **Anithec0der** 1 year, 2 months ago

Selected Answer: B

yeah B is the correct answer cause in the current code it will look for duplicates in the currentDF based on composite keys and not for the duplicates which are already in the target table. if we want to insert for the rows which are not there in target table then we can make use of Merge Into statement of databricks.

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 4 months ago

Selected Answer: B

Append method does not take in consideration any key in the target table, it simply add all rows of the input table to the target table.

upvoted 1 times

🗳️ 👤 **panya** 1 year, 7 months ago

Yes it should be B

upvoted 1 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

B. Each write to the orders table will only contain unique records, but newly written records may have duplicates already present in the target table.

Using merge this problem would not happen

upvoted 1 times

🗳️ 👤 **DavidRou** 1 year, 11 months ago

Selected Answer: B

B is the right answer. The above code only remove duplicates from the batch that is processed, no logic is applied to already saved records.

upvoted 1 times

🗳️ 👤 **Jay_98_11** 2 years ago

Selected Answer: B

B is correct

upvoted 1 times

🗳️ 👤 **5ffcd04** 2 years, 1 month ago

Selected Answer: B

Answer B

upvoted 1 times

🗳️ 👤 **kz_data** 2 years, 1 month ago

Selected Answer: B

B is correct

upvoted 1 times

  **vivekla** 2 years, 2 months ago

correct B

upvoted 1 times

  **sturcu** 2 years, 3 months ago

Selected Answer: B

Correct

upvoted 1 times

  **Starvosxant** 2 years, 4 months ago

Correct. B

upvoted 1 times

A junior member of the data engineering team is exploring the language interoperability of Databricks notebooks. The intended outcome of the below code is to register a view of all sales that occurred in countries on the continent of Africa that appear in the geo_lookup table. Before executing the code, running SHOW TABLES on the current database indicates the database contains only two tables: geo_lookup and sales.

Cmd 1

```
%python
countries_af = [x[0] for x in
spark.table("geo_lookup").filter("continent='AF'").select("country").collect()]
```

Cmd 2

```
%sql
CREATE VIEW sales_af AS
SELECT *
FROM sales
WHERE city IN countries_af
AND CONTINENT = "AF"
```

Which statement correctly describes the outcome of executing these command cells in order in an interactive notebook?

- A. Both commands will succeed. Executing show tables will show that countries_af and sales_af have been registered as views.
- B. Cmd 1 will succeed. Cmd 2 will search all accessible databases for a table or view named countries_af: if this entity exists, Cmd 2 will succeed.
- C. Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable representing a PySpark DataFrame.
- D. Both commands will fail. No new variables, tables, or views will be created.
- E. Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable containing a list of strings.

Suggested Answer: E

Community vote distribution

E (88%)

12%

 **aragorn_brego** Highly Voted 2 years, 2 months ago

Selected Answer: E

Cmd 1 is a PySpark command that collects the list of countries from the 'geo_lookup' table where the continent is Africa ('AF'). This command will execute successfully, resulting in countries_af being a list of country names (strings) in Python's local memory.

Cmd 2 is an SQL command intended to create a view named 'sales_af' from the 'sales' table, filtered by the cities in the countries_af list. However, this will fail because the countries_af variable exists in the Python environment and is not recognized in the SQL context. SQL does not have access to Python variables directly; they are two separate execution contexts within a Databricks notebook. There is no table or view named countries_af that SQL can reference; it is merely a Python list variable.

The other options are incorrect because they either assume cross-contextual operation between Python and SQL within a Databricks notebook (which is not possible in the way described in the commands), or they do not correctly interpret the outcome of running the commands.

upvoted 15 times

 **freely** 1 year, 1 month ago

I mean without specifying the catalog and the schema in a unity catalog context ? this will only succeed if the table is in the default catalog and schema

upvoted 1 times

 **_Lukas_** Most Recent 3 months ago

Selected Answer: E

Correct:

E - Cmd 1 creates a Python list of country names by collecting data from the 'geo_lookup' table and extracting the first column value from each Row object.

Cmd 2 attempts to directly reference 'countries_af' in a SQL query using the %sql magic command. However, variables defined in one language are not accessible in another language's. SQL magic cells cannot directly access Python variables.

Incorrect:

A - Cmd 2 will fail because Python variables are not accessible in SQL cells.

B - Cmd 2 will not succeed.

C - The Python variable does not represent a PySpark DataFrame.

D - The first command will not fail.

How to correctly share variables between Python and SQL:

Use `spark.sql()` with f-strings in Python

```
spark.sql(f"""
CREATE VIEW sales_af AS
SELECT * FROM sales WHERE city IN {tuple(countries_af)}
""")
```

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: E

suretly E

upvoted 1 times

  **benni_ale** 1 year, 4 months ago

Selected Answer: E

E , the collect method outputs strings so the python variable bill be a list of string which should not be called as a spark table as in cmd 2

upvoted 1 times

  **imatheushenrique** 1 year, 8 months ago

E. Cmd 1 will succeed and Cmd 2 will fail. `countries_af` will be a Python variable containing a list of strings.

upvoted 1 times

  **juliom6** 1 year, 10 months ago

Selected Answer: E

E is correct.

```
%sql
```

```
create table geo_lookup (continent varchar(2), country varchar(15));
```

```
insert into geo_lookup (continent, country) values
```

```
('AF','Nigeria'),
```

```
('AF','Kenya');
```

```
create table sales (city varchar(15), continent varchar(2));
```

```
insert into sales (city, continent) values
```

```
('Nigeria','AF'),
```

```
('Kenya','AF');
```

```
%python
```

```
countries_af = [x[0] for x in spark.table('geo_lookup').filter("continent='AF'").select('country').collect()]
```

```
%sql
```

```
create view sales_af as
```

```
select *
```

```
from sales
```

```
where city in countries_af
```

```
and continent = "AF";
```

ParseException: [PARSE_SYNTAX_ERROR] Syntax error at or near 'in'.(line 4, pos 11)

i.e. `countries_af` is a python list of strings and can't be used inside a sql statement

upvoted 3 times

  **AndreFR** 1 year, 5 months ago

```
%python
```

```
print(countries_af)
```

```
type(countries_af)
```

upvoted 1 times

🗳️ 👤 **leopedroso1** 1 year, 11 months ago

By simulating this code in databricks we can see an error being thrown in the SQL statement

ParseException:

[PARSE_SYNTAX_ERROR] Syntax error at or near 'IN'.(line 1, pos 38)

== SQL ==

```
SELECT * FROM backup.sales WHERE CITY IN countries_af AND CONTINENT = "AF"
```

upvoted 1 times

🗳️ 👤 **RiktRikt007** 1 year, 12 months ago

Selected Answer: B

B shows the actual flow of spark sql, where E shows the question context, i mean from databricks point of view E never looked, it's true that question state that database has no other tables, so ?? that mean databricks will not check for that particular table ? it will right ? i also confused by "database has no other database statement" and E and B both are right, but again B state "if countries table exists then command 2 will run" here "if" used, but question want to describe the language interoperability, so most of us selected E

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 4 months ago

how could it succed if all people tested sql parse syntax error?

upvoted 1 times

🗳️ 👤 **PrashantTiwari** 2 years ago

E is correct

upvoted 2 times

🗳️ 👤 **Jay_98_11** 2 years ago

Selected Answer: E

vote for E

upvoted 2 times

🗳️ 👤 **kz_data** 2 years ago

Selected Answer: E

E is correct answer

upvoted 1 times

🗳️ 👤 **ismoshkov** 2 years, 3 months ago

Selected Answer: B

<https://docs.databricks.com/en/notebooks/notebooks-code.html#mix-languages>

Variables defined in one language (and hence in the REPL for that language) are not available in the REPL of another language

upvoted 1 times

🗳️ 👤 **Naveenkm** 2 years, 2 months ago

It is mentioned there exists only 2 objects in database. so B is not an option

upvoted 1 times

🗳️ 👤 **Karen1232123** 2 years, 3 months ago

even if it exists, a table or a view won't work in cmd 2

upvoted 2 times

🗳️ 👤 **sturcu** 2 years, 3 months ago

Selected Answer: E

correct

upvoted 1 times

🗳️ 👤 **lucasasterio** 2 years, 5 months ago

Selected Answer: E

correct

upvoted 2 times

🗳️ 👤 **Eertyy** 2 years, 5 months ago

E is right nswer

upvoted 2 times

A Delta table of weather records is partitioned by date and has the below schema: date DATE, device_id INT, temp FLOAT, latitude FLOAT, longitude FLOAT

To find all the records from within the Arctic Circle, you execute a query with the below filter: latitude > 66.3

Which statement describes how the Delta engine identifies which files to load?

- A. All records are cached to an operational database and then the filter is applied
- B. The Parquet file footers are scanned for min and max statistics for the latitude column
- C. All records are cached to attached storage and then the filter is applied
- D. The Delta log is scanned for min and max statistics for the latitude column
- E. The Hive metastore is scanned for min and max statistics for the latitude column

Suggested Answer: B

Community vote distribution

D (100%)

 **taif12340** Highly Voted 2 years, 5 months ago

Answer D:

In the Transaction log, Delta Lake captures statistics for each data file of the table. These statistics indicate per file:

- Total number of records
- Minimum value in each column of the first 32 columns of the table
- Maximum value in each column of the first 32 columns of the table
- Null value counts for in each column of the first 32 columns of the table

When a query with a selective filter is executed against the table, the query optimizer uses these statistics to generate the query result. It leverages them to identify data files that may contain records matching the conditional filter.

For the SELECT query in the question, The transaction log is scanned for min and max statistics for the price column

upvoted 23 times

 **_Lukas_** Most Recent 3 months ago

Selected Answer: D

Correct:

D - Delta Lake stores file-level statistics including minimum, maximum, and null counts for the first 32 columns by default in the Delta transaction log. The Delta engine scans the Delta transaction log to identify which Parquet files could possibly contain records with latitude > 66.3.

Incorrect:

A, C - These options incorrectly suggest caching all records before filtering, which would be inefficient and contradicts how Delta operates.

B - Delta doesn't scan individual Parquet file footers. Instead, it reads the aggregated statistics from the centralized Delta transaction log, which is much more efficient.

E - The Hive metastore doesn't store min/max column statistics for data skipping.

upvoted 1 times

 **7afe201** 7 months, 3 weeks ago

Selected Answer: D

Since the table is not partitioned on Latitude Delta log is used. If latitude was a partition column then Parquet footers would come into play.

upvoted 1 times

 **Gaurav207** 8 months, 4 weeks ago

Selected Answer: D

In the Transaction log, Delta Lake captures statistics for each data file of the table. These statistics indicate per file:

- Total number of records
- Minimum value in each column of the first 32 columns of the table
- Maximum value in each column of the first 32 columns of the table
- Null value counts for in each column of the first 32 columns of the table

upvoted 1 times

🗨️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

suretly D

upvoted 1 times

🗨️ 👤 **JoG1221** 9 months, 3 weeks ago

Selected Answer: D

elta extracts stats from the Parquet footers at write time and stores them in _delta_log.

At query time, Delta reads the stats from the log instead of scanning file footers = faster performance. This is what enables efficient data skipping and query optimization.

upvoted 1 times

🗨️ 👤 **johnserafim** 11 months, 1 week ago

Selected Answer: B

B is correct!

Delta Lake stores min/max statistics for each column in the Parquet file footers. The engine scans these footers to determine if a file contains any data that satisfies the latitude > 66.3 condition. If the minimum latitude in a file is greater than 66.3, the file is loaded. If the maximum latitude is less than or equal to 66.3, the file is skipped.

upvoted 3 times

🗨️ 👤 **akashdesarda** 1 year, 4 months ago

Selected Answer: D

Above mentioned points are correct. If the table was just parquet table then parquet file footer have been used. But since this is Delta table, then delta log is used to scan & skip files. It uses stats written in transaction log.

upvoted 3 times

🗨️ 👤 **AndreFR** 1 year, 5 months ago

Answer D :

Delta data skipping automatically collects the stats (min, max, etc.) for the first 32 columns for each underlying Parquet file when you write data into a Delta table. Databricks takes advantage of this information (minimum and maximum values) at query time to skip unnecessary files in order to speed up the queries.

<https://www.databricks.com/discover/pages/optimize-data-workloads-guide#delta-data>

upvoted 2 times

🗨️ 👤 **saravanan289** 1 year, 5 months ago

Selected Answer: D

Delta table stores file statistics in transaction log

upvoted 2 times

🗨️ 👤 **03355a2** 1 year, 7 months ago

Selected Answer: D

No explanation needed, this is where the information is stored.

upvoted 2 times

🗨️ 👤 **imatheushenrique** 1 year, 8 months ago

D. The Delta log is scanned for min and max statistics for the latitude column

upvoted 1 times

🗨️ 👤 **coercion** 1 year, 8 months ago

Selected Answer: D

Delta log collects statistics like min value, max value, no of records, no of files for each transaction that happens on the table for the first 32 columns (default value)

upvoted 1 times

🗨️ 👤 **Tayari** 1 year, 9 months ago

Selected Answer: D

D is the answer

upvoted 1 times

🗨️ 👤 **arik90** 1 year, 10 months ago

Selected Answer: D

Based on Docu is D I don't know why here is showing B
upvoted 1 times

  **alexvno** 1 year, 11 months ago

Selected Answer: D

Delta log first
upvoted 1 times

  **DavidRou** 1 year, 11 months ago

Selected Answer: D

Statistics on first 32 columns of a table are computed and written in the Delta Log by default.
upvoted 1 times

The data engineering team has configured a job to process customer requests to be forgotten (have their data deleted). All user data that needs to be deleted is stored in Delta Lake tables using default table settings.

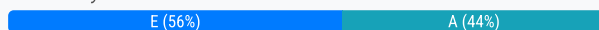
The team has decided to process all deletions from the previous week as a batch job at 1am each Sunday. The total duration of this job is less than one hour. Every Monday at 3am, a batch job executes a series of VACUUM commands on all Delta Lake tables throughout the organization. The compliance officer has recently learned about Delta Lake's time travel functionality. They are concerned that this might allow continued access to deleted data.

Assuming all delete logic is correctly implemented, which statement correctly addresses this concern?

- A. Because the VACUUM command permanently deletes all files containing deleted records, deleted records may be accessible with time travel for around 24 hours.
- B. Because the default data retention threshold is 24 hours, data files containing deleted records will be retained until the VACUUM job is run the following day.
- C. Because Delta Lake time travel provides full access to the entire history of a table, deleted records can always be recreated by users with full admin privileges.
- D. Because Delta Lake's delete statements have ACID guarantees, deleted records will be permanently purged from all storage systems as soon as a delete job completes.
- E. Because the default data retention threshold is 7 days, data files containing deleted records will be retained until the VACUUM job is run 8 days later.

Suggested Answer: A

Community vote distribution



asmayassineg Highly Voted 2 years, 6 months ago

Answer is E, default retention period is 7 days <https://learn.microsoft.com/en-us/azure/databricks/delta/vacuum>
upvoted 24 times

mardigras Highly Voted 1 year, 11 months ago

Selected Answer: A

The answer has to be A.

The deletion is done on Sunday 1am and then the next day Monday 3am, VACUUM was initiated, so one can only time travel for about 24 hours.
upvoted 14 times

csrazdan 1 year, 5 months ago

The default retention threshold for time travel is 7 days. VACUUM which is executed on Monday 3 am will remove history for changes where time travel has expired for previous 7 days.

upvoted 10 times

benni_ale 1 year, 3 months ago

Exactly!

upvoted 2 times

Lukas Most Recent 2 months, 4 weeks ago

Selected Answer: E

Correct:

E - Using VACUUM operation data is permanently removed that are older than 7 days by default.

Sunday 1am - job completes, marking records as deleted in the transaction log (this is how the delete works in delta lake)

Monday 3am (26 hours later): first VACUUM runs, but the deleted data files are less than 7 days old (only 1 day and 2 hours), so they are not removed.

Next Monday 3 am (8 days later counting from Sunday) - VACUUM runs again, and now the files are older than the 7-day threshold, so they are permanently deleted.

Incorrect:

A, B - Default data retention threshold is 7 days, not 24 hours.

C - Time travel is limited by VACUUM operations. Deleted records are not accessible after this operation covers those records.

D - The DELETE operations are logical in delta lake, not immediate physical deletions.

upvoted 1 times

🗳️ 👤 **roopesh1997** 6 months, 1 week ago

Selected Answer: E

Delta Lake's time travel feature allows users to access previous versions of a table. Deleted data is not physically removed from storage immediately – instead, the files are logically marked for deletion, but retained for 7 days by default to support time travel and recovery.

The VACUUM command is responsible for physically removing these obsolete files. However, to prevent accidental data loss, Delta Lake enforces a default retention threshold of 7 days (168 hours), meaning you cannot vacuum files younger than 7 days unless you override the retention check with a flag like retain 0 hours (which is strongly discouraged in production).

So even if a delete is issued on Sunday at 1am and VACUUM is run on Monday at 3am, the deleted files won't be physically removed unless the retention threshold is overridden – hence, they remain accessible via time travel for up to 7 days.

upvoted 1 times

🗳️ 👤 **79f0e18** 7 months, 1 week ago

Selected Answer: B

Since the VACUUM job runs every Monday at 3am, and deletes happen on Sundays at 1am, the deleted data files remain accessible via time travel for approximately 24 hours (until VACUUM removes them).

upvoted 2 times

🗳️ 👤 **paluskapter** 8 months ago

Selected Answer: E

Because of the retention period of 7days, vacuum won't delete it the next day.

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: E

suretly E

upvoted 1 times

🗳️ 👤 **Deep92** 10 months, 1 week ago

Selected Answer: E

Team configured weekly deletion and vacuum will delete weekly data. So this is correct for me.

upvoted 1 times

🗳️ 👤 **Tedet** 11 months, 2 weeks ago

Selected Answer: E

Read question carefully: "all deletions from the previous week", "Every Monday at 3am, a batch job executes a series of VACUUM commands" Logically 8th day.

Apply basics: Default retention threshold for VACUUM is 7 days beyond which on running VACUUM data will be purged.

upvoted 1 times

🗳️ 👤 **fabiospont** 1 year ago

Selected Answer: E

E is correct, because the VACUUM retention is of 168h or 7 days, after statments of deletions.

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: E

Saturday delete puts the deleted records in the transaction log. The retention clock starts ticking. Since the default for keeping the history is 168 hours (7 days), by no means the following Monday vacuum removes since the clock did not reach 168 hours (7 days) - it is at hour 26.

upvoted 2 times

🗳️ 👤 **Anithec0der** 1 year, 1 month ago

Selected Answer: E

I was also thinking in the same way that data will be deleted immediate after the vaccum command is run but it actually logically deletes the data and not physically till the 7 day from the ask of vaccum command. so E is perfect.

upvoted 1 times

🗳️ 👤 **AlejandroU** 1 year, 2 months ago

Selected Answer: E

Answer E. The retention period for time travel queries in Delta Lake is controlled by a 7-day default, not 24 hours. Hence, the statement (Option A) that deleted records may be accessible for around 24 hours is incorrect in the context of Delta Lake's default retention period.

upvoted 1 times

🗨️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: E

Default retention period is 7 days so the vacuum command won't delete the files corresponding to deleted rows at Sunday 1 am but the ones of the previous week instead.

upvoted 1 times

🗨️ 👤 **tangerine141** 1 year, 4 months ago

Selected Answer: E

Delta Lake's default retention threshold for old data files (which allows time travel) is 7 days. This means that even after records are deleted, the files that previously contained those records are kept for 7 days before they are eligible for permanent deletion by the VACUUM command.

The VACUUM command is responsible for permanently deleting the old data files after the retention period. Since the job runs every Monday, this means that data deleted during the previous week will not be fully purged until after the retention period has passed (which would be 8 days after the deletion, considering the weekly processing).

upvoted 2 times

🗨️ 👤 **akashdesarda** 1 year, 4 months ago

Selected Answer: E

Delete job is running as batch job for all requests made current week on Sunday & Vacuum is ran next day . Since there is no mention of change is retention period then it is 7 days. Vacuum will delete data older than 7 days, i.e. it will delete data of previous week & not current week. Current weeks data will be removed in next week's vacuum job.

upvoted 1 times

🗨️ 👤 **fe3b2fc** 1 year, 5 months ago

Selected Answer: E

From the documentation.

"The default retention threshold for data files after running VACUUM is 7 days."

It doesn't matter if VACUUM is ran the following day, the retention period on a default setup is still 7 days after they do the VACUUM on Monday.

upvoted 3 times

A junior data engineer has configured a workload that posts the following JSON to the Databricks REST API endpoint 2.0/jobs/create.

```
{
  "name": "Ingest new data",
  "existing_cluster_id": "6015-954420-peace720",
  "notebook_task": {
    "notebook_path": "/Prod/ingest.py"
  }
}
```

Assuming that all configurations and referenced resources are available, which statement describes the result of executing this workload three times?

- A. Three new jobs named "Ingest new data" will be defined in the workspace, and they will each run once daily.
- B. The logic defined in the referenced notebook will be executed three times on new clusters with the configurations of the provided cluster ID.
- C. Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.
- D. One new job named "Ingest new data" will be defined in the workspace, but it will not be executed.
- E. The logic defined in the referenced notebook will be executed three times on the referenced existing all purpose cluster.

Suggested Answer: C

Community vote distribution

C (100%)

 **arekm** Highly Voted 1 year, 1 month ago

Selected Answer: C

You can totally create 3 jobs with the same name using the UI. REST API is no different. Since no schedule information is in the json, it will not run.
upvoted 5 times

 **Ral17** Most Recent 2 months, 1 week ago

Selected Answer: C

Since no schedule is mentioned, 3 jobs with the same name will be created.
upvoted 1 times

 **_Lukas_** 2 months, 1 week ago

Selected Answer: C

Correct:

C - Each call creates a new job definition in the workspace (Databricks allows duplicate job names, each with a different, unique job_id). It does not trigger execution.

Incorrect:

- A - They will not run daily because there is no schedule definition.
 - B - The endpoint does not execute any logic' it only registers the job definition.
 - D - This option creates three separate job definitions, not one.
 - E - The endpoint does not execute anything.
- upvoted 1 times

 **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

 **AlejandroU** 1 year, 2 months ago

Selected Answer: C

Answer C. 3 new jobs with the same name will be created with each API call, but they won't be executed unless further configuration is made for scheduling or triggering.

upvoted 1 times

 **akashdesarda** 1 year, 4 months ago

Selected Answer: C

Databricks will keep on creating new jobs if you keep running create rept api. Each will have the same name but a different ID. Also no trigger/schedule is mentioned so they wont run.

upvoted 1 times

🗉 👤 **imatheushenrique** 1 year, 8 months ago

C. Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.

upvoted 1 times

🗉 👤 **coercion** 1 year, 8 months ago

Selected Answer: C

Learnt new thing : DBX can have duplicate job names (Job ID will be different). So three jobs will be created with three job ids but it will not run as no schedule is mentioned.

upvoted 3 times

🗉 👤 **franciscodm** 1 year, 11 months ago

C for sure

upvoted 1 times

🗉 👤 **spaceexplorer** 2 years ago

Selected Answer: C

Correct answer is C

upvoted 1 times

🗉 👤 **Jay_98_11** 2 years ago

Selected Answer: C

C is correct

upvoted 1 times

🗉 👤 **kz_data** 2 years ago

Selected Answer: C

Correct answer is C

upvoted 1 times

🗉 👤 **sturcu** 2 years, 3 months ago

Selected Answer: C

databricks jobs create will create a new job with the same name each time it is run.

In order to overwrite the extsting job you need to run databricks jobs reset

upvoted 3 times

🗉 👤 **bob_** 2 years, 4 months ago

Answer is correct. The 3 API calls create 3 jobs with the same name but different job ids. There is no schedule defined so will not execute.

upvoted 3 times

🗉 👤 **Eertyy** 2 years, 4 months ago

correct answer is A, because an api can create can create same job with same name if executed thrice

upvoted 2 times

🗉 👤 **Starvosxant** 2 years, 4 months ago

Ok. So Tell me the schedule these Jobs will run?

Dont know? Why? Maybe because it is not specified, or even configured. So the answer is correct. Create 3 Jobs but none will be executed.

upvoted 2 times

🗉 👤 **Eertyy** 2 years, 4 months ago

to add more: when you execute this workload three times, it will define three new jobs in the workspace, each with the name "Ingest new data."

These jobs can be scheduled to run daily or at a specified frequency, depending on how they are configured.

upvoted 2 times

🗉 👤 **mwypme** 2 years, 4 months ago

therefore answer: D

upvoted 2 times

🗉 👤 **mwypme** 2 years, 4 months ago

only one job, why 3? - because there 3 lines of JSON??

answer should be:

One new job named "Ingest new data" will be defined in the workspace, but it will not be executed.

upvoted 1 times

  **vsydoriak99** 2 years, 4 months ago

Because the create command was run 3 times. Databricks can have several jobs with the same name
upvoted 3 times

An upstream system is emitting change data capture (CDC) logs that are being written to a cloud object storage directory. Each record in the log indicates the change type (insert, update, or delete) and the values for each field after the change. The source table has a primary key identified by the field `pk_id`.

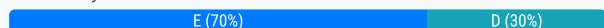
For auditing purposes, the data governance team wishes to maintain a full record of all values that have ever been valid in the source system. For analytical purposes, only the most recent value for each record needs to be recorded. The Databricks job to ingest these records occurs once per hour, but each individual record may have changed multiple times over the course of an hour.

Which solution meets these requirements?

- A. Create a separate history table for each `pk_id` resolve the current state of the table by running a union all filtering the history tables for the most recent state.
- B. Use `MERGE INTO` to insert, update, or delete the most recent entry for each `pk_id` into a bronze table, then propagate all changes throughout the system.
- C. Iterate through an ordered set of changes to the table, applying each in turn; rely on Delta Lake's versioning ability to create an audit log.
- D. Use Delta Lake's change data feed to automatically process CDC data from an external system, propagating all changes to all dependent tables in the Lakehouse.
- E. Ingest all log information into a bronze table; use `MERGE INTO` to insert, update, or delete the most recent entry for each `pk_id` into a silver table to recreate the current table state.

Suggested Answer: E

Community vote distribution



Eertyy Highly Voted 2 years, 5 months ago

The answer given is correct
upvoted 10 times

Eertyy 2 years, 4 months ago

I want to correct my response. It seems the right answer Option D, it leverages Delta Lake's built-in capabilities for handling CDC data. It is designed to efficiently capture, process, and propagate changes, making it a more robust and scalable solution, particularly for large-scale data scenarios with frequent updates and auditing requirements.
upvoted 4 times

sturcu 2 years, 3 months ago

the D states: process CDC data from an external system. so this delta CDF.
upvoted 1 times

Starvosxant 2 years, 4 months ago

Databricks is NOT able to process CDC alone. It needs a intermediare Tool to make it on an object storage and then ingest it.
So how can be D?
upvoted 5 times

Sriramiyer92 1 year, 1 month ago

Actually you were correct in the first go. If you are dealing with small amount of data to changes, CDC is it way to go. But not in this case. (Keywords: For auditing purposes, the data governance team wishes to maintain a full record of all values that have ever been valid in the source system.) Answer is E in that case.
upvoted 1 times

fabiospont 1 year ago

Option D
upvoted 2 times

imatheushenrique Highly Voted 1 year, 8 months ago

E.
This is the correct answer because it meets the requirements of maintaining a full record of all values that have ever been valid in the source system and recreating the current table state with only the most recent value for each record. The code ingests all log information into a bronze table, which preserves the raw CDC data as it is. Then, it uses merge into to perform an upsert operation on a silver table, which means it will insert new records

or update or delete existing records based on the change type and the pk_id columns. This way, the silver table will always reflect the current state of the source table, while the bronze table will keep the history of all changes.

upvoted 7 times

  **_Lukas_** Most Recent 2 months, 4 weeks ago

Selected Answer: E

Correct:

E - satisfies two needs:

Bronze table - auditing purposes; keeps everything ever recorded.

Silver table - analytical purposes; contains only the latest record for each 'pk_id'.

MERGE INTO efficiently update the silver table each hour with the latest changes from bronze.

Incorrect:

A - Create separate history tables per 'pk_id' is very inefficient and messy.

B - Good for the current state, but doesn't store the full history for auditing.

C - Delta Lake table versioning tracks table-level changes, not individual record-level audit history for compliance.

D - Delta's change data feed only tracks changes within Delta tables, not external CDC logs.

upvoted 2 times

  **roopesh1997** 6 months, 1 week ago

Selected Answer: E

This question is about maintaining both an audit log and a current state of a table using CDC (Change Data Capture) data in a Lakehouse architecture.

Bronze table: This is your raw, append-only ingestion layer. Here you capture all CDC records (inserts, updates, deletes) as they come in, with full historical detail. This satisfies the audit requirement.

Silver table: This layer reflects the current state of each record (i.e., the latest version by pk_id), typically using MERGE INTO. This satisfies the analytics requirement.

Since each record may have changed multiple times in an hour, and the job runs hourly, you must process the full CDC batch and consolidate the changes per pk_id – that's exactly what the MERGE INTO operation is designed for.

upvoted 1 times

  **roopesh1997** 6 months, 1 week ago

Selected Answer: E

This question is about maintaining both an audit log and a current state of a table using CDC (Change Data Capture) data in a Lakehouse architecture.

Bronze table: This is your raw, append-only ingestion layer. Here you capture all CDC records (inserts, updates, deletes) as they come in, with full historical detail. This satisfies the audit requirement.

Silver table: This layer reflects the current state of each record (i.e., the latest version by pk_id), typically using MERGE INTO. This satisfies the analytics requirement.

Since each record may have changed multiple times in an hour, and the job runs hourly, you must process the full CDC batch and consolidate the changes per pk_id – that's exactly what the MERGE INTO operation is designed for.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: E

Surely E

upvoted 1 times

  **Tedet** 11 months, 2 weeks ago

Selected Answer: E

Bronze Table (Raw Ingest): You start by ingesting all the change data capture (CDC) records into a bronze table.

Silver Table (Processed State): The silver table represents the most recent state of the data. You would use the MERGE INTO command to process the changes from the bronze table and update the silver table accordingly.

Audit Trail: Since you're ingesting all the data into the bronze table, you maintain a full history of changes that have occurred over time, which satisfies the auditing requirement.

upvoted 2 times

🗳️ 👤 **fabiospont** 1 year ago

D is the option more indicated. CDC is the most effeciently.

upvoted 1 times

🗳️ 👤 **mwynn** 1 year, 1 month ago

Selected Answer: E

If getting External CDC Data (Kafka, etc) no need for CDF! Just ingest to Bronze with (pipelines.reset.allowed = false)

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: E

The question names 2 requirements to keep the data

- archival with all records

- querying with only the currently valid values

CDF is not designed as a permanent storage for archival purposes. It keeps the data to propagate it to downstream applications / workloads. CDF is also purged with the vacuum command, so this would make a very unreliable archival.

Medallion architecture that Databricks promotes seems to be a clear winner.

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

On top of that - CDC with CDF is not automatic. You still need SQL or Python to read the changes and put them somewhere.

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: E

You can only read the change data feed for enabled tables. You must explicitly enable the change data feed option using one of the following methods: TBLPROPERTIES (delta.enableChangeDataFeed = true) . this means it is a delta feature or in other words it is a feature supported by delta tables. the data to process in the question is external so it is not a delta table => can't be B... Hopefully I am correct.

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 2 months ago

i meant can't be D

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: E

E . databricks cdc is not set to process external cdc. if u have external cdc u could send to bronze for auditing purposes and use bronze to get silver where u have only valid records

upvoted 1 times

🗳️ 👤 **databrick_work** 1 year, 4 months ago

E is correct

upvoted 1 times

🗳️ 👤 **spaceexplorer** 2 years ago

Selected Answer: E

The answer is E

upvoted 2 times

🗳️ 👤 **RafaelCFC** 2 years ago

Selected Answer: E

Complimenting kz_data's response, be aware that the data that is being consumed is not a Databrick's CDC data feed object, but rather, CDC coming from somewhere else, that is, just regular data. So, indeed, it can't be processed without another tool.

upvoted 1 times

🗳️ 👤 **kz_data** 2 years ago

Selected Answer: E

Answer E is correct, as the CDC captured from the external database may contain duplicates for the same pk_id (key) due to multiple updates within the processed hour, we need to take the most recent update for the pk_id, and then MERGE into a silver table.

upvoted 2 times

🗳️ 👤 **a560fe1** 2 years, 1 month ago

CDF captures changes only from a Delta table and is only forward-looking once enabled. The CDC logs are writing to object storage. So you would need to ingest those and merge into downstream tables, hence the answer is E

upvoted 2 times

An hourly batch job is configured to ingest data files from a cloud object storage container where each batch represent all records produced by the source system in a given hour. The batch job to process these records into the Lakehouse is sufficiently delayed to ensure no late-arriving data is missed. The user_id field represents a unique key for the data, which has the following schema: user_id BIGINT, username STRING, user_utc STRING, user_region STRING, last_login BIGINT, auto_pay BOOLEAN, last_updated BIGINT

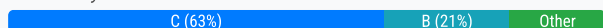
New records are all ingested into a table named account_history which maintains a full record of all data in the same schema as the source. The next table in the system is named account_current and is implemented as a Type 1 table representing the most recent value for each unique user_id.

Assuming there are millions of user accounts and tens of thousands of records processed hourly, which implementation can be used to efficiently update the described account_current table as part of each hourly batch job?

- A. Use Auto Loader to subscribe to new files in the account_history directory; configure a Structured Streaming trigger once job to batch update newly detected files into the account_current table.
- B. Overwrite the account_current table with each batch using the results of a query against the account_history table grouping by user_id and filtering for the max value of last_updated.
- C. Filter records in account_history using the last_updated field and the most recent hour processed, as well as the max last_login by user_id write a merge statement to update or insert the most recent value for each user_id.
- D. Use Delta Lake version history to get the difference between the latest version of account_history and one version prior, then write these records to account_current.
- E. Filter records in account_history using the last_updated field and the most recent hour processed, making sure to deduplicate on username; write a merge statement to update or insert the most recent value for each username.

Suggested Answer: C

Community vote distribution



🗳️ 👤 **RafaelCFC** Highly Voted 👍 2 years, 1 month ago

Selected Answer: C

My reasoning is thus:

The application is based on batch processes, so A is wrong.

Overwriting the table would destroy the Type 1 SCD behavior, so B is wrong.

Comparing versions of account_history would not be efficient, as the whole data would be scanned, so D is wrong. 'username' is not a key column, so we have no guarantee that it's unique, thus de-duplicating by it can yield wrongly grouped sets of rows, so E is not a safe bet, with the information we know.

C is the best option.

upvoted 15 times

🗳️ 👤 **terrku** Highly Voted 👍 1 year, 10 months ago

Selected Answer: B

Type 1 table means the behavior is overwriting.

upvoted 10 times

🗳️ 👤 **_Lukas_** Most Recent 🕒 2 months, 4 weeks ago

Selected Answer: C

Correct:

C - We process only tens of thousands of new records instead of scanning millions of historical records.

Using the max 'last_login' per user ensures merging only the most recent state for each user from that batch.

The MERGE statement efficiently handles both updates and inserts (SCD Type 1).

Incorrect:

A - The 'account_history' is a table, not a file directory. Auto Loader subscribes to file directories in object storage, not Delta Tables.

B - Overwriting the entire table by scanning all of 'account_history' records every hour is extremely inefficient when processing millions of records to update only tens of thousands.

D - Using Delta version history adds unnecessary complexity and overhead compared to direct timestamp filtering.

E - Username may not be unique across users. The 'user_id' is defined as the primary key in the schema.

upvoted 1 times

🗨️ **mansnik8181** 6 months ago

Selected Answer: C

Many people who are fixated on the "overwrite" in option B have overlooked the fact, that the overwrite is happening by querying the history table, grouping on user_id and then filtering (probably a having clause) the records on the max(last_updated). What do you think will happen to the other records which have not been updated in the last hour?. they will get overwritten and this will lead to missing records.

Option C is the best option here.

upvoted 1 times

🗨️ **363c4c5** 7 months ago

Selected Answer: C

It depends on whether we receive the full list of user accounts each time. It says: "each batch represents all records produced by the source system in a given hour", so if we use option B, we might miss some accounts.

upvoted 2 times

🗨️ **thotwielder** 8 months ago

Selected Answer: B

c: wrong. it is using last_login, but the key is user_id, no reason to group by last_login.

b: correct. group by key (user_id), and get the latest record. overwrite is very efficient.

upvoted 2 times

🗨️ **thotwielder** 8 months ago

update. to clarify c is wrong not because group by last_login. c is wrong because merge is not as efficient as overwrite because there are millions of user which means the account_current table is huge and merge needs to compare source and target table which is time consuming compared with simple overwrite.

upvoted 1 times

🗨️ **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

🗨️ **JoG1221** 9 months, 3 weeks ago

Selected Answer: C

This approach is:

Efficient: Only processes incremental data, not the full table

Accurate: Ensures latest record per user_id

Scalable: Works well with millions of users and hourly ingestion

upvoted 1 times

🗨️ **Tedet** 11 months, 2 weeks ago

Selected Answer: C

This option proposes filtering the records in the account_history table for the most recent records based on the last_updated field, which is exactly what you want to do to get the most recent value.

The use of a MERGE statement ensures that only the most recent records are inserted or updated in the account_current table.

This method avoids full overwrites of the account_current table and only updates records that have actually changed, which is efficient for large datasets.

Conclusion: This is the most efficient approach because it ensures only the most recent data is merged, and it avoids unnecessary full table rebuilds.

upvoted 1 times

🗨️ **Ananth4Sap** 1 year, 3 months ago

C is correct because

B is wrong as it says filtering the max value of last updated + overwriting we will miss some valid records.

two valid scenarios: 1-Filtering the max value of last updated +merging (Option-c)

2.use window function on last update, filter and then overwrite (no options)

upvoted 3 times

🗨️ **benni_ale** 1 year, 3 months ago

Selected Answer: C

A. NO. Batch job required so AutoLoader and StructuredStreaming unnecessarily complex solutions.

B. NO. A full overwrite of the table is not efficient.

C. YES. Seems it is filtering and merging on the id by using as less data as reasonable in the merge statement, why not?

D. NO. Difference operation is very ineffecient for this purpose

E. NO. Username is not key

upvoted 3 times

🗨️ 👤 **Dhusanth** 1 year, 6 months ago

Selected Answer: C

C is correct

upvoted 1 times

🗨️ 👤 **faraaz132** 1 year, 6 months ago

C is correct because:

A record might have multiple changes and we need to select the most recent change that happened on that record. For that we will use max Log in date and rank it using window function, then we filter on rank=1 and use it for UPSERT operation.

upvoted 1 times

🗨️ 👤 **Karunakaran_R** 1 year, 8 months ago

I think B ,Type 1 table must overwrite the data

upvoted 1 times

🗨️ 👤 **Freyr** 1 year, 8 months ago

C is correct.

A Type 1 table means that it performs an "upsert" operation without maintaining history, based on the merge condition. This means that new records are inserted, and existing records are updated. As a result, the merge process does not retain historical records.

Therefore, the correct answer is C.

upvoted 3 times

🗨️ 👤 **PrashantTiwari** 1 year, 12 months ago

C is correct

upvoted 1 times

🗨️ 👤 **DAN_H** 2 years ago

Selected Answer: C

answer is C

upvoted 2 times

A table in the Lakehouse named `customer_churn_params` is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

The churn prediction model used by the ML team is fairly stable in production. The team is only interested in making predictions on records that have changed in the past 24 hours.

Which approach would simplify the identification of these changed records?

- A. Apply the churn model to all rows in the `customer_churn_params` table, but implement logic to perform an upsert into the predictions table that ignores rows where predictions have not changed.
- B. Convert the batch job to a Structured Streaming job using the complete output mode; configure a Structured Streaming job to read from the `customer_churn_params` table and incrementally predict against the churn model.
- C. Calculate the difference between the previous model predictions and the current `customer_churn_params` on a key identifying unique customers before making new predictions; only make predictions on those customers not in the previous predictions.
- D. Modify the overwrite logic to include a field populated by calling `spark.sql.functions.current_timestamp()` as data are being written; use this field to identify records written on a particular date.
- E. Replace the current overwrite logic with a merge statement to modify only those records that have changed; write logic to make predictions on the changed records identified by the change data feed.

Suggested Answer: E

Community vote distribution

E (100%)

  **Eertyy** Highly Voted 2 years, 5 months ago

E is right answer
upvoted 7 times

  **_Lukas_** Most Recent 2 months, 1 week ago

Selected Answer: E

Correct:

E - By replacing overwrite with a MERGE statement, only records with actual data changes are modified. By enabling CDF on the table, it automatically captures row changes, logging row changes in additional CDF metadata columns.

Incorrect:

- A - This still requires predicting on all customers, which is exactly what we want to avoid.
 - B - Converting batch job to streaming adds unnecessary complexity for once-nightly workload. This doesn't help identify what changed.
 - C - It compares predictions, not customer data. Changes in data don't always cause prediction changes, and vice versa.
 - D - Overwriting the whole table gives every row the same timestamp so it doesn't help.
- upvoted 2 times

  **_Lukas_** 2 months, 1 week ago

Summary:

- There is a table with customer info 'customer_churn_params'.
- This table is recreated every night from scratch.
- The ML team only wants to run their churn model on customers whose data changed in the past day.
- But because the table is overwritten, it's hard to tell which customers actually changed.

Simplified question:

How can we easily figure out which customers' data changed since yesterday, so the ML team doesn't need to run the model on everyone?
upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: E

suretly E
upvoted 1 times

🗳️ 👤 **JoG1221** 9 months, 3 weeks ago

Selected Answer: E

Option E aligns with Delta Lake best practices:

Efficient updates using MERGE

Precise tracking with Change Data Feed

Streamlined ML inference on only updated records

upvoted 1 times

🗳️ 👤 **AlHerd** 10 months, 2 weeks ago

Selected Answer: E

E.

While both D and E look right D only adds a timestamp but doesn't track whether the record content actually changed, leading to false positives.

upvoted 1 times

🗳️ 👤 **Tedet** 11 months, 2 weeks ago

Selected Answer: D

Evaluation:

Adding a `current_timestamp()` field to each record during the overwrite allows you to track when each record was written.

This makes it easy to identify records that have been updated or inserted recently by filtering on this timestamp field (e.g., filtering for records written in the past 24 hours).

This approach simplifies identifying recently changed records because you can easily filter for the most recent data and then run churn predictions only on those records.

Conclusion: This is a simple and efficient solution. It allows you to track changes by using a timestamp, making it easy to filter and predict only on changed records without complex logic.

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: E

A, B, and C don't make sense. Adding a timestamp with an overwrite logic that overwrites everything does not make sense - all records would have a timestamp from the last night. That would be not helpful in identifying what changed.

E is correct. Only write changes, use CDF to identify the changes and apply the model.

upvoted 2 times

🗳️ 👤 **Sriramiyer92** 1 year, 1 month ago

Selected Answer: E

While both E and D are correct.

E is more accurate, given the scenario

upvoted 1 times

🗳️ 👤 **janeZ** 1 year, 2 months ago

Selected Answer: D

D is the right answer

upvoted 1 times

🗳️ 👤 **Melik3** 1 year, 6 months ago

I don't understand why E is correct. With E we are updating only data needed but we are then doing prediction on the whole table which means that we are doing again predictions on not changing records which is not efficient

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 3 months ago

"write logic to make predictions on the CHANGED records identified by the change data feed". the only thing partially wrong about E is that it has never been stated that the table has a change data feed enables.

upvoted 1 times

🗳️ 👤 **Tedet** 11 months, 2 weeks ago

You are 100pc correct Melik3. Reason being consequences of E are below.

A merge statement ensures that only the records that have changed are updated, but it doesn't directly address how to identify which records have changed within the last 24 hours.

Using a change data feed can help track changes, but it may not be the most efficient method unless the infrastructure is set up for real-time change tracking.

The complexity of managing and using the change data feed for just 24-hour changes might introduce unnecessary overhead.

Conclusion: This is a good option, but it could be more complex to implement than simply adding a `current_timestamp()` field.

upvoted 1 times

  **leopedroso1** 1 year, 11 months ago

E is the correct one. By removing overwrite with merge, this will lead to an UPSERT causing updating only the data needed (When Matched Update + When not mached insert clauses). Then, with the CDC the capability of identifying is also satisfied.

upvoted 2 times

  **AziLa** 2 years ago

correct ans is E

upvoted 1 times

  **kz_data** 2 years ago

Selected Answer: E

E is correct

upvoted 1 times

  **sturcu** 2 years, 3 months ago

Selected Answer: E

E is Correct

upvoted 3 times

A table is registered with the following code:

```
CREATE TABLE recent_orders AS (
  SELECT a.user_id, a.email, b.order_id, b.order_date
  FROM
    (SELECT user_id, email
     FROM users) a
  INNER JOIN
    (SELECT user_id, order_id, order_date
     FROM orders
     WHERE order_date >= (current_date() - 7)) b
  ON a.user_id = b.user_id
)
```

Both users and orders are Delta Lake tables. Which statement describes the results of querying recent_orders?

- A. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.
- B. All logic will execute when the table is defined and store the result of joining tables to the DBFS; this stored data will be returned when the table is queried.
- C. Results will be computed and cached when the table is defined; these cached results will incrementally update as new records are inserted into source tables.
- D. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.
- E. The versions of each source table will be stored in the table transaction log; query results will be saved to DBFS with each query.

Suggested Answer: D

Community vote distribution

B (81%)

D (19%)

practitioner Highly Voted 1 year, 5 months ago

Selected Answer: D

The question says "Which statement describes the results of querying recent_orders?"

The question doesn't ask about the code snippet itself. This question is about the logic of "select * from recent_orders" after the creation of recent_orders.

answer is D

D is the right answer

upvoted 22 times

fe3b2fc 1 year, 5 months ago

Not sure how so many people misunderstood the actual question. It already says at the top that the table is registered as the code given, they're not executing the code again.

upvoted 3 times

Onobhas01 1 year, 5 months ago

Why are people so fixed on how the table was created, question says what happens when a query is run against the table.

upvoted 2 times

benni_ale 1 year, 3 months ago

option D mentions "return the result of joining the valid versions of the source tables" but that's not true. when u interrogate the table resulting from a join of two tables u are not re-performing joins operations at query time anymore an the version is the one from the last time the CTAS statement was executed

upvoted 5 times

HairyTorso 1 year, 1 month ago

As Benni_ale said - CTAS can be used to create a snapshot of a table. The data will be stored and not updated further, unless table is explicitly updated.

upvoted 2 times

🗄️ 👤 **asmayassineg** Highly Voted 2 years, 6 months ago

Correct answer is B. table is created and data of join will be stored on DBFS and it will be returned on query time
upvoted 21 times

🗄️ 👤 **_Lukas_** Most Recent 2 months, 1 week ago

Selected Answer: B

Correct:

B - This command creates a table. Delta lake writes the results of the SQL query as Parquet files to DBFS. The table becomes a static snapshot of the data at creation time.

Incorrect:

A, D - Both describe VIEW behavior, where the query logic executes each time the view is queried.

C - CTAS creates a static snapshot that does not incrementally update.

E - The transaction log tracks changes to the 'recent_orders' table, not versions of the source table. Results are saved once at creation time.

upvoted 1 times

🗄️ 👤 **2b2fa03** 3 months, 2 weeks ago

Selected Answer: B

In the Question they have mentioned that Correct statement that defines the querying of recent_order table.

Now, The table is already created that is existing table(registered table), So the create query is already executed and table is available in DBFS.

Hence, when the querying the recent_orders table, it fetches from DBFS the existing table.

upvoted 2 times

🗄️ 👤 **gloomy_marmot** 6 months, 2 weeks ago

Selected Answer: B

Correct answer is B, because:

1) Table "recent_orders" was already created by provided SQL.

2) The question is "Which statement describes the results of querying recent_orders?"

It's mean that D isn't correct: "All logic will execute at query time and return the result of joining the VALID versions of the source tables at THE TIME the query began."

So D means that the joining executes at the time of another query to the table.

upvoted 1 times

🗄️ 👤 **Vitality** 8 months ago

Selected Answer: D

Definitely D, c'mon guys.

upvoted 1 times

🗄️ 👤 **363c4c5** 6 months, 3 weeks ago

Definitely B, c'mon guys.

for CTAS joins worked when table was defined, when you query it it will not do anything, just return data

upvoted 1 times

🗄️ 👤 **c315d10** 8 months, 1 week ago

Selected Answer: D

d is correct

upvoted 1 times

🗄️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

🗄️ 👤 **JoG1221** 9 months, 3 weeks ago

Selected Answer: B

It materializes the result and saves the data into a Delta Lake table stored on DBFS (or other compatible storage).

upvoted 1 times

🗄️ 👤 **lakime** 10 months, 2 weeks ago

Selected Answer: D

It's CTAAS, nothing is being stored in DBFS...

upvoted 1 times

🗨️ 👤 **Tedet** 11 months, 2 weeks ago

Selected Answer: B

It's CTAS. So snapshot at the time of creation will be returned.

upvoted 3 times

🗨️ 👤 **ptty** 11 months, 2 weeks ago

Selected Answer: B

It's B because this is different from creating a view (which would use CREATE VIEW instead), where the query logic would be executed each time the view is accessed.

upvoted 3 times

🗨️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: B

B - it is a table.

Query time answers assume we are talking about a view, which we aren't. Table is not automatically updated whenever the tables used in CTAS change - it is a standalone entity.

upvoted 2 times

🗨️ 👤 **Sriramiyer92** 1 year, 1 month ago

Selected Answer: B

The answer is B.

Pls note it is CTAS statement and not a subquery.

upvoted 1 times

🗨️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: B

B is correct

upvoted 1 times

🗨️ 👤 **nedlo** 1 year, 3 months ago

Selected Answer: B

Only logic there is inside create statemetn and it will execute once while executing "create table" statement. Further select queries will only select any data that was inserted during create table statement , data wont by updated automatically. So B

upvoted 1 times

🗨️ 👤 **benni_ale** 1 year, 3 months ago

i think B

upvoted 1 times

A production workload incrementally applies updates from an external Change Data Capture feed to a Delta Lake table as an always-on Structured Stream job. When data was initially migrated for this table, OPTIMIZE was executed and most data files were resized to 1 GB. Auto Optimize and Auto Compaction were both turned on for the streaming production job. Recent review of data files shows that most data files are under 64 MB, although each partition in the table contains at least 1 GB of data and the total table size is over 10 TB.

Which of the following likely explains these smaller file sizes?

- A. Databricks has autotuned to a smaller target file size to reduce duration of MERGE operations
- B. Z-order indices calculated on the table are preventing file compaction
- C. Bloom filter indices calculated on the table are preventing file compaction
- D. Databricks has autotuned to a smaller target file size based on the overall size of data in the table
- E. Databricks has autotuned to a smaller target file size based on the amount of data in each partition

Suggested Answer: A

Community vote distribution

A (100%)

 **cotardo2077** Highly Voted 2 years, 5 months ago

Selected Answer: A

<https://docs.databricks.com/en/delta/tune-file-size.html#autotune-table 'Autotune file size based on workload'>

upvoted 13 times

 **Vitality** 8 months ago

MERGE operations overall benefit from fewer, larger files, so reducing file size to improve MERGE performance would be counterintuitive. E can be the only correct answer.

upvoted 1 times

 **meatpoof** 1 year ago

Your source doesn't support your answer. It doesn't mention anything about autotuning to increase the speed of merges

upvoted 3 times

 **_Lukas_** Most Recent 2 months, 1 week ago

Selected Answer: A

Correct:

A - Databricks detected that you're performing frequent MERGE operations (updating existing records) and automatically switched to smaller files to make those updates faster.

Incorrect:

B, C - Z-order or Bloom filter indices do not prevent file compaction. They are optimization features.

D - The overall size of the data in the table is around 10 TB, so it should generate 1 GB files, not 64 MB files.

E - Partition size doesn't drive this decision, but workload type does.

upvoted 1 times

 **_Lukas_** 2 months, 1 week ago

The situation:

- You have a large Delta Lake table (> 10 TB).
- It's updated continuously by a Structured Streaming job using CDC data.
- When the data was first loaded, you ran 'OPTIMIZE', and the files were around 1 GB each - that's ideal for analytical reads.
- Now, even though each partition has plenty of data (>= 1GB), most files have shrunk to under 64 MB
- Auto Optimize and Auto Compaction are turned on.

upvoted 1 times

 **gizzamo** 4 months, 4 weeks ago

Selected Answer: A

A is correct.

Reason: for merge-heavy, streaming workloads Databricks (and the guidance docs) recommend—and will autotune toward—smaller target file sizes (often in the 16–64 MB range) to reduce the amount of data scanned/rewritten during MERGE operations, which shortens MERGE duration; that explains why you see many ~64 MB files despite earlier OPTIMIZE to 1 GB.

<https://www.databricks.com/discover/pages/optimize-data-workloads-guide>

upvoted 1 times

🗲️ 👤 **2262b46** 6 months, 1 week ago

Selected Answer: E

Should be E

upvoted 1 times

🗲️ 👤 **Billybob0604** 6 months, 2 weeks ago

Selected Answer: E

Databricks Auto Optimize/Auto Compaction bases its file sizing decisions primarily on partition size, not just the type of operation (e.g., MERGE, INSERT, UPDATE).

upvoted 1 times

🗲️ 👤 **363c4c5** 6 months, 3 weeks ago

Selected Answer: A

It could be D, but for 100 TB it remains under 64 MB. Databricks autotunes the target file size based on the overall size of the table, and for tables larger than 10 TB, the target file size should be 1 GB. So, the answer is A.

upvoted 1 times

🗲️ 👤 **a67e3d1** 6 months, 4 weeks ago

Selected Answer: A

Autotune file size based on workload

Databricks recommends setting the table property `delta.tuneFileSizesForRewrites` to true for all tables that are targeted by many MERGE or DML operations, regardless of Databricks Runtime, Unity Catalog, or other optimizations. When set to true, the target file size for the table is set to a much lower threshold, which accelerates write-intensive operations.

If not explicitly set, Databricks automatically detects if 9 out of last 10 previous operations on a Delta table were MERGE operations and sets this table property to true. You must explicitly set this property to false to avoid this behavior.

upvoted 1 times

🗲️ 👤 **79f0e18** 7 months, 1 week ago

Selected Answer: E

Databricks' Auto Optimize and Auto Compaction features dynamically tune file sizes based on the data volume per partition, not the entire table size.

Why the other options are incorrect:

A. Databricks doesn't reduce file size to optimize MERGE performance. Smaller files actually hurt MERGE performance due to file proliferation.

B. Z-Ordering affects read performance and OPTIMIZE behavior, but it doesn't prevent Auto Compaction from running or tuning file size.

C. Bloom filters are for indexing string fields for faster point lookups and don't affect file size or compaction logic.

D. Autotuning is not based on overall table size, but rather the individual partition size and data volume written in micro-batches.

upvoted 2 times

🗲️ 👤 **Vitality** 8 months ago

Selected Answer: E

Definitely E. Auto Compaction is tuning file sizes based on the data volume per partition. This helps optimize performance for streaming workloads, where smaller files can reduce latency and improve the efficiency of incremental updates.

upvoted 1 times

🗲️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: A

Surely A

upvoted 1 times

🗲️ 👤 **JoG1221** 9 months, 3 weeks ago

Selected Answer: E

Databricks has autotuned to a smaller target file size based on the amount of data in each partition

upvoted 1 times

🗄️ 👤 **JoG1221** 9 months, 3 weeks ago

Selected Answer: D

File size tuning is not based on total table size or merge ops, but on partition-level dynamics.

upvoted 2 times

🗄️ 👤 **JoG1221** 9 months, 3 weeks ago

Answer is E

upvoted 1 times

🗄️ 👤 **kishanu** 10 months, 1 week ago

Selected Answer: E

Databricks Auto Optimize and Auto Compaction features are designed to optimize file sizes dynamically for better performance and efficiency in Delta Lake. These features do not use a fixed target file size like 1 GB, but instead autotune file sizes based on partition-level characteristics.

In this case:

Each partition has at least 1 GB of data, and the overall table is large (10+ TB), but...

You see many small files <64 MB, which seems suboptimal at first.

However, Databricks may intentionally use smaller file sizes within partitions when:

The data change rate is high (as in a streaming CDC feed).

Smaller file sizes help with faster read times, reduced shuffle, and quicker MERGE operations during structured streaming.

The amount of new data added per batch or microbatch is small, leading to many smaller files, especially when auto compaction determines this improves job performance at runtime.

This makes option E the most accurate description of what's happening.

upvoted 2 times

🗄️ 👤 **AIHerd** 10 months, 2 weeks ago

Selected Answer: A

An always-on Structured Streaming job that applies updates from a Change Data Capture (CDC) feed uses frequent MERGE operations to apply changes (inserts, updates, deletes) to the Delta table.

Because these MERGE operations are constant and high-frequency, Databricks may autotune to a smaller target file size to reduce the duration and overhead of each merge. This behaviour is described explicitly in the documentation.

So, with this in view, the correct answer is A

upvoted 1 times

🗄️ 👤 **EZZALDIN** 10 months, 2 weeks ago

Selected Answer: E

The primary goal of Auto Optimize and Auto Compaction in a streaming job isn't specifically to reduce MERGE duration. Instead, these features adjust file sizes based on the incremental volume of data being ingested in each micro-batch within a partition. Even though each partition contains around 1 GB of data (from the original OPTIMIZE), the streaming job writes small batches that are compacted into smaller files (often under 64 MB) because that's the amount of new data per batch.

So, Option E is more accurate: Databricks auto-tunes the target file size based on the amount of data in each partition (from each micro-batch), not specifically to speed up MERGE operations.

upvoted 1 times

🗄️ 👤 **Tedet** 11 months, 2 weeks ago

Selected Answer: E

Option E is more accurate because Delta Lake's Auto Optimize and Auto Compaction are designed to adjust file sizes based on the streaming data partitioning, which inherently leads to smaller files over time. The system auto-tunes file sizes as new, incremental data is ingested and partitioned. Option A is plausible, but optimizing file sizes for MERGE operations is not the core focus of Auto Optimize in this case. The system's auto-tuning mechanism is more about managing file sizes based on the streaming data's partition size and maintaining efficient reads/writes, rather than directly optimizing for MERGE performance.

upvoted 3 times

🗄️ 👤 **Tedet** 11 months, 2 weeks ago

Selected Answer: A

Options Behavior

auto (recommended) Tunes target file size while respecting other autotuning functionality. Requires Databricks Runtime 10.4 LTS or above.

legacy Alias for true. Requires Databricks Runtime 10.4 LTS or above.

true Use 128 MB as the target file size. No dynamic sizing.

false Turns off auto compaction. Can be set at the session level to override auto compaction for all Delta tables modified in the workload.

upvoted 1 times

Which statement regarding stream-static joins and static Delta tables is correct?

- A. Each microbatch of a stream-static join will use the most recent version of the static Delta table as of each microbatch.
- B. Each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization.
- C. The checkpoint directory will be used to track state information for the unique keys present in the join.
- D. Stream-static joins cannot use static Delta tables because of consistency issues.
- E. The checkpoint directory will be used to track updates to the static Delta table.

Suggested Answer: A

Community vote distribution

A (100%)

  **Eertyy** Highly Voted 2 years, 4 months ago

B is the right answer as Option B is more typical for stream-static joins, as it provides a consistent static DataFrame snapshot for the entire job's duration. Option A might be suitable in specialized cases where you need real-time updates of the static DataFrame for each microbatch.

upvoted 12 times

  **hamzaKhribi** 2 years, 2 months ago

Answer is A, When Azure Databricks processes a micro-batch of data in a stream-static join, the latest valid version of data from the static Delta table joins with the records present in the current micro-batch

from <https://learn.microsoft.com/en-us/azure/databricks/structured-streaming/delta-lake>

upvoted 13 times

  **arekm** 1 year, 1 month ago

The explanation suggests the author would like the stream-static join to work in this way. However, it works as it does - see the first sentence in here: <https://learn.microsoft.com/en-us/azure/databricks/transform/join#stream-static>

upvoted 3 times

  **BrianNguyen95** Highly Voted 2 years, 5 months ago

correct answer is A

upvoted 6 times

  **_Lukas_** Most Recent 2 months, 1 week ago

Selected Answer: A

Correct:

A - Each time Databricks processes a new batch of streaming data (a microbatch), it grabs the latest version of the static Delta table and joins it with that batch.

Incorrect:

B - The static table is not fixed at job initialization - it's refreshed with each microbatch.

C - Stream-static joins don't maintain state for join keys.

D - Stream-static joins explicitly support static Delta tables.

E - Checkpoints don't track static table updates.

upvoted 1 times

  **_Lukas_** 2 months, 1 week ago

How Stream-Static Joins work:

Imagine you're processing incoming orders (streaming data) and need to add customer information from a customer database (static Delta table):

11:00 AM: New orders arrive → Databricks reads the current customer table and joins it

11:05 AM: More orders arrive → Databricks reads the customer table again (now with any updates) and joins it

11:10 AM: Even more orders → Databricks reads the customer table again and joins it

If a customer updates their address at 11:02 AM, any orders processed at 11:05 AM or later will automatically have the new address.

upvoted 1 times

  **8c1d06f** 4 months ago

Selected Answer: B

In a stream-static join, the static side is read once at query start (a single Delta snapshot) and reused for all microbatches. Changes to that static table aren't picked up until you restart the streaming job (or explicitly re-read it). The checkpoint tracks streaming progress/state, not updates to the static table

upvoted 1 times

  **8c1d06f** 4 months ago

Selected Answer: B

In a stream-static join, the static side is read once at query start (a single Delta snapshot) and reused for all microbatches. Changes to that static table aren't picked up until you restart the streaming job (or explicitly re-read it). The checkpoint tracks streaming progress/state, not updates to the static table—so A, C, and E are incorrect, and D is false.

upvoted 1 times

  **e671267** 4 months, 3 weeks ago

Selected Answer: B

<https://docs.databricks.com/gcp/en/transform/join#:~:text=A%20stream%2Dstatic%20join%20joins,in%20the%20current%20micro%2Dbatch>.

upvoted 1 times

  **markp123** 5 months, 4 weeks ago

Selected Answer: B

Voting B. Was torn between A and B. When joining a streaming DataFrame with a static Delta table, the static side is loaded once when the query starts and remains fixed throughout the job's (streams) lifetime. Until the stream is restarted.

upvoted 1 times

  **Eniyan** 6 months, 1 week ago

Selected Answer: A

statement B describes a scenario where the static table acts like a fixed snapshot taken only at the very start of the streaming job, which is not how Databricks stream-static joins with Delta tables are designed to function. The design allows the streaming job to dynamically pick up the most current state of the static Delta table for each new batch of streaming data, ensuring that your joined results are always up-to-date with your reference data.

upvoted 1 times

  **Billybob0604** 6 months, 2 weeks ago

Selected Answer: B

When joining a streaming DataFrame with a static Delta table, the static side is loaded once when the query starts and remains fixed throughout the job's lifetime.

upvoted 1 times

  **a67e3d1** 6 months, 4 weeks ago

Selected Answer: A

from docs

A stream-static join joins the latest valid version of a Delta table (the static data) to a data stream using a stateless join.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: A

Surely A

upvoted 1 times

  **JoG1221** 9 months, 3 weeks ago

Selected Answer: B

In a stream-static join, Spark treats the static Delta table as a constant snapshot at the time the streaming query starts. This means:

The static table is loaded once when the stream starts.

All micro-batches of the stream will join with that same version of the static table.

Any updates made to the static table after the stream starts will not be reflected in the join, unless the stream is restarted.

upvoted 1 times

  **arekm** 1 year, 1 month ago

Selected Answer: A

A is correct, see: <https://learn.microsoft.com/en-us/azure/databricks/transform/join#stream-static>

upvoted 3 times

  **Sriramiyer92** 1 year, 1 month ago

Selected Answer: B

A stream-static join joins the latest valid version of a Delta table (the static data) to a data stream using a stateless join.

When Databricks processes a micro-batch of data in a stream-static join, the latest valid version of data from the static Delta table joins with the records present in the current micro-batch. Because the join is stateless, you do not need to configure watermarking and can process results with low latency. The data in the static Delta table used in the join should be slowly-changing.

upvoted 1 times

  **Sriramiyer92** 1 year, 1 month ago

Selected Answer: A

<https://docs.databricks.com/en/transform/join.html#stream-static>

upvoted 1 times

  **akashdesarda** 1 year, 4 months ago

Selected Answer: A

This is straight from docs, "A stream-static join joins the latest valid version of a Delta table (the static data) to a data stream using a stateless join.

When Azure Databricks processes a micro-batch of data in a stream-static join, the latest valid version of data from the static Delta table joins with the records present in the current micro-batch. Because the join is stateless, you do not need to configure watermarking and can process results with low latency. The data in the static Delta table used in the join should be slowly-changing."

<https://learn.microsoft.com/en-us/azure/databricks/transform/join#stream-static>

upvoted 1 times

  **kz_data** 2 years ago

Selected Answer: A

correct answer is A

upvoted 1 times

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Events are recorded once per minute per device.

Streaming DataFrame df has the following schema:

"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"

Code block:

```
df.withWatermark("event_time", "10 minutes")
  .groupBy(
    _____
    "device_id"
  )
  .agg(
    avg("temp").alias("avg_temp"),
    avg("humidity").alias("avg_humidity")
  )
  .writeStream
  .format("delta")
  .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

- A. to_interval("event_time", "5 minutes").alias("time")
- B. window("event_time", "5 minutes").alias("time")
- C. "event_time"
- D. window("event_time", "10 minutes").alias("time")
- E. lag("event_time", "10 minutes").alias("time")

Suggested Answer: B

Community vote distribution

B (100%)

  **_Lukas_** 2 months, 1 week ago

Selected Answer: B

Correct:

B - The 'window()' function creates time-based intervals for grouping streaming data. When you specify window("event_time", "5 minutes"), Spark automatically divides the event timeline into non-overlapping 5-minute buckets and groups events that fall within each bucket.

Incorrect:

- A - 'to_interval' doesn't exist as a standard Spark function for windowing.
 - C - Using just 'event_time' would group by exact timestamps instead of intervals.
 - D - This uses 10 minutes instead of the required 5 minutes.
 - E - The 'lag' function is for accessing previous row values, not for creating time windows.
- upvoted 1 times

  **42f87fd** 7 months, 3 weeks ago

Selected Answer: B

non-overlapping 5 minute interval --> so use window("event_time", "5 minutes")

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

Surely B

upvoted 1 times

  **imatheushenrique** 1 year, 2 months ago

B. window("event_time", "5 minutes").alias("time")

In Structured Streaming, expressing such windows on event-time is simply performing a special grouping using the window() function. For example, counts over 5 minute tumbling (non-overlapping) windows on the eventTime column in the event is as following.

upvoted 4 times

🗨️ 👤 **Jay_98_11** 1 year, 6 months ago

Selected Answer: B

correct B

upvoted 2 times

🗨️ 👤 **kz_data** 1 year, 7 months ago

Selected Answer: B

B is correct

upvoted 1 times

🗨️ 👤 **BIKRAM063** 1 year, 9 months ago

Selected Answer: B

Window of 5 mins

upvoted 2 times

🗨️ 👤 **sturcu** 1 year, 10 months ago

Selected Answer: B

B is correct:

<https://www.databricks.com/blog/2017/05/08/event-time-aggregation-watermarking-apache-sparks-structured-streaming.html>

upvoted 2 times

🗨️ 👤 **Eertyy** 1 year, 10 months ago

answer is B

upvoted 2 times

🗨️ 👤 **thxsgod** 1 year, 11 months ago

Selected Answer: B

Correct, B.

upvoted 4 times

A data architect has designed a system in which two Structured Streaming jobs will concurrently write to a single bronze Delta table. Each job is subscribing to a different topic from an Apache Kafka source, but they will write data with the same schema. To keep the directory structure simple, a data engineer has decided to nest a checkpoint directory to be shared by both streams.

The proposed directory structure is displayed below:

```
./bronze
├── __checkpoint
├── __delta_log
├── year_week=2020_01
├── year_week=2020_02
└── ...
```

Which statement describes whether this checkpoint directory structure is valid for the given scenario and why?

- A. No; Delta Lake manages streaming checkpoints in the transaction log.
- B. Yes; both of the streams can share a single checkpoint directory.
- C. No; only one stream can write to a Delta Lake table.
- D. Yes; Delta Lake supports infinite concurrent writers.
- E. No; each of the streams needs to have its own checkpoint directory.

Suggested Answer: E

Community vote distribution

E (90%)

10%

 **thxgod** Highly Voted 2 years, 5 months ago

Selected Answer: E

Correct, E.

Source:

[https://docs.databricks.com/en/optimizations/isolation-](https://docs.databricks.com/en/optimizations/isolation-level.html#:~:text=If%20a%20streaming%20query%20using%20the%20same%20checkpoint%20location%20is%20started%20multiple%20times%20concurrent)

[level.html#:~:text=If%20a%20streaming%20query%20using%20the%20same%20checkpoint%20location%20is%20started%20multiple%20times%20concurrent](https://docs.databricks.com/en/optimizations/isolation-level.html#:~:text=If%20a%20streaming%20query%20using%20the%20same%20checkpoint%20location%20is%20started%20multiple%20times%20concurrent)
upvoted 11 times

 **_Lukas_** Most Recent 2 months, 3 weeks ago

Selected Answer: E

Correct:

E - Each Structured Streaming query must have its own unique checkpoint location. The checkpoint directory tracks query-specific state information, including which records have been processed and the exact progress of the particular stream.

If two streaming jobs share the same checkpoint directory, they would corrupt each other's state tracking, leading to data inconsistency and processing errors.

Incorrect:

A - Streaming checkpoints are managed separately from the Delta transaction log.

B - Sharing checkpoints between streams causes state conflicts.

C - Delta Lake explicitly supports multiple concurrent writers to the same table.

D - Delta supports concurrent writes but doesn't address the checkpoint requirement correctly.

upvoted 1 times

 **KadELbied** 9 months, 1 week ago

Selected Answer: E

Surely E

upvoted 1 times

 **benni_ale** 1 year, 3 months ago

Selected Answer: E

E is the correct

upvoted 1 times

 **imatheushenrique** 1 year, 8 months ago

E. No; each of the streams needs to have its own checkpoint directory.

The checkpoint directory is 1 to 1

upvoted 2 times

  **svik** 1 year, 9 months ago

Selected Answer: B

It is not clear from the question that year_week=2020_01 and year_week=2020_02 are used by stream 1 and stream 2 respectively. If they use the common parent checkpoint directory with individual sub folders for checkpointing, that should work fine. In that case the answer should be B

upvoted 2 times

  **Kill9** 1 year, 7 months ago

That are table partitions. They are not used to build checkpoint adress. The adress finish at /bronze

upvoted 1 times

  **Jay_98_11** 2 years ago

Selected Answer: E

correct E

upvoted 1 times

  **kz_data** 2 years ago

Selected Answer: E

E is correct

upvoted 1 times

  **sturcu** 2 years, 3 months ago

E is correct.

If user wants 1 checkpoint directory then he needs to unions streams before writing.

upvoted 2 times

  **Eertyy** 2 years, 5 months ago

answer is correct

upvoted 3 times

A Structured Streaming job deployed to production has been experiencing delays during peak hours of the day. At present, during normal execution, each microbatch of data is processed in less than 3 seconds. During peak hours of the day, execution time for each microbatch becomes very inconsistent, sometimes exceeding 30 seconds. The streaming write is currently configured with a trigger interval of 10 seconds. Holding all other variables constant and assuming records need to be processed in less than 10 seconds, which adjustment will meet the requirement?

- A. Decrease the trigger interval to 5 seconds; triggering batches more frequently allows idle executors to begin processing the next batch while longer running tasks from previous batches finish.
- B. Increase the trigger interval to 30 seconds; setting the trigger interval near the maximum execution time observed for each batch is always best practice to ensure no records are dropped.
- C. The trigger interval cannot be modified without modifying the checkpoint directory; to maintain the current stream state, increase the number of shuffle partitions to maximize parallelism.
- D. Use the trigger once option and configure a Databricks job to execute the query every 10 seconds; this ensures all backlogged records are processed with each batch.
- E. Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.

Suggested Answer: D

Community vote distribution

E (100%)

 **RafaelCFC** Highly Voted 2 years, 1 month ago

Selected Answer: E

I believe this is a case of the least bad option, not exactly the best option possible.

- A is wrong because in Streaming you very rarely have any executors idle, as all cores are engaged in processing the window of data;
- B is wrong because triggering every 30s will not meet the 10s target processing interval;
- C is wrong in two manners: increasing shuffle partitions to any number above the number of available cores in the cluster will worsen performance in streaming; also, the checkpoint folder has no connection with trigger time.
- D is wrong because, keeping all other things the same as described by the problem, keeping the trigger time as 10s will not change the underlying conditions of the delay (i.e.: too much data to be processed in a timely manner).

E is the only option that might improve processing time.

upvoted 10 times

 **arekm** 1 year, 1 month ago

With one addition to A explanation - micro-batches are sequential by design.

upvoted 2 times

 **_Lukas_** Most Recent 2 months, 1 week ago

Selected Answer: E

Correct:

E - During peak hours, more data arrives between trigger intervals, creating larger batches that take longer to process. By decreasing the trigger interval from 10 seconds to 5 seconds, each microbatch contains fewer records.

Incorrect:

- A - Structured Streaming processes one microbatch at a time, not concurrently.
- B - Increasing the interval would actually make batches even larger during peak hours.
- C - This option falsely claims you must modify the checkpoint directory to change trigger intervals.
- D - This uses trigger-once, which doesn't address the root cause of large batch sizes during peaks.

upvoted 1 times

 **KadELbied** 9 months, 1 week ago

Selected Answer: E

suretly E

upvoted 1 times

🗨️ **arekm** 1 year, 1 month ago

Selected Answer: E

Answer E, see explanation by RafaelCFC.

upvoted 1 times

🗨️ **ASRCA** 1 year, 1 month ago

Selected Answer: A

Option A emphasizes utilizing idle executors to begin processing the next batch while longer-running tasks from previous batches finish. This approach can help maintain a steady flow of data processing and reduce the likelihood of bottlenecks.

upvoted 1 times

🗨️ **arekm** 1 year, 1 month ago

Structured streaming processes batches in sequence. It does so since it guarantees exactly once processing, see:

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

upvoted 1 times

🗨️ **Thameur01** 1 year, 2 months ago

Selected Answer: B

If microbatch execution occasionally exceeds 30 seconds, a trigger interval of 5 seconds would cause multiple batches to queue up while the previous batch is still running. This would exacerbate the delays and potentially lead to backpressure and failure.

B is the best option in this case.

If we assume for sure that execution time should be less than 10s, then in that case a 5s interval will make more sense and E should be the best answer.

upvoted 1 times

🗨️ **wdeleersnyder** 1 year, 6 months ago

In Databricks Runtime 11.3 LTS and above, the Trigger.Once setting is deprecated. Databricks recommends you use Trigger.AvailableNow for all incremental batch processing workloads.

<https://docs.databricks.com/en/structured-streaming/triggers.html>

Doesn't seem like E is a valid and recommended option given that it is deprecated.

upvoted 2 times

🗨️ **wdeleersnyder** 1 year, 6 months ago

Ooops, I mean, D.

upvoted 2 times

🗨️ **imatheushenrique** 1 year, 8 months ago

Considering the best option for performance gain is:

E. Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.

upvoted 2 times

🗨️ **ojudz08** 1 year, 11 months ago

Selected Answer: E

E is the answer.

Enable the settings uses the 128 MB as the target file size

<https://learn.microsoft.com/en-us/azure/databricks/delta/tune-file-size>

upvoted 2 times

🗨️ **DAN_H** 2 years ago

Selected Answer: E

E is correct as A is wrong because in Streaming you very rarely have any executors idle

upvoted 2 times

🗨️ **kz_data** 2 years ago

Selected Answer: E

I think is E is correct

upvoted 1 times

🗨️ **ervinshang** 2 years, 1 month ago

Selected Answer: E

correct answer is E

upvoted 1 times

  **ofed** 2 years, 3 months ago

Only C. Even if you trigger more frequently you decrease both load and time for this load. E doesn't change anything.

upvoted 1 times

  **sturcu** 2 years, 3 months ago

Selected Answer: E

Changing trigger interval to "one" will cause this to be a "batch" and will not execute in microbranches. This will not help at all

upvoted 4 times

  **Eertyy** 2 years, 4 months ago

correct answer is E

upvoted 1 times

  **azurearch** 2 years, 5 months ago

sorry, the caveat is holding all other variables constant.. that means we are not allowed to change trigger intervals. is C the answer then

upvoted 1 times

  **azurearch** 2 years, 5 months ago

what if in between those 5 seconds trigger interval if there are more records, that would still increase the time it takes to process.. i doubt E is correct. I will go with answer D. it is not to execute all queries within 10 secs. it is to execute trigger now batch every 10 seconds.

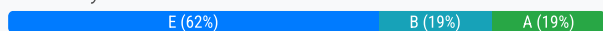
upvoted 1 times

Which statement describes Delta Lake Auto Compaction?

- A. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 1 GB.
- B. Before a Jobs cluster terminates, OPTIMIZE is executed on all tables modified during the most recent job.
- C. Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.
- D. Data is queued in a messaging bus instead of committing data directly to memory; all data is committed from the messaging bus in one batch once the job is complete.
- E. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 128 MB.

Suggested Answer: A

Community vote distribution



aragorn_brego Highly Voted 2 years, 2 months ago

Selected Answer: A

Delta Lake's Auto Compaction feature is designed to improve the efficiency of data storage by reducing the number of small files in a Delta table. After data is written to a Delta table, an asynchronous job can be triggered to evaluate the file sizes. If it determines that there are a significant number of small files, it will automatically run the OPTIMIZE command, which coalesces these small files into larger ones, typically aiming for files around 1 GB in size for optimal performance.

E is incorrect because the statement is similar to A but with an incorrect default file size target.
upvoted 5 times

Kill9 1 year, 7 months ago

Table property delta.autoOptimize.autoCompact target 128 mb. For table property delta.tuneFileSizesForRewrites, tables larger than 10 TB, the target file size is 1 GB.

<https://learn.microsoft.com/en-us/azure/databricks/delta/tune-file-size>

upvoted 4 times

Lukas Most Recent 2 months, 3 weeks ago

Selected Answer: E

Correct:

E - Auto Compaction automatically combines small files within Delta table partitions after a write operation has succeeded.

The process works as follows:

- After each write, the system evaluates partition health to detect excessive file fragmentation
- If the partition has too many small files, compaction is triggered.
- An OPTIMIZE operation is executed with a default target file size of 128 MB.
- Only files that haven't been compacted previously are processed.

Incorrect:

A - This option incorrectly states a default of 1 GB instead of 128 MB.

B - This describes cluster termination behavior, not auto compaction.

C - This describes optimized writes, which is a different feature that optimizes before writing.

D - This describes a messaging queue pattern unrelated to Delta Lake auto compaction.

upvoted 1 times

KadELbied 9 months, 1 week ago

Selected Answer: E

suretly E

upvoted 1 times

RandomForest 1 year ago

Selected Answer: E

Delta Lake Auto Compaction is a feature that automatically detects opportunities to optimize small files. When a write operation is completed, an asynchronous job assesses whether the resulting files can be compacted into larger files (the default target size is 128 MB). If compaction is needed, the system executes an OPTIMIZE job in the background to improve file size and query performance.

This feature reduces the overhead of managing small files manually and improves storage and query efficiency. It aligns with Delta Lake's goal of simplifying and optimizing data lake performance.

upvoted 3 times

🗳️ 👤 **mwynn** 1 year ago

Selected Answer: E

I think it is E because they are just asking us to generally describe the feature - here's some info I gleaned from a DB Academy video:

- Compact small files on write with auto-optimize (tries to achieve file size of 128 MB)
- Auto-Compact launches a new job after execution of first Spark job (i.e. async), where it will try to compress files closer to 128 MB

upvoted 4 times

🗳️ 👤 **pallazoj** 11 months, 4 weeks ago

This is true. I just heard the same statement in Databricks Academy video. Advanced Data Engineering with Databricks/Section5/Lesson1:Designing the foundation from 4:00 into the video!

upvoted 1 times

🗳️ 👤 **Nicks_name** 1 year, 1 month ago

Selected Answer: E

typo in databricks documentation about sync job, but default size is explicitly mentioned as 128

upvoted 1 times

🗳️ 👤 **carah** 1 year, 1 month ago

Selected Answer: B

Table property: delta.autoOptimize.autoCompact

B. correct, although <https://docs.databricks.com/en/delta/tune-file-size.html#auto-compaction-for-delta-lake-on-databricks> does not mention OPTIMIZE, it is best option

A., E. wrong, auto compaction runs synchronously

C. wrong, it describes Table setting: delta.autoOptimize.optimizeWrite

D. wrong, not related to file compaction

upvoted 3 times

🗳️ 👤 **arekm** 1 year, 1 month ago

The problem I have with B is that it says - on all tables. That depends on whether we use spark settings or table settings.

However, I still believe the asynchronous in A and E was meant to be synchronous (it is a typo). If it was not, then you are right :)

upvoted 1 times

🗳️ 👤 **vish9** 1 year, 3 months ago

There appears to be a typo in databricks documentation

upvoted 3 times

🗳️ 👤 **rrprofessional** 1 year, 3 months ago

Enable auto compaction. By default will use 128 MB as the target file size.

upvoted 1 times

🗳️ 👤 **akashdesarda** 1 year, 4 months ago

Selected Answer: B

If you go through this docs - then one thing is clear that it is not async job, so we have to eliminate A & C. D is wrong. It has no special job wrt the partition. Also file size of 128 MB is legacy config, latest one is dynamic. So we are left with B

upvoted 3 times

🗳️ 👤 **mouthwash** 1 year, 1 month ago

This. Don't be fooled by the typo answers, typo is inserted for a reason. It makes the answer wrong.

upvoted 1 times

🗳️ 👤 **pk07** 1 year, 4 months ago

Selected Answer: E

<https://docs.databricks.com/en/delta/tune-file-size.html>

upvoted 2 times

🗳️ 👤 **partha1022** 1 year, 5 months ago

Selected Answer: B

Auto compaction is synchronous job.

upvoted 2 times

🗳️ 👤 **Shailly** 1 year, 6 months ago

Selected Answer: B

A and E are wrong because auto compaction is synchronous operation!

I vote for B

As per documentation - "Auto compaction occurs after a write to a table has succeeded and runs synchronously on the cluster that has performed the write. Auto compaction only compacts files that haven't been compacted previously."

<https://docs.delta.io/latest/optimizations-oss.html>

upvoted 4 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

E. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 128 MB.

<https://community.databricks.com/t5/data-engineering/what-is-the-difference-between-optimize-and-auto-optimize/td-p/21189>

upvoted 1 times

🗳️ 👤 **ojudz08** 1 year, 11 months ago

Selected Answer: E

E is the answer.

Enable the settings uses the 128 MB as the target file size

<https://learn.microsoft.com/en-us/azure/databricks/delta/tune-file-size>

upvoted 2 times

🗳️ 👤 **DAN_H** 2 years ago

Selected Answer: E

default file size is 128MB in auto compaction

upvoted 1 times

🗳️ 👤 **kz_data** 2 years ago

E is correct as the default file size is 128MB in auto compaction, not 1GB as normal OPTIMIZE statement.

upvoted 1 times

Which statement characterizes the general programming model used by Spark Structured Streaming?

- A. Structured Streaming leverages the parallel processing of GPUs to achieve highly parallel data throughput.
- B. Structured Streaming is implemented as a messaging bus and is derived from Apache Kafka.
- C. Structured Streaming uses specialized hardware and I/O streams to achieve sub-second latency for data transfer.
- D. Structured Streaming models new data arriving in a data stream as new rows appended to an unbounded table.
- E. Structured Streaming relies on a distributed network of nodes that hold incremental state values for cached stages.

Suggested Answer: D

Community vote distribution

D (100%)

🗳️ 👤 **8605246** Highly Voted 2 years ago

correct; The key idea in Structured Streaming is to treat a live data stream as a table that is being continuously appended. This leads to a new stream processing model that is very similar to a batch processing model. You will express your streaming computation as standard batch-like query as on a static table, and Spark runs it as an incremental query on the unbounded input table. Let's understand this model in more detail.

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

upvoted 5 times

🗳️ 👤 **_Lukas_** Most Recent 2 months, 3 weeks ago

Selected Answer: D

Correct:

D - The fundamental concept behind Spark Structured Streaming is to treat a live data stream as a table that is being continuously appended.

This programming model works as follows:

- The input data stream is treated as an 'Input Table'
- Every new batch of data is seen as new rows appended to that table.
- Your query stays active and Spark refreshes the results whenever new data comes in. Spark executes it as an incremental query on the unbounded input table.

Incorrect:

A - Structured Streaming uses standard CPU-based distributed processing, not GPU acceleration.

B - It's built on the Spark SQL engine, not derived from Kafka.

C - This option doesn't describe the programming model at all.

E - This option describes distributed processing mechanics but misses the core abstraction of the unbounded table model.

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

Surely D

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: D

D - see explanation of 8605246

upvoted 1 times

🗳️ 👤 **imatheushenrique** 1 year, 2 months ago

D. Structured Streaming models new data arriving in a data stream as new rows appended to an unbounded table.

upvoted 2 times

🗳️ 👤 **mardigras** 1 year, 5 months ago

Selected Answer: D

Yes. answer is D

upvoted 2 times

🗳️ 👤 **Jay_98_11** 1 year, 6 months ago

Selected Answer: D

vote for D

upvoted 1 times

  **sturcu** 1 year, 10 months ago

Selected Answer: D

Correct.

Structured streaming needs to be considered as a table with append

upvoted 2 times

Which configuration parameter directly affects the size of a spark-partition upon ingestion of data into Spark?

- A. spark.sql.files.maxPartitionBytes
- B. spark.sql.autoBroadcastJoinThreshold
- C. spark.sql.files.openCostInBytes
- D. spark.sql.adaptive.coalescePartitions.minPartitionNum
- E. spark.sql.adaptive.advisoryPartitionSizeInBytes

Suggested Answer: A

Community vote distribution

A (100%)

🗳️ 👤 **8605246** Highly Voted 📈 2 years ago

correct; The maximum number of bytes to pack into a single partition when reading files. This configuration is effective only when using file-based sources such as Parquet, JSON and ORC.

<https://spark.apache.org/docs/latest/sql-performance-tuning.html>

upvoted 5 times

🗳️ 👤 **_Lukas_** Most Recent 🕒 2 months, 1 week ago

Selected Answer: A

Correct:

A - This configuration parameter directly controls the target maximum size (in bytes) of each partition during file reading. Default: 128 MB.

Incorrect:

B - This happens during join operations, not when reading files initially.

C - This option doesn't directly control partition size. It just helps Spark make smarter decisions about grouping small files together.

D - This happens after shuffles, not during initial file reading. The sequence is: read files -> process data -> shuffle -> then this parameter applies.

E - Like option D, this is about shuffle partitions during query execution, not about reading files.

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: A

suretly A

upvoted 1 times

🗳️ 👤 **Jay_98_11** 1 year, 6 months ago

Selected Answer: A

correct

upvoted 3 times

🗳️ 👤 **sturcu** 1 year, 10 months ago

Selected Answer: A

from the provided list, this fits best.

In reality partition size/number can be influenced my many settings

upvoted 1 times

A Spark job is taking longer than expected. Using the Spark UI, a data engineer notes that the Min, Median, and Max Durations for tasks in a particular stage show the minimum and median time to complete a task as roughly the same, but the max duration for a task to be roughly 100 times as long as the minimum.

Which situation is causing increased duration of the overall job?

- A. Task queueing resulting from improper thread pool assignment.
- B. Spill resulting from attached volume storage being too small.
- C. Network latency due to some cluster nodes being in different regions from the source data
- D. Skew caused by more data being assigned to a subset of spark-partitions.
- E. Credential validation errors while pulling data from an external system.

Suggested Answer: D

Community vote distribution

D (100%)

  **_Lukas_** 2 months, 1 week ago

Selected Answer: D

Correct:

D - This pattern described is a signature of data skew. This tells you that:

Most tasks finish quickly (hence the low median)

A few tasks take dramatically longer (the high max)

Some partitions have much more data than others

Incorrect:

A - If tasks were queueing due to thread pool issues, you'd see consistent delays across many tasks, not just a few outliers.

B - If storage was too small and causing spill you'd see this problem across many tasks, not just one or two outliers.

C - Network latency affects tasks consistently based on location. If certain worker nodes are in a different region, all tasks running on those specific nodes would be slow.

E - Authentication errors would cause task failures, not just slownesses.

upvoted 1 times

  **a68856c** 3 months ago

Selected Answer: D

Data distribution in Spark partitions: Uneven data distribution can lead to some partitions being much larger than others, causing performance bottlenecks.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: D

Suretly D

upvoted 1 times

  **arekm** 1 year, 1 month ago

Selected Answer: D

D - other answers don't make sense. In particular C - all nodes of the cluster must be in the same region (at least on AWS and Azure; GCP - I don't know, but they have networks spanning regions, so maybe it is possible).

upvoted 2 times

  **benni_ale** 1 year, 2 months ago

Selected Answer: D

D is correct

upvoted 1 times

  **AndreFR** 1 year, 5 months ago

A excluded because task queueing does not increase the duration of a task
B excluded, spill is writing to storage when a memory is insufficient (not storage insufficient)
C excluded, region cannot have a 100 times impact on duration
E excluded, no errors mentioned in question
upvoted 1 times

🗨️ 👤 **imatheushenrique** 1 year, 8 months ago

D. Skew caused by more data being assigned to a subset of spark-partitions.
upvoted 1 times

🗨️ 👤 **vikram12apr** 1 year, 11 months ago

Selected Answer: D

because a particular executors are executing majority of data while rest are processing very less. The total execution time depends upon the slowest executors.
Answer is D.
upvoted 3 times

🗨️ 👤 **Jay_98_11** 2 years ago

Selected Answer: D

correct
upvoted 1 times

🗨️ 👤 **kz_data** 2 years ago

Selected Answer: D

I think D is correct
upvoted 1 times

🗨️ 👤 **sturcu** 2 years, 3 months ago

Selected Answer: D

D is correct
upvoted 1 times

🗨️ 👤 **Eertyy** 2 years, 4 months ago

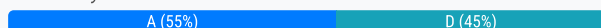
D is the correct answer
upvoted 3 times

Each configuration below is identical to the extent that each cluster has 400 GB total of RAM, 160 total cores and only one Executor per VM. Given a job with at least one wide transformation, which of the following cluster configurations will result in maximum performance?

- A. • Total VMs; 1
 - 400 GB per Executor
 - 160 Cores / Executor
- B. • Total VMs: 8
 - 50 GB per Executor
 - 20 Cores / Executor
- C. • Total VMs: 16
 - 25 GB per Executor
 - 10 Cores/Executor
- D. • Total VMs: 4
 - 100 GB per Executor
 - 40 Cores/Executor
- E. • Total VMs: 2
 - 200 GB per Executor
 - 80 Cores / Executor

Suggested Answer: B

Community vote distribution



robson90 Highly Voted 2 years, 5 months ago

Option A, question is about maximum performance. Wide transformation will result in often expensive shuffle. With one executor this problem will be resolved. <https://docs.databricks.com/en/clusters/cluster-config-best-practices.html#complex-batch-etl>
upvoted 44 times

dp_learner 2 years, 3 months ago

source : <https://docs.databricks.com/en/clusters/cluster-config-best-practices.html>
upvoted 3 times

Ashok_Choudhary_CT Highly Voted 10 months, 1 week ago

Selected Answer: C

How Option (C) Excels?

- ✓ More Executors (16 vs. 8 in Option B) → Faster parallel execution.
- ✓ Fewer Cores per Executor (10 vs. 20 in Option B) → Prevents CPU contention and scheduling delays.
- ✓ Better Memory Management (25GB vs. 50GB in Option B) → Reduces GC overhead.

Final Verdict

Option (C) is the "Best" configuration for handling a job with wide transformations.
upvoted 7 times

Lukas Most Recent 2 months, 1 week ago

Selected Answer: C

Correct:

C - Maximum number of executors enables highest parallelism for processing shuffle partitions simultaneously, making it optimal for wide transformations despite smaller individual executor size.

Incorrect:

- A - Single executor cannot distribute shuffle operations across multiple nodes, eliminating parallelism benefits essential for wide transformations.
- B - Eight executors provide good but not the highest parallelism for shuffle operations with well-balanced memory and core allocation per executor.
- D - Only four executors create shuffle bottlenecks as data concentrates on too few nodes.
- E - Two executors provide minimal parallelism, forcing most shuffle data to move between just two nodes.

upvoted 2 times

🗨️ 👤 **Ral17** 2 months, 2 weeks ago

Selected Answer: C

For wide transformations (shuffles), you want:

More executors = Better shuffle parallelism

More network interfaces = Higher aggregate bandwidth

Distributed I/O = Parallel shuffle reads/writes. 16x network bandwidth: Shuffle data distributed across 16 network interfaces

✓ True parallelism: 16 nodes simultaneously reading/writing shuffle data

✓ Better fault tolerance: Losing 1 executor = only 6% of work lost vs 100%

✓ Reasonable cores/executor: 10 cores (not ideal, but better than 160!)

upvoted 2 times

🗨️ 👤 **8c1d06f** 4 months ago

Selected Answer: C

For jobs with wide transformations (shuffles), you generally get better performance from more (smaller) executors rather than a few huge ones. This increases parallelism across shuffle stages, reduces GC pressure per executor, and improves disk/network throughput during shuffle. Among the options, 10 cores/executor is the smallest (closest to best practice of ~5–6 cores/executor) and gives the most executors (16), so it's the best of the set.

upvoted 2 times

🗨️ 👤 **ealpuche** 5 months, 3 weeks ago

Selected Answer: C

More nodes, so we have more capacity to make that job parallel, so it takes less time

upvoted 2 times

🗨️ 👤 **363c4c5** 6 months, 3 weeks ago

Selected Answer: B

Databricks Assistant:

Parallelism: Wide transformations (such as joins, aggregations, etc.) benefit from higher parallelism. Having more executors with fewer cores each allows for better distribution of tasks and more efficient use of resources.

Resource Utilization: You need to achieve a good balance between the number of executors and the resources allocated to each executor. This configuration allows for efficient task execution and minimizes the risk of resource contention.

Avoiding Bottlenecks: Configurations with fewer VMs but more cores per executor can lead to bottlenecks, as a single executor might become a point of contention. Conversely, configurations with too many VMs but very few cores per executor might not fully utilize the available resources.

upvoted 1 times

🗨️ 👤 **086ce7d** 6 months, 3 weeks ago

Selected Answer: A

It's max performance, so A

upvoted 1 times

🗨️ 👤 **happyhelppy** 6 months, 4 weeks ago

Selected Answer: A

See aswer from robson90

upvoted 2 times

🗨️ 👤 **a67e3d1** 6 months, 4 weeks ago

Selected Answer: A

one worker no shuffle, if we target wide tr explicitly, if we go for balance than c

upvoted 1 times

🗨️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: A

Suretly A

upvoted 1 times

🗨️ 👤 **capt2101akash** 10 months, 2 weeks ago

Selected Answer: A

The question talks about higher performance for one large wide transformation. This needs fewer large VMs/Executor. Therefore, one needs to choose the largest possible option.

upvoted 1 times

🗨️ 👤 **shaswat1404** 12 months ago

Selected Answer: C

overly large executors are bad due to large Garbage Collection (GC) overhead and inefficient parallelism

option C provides the best balance of parallelism, memory utilization and performance efficiency

upvoted 3 times

🗨️ 👤 **fabiospont** 1 year ago

Selected Answer: A

A is correct only one VM per Job.

upvoted 1 times

🗨️ 👤 **HairyTorso** 1 year, 1 month ago

Selected Answer: B

Number of workers

Choosing the right number of workers requires some trials and iterations to figure out the compute and memory needs of a Spark job. Here are some guidelines to help you start:

Never choose a single worker for a production job, as it will be the single point for failure

Start with 2-4 workers for small workloads (for example, a job with no wide transformations like joins and aggregations)

Start with 8-10 workers for medium to big workloads that involve wide transformations like joins and aggregations, then scale up if necessary

<https://www.databricks.com/discover/pages/optimize-data-workloads-guide#number-workers>

upvoted 4 times

🗨️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: A

Maximum performance - A guarantees no shuffles between nodes in the cluster. Only processes on one VM.

upvoted 1 times

🗨️ 👤 **AlejandroU** 1 year, 1 month ago

Selected Answer: B

Answer B offers a good balance with 8 executors, providing a decent amount of memory and cores per executor, allowing for significant parallel processing.

Option C increases the number of executors further but at the cost of reduced memory and cores per executor, which might not be as effective for wide transformations.

upvoted 1 times

🗨️ 👤 **arekm** 1 year, 1 month ago

The question is about maximum performance.

upvoted 1 times

A junior data engineer on your team has implemented the following code block.

```

MERGE INTO events
USING new_events
ON events.event_id = new_events.event_id
WHEN NOT MATCHED
  INSERT *

```

The view new_events contains a batch of records with the same schema as the events Delta table. The event_id field serves as a unique key for this table.

When this query is executed, what will happen with new records that have the same event_id as an existing record?

- A. They are merged.
- B. They are ignored.
- C. They are updated.
- D. They are inserted.
- E. They are deleted.

Suggested Answer: B

Community vote distribution

B (100%)

🗳️ 👤 **Ral17** 2 months, 2 weeks ago

Selected Answer: C

For wide transformations (shuffles), you want:

More executors = Better shuffle parallelism

More network interfaces = Higher aggregate bandwidth

Distributed I/O = Parallel shuffle reads/writes. 16x network bandwidth: Shuffle data distributed across 16 network interfaces

- ✓ True parallelism: 16 nodes simultaneously reading/writing shuffle data
- ✓ Better fault tolerance: Losing 1 executor = only 6% of work lost vs 100%
- ✓ Reasonable cores/executor: 10 cores (not ideal, but better than 160!)

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

surely B

upvoted 1 times

🗳️ 👤 **Ashish7singh2020** 1 year ago

Selected Answer: B

merge will work if no match

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: B

No WHEN MATCHED section in MERGE, hence no action on those records, hence ignore - answer B.

upvoted 2 times

🗳️ 👤 **Shakmak** 1 year, 2 months ago

Selected Answer: B

B is a correct Answer

upvoted 1 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

B. They are ignored.

Because there is not mention so there is no WHEN statement for this condition

upvoted 2 times

🗨️ 👤 **PrashantTiwari** 1 year, 12 months ago

B is correct

upvoted 2 times

🗨️ 👤 **kz_data** 2 years ago

Selected Answer: B

B is correct

upvoted 1 times

🗨️ 👤 **alexvno** 2 years, 3 months ago

Selected Answer: B

Ignored

upvoted 1 times

🗨️ 👤 **rairaix** 2 years, 4 months ago

Selected Answer: B

The answer is correct. "If none of the WHEN MATCHED conditions evaluate to true for a source and target row pair that matches the merge_condition, then the default action is applied." [https://docs.databricks.com/en/sql/language-manual/delta-merge-](https://docs.databricks.com/en/sql/language-manual/delta-merge-into.html#:~:text=If%20none%20of%20the%20WHEN%20MATCHED%20conditions%20evaluate%20to%20true%20for%20a%20source%20and%20target%20row)

[into.html#:~:text=If%20none%20of%20the%20WHEN%20MATCHED%20conditions%20evaluate%20to%20true%20for%20a%20source%20and%20target%20row](https://docs.databricks.com/en/sql/language-manual/delta-merge-into.html#:~:text=If%20none%20of%20the%20WHEN%20MATCHED%20conditions%20evaluate%20to%20true%20for%20a%20source%20and%20target%20row)

upvoted 3 times

A junior data engineer seeks to leverage Delta Lake's Change Data Feed functionality to create a Type 1 table representing all of the values that have ever been valid for all rows in a bronze table created with the property `delta.enableChangeDataFeed = true`. They plan to execute the following code as a daily job:

```
from pyspark.sql.functions import col

(spark.read.format("delta")
 .option("readChangeFeed", "true")
 .option("startingVersion", 0)
 .table("bronze")
 .filter(col("_change_type").isin(["update_postimage", "insert"]))
 .write
 .mode("append")
 .table("bronze_history_type1")
)
```

Which statement describes the execution and results of running the above query multiple times?

- A. Each time the job is executed, newly updated records will be merged into the target table, overwriting previous values with the same primary keys.
- B. Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.
- C. Each time the job is executed, the target table will be overwritten using the entire history of inserted or updated records, giving the desired result.
- D. Each time the job is executed, the differences between the original and current versions are calculated; this may result in duplicate entries for some records.
- E. Each time the job is executed, only those records that have been inserted or updated since the last execution will be appended to the target table, giving the desired result.

Suggested Answer: B

Community vote distribution

B (83%)

E (17%)

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

select B

upvoted 2 times

🗳️ 👤 **Chugs** 11 months, 1 week ago

Selected Answer: B

As update type is insert and update so B is correct option.

upvoted 1 times

🗳️ 👤 **asdsadasdas** 11 months, 4 weeks ago

Selected Answer: B

B, Spark.read reads the entire table every time processed. If it was readstream then E would be answer

upvoted 3 times

🗳️ 👤 **Ashish7singh2020** 1 year ago

Selected Answer: B

since start version is 0

upvoted 1 times

🗳️ 👤 **akashdesarda** 1 year, 3 months ago

Selected Answer: B

The starting version is 0, that means in every version entire data will be fetched. It is then append.

upvoted 1 times

🗳️ 👤 **faraaz132** 1 year, 6 months ago

Correct Answer: B (not E)

Although it was pretty obvious to me, I still wrote the code to check and yes, it will append the entire change during every write since starting version is mentioned as 0.

If in doubt, code it yourselves

upvoted 2 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

("startingVersion", 0) that means the entire history of table will be read so B.

upvoted 3 times

🗳️ 👤 **PrashantTiwari** 1 year, 12 months ago

B is correct

upvoted 2 times

🗳️ 👤 **kz_data** 2 years ago

Selected Answer: B

B is correct

upvoted 2 times

🗳️ 👤 **5ffcd04** 2 years, 1 month ago

Selected Answer: B

Correct B

upvoted 1 times

🗳️ 👤 **azurelearn2020** 2 years, 2 months ago

Selected Answer: B

correct answer is B.

upvoted 1 times

🗳️ 👤 **[Removed]** 2 years, 2 months ago

Selected Answer: E

Considering that we are talking about Change Data Feed and the code is filtering by ["update_postimage", "insert"] the column "_change_type", I would go with the option E.

Reference:

https://docs.delta.io/latest/delta-change-data-feed.html#:~:text=_change_type,update_preimage%20%2C%20update_postimage

upvoted 1 times

🗳️ 👤 **5ffcd04** 2 years, 1 month ago

Notice option ("startingVersion", 0), which will bring all changes from beginning. Hence Answer is B.

upvoted 6 times

🗳️ 👤 **jyothsna12496** 2 years, 3 months ago

why is it Not E. It gets newly inserted or updated records

upvoted 1 times

🗳️ 👤 **5ffcd04** 2 years, 1 month ago

Notice .option ("startingVersion", 0), which will bring all changes from beginning. Hence Answer is B.

upvoted 1 times

🗳️ 👤 **sturcu** 2 years, 3 months ago

Selected Answer: B

correct

upvoted 1 times

🗳️ 👤 **azurearch** 2 years, 5 months ago

B is the right answer, sorry.

upvoted 2 times

🗳️ 👤 **azurearch** 2 years, 5 months ago

answer is A, because there is a filter as asmayassineg said. Filter filters only existing records from change feed

upvoted 1 times

🗳️ 👤 **asmayassineg** 2 years, 6 months ago

sorry, answer is correct B.
upvoted 2 times

A new data engineer notices that a critical field was omitted from an application that writes its Kafka source to Delta Lake. This happened even though the critical field was in the Kafka source. That field was further missing from data written to dependent, long-term storage. The retention threshold on the Kafka service is seven days. The pipeline has been in production for three months.

Which describes how Delta Lake can help to avoid data loss of this nature in the future?

- A. The Delta log and Structured Streaming checkpoints record the full history of the Kafka producer.
- B. Delta Lake schema evolution can retroactively calculate the correct value for newly added fields, as long as the data was in the original source.
- C. Delta Lake automatically checks that all fields present in the source data are included in the ingestion layer.
- D. Data can never be permanently dropped or deleted from Delta Lake, so data loss is not possible under any circumstance.
- E. Ingesting all raw data and metadata from Kafka to a bronze Delta table creates a permanent, replayable history of the data state.

Suggested Answer: E

Community vote distribution

E (100%)

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: E

select E

upvoted 1 times

🗳️ 👤 **kishanu** 10 months, 1 week ago

Selected Answer: E

E is the right answer, as the table in bronze can be replayed again when required.

upvoted 1 times

🗳️ 👤 **Tedet** 11 months, 1 week ago

Selected Answer: A

Considering the Databricks documentation on change feed and your need to process new records that have not been processed yet, Option A might actually be a better fit since you're looking for a streaming solution that can continuously monitor new records. The change feed (Option D) works for batch processing changes from a specific version, which isn't ideal for real-time streaming.

upvoted 1 times

🗳️ 👤 **HairyTorso** 1 year, 1 month ago

Selected Answer: E

E lgtm

upvoted 1 times

🗳️ 👤 **Anithec0der** 1 year, 1 month ago

Selected Answer: E

When we design pipeline, we will have to make sure data from source will be present there in the raw layer/bronze layer and the transformation we make should be done in refine and enterprise layer so by this way we can tackle this kind of situation where the necessary column was not replicated in previous runs of pipeline and we can create new column based on raw data we have.

upvoted 3 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

Medallion Architecture is named in E. (Ingesting all raw data and metadata from Kafka to a bronze Delta table creates a permanent, replayable history of the data state.)

upvoted 3 times

🗳️ 👤 **ojudz08** 1 year, 11 months ago

Selected Answer: E

E is correct

upvoted 2 times

🗳️ 👤 **DAN_H** 2 years ago

Selected Answer: E

I think E is correct

upvoted 1 times

  **kz_data** 2 years ago

Selected Answer: E

I think E is correct

upvoted 1 times

  **alexvno** 2 years, 3 months ago

Selected Answer: E

Looks good - E

upvoted 2 times

A nightly job ingests data into a Delta Lake table using the following code:

```
from pyspark.sql.functions import current_timestamp, input_file_name, col
from pyspark.sql.column import Column

def ingest_daily_batch(time_col: Column, year:int, month:int, day:int):
    (spark.read
     .format("parquet")
     .load(f"/mnt/daily_batch/{year}/{month}/{day}")
     .select("time_col.alias('ingest_time'),
            input_file_name().alias('source_file')
            )
     .write
     .mode("append")
     .saveAsTable("bronze"))
```

The next step in the pipeline requires a function that returns an object that can be used to manipulate new records that have not yet been processed to the next table in the pipeline.

Which code snippet completes this function definition?

def new_records():

A. return spark.readStream.table("bronze")

B. return spark.readStream.load("bronze")

C.

```
return (spark.read
        .table("bronze")
        .filter(col("ingest_time") == current_timestamp())
        )
```

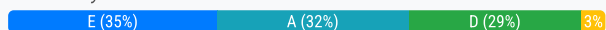
D. return spark.read.option("readChangeFeed", "true").table("bronze")

E.

```
return (spark.read
        .table("bronze")
        .filter(col("source_file") == f"/mnt/daily_batch/{year}/{month}/{day}")
        )
```

Suggested Answer: D

Community vote distribution



AzureDE2522 Highly Voted 2 years, 2 months ago

Selected Answer: D

not providing a starting version/timestamp will result in the latest snapshot being fetched first

```
spark.readStream.format("delta") \
.option("readChangeFeed", "true") \
.table("myDeltaTable")
```

Please refer:

<https://docs.databricks.com/en/delta/delta-change-data-feed.html>

upvoted 15 times

shaojunni 1 year, 3 months ago

readChangeFeed is disabled by default.

upvoted 2 times

arekm 1 year, 1 month ago

Answer D would require specifying the start and (optionally) the end version for reading data from CDF. So D does not seem to be correct.

upvoted 3 times

t_d_v 1 year, 5 months ago

There is no stream in option D

upvoted 4 times

GHill1982 1 year, 5 months ago

You can read Delta Lake Change Data Feed without using a stream. You can use batch queries to read the change data feed by setting the readChangeFeed option to true.

upvoted 2 times

  **arekm** 1 year, 1 month ago

CDF without a stream requires a starting version at the minimum.

upvoted 2 times

  **gloomy_marmot** 6 months, 1 week ago

But "A starting version is required for batch reads, and many batch patterns can benefit from setting an optional ending version."

<https://docs.databricks.com/aws/en/delta/delta-change-data-feed?language=Python#should-i-specify-a-starting-version>

So looks like A is correct

upvoted 1 times

  **Laraujo2022** Highly Voted 2 years, 2 months ago

In my opinion E is not correct because we do not see parameters pass within to the function (year, month and day)... the function is def new_records():

upvoted 9 times

  **_Lukas_** Most Recent 2 months, 3 weeks ago

Selected Answer: A

Correct:

A - This code snippet correctly returns a streaming DataFrame that incrementally processes new records that haven't been processed yet.

Incorrect:

B - This expects a file path, not a table name.

C - This is batch processing, not streaming, and current_timestamp() evaluates at query execution time, which does not match the ingestion time accurately.

D - The task doesn't mention that Change Data Feed is enabled, so we cannot assume that we can use readChangeFeed option. Command to enable it: ALTER TABLE bronze SET TBLPROPERTIES (delta.enableChangeDataFeed = true).

E - This is batch processing without tracking what's been processed, and the year/month/day variables are not in scope within the function new_records().

upvoted 1 times

  **gloomy_marmot** 6 months, 1 week ago

Selected Answer: A

A is right

B isn't correct(should be table instead of load)

C isn't correct because is used wrong filter

D isn't correct because "A starting version is required for batch reads, and many batch patterns can benefit from setting an optional ending version."

<https://docs.databricks.com/aws/en/delta/delta-change-data-feed?language=Python#should-i-specify-a-starting-version>

E isn't correct because is used wrong filter(it's folder, not filename)

upvoted 1 times

  **podsiqueira** 6 months, 1 week ago

Selected Answer: D

The question is asking how to retrieve new records from a Delta Lake Bronze table that haven't yet been processed further in the pipeline. This is a classic CDC use case, where the next layer wants to track only new changes to the data (inserts, updates, deletes).

upvoted 1 times

  **ConquerorAlpha** 7 months, 2 weeks ago

Selected Answer: E

The question is clearly saying that the table is being written in batch format, then how you can read is using streaming code, hence option E is correct there with the proper filter to read the current date data. Please think logically.

upvoted 1 times

  **BryOs** 8 months, 1 week ago

Selected Answer: A

It's Option A because it will allow you to get the latest changes even if CDF isn't enabled on the table. Option D would fail if CDF isn't enabled on the table. The statement doesn't indicate if CDF is active.

upvoted 1 times

  **KadELbied** 9 months ago

Selected Answer: A

Surely A

upvoted 1 times

🗨️ **AIHerd** 10 months, 2 weeks ago

Selected Answer: A

Option A is best because it creates a streaming source that reads only new appended data from the "bronze" table incrementally. Even if ingestion is done in batch, using `spark.readStream.table("bronze")` lets downstream processing treat the table as a live data stream.

upvoted 2 times

🗨️ **Tedet** 11 months, 1 week ago

Selected Answer: D

Explanation: This is the best option for Delta Lake, as it uses the `readChangeFeed` option. This option is specifically designed to read only the new changes (insertions, updates, or deletions) since the last read, which is exactly what is needed when you want to handle new records that have not yet been processed. This ensures that only records that are new or changed since the last read are returned.

Conclusion: This is the correct choice, as it ensures that only new records are read.

upvoted 1 times

🗨️ **asdsadasdas** 11 months, 4 weeks ago

Selected Answer: A

"manipulate new records that have *not yet been processed* to the next table " readstream can incrementally pick data yet to be processed. with D the issue is `spark.read` it will read the entire table

upvoted 1 times

🗨️ **asdsadasdas** 11 months, 4 weeks ago

Batch (read) Reads all available CDF history starting from the earliest retained version May load too much data or fail if old versions are deleted

Streaming (readStream) Starts from the latest version unless a checkpoint exists

upvoted 1 times

🗨️ **shaswat1404** 12 months ago

Selected Answer: E

in option A and B assume steaming ingestio~~pn~~ but ingestion is in batch mode

in option C `current_timestamp` is used which is dynamic and changes every time the query is executed therefore it wont correctly filter records ingested in the last batch

in option D it only works if `delta.enableChangeDataFeed = true` was set on the table before the ingestion (its disabled by default and given query does not set this option as true) therefore this option is in valid

option E is correct as it correctly filters from the most recent batch as it uses file path to retrieve only data from the latest ingestion column `source_file` was created specifically for this purpose ensuring the function returns onle new records..

upvoted 2 times

🗨️ **arekm** 1 year, 1 month ago

Selected Answer: A

You can read data from the delta table using structured streaming. You have 2 options:

- without CDF - only process new rows (without updates and deletes)

- with CDF - all changes to the data, i.e. insert, update, delete.

Answer A uses the first option. However, in the question they talk about "new records". So using streaming for new records is OK. Answer A is correct.

upvoted 2 times

🗨️ **arekm** 1 year, 1 month ago

At first I thought of answer D. However, after checking in the docs I learned that starting version is a must while reading from CDF using batch pattern.

upvoted 1 times

🗨️ **sgerin** 1 year, 1 month ago

Selected Answer: E

New records will be filtered for D /

upvoted 1 times

🗨️ **temple1305** 1 year, 1 month ago

Selected Answer: D

New records will be filtered for D -

example <https://delta.io/blog/2023-07-14-delta-lake-change-data-feed-cdf/>

upvoted 1 times

🗨️ 👤 **AlejandroU** 1 year, 1 month ago

Selected Answer: A

Answer A. A better approach would involve streaming directly from the Delta table (Option A), possibly along with using metadata like ingest_time to track new records more accurately.

It might be better to rely on the streaming process itself rather than trying to filter based on the file path (option E).

upvoted 1 times

🗨️ 👤 **Thameur01** 1 year, 2 months ago

Selected Answer: E

Using the source_file metadata field allows you to filter new records ingested from specific files.

E is the most robust and reliable option for tracking and working with new records in this batch ingestion pipeline.

upvoted 1 times

A junior data engineer is working to implement logic for a Lakehouse table named `silver_device_recordings`. The source data contains 100 unique fields in a highly nested JSON structure.

The `silver_device_recordings` table will be used downstream to power several production monitoring dashboards and a production model. At present, 45 of the 100 fields are being used in at least one of these applications.

The data engineer is trying to determine the best approach for dealing with schema declaration given the highly-nested structure of the data and the numerous fields.

Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

- A. The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.
- B. Because Delta Lake uses Parquet for data storage, data types can be easily evolved by just modifying file footer information in place.
- C. Human labor in writing code is the largest cost associated with data engineering workloads; as such, automating table declaration logic should be a priority in all migration workloads.
- D. Because Databricks will infer schema using types that allow all observed data to be processed, setting types manually provides greater assurance of data quality enforcement.
- E. Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.

Suggested Answer: D

Community vote distribution

D (100%)

 **RafaelCFC** Highly Voted 1 year, 7 months ago

Selected Answer: D

A is wrong, because Tungsten is a project around improving Spark's efficiency on memory and CPU usage;

B is wrong because Parquet does not support file editing, it only supports overwrite and create operations by itself;

C is wrong because completely automating schema declaration for tables will incur in reduced previsibility for data types and data quality;

E is false because unlucky sampling can yield bad inferences by Spark;

upvoted 14 times

 **hal2401me** Highly Voted 1 year, 4 months ago

from my exam today, both C & D are no longer available, so they can't be correct.

E & A are available. E states "always accurate" so I hesitate to choose it.

There is a new option stating like "delta lake indexes first 32column in delta log for Z order and optimization"(not sure I remember exactly, it looks statementfully correct). and I chosed this "new" option. Because, this should impact the schema decision by putting high-usage field in the first 32 columns.

upvoted 7 times

 **_Lukas_** Most Recent 2 months, 1 week ago

Selected Answer: D

Correct:

D - Schema inference uses safe types to accommodate all observed data without failing. Manual schema declaration allows setting precise types to enforce data quality at ingestion.

Incorrect:

A - Storing JSON as strings is less efficient than properly typed structured data (data skipping, predicate pushdown etc.).

B - You cannot evolve schema by modifying Parquet file footers.

C - Explicit schema definition is often worth the manual effort to prevent type mismatches and data quality issues.

E - Schema inference has no knowledge of downstream requirements.

upvoted 1 times

 **Tedet** 11 months, 1 week ago

Selected Answer: D

Explanation: Databricks can infer schema when reading data, but automatic schema inference doesn't always guarantee the accuracy of data types. For complex or highly-nested structures, schema inference might not always align with the actual data quality or the needs of downstream applications, and manual type definition ensures that the schema is more consistent and predictable. While automatic inference is useful for quick analysis or exploratory work, manual schema definition provides better data quality assurance in production workloads, especially when dealing with large, complex data structures.

Conclusion: This statement correctly emphasizes the importance of manual schema declaration to ensure data quality enforcement and consistency, especially when dealing with complex structures. Best option.

upvoted 1 times

  **guillesd** 1 year, 6 months ago

Selected Answer: D

Only answer that makes sense

upvoted 1 times

  **AziLa** 1 year, 6 months ago

Correct Ans is D

upvoted 1 times

  **sturcu** 1 year, 9 months ago

Selected Answer: D

correct

upvoted 2 times

  **hammer_1234_h** 1 year, 11 months ago

D is correct.

we can use `schema hint` to enforce the schema information that we know and expect on an inferred schema.

upvoted 2 times

The data engineering team maintains the following code:

```
accountDF = spark.table("accounts")
orderDF = spark.table("orders")
itemDF = spark.table("items")

orderWithItemDF = (orderDF.join(
    itemDF,
    orderDF.itemID == itemDF.itemID)
    .select(
        orderDF.accountID,
        orderDF.itemID,
        itemDF.itemName))

finalDF = (accountDF.join(
    orderWithItemDF,
    accountDF.accountID == orderWithItemDF.accountID)
    .select(
        orderWithItemDF["*"],
        accountDF.city))

(finalDF.write
    .mode("overwrite")
    .table("enriched_itemized_orders_by_account"))
```

Assuming that this code produces logically correct results and the data in the source tables has been de-duplicated and validated, which statement describes what will occur when this code is executed?

- A. A batch job will update the enriched_itemized_orders_by_account table, replacing only those rows that have different values than the current version of the table, using accountID as the primary key.
- B. The enriched_itemized_orders_by_account table will be overwritten using the current valid version of data in each of the three tables referenced in the join logic.
- C. An incremental job will leverage information in the state store to identify unjoined rows in the source tables and write these rows to the enriched_itemized_orders_by_account table.
- D. An incremental job will detect if new rows have been written to any of the source tables; if new rows are detected, all results will be recalculated and used to overwrite the enriched_itemized_orders_by_account table.
- E. No computation will occur until enriched_itemized_orders_by_account is queried; upon query materialization, results will be calculated using the current valid version of data in each of the three tables referenced in the join logic.

Suggested Answer: B

Community vote distribution

B (100%)

arekm 1 year, 1 month ago

Selected Answer: B

B - it is a batch overwrite, which means: whatever was there is gone.

upvoted 1 times

nedlo 1 year, 3 months ago

Selected Answer: B

i agree. Cannot be E because write itself is action

upvoted 1 times

AndreFR 1 year, 5 months ago

Selected Answer: B

B because code has : .mode("Overwrite")

upvoted 2 times

imatheushenrique 1 year, 8 months ago

B is correct

upvoted 1 times

  **AziLa** 2 years ago

Correct Ans is B

upvoted 2 times

  **Jay_98_11** 2 years ago

Selected Answer: B

correct

upvoted 2 times

  **sturcu** 2 years, 3 months ago

Selected Answer: B

B is correct

upvoted 3 times

The data engineering team is migrating an enterprise system with thousands of tables and views into the Lakehouse. They plan to implement the target architecture using a series of bronze, silver, and gold tables. Bronze tables will almost exclusively be used by production data engineering workloads, while silver tables will be used to support both data engineering and machine learning workloads. Gold tables will largely serve business intelligence and reporting purposes. While personal identifying information (PII) exists in all tiers of data, pseudonymization and anonymization rules are in place for all data at the silver and gold levels.

The organization is interested in reducing security concerns while maximizing the ability to collaborate across diverse teams.

Which statement exemplifies best practices for implementing this system?

- A. Isolating tables in separate databases based on data quality tiers allows for easy permissions management through database ACLs and allows physical separation of default storage locations for managed tables.
- B. Because databases on Databricks are merely a logical construct, choices around database organization do not impact security or discoverability in the Lakehouse.
- C. Storing all production tables in a single database provides a unified view of all data assets available throughout the Lakehouse, simplifying discoverability by granting all users view privileges on this database.
- D. Working in the default Databricks database provides the greatest security when working with managed tables, as these will be created in the DBFS root.
- E. Because all tables must live in the same storage containers used for the database they're created in, organizations should be prepared to create between dozens and thousands of databases depending on their data isolation requirements.

Suggested Answer: A

Community vote distribution

A (100%)

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: A

suretly A

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: A

A - most logical

B - it is a logical construct, but under default settings tables are stored where the database is so there is a security component to it

C - never a good idea to store everything in one db since db allows to group tables with similar area of interest and allows to manage permissions (like groups in Entra and assigning permissions to groups)

D - not default database does not mean we cannot use managed tables and you can specify your location still; I do not think that storing anything on DBFS is a good idea - even Databricks suggests to use workspaces for your code, not to mention the data.

E - thousand databases - nonsense; you can specify the location of individual tables.

upvoted 3 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Correction to D explanation - apparently the default location is bfs

upvoted 1 times

🗳️ 👤 **strayda** 1 year, 7 months ago

Selected Answer: A

The most logical answer is A

upvoted 1 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

A is correct

upvoted 2 times

🗳️ 👤 **ojudz08** 1 year, 11 months ago

Selected Answer: A

answer is A

upvoted 1 times

  **AziLa** 2 years ago

Correct Ans is A

upvoted 2 times

  **Enduresoul** 2 years, 2 months ago

Selected Answer: A

A is correct

upvoted 2 times

The data architect has mandated that all tables in the Lakehouse should be configured as external Delta Lake tables. Which approach will ensure that this requirement is met?

- A. Whenever a database is being created, make sure that the LOCATION keyword is used
- B. When configuring an external data warehouse for all table storage, leverage Databricks for all ELT.
- C. Whenever a table is being created, make sure that the LOCATION keyword is used.
- D. When tables are created, make sure that the EXTERNAL keyword is used in the CREATE TABLE statement.
- E. When the workspace is being configured, make sure that external cloud object storage has been mounted.

Suggested Answer: C

Community vote distribution

C (100%)

  **_Lukas_** 2 months, 3 weeks ago

Selected Answer: C

Correct:

C - In Databricks, an external table is specifically defined as a table that references an external storage path by using the 'LOCATION' clause, which makes the table external instead of managed.

Incorrect:

A - Setting a database or schema 'LOCATION' only changes the default root path for managed tables in that schema. Tables created without their own 'LOCATION' are still managed, not external.

B - Using an external warehouse and doing ELT there does not create Delta Lake tables in the Lakehouse at all.

D - Using 'EXTERNAL' without 'LOCATION' is not sufficient, and in practice you must specify a path.

E - Mounting only exposes storage paths into the workspace. It does not control whether tables are created as managed or external.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

  **Sriramiyer92** 1 year, 1 month ago

Selected Answer: C

Note: External keyword is not mandatory.

Location is mandatory the presence implies, that the table is external

upvoted 2 times

  **carah** 1 year, 1 month ago

Selected Answer: C

A. is not correct:

having schema with LOCATION

CREATE SCHEMA my_schema

LOCATION 's3://<bucket-path>/my_schema';

Table Location Scenarios:

Table Without LOCATION:

CREATE TABLE my_schema.my_table (id INT);

The table will be stored in the default warehouse directory (e.g., dbfs:/user/hive/warehouse/), not the schema's LOCATION.

Table With Explicit LOCATION: If you want the table to be stored under the schema's LOCATION, you need to specify the location explicitly:

```
CREATE TABLE my_schema.my_table (id INT)
LOCATION 's3://<bucket-path>/my_schema/my_table/';
```

So, if you want all tables under the schema to use the schema's LOCATION, explicitly specify the LOCATION for each table during creation.
upvoted 3 times

🗳️ 👤 **y2kal** 1 year, 3 months ago

It should be A, as the question states "all tables". Once an external DB is created, then all the tables in that would be by default be external.
upvoted 1 times

🗳️ 👤 **akashdesarda** 1 year, 3 months ago

Selected Answer: A

A is correct. If a database is created using location keyword then by default all the tables created in it will use that location. They follows <provided location>/_unity_catalog/tables/<uuid>
upvoted 1 times

🗳️ 👤 **leopedroso1** 1 year, 11 months ago

C is the correct answer. According to the documentation only the LOCATION is needed to make a table external. Moreover, we can also assume the keyword EXTERNAL is optional in the SQL statement.

<https://docs.databricks.com/en/sql/language-manual/sql-ref-external-tables.html>

upvoted 2 times

🗳️ 👤 **CY** 1 year, 12 months ago

'A' seems more appropriate.

All the tables in Delta lake house should be marked as external.. which can be achieved using location keyword at database level instead of each table level.

upvoted 3 times

🗳️ 👤 **Yogi05** 2 years, 1 month ago

Why not D? i know both C and D are same, but D is more precise

upvoted 2 times

🗳️ 👤 **Yogi05** 2 years, 1 month ago

my bad. D is having EXTERNAL keyword, got confused. C is correct answer

upvoted 2 times

🗳️ 👤 **Laraujo2022** 2 years, 2 months ago

If you set a location in a database level, all tables under this database are automatically external table, in my opinion is A is correct.

upvoted 1 times

🗳️ 👤 **Isio05** 1 year, 8 months ago

According to what I've found in Databricks forums: "Database location and Table location are independent". So it looks like specifying location at DB level is not sufficient as tables will be still created as managed ones.

upvoted 3 times

🗳️ 👤 **Quadronoid** 2 years, 3 months ago

Selected Answer: C

C is correct. Location keyword should be in create script of the table

upvoted 4 times

🗳️ 👤 **mouad_attaqi** 2 years, 3 months ago

C is correct, the key word to be used is Location, the keyword external is optional

upvoted 3 times

🗳️ 👤 **chokthewa** 2 years, 3 months ago

The correct is D

upvoted 2 times

🗳️ 👤 **mht3336** 2 years ago

there is no EXTERNAL key word in databricks, however it is there for other systems like Oracle, Hive, Cassandra etc.

upvoted 1 times

🗳️ 👤 **Dusica** 1 year, 8 months ago

and microsoft synapse

upvoted 1 times

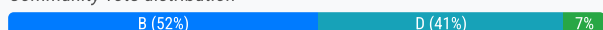
To reduce storage and compute costs, the data engineering team has been tasked with curating a series of aggregate tables leveraged by business intelligence dashboards, customer-facing applications, production machine learning models, and ad hoc analytical queries. The data engineering team has been made aware of new requirements from a customer-facing application, which is the only downstream workload they manage entirely. As a result, an aggregate table used by numerous teams across the organization will need to have a number of fields renamed, and additional fields will also be added.

Which of the solutions addresses the situation while minimally interrupting other teams in the organization without increasing the number of tables that need to be managed?

- A. Send all users notice that the schema for the table will be changing; include in the communication the logic necessary to revert the new table schema to match historic queries.
- B. Configure a new table with all the requisite fields and new names and use this as the source for the customer-facing application; create a view that maintains the original data schema and table name by aliasing select fields from the new table.
- C. Create a new table with the required schema and new fields and use Delta Lake's deep clone functionality to sync up changes committed to one table to the corresponding table.
- D. Replace the current table definition with a logical view defined with the query logic currently writing the aggregate table; create a new table to power the customer-facing application.
- E. Add a table comment warning all users that the table schema and field names will be changing on a given date; overwrite the table in place to the specifications of the customer-facing application.

Suggested Answer: B

Community vote distribution



alexvno **Highly Voted** 1 year, 11 months ago

Selected Answer: D

Create view. Can't be B as -> without increasing the number of tables that need to be managed

upvoted 7 times

carlosmps 1 year, 2 months ago

But option B does not increase the number of tables to maintain; in fact, it replaces the source table. The question states that it should not increase, and from that perspective, the NUMBER of tables does not increase. It only replaces the source and creates a view.

upvoted 1 times

EZZALDIN 11 months, 3 weeks ago

Replaces the current table with a view, but still creates a separate table for the customer app, this doesn't cleanly separate the two schemas.

upvoted 1 times

guillesd **Highly Voted** 2 years ago

Selected Answer: B

B makes way more sense, the number of tables managed do not increase since the old table won't be used anymore, then the view on top of this table is not another table to manage, just maintains the "original API" of the table to avoid breaking changes in downstream applications

upvoted 7 times

Ral17 **Most Recent** 2 months, 2 weeks ago

Selected Answer: B

I agree with Option B. This is the correct approach using views as an abstraction layer.

Why Option B is Correct

The Strategy

Create new table with the updated schema (renamed fields + new fields) → Powers customer-facing app

Create a view with the original schema/field names → Maintains backward compatibility for all other teams

Zero disruption to BI dashboards, ML models, and ad hoc queries

Single source of truth (only one physical table)

upvoted 1 times

🗨️ 👤 **gizzamo** 4 months, 3 weeks ago

Selected Answer: D

Doesn't Increase Table Count: vs Option B

Removes 1 table (current aggregate table becomes a view)

Adds 1 table (new table for customer-facing app)

Net change: 0 tables ✅

Minimally Interrupts Other Teams:

Original table name preserved as a view

Existing queries continue working against the view

No schema changes for existing consumers

Other teams see no difference in their workflows ✅

Supports New Requirements:

New table has the required field names and additional fields

Customer-facing application gets exactly what it needs

Optimized for the specific use case ✅

upvoted 1 times

🗨️ 👤 **363c4c5** 6 months, 3 weeks ago

Selected Answer: B

D replicates the business logic – we don't need that. It's enough to just create a view on top of the new table as mentioned in B.

upvoted 1 times

🗨️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

🗨️ 👤 **EZZALDIN** 11 months, 3 weeks ago

Selected Answer: B

it recommends creating a view that maps the new table back to the original schema. This view lets other teams continue using the table as they always have, with no changes to their queries.

upvoted 1 times

🗨️ 👤 **RandomForest** 1 year ago

Selected Answer: B

This approach achieves the following key goals:

1. Minimizes Disruption: By creating a view that mirrors the original schema, existing workloads that depend on the current schema remain uninterrupted.

Other teams can continue their queries without needing to adjust their logic for the schema change.

2. Meets New Requirements: The new table accommodates the changes required by the customer-facing application, ensuring that the application's updated requirements are fulfilled.

3. Avoids Table Duplication: Instead of maintaining multiple tables for the same dataset, this approach uses a combination of a new table and a view, reducing the overall management burden.

4Flexibility for Future Changes: Views can be adjusted as needed, providing a layer of abstraction. Future schema updates can be handled similarly without directly impacting dependent systems.

upvoted 1 times

🗨️ 👤 **HairyTorso** 1 year, 1 month ago

Selected Answer: B

Create view -> number of tables stay the same. Option D has overhead

upvoted 1 times

🗨️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: B

B - but I was wondering between B & D.

I do not like D since you replace the table with a view (query costs + you need to change the currently working workflow). Additionally, you create a table that does similar thing to the view - why?

upvoted 1 times

🗨️ 👤 **Nicks_name** 1 year, 2 months ago

E. not D because, by converting the aggregate table into a view, might introduce performance overheads as every access now potentially involves running complex query logic to reconstruct the desired dataset on-the-fly. This might not be ideal for performance-sensitive applications like business intelligence dashboards.

upvoted 1 times

🗨️ 👤 **vish007** 1 year, 2 months ago

Selected Answer: B

Option D will increase the Compute cost significantly as all the downstream teams will run the view which has logic for Aggregate table.

Option B make more sense with less impact to storage and compute cost which is the original ask for the data engineering team in the question.

upvoted 2 times

🗨️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: D

I am not sure whether B or D... I believe B increases the number of managed Tables as it states that a CREATE TABLE statement is run before a CREATE VIEW ... the fact that the CREATE VIEW will replace the current table is not really specified... still one could argue that it would be dumb not to do it but at this point i would say that D is more precise

upvoted 2 times

🗨️ 👤 **b.b.da.costa** 1 year, 3 months ago

The problem with this question is if the order of the sentence matters.

B: Create a table then create a view. Teams are interrupted after the creation of the table.

D: Create a view then create a table. Teams are not interrupted because they are consuming the view first.

upvoted 1 times

🗨️ 👤 **benni_ale** 1 year, 2 months ago

Also B does it really not increase the number of written tables? It states that a CREATE TABLE is run and CREATE VIEW is run... Nothing really points to the fact that the view will replace the table... Indeed I would opt for D

upvoted 1 times

🗨️ 👤 **vish007** 1 year, 2 months ago

Option D will increase the Compute cost significantly as all the downstream teams will run the view which has logic for Aggregate table.

Option B make more sense with less impact to storage and compute cost which is the original ask for the data engineering team in the question.

upvoted 1 times

🗨️ 👤 **kimberlysmith** 1 year, 3 months ago

Selected Answer: B

B is Correct. It does not create additional tables. The view mimics the old schema so not to interrupt downstream consumers. It ensures the aggregates are persisted to save on compute.

D is incorrect mostly due to the aggregates being baked into the view which is not optimal as each time downstream users query the view the joins and aggregates have to be recomputed.

upvoted 1 times

🗨️ 👤 **shaojunni** 1 year, 3 months ago

Selected Answer: D

D will not increase the number of table. It will create a new table and replace the aggregation table with a view. B will create a new table, a new view match old table name and schema, aggregation table still there.

upvoted 1 times

🗨️ 👤 **KB_Ai_Champ** 1 year, 4 months ago

option D is correct

docs : <https://docs.databricks.com/en/delta/update-schema.html>

also they specifically says that they dont want to increase managed tables!

upvoted 2 times

🗨️ 👤 **KB_Ai_Champ** 1 year, 4 months ago

Reasons :

No Increase in Managed Tables: By replacing the current table with a view, you maintain the same number of managed tables.

Backward Compatibility: The view can mimic the original table's schema, ensuring that existing queries and applications continue to function without modification.

Dedicated Table for New Requirements: The new table can be tailored to meet the specific needs of the customer-facing application without affecting other users.

upvoted 2 times

A Delta Lake table representing metadata about content posts from users has the following schema: user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude FLOAT, post_time TIMESTAMP, date DATE

This table is partitioned by the date column. A query is run with the following filter: longitude < 20 & longitude > -20

Which statement describes how data will be filtered?

- A. Statistics in the Delta Log will be used to identify partitions that might include files in the filtered range.
- B. No file skipping will occur because the optimizer does not know the relationship between the partition column and the longitude.
- C. The Delta Engine will use row-level statistics in the transaction log to identify the files that meet the filter criteria.
- D. Statistics in the Delta Log will be used to identify data files that might include records in the filtered range.
- E. The Delta Engine will scan the parquet file footers to identify each row that meets the filter criteria.

Suggested Answer: D

Community vote distribution

D (91%)

9%

 **Enduresoul** Highly Voted 2 years, 2 months ago

Selected Answer: D

D is correct. A partition can include multiple files. And the statistics are collected for each file.
upvoted 12 times

 **KadELbied** Most Recent 9 months, 1 week ago

Selected Answer: D

suretly D
upvoted 1 times

 **AlejandroU** 1 year, 1 month ago

Selected Answer: B

Answer B. Single Comparison Filter (e.g., latitude > 66.3): File skipping is highly efficient because Delta can use min/max statistics to directly eliminate files that don't meet the condition.
Range Filters (e.g., longitude < 20 AND longitude > -20): File skipping is still possible but less efficient, because Delta has to evaluate whether any records in the file might meet the condition, even if the min and max values of the column in the file overlap with the filter range.
So in summary, file skipping works best with single comparisons like latitude > 66.3 but is less effective with range filters like longitude < 20 AND longitude > -20.
upvoted 1 times

 **Sriramiyer92** 1 year, 1 month ago

Selected Answer: D

Do not get confused between option c and d. Given answer is correct.
upvoted 1 times

 **hebied** 1 year, 2 months ago

Selected Answer: D

D is more suitable
upvoted 1 times

 **AndreFR** 1 year, 5 months ago

Selected Answer: D

Min and max values of each parquet file are stored in Delta Logs
Delta data skipping automatically collects the stats (min, max, etc.) for the first 32 columns for each underlying Parquet file when you write data into a Delta table. Databricks takes advantage of this information (minimum and maximum values) at query time to skip unnecessary files in order to speed up the queries.
<https://www.databricks.com/discover/pages/optimize-data-workloads-guide#delta-data>
upvoted 2 times

 **AziLa** 2 years ago

Correct Ans is D

upvoted 2 times

  **Quadronoid** 2 years, 3 months ago

Selected Answer: C

I guess C option is right since transaction log contains information about max/min values of first 32 columns, it can be used in order to filter files.

upvoted 1 times

  **Quadronoid** 2 years, 3 months ago

I reread the question and thing that I made a mistake, in option C there is information about row-level statistics, but, I guess, statistics in Delta Log it is more less about columns. So, now D looks fine for me.

upvoted 4 times

  **sturcu** 2 years, 3 months ago

Selected Answer: D

D is Correct

upvoted 3 times

A small company based in the United States has recently contracted a consulting firm in India to implement several new data engineering pipelines to power artificial intelligence applications. All the company's data is stored in regional cloud storage in the United States. The workspace administrator at the company is uncertain about where the Databricks workspace used by the contractors should be deployed. Assuming that all data governance considerations are accounted for, which statement accurately informs this decision?

- A. Databricks runs HDFS on cloud volume storage; as such, cloud virtual machines must be deployed in the region where the data is stored.
- B. Databricks workspaces do not rely on any regional infrastructure; as such, the decision should be made based upon what is most convenient for the workspace administrator.
- C. Cross-region reads and writes can incur significant costs and latency; whenever possible, compute should be deployed in the same region the data is stored.
- D. Databricks leverages user workstations as the driver during interactive development; as such, users should always use a workspace deployed in a region they are physically near.
- E. Databricks notebooks send all executable code from the user's browser to virtual machines over the open internet; whenever possible, choosing a workspace region near the end users is the most secure.

Suggested Answer: C

Community vote distribution

C (83%)

B (17%)

  **_Lukas_** 2 months, 2 weeks ago

Selected Answer: C

Correct:

C - The main practical concern is cross-region data access. If compute runs in a different region from the data, you pay extra for data transfer and suffer higher latency.

Databricks clusters read data directly from cloud object storage, so placing compute near the data is the standard practice.

Incorrect:

A - Databricks doesn't use HDFS. It uses cloud object storage directly.

B - Workspaces are region-specific and rely heavily on regional cloud infrastructure.

D - Drivers and compute run in the cloud, not on user machines. Physical user location doesn't dictate region choice.

E - Notebook code runs on cloud VMs in the workspace region. Choosing region for "security of code transit" is irrelevant.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

  **RandomForest** 1 year ago

Selected Answer: C

C is the correct answer.

upvoted 1 times

  **imatheushenrique** 1 year, 2 months ago

(C)

The decision is about where the Databricks workspace used by the contractors should be deployed. The contractors are based in India, while all the company's data is stored in regional cloud storage in the United States. When choosing a region for deploying a Databricks workspace, one of the important factors to consider is the proximity to the data sources and sinks. Cross-region reads and writes can incur significant costs and latency due to network bandwidth and data transfer fees. Therefore, whenever possible, compute should be deployed in the same region the data is stored to optimize performance and reduce costs

upvoted 2 times

  **spaceexplorer** 1 year, 6 months ago

Selected Answer: C

C is the answer.

upvoted 3 times

🗨️ 👤 **RafaelCFC** 1 year, 7 months ago

Selected Answer: C

An important part of data governance is usage cost, and, as a general data engineering practice, egress costs related to moving data between regions is always an important consideration. Having the workspaces located in a different region than the contractors will incur to them in very little nuisance, while greatly saving in this sense.

upvoted 2 times

🗨️ 👤 **Patito** 1 year, 7 months ago

Selected Answer: B

From where data engineering team develops pipelines is independent of where the data objects reside in the cloud storage.

upvoted 1 times

🗨️ 👤 **coercion** 1 year, 2 months ago

These pipelines will create clusters (machines) which will reside in a different region than the data and that will cause latency issues. So C should be the correct option.

upvoted 2 times

🗨️ 👤 **chokthewa** 1 year, 9 months ago

C is correct.

upvoted 2 times

The downstream consumers of a Delta Lake table have been complaining about data quality issues impacting performance in their applications. Specifically, they have complained that invalid latitude and longitude values in the activity_details table have been breaking their ability to use other geolocation processes.

A junior engineer has written the following code to add CHECK constraints to the Delta Lake table:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
  latitude >= -90 AND
  latitude <= 90 AND
  longitude >= -180 AND
  longitude <= 180);
```

A senior engineer has confirmed the above logic is correct and the valid ranges for latitude and longitude are provided, but the code fails when executed.

Which statement explains the cause of this failure?

- A. Because another team uses this table to support a frequently running application, two-phase locking is preventing the operation from committing.
- B. The activity_details table already exists; CHECK constraints can only be added during initial table creation.
- C. The activity_details table already contains records that violate the constraints; all existing data must pass CHECK constraints in order to add them to an existing table.
- D. The activity_details table already contains records; CHECK constraints can only be added prior to inserting values into a table.
- E. The current table schema does not contain the field valid_coordinates; schema evolution will need to be enabled before altering the table to add a constraint.

Suggested Answer: B

Community vote distribution

C (100%)

🗳️ 👤 **8605246** Highly Voted 📈 2 years, 6 months ago

incorrect the correct option is C, with constraints, if added to an existing table the existing data in the table must be consistent with the constraint otherwise it fails

<https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-alter-table.html#add-constraint>

upvoted 13 times

🗳️ 👤 **KadELbied** Most Recent ⌚ 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

🗳️ 👤 **AndreFR** 1 year, 5 months ago

Selected Answer: C

-- CREATE TABLE

create table test_constraint (t1 varchar(2), n1 int);

-- ADD VALUE

insert into test_constraint values ('v3', 3);

-- ADD CONSTRAINT VIOLATED BY CURRENT DATA

-- should throw error : 1 row in spark_catalog.default.test_constraint violate the new CHECK constraint (n1 < 3)

alter table test_constraint add constraint valid_n1 check (n1 < 3);

-- ADD CONSTRAINT NOT VIOLATED BY CURRENT DATA (no error)

alter table test_constraint add constraint valid_n1 check (n1 < 100);

upvoted 1 times

🗨️ 👤 **faraaz132** 1 year, 6 months ago

Selected Answer: C

C is correct.

upvoted 1 times

🗨️ 👤 **PrashantTiwari** 1 year, 12 months ago

C is correct

upvoted 1 times

🗨️ 👤 **DAN_H** 2 years ago

correct ans is C

upvoted 1 times

🗨️ 👤 **AziLa** 2 years ago

correct ans is C

upvoted 1 times

🗨️ 👤 **Jay_98_11** 2 years ago

Selected Answer: C

correct

upvoted 1 times

🗨️ 👤 **kz_data** 2 years ago

Selected Answer: C

C is the correct answer

upvoted 1 times

🗨️ 👤 **Patito** 2 years, 1 month ago

Selected Answer: C

C is correct

upvoted 1 times

🗨️ 👤 **hamzaKhribi** 2 years, 2 months ago

Selected Answer: C

C is correct

upvoted 1 times

🗨️ 👤 **Enduresoul** 2 years, 2 months ago

Selected Answer: C

C is correct

upvoted 1 times

🗨️ 👤 **aragorn_brego** 2 years, 2 months ago

Selected Answer: C

When adding a CHECK constraint to an existing table, the operation will fail if there are any rows in the table that do not meet the constraint. Before a CHECK constraint can be added, the data already in the table must be validated to ensure that it complies with the constraint conditions. If any existing records violate the new constraints, they must be corrected or removed before the ALTER TABLE command can be successfully executed.

upvoted 2 times

🗨️ 👤 **BIKRAM063** 2 years, 3 months ago

Selected Answer: C

Correct option C : existing data violated check constraint condition

upvoted 1 times

🗨️ 👤 **Quadronoid** 2 years, 3 months ago

Selected Answer: C

Right answer is C

upvoted 1 times

🗨️ 👤 **sturcu** 2 years, 3 months ago

Selected Answer: C

C - table already has data

upvoted 1 times

🗨️ 👤 **MarceloManhaes** 2 years, 4 months ago

Yes the correct is option C

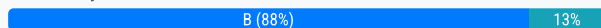
upvoted 1 times

Which of the following is true of Delta Lake and the Lakehouse?

- A. Because Parquet compresses data row by row, strings will only be compressed when a character is repeated multiple times.
- B. Delta Lake automatically collects statistics on the first 32 columns of each table which are leveraged in data skipping based on query filters.
- C. Views in the Lakehouse maintain a valid cache of the most recent versions of source tables at all times.
- D. Primary and foreign key constraints can be leveraged to ensure duplicate values are never entered into a dimension table.
- E. Z-order can only be applied to numeric values stored in Delta Lake tables.

Suggested Answer: B

Community vote distribution



Lukas 2 months, 2 weeks ago

Selected Answer: B

Correct:

B - Delta Lake does automatically collect statistics on the first 32 columns, and these statistics are used for data skipping, which reduced the amount of data scanned during queries.

Incorrect:

A - Parquet is a columnar, not row-by-row format. It uses various encodings and compresses much more than just repeated characters in strings.

C - Regular views are just saved SQL definitions. They do not maintain a physical cache of the latest table data.

D - Primary and foreign key constraints in Delta are not used to physically enforce uniqueness or referential integrity they way a traditional OLTP database would.

E - Z-ordering can be applied to various column types (including strings), not only numeric columns.

upvoted 2 times

KadELbied 9 months, 1 week ago

Selected Answer: B

Surely B

upvoted 1 times

SRV_33 1 year ago

Selected Answer: B

Complete statement is correct only in this option

upvoted 1 times

PrashantTiwari 1 year, 12 months ago

B is correct

upvoted 1 times

guillesd 2 years ago

Selected Answer: B

B is correct

upvoted 2 times

spaceexplorer 2 years ago

Selected Answer: B

B is correct

upvoted 1 times

Crocjun 2 years, 1 month ago

Can anyone explain why D is not correct?

upvoted 1 times

cryptoflam 2 years, 1 month ago

Because Primary & Foreign Key information is not enforced.

"Primary and foreign keys are informational only and are not enforced" from:

<https://docs.databricks.com/en/tables/constraints.html#declare-primary-key-and-foreign-key-relationships>

upvoted 2 times

  **decisiontree** 1 year, 1 month ago

Foreign key constraints have nothing to do with the duplicate values.

upvoted 1 times

  **Patito** 2 years, 1 month ago

Selected Answer: B

B is correct since statistics are collected for the first 32 columns and stored in the transaction log.

upvoted 3 times

  **AndreFR** 1 year, 5 months ago

<https://www.databricks.com/discover/pages/optimize-data-workloads-guide#delta-data>

Delta data skipping automatically collects the stats (min, max, etc.) for the first 32 columns for each underlying Parquet file when you write data into a Delta table. Databricks takes advantage of this information (minimum and maximum values) at query time to skip unnecessary files in order to speed up the queries.

upvoted 1 times

  **ervinshang** 2 years, 1 month ago

Selected Answer: B

B is correct,

C is error, can't have new cache in view

upvoted 1 times

  **f728f7f** 2 years, 1 month ago

Selected Answer: C

C is correct

upvoted 1 times

  **chokthewa** 2 years, 3 months ago

B is correct.

<https://docs.delta.io/2.0.0/table-properties.html>

upvoted 1 times

The view updates represents an incremental batch of all newly ingested data to be inserted or updated in the customers table.

The following logic is used to process these records.

```

MERGE INTO customers
USING (
  SELECT updates.customer_id as merge_key, updates.*
  FROM updates

  UNION ALL

  SELECT NULL as merge_key, updates.*
  FROM updates JOIN customers
  ON updates.customer_id = customers.customer_id
  WHERE customers.current = true AND updates.address <> customers.address
) staged_updates
ON customers.customer_id = mergeKey
WHEN MATCHED AND customers.current = true AND customers.address <> staged_updates.address THEN
  UPDATE SET current = false, end_date = staged_updates.effective_date
WHEN NOT MATCHED THEN
  INSERT(customer_id, address, current, effective_date, end_date)
  VALUES(staged_updates.customer_id, staged_updates.address, true, staged_updates.effective_date,
  null)

```

Which statement describes this implementation?

- A. The customers table is implemented as a Type 3 table; old values are maintained as a new column alongside the current value.
- B. The customers table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.
- C. The customers table is implemented as a Type 0 table; all writes are append only with no changes to existing values.
- D. The customers table is implemented as a Type 1 table; old values are overwritten by new values and no history is maintained.
- E. The customers table is implemented as a Type 2 table; old values are overwritten and new customers are appended.

Suggested Answer: B

Community vote distribution

B (100%)

  **_Lukas_** 2 months, 2 weeks ago

Selected Answer: B

Correct:

B - In the 'WHEN MATCHED' clause, the existing row is updated to 'current = false' and given an 'end_date', which 'closes' the old version but keeps it in the table as history.

In the 'WHEN NOT MATCHED' clause, a new row is inserted with 'current = true', a new 'effective_date' and 'end_date = null', representing the latest version of that customer.

This pattern (expire old row + insert new current row with effective/end dates) is exactly the definition of a Slowly Changing Dimension Type 2 implementation.

Incorrect:

A - Old values are maintained as a rows, not as new columns.

C - Old values are marked by setting 'current = false' and specifying an 'end_date'.

D - History is maintained.

E - This contradicts Type 2, as Type 2 never overwrites old values.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

Surely B

upvoted 1 times

  **Tayari** 1 year, 3 months ago

B is correct

upvoted 1 times

  **imatheushenrique** 1 year, 8 months ago

B. The customers table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.

A Type 1 table does not track changes in dimensional attributes - the new value overwrites the existing value. Here, we do not preserve historical changes in data.

A Type 2 Table tracks change over time by creating new rows for each change. A new dimension record is inserted with a high-end date or one with NULL. The previous record is "closed" with an end date. This approach maintains a complete history of changes and allows for as-was reporting use cases.

A data warehousing method called Slowly Changing Dimension (SCD) Type 3 is used to track both the old and new values while managing historical changes in data over time. To reflect the historical and present values of an attribute, SCD Type 3 keeps two extra columns in the dimension table.

upvoted 3 times

  **spaceexplorer** 2 years ago

Selected Answer: B

B is correct

upvoted 3 times

  **kz_data** 2 years ago

Selected Answer: B

B is correct

upvoted 1 times

  **chokthewa** 2 years, 3 months ago

B is correct.

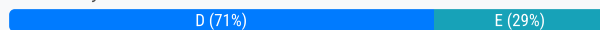
upvoted 2 times

The DevOps team has configured a production workload as a collection of notebooks scheduled to run daily using the Jobs UI. A new data engineering hire is onboarding to the team and has requested access to one of these notebooks to review the production logic. What are the maximum notebook permissions that can be granted to the user without allowing accidental changes to production code or data?

- A. Can Manage
- B. Can Edit
- C. No permissions
- D. Can Read
- E. Can Run

Suggested Answer: D

Community vote distribution



🗨️ 👤 **_Lukas_** 2 months, 2 weeks ago

Selected Answer: D

Correct:

D - With 'Can Read', the engineer can view all notebook cells and logic and add comments, which is enough to review how the production job works. Can Read does not allow attaching a cluster or running commands, so the user cannot accidentally execute code that might modify production data or state.

Incorrect:

A, B - 'Can Manage' or 'Can Edit' would allow changing code, configuration, or permissions, making accidental changes to production logic or access highly likely.

C - No permission does not allow to review the production logic.

E - 'Can Run' would let the user attach the notebook to compute and execute cells, which could trigger production writes by mistake.

upvoted 2 times

🗨️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

Surely D

upvoted 1 times

🗨️ 👤 **AlHerd** 10 months, 2 weeks ago

Selected Answer: D

We don't want to person to run the code in the notebooks as this might change the data, so "can read" is best.

upvoted 1 times

🗨️ 👤 **RasipalayamRDK** 1 year ago

Selected Answer: E

Maximum permission Can Run .<https://docs.databricks.com/en/notebooks/notebooks-collaborate.html>

upvoted 1 times

🗨️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: D

I change my mind - answer D. D would still allow to change data (provided the notebook changes the data), however it would require using importing from other notebook, which is more of a nuisance than just clicking Run in the notebook.

upvoted 1 times

🗨️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: E

Answer E - the question states "maximum notebook permissions", which means "Can Run", see:

<https://docs.databricks.com/en/security/auth/access-control/index.html#notebook-acls>

upvoted 1 times

🗨️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: D

It happens that by only running a production workload scheduled to run only once a day, some directories got removed. I would say D as by only running a program when you should not you could commit changes to production data.

upvoted 1 times

  **nedlo** 1 year, 3 months ago

Selected Answer: D

Can Read, because Running could have changed DATA

upvoted 2 times

  **olly24** 1 year, 5 months ago

Correct answer is C. <https://learn.microsoft.com/en-us/azure/databricks/security/auth/access-control/#--notebook-acls>

upvoted 1 times

  **Melik3** 1 year, 5 months ago

Selected Answer: E

can run is the correct answer here because the question asked for the maximum possible permission without editing.

upvoted 2 times

  **shynkary** 1 year, 5 months ago

disagree, it says "changes to production code or data". Can Run permission allows to run a workflow which will cause changes in data

upvoted 3 times

  **AndreFR** 1 year, 5 months ago

Selected Answer: D

<https://docs.databricks.com/en/security/auth/access-control/index.html#notebook-acls>

upvoted 2 times

  **alexvno** 2 years, 2 months ago

Selected Answer: D

Correct

upvoted 3 times

  **Quadronoid** 2 years, 3 months ago

Selected Answer: D

Correct, D

upvoted 1 times

A table named `user_ltv` is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The `user_ltv` table has the following schema:

email STRING, age INT, ltv INT

The following view definition is executed:

```
CREATE VIEW email_ltv AS
SELECT
CASE WHEN
    is_member('marketing') THEN email
    ELSE 'REDACTED'
END AS email,
    ltv
FROM user_ltv
```

An analyst who is not a member of the marketing group executes the following query:

```
SELECT * FROM email_ltv -
```

Which statement describes the results returned by this query?

- A. Three columns will be returned, but one column will be named "REDACTED" and contain only null values.
- B. Only the email and ltv columns will be returned; the email column will contain all null values.
- C. The email and ltv columns will be returned with the values in `user_ltv`.
- D. The email, age, and ltv columns will be returned with the values in `user_ltv`.
- E. Only the email and ltv columns will be returned; the email column will contain the string "REDACTED" in each row.

Suggested Answer: E

Community vote distribution

E (100%)

  **_Lukas_** 2 months, 2 weeks ago

Selected Answer: E

Correct:

E - The view selects only two columns: a CASE expression aliased as 'email' plus 'ltv', so the view schema is just 'email', 'ltv'.

For a user who is not in the 'marketing' group, `is_member('marketing')` is false, so the CASE expression always returns the literal 'REDACTED' for every row, while 'ltv' is passed through unchanged.

Incorrect:

A - The 'age' column is not listed in the view definition, so the view returns two columns, not three.

B - The 'email' column will not contain null values, because the CASE expression specifies an ELSE condition for people who are not in the 'marketing' group.

C - The 'email' column will contain 'REDACTED' values for people who are not in the 'marketing' group.

D - The 'age' column will not be returned.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: E

suretly E

upvoted 1 times

  **AndreFR** 1 year, 5 months ago

Selected Answer: E

A, D incorrect because 2 columns email & ltv are returned.

B incorrect because email will not always contain null values (unless email is null)

The user is not a member of "marketing", so 3 is the correct answer. If the user were a member of "marketing" group, correct answer would have been C

upvoted 2 times

🗲️ 👤 **Isio05** 1 year, 8 months ago

Selected Answer: E

E, only email column is selected and is not allowed to be viewed by the user

upvoted 1 times

🗲️ 👤 **alexvno** 2 years, 2 months ago

Selected Answer: E

sure E

upvoted 2 times

🗲️ 👤 **ismoshkov** 2 years, 3 months ago

Selected Answer: E

E is correct

upvoted 2 times

🗲️ 👤 **chokthewa** 2 years, 3 months ago

E is correct.

upvoted 1 times

The data governance team has instituted a requirement that all tables containing Personal Identifiable Information (PH) must be clearly annotated. This includes adding column comments, table comments, and setting the custom table property "contains_pii" = true.

The following SQL DDL statement is executed to create a new table:

```
CREATE TABLE dev.pii_test
(id INT, name STRING COMMENT "PII")
COMMENT "Contains PII"
TBLPROPERTIES ('contains_pii' = True)
```

Which command allows manual confirmation that these three requirements have been met?

- A. DESCRIBE EXTENDED dev.pii_test
- B. DESCRIBE DETAIL dev.pii_test
- C. SHOW TBLPROPERTIES dev.pii_test
- D. DESCRIBE HISTORY dev.pii_test
- E. SHOW TABLES dev

Suggested Answer: A

Community vote distribution

A (100%)

🗳️ **_Lukas_** 2 months, 2 weeks ago

Selected Answer: A

Correct:

A - 'DESCRIBE EXTENDED' returns column name, data type, and column comment, so it lets you verify that 'name STRING COMMENT "PII"' is present. In the extended metadata section, it also shows the table comment ("Contains PII") and the table_properties map, where you can confirm that 'contains_pii' = true is set.

Incorrect:

- B - This option focuses on high-level table metadata but does not list per-column comments.
- C - This option only shows table properties and not column or table comments.
- D - This option shows operation history, not comments or custom properties.
- E - This option lists tables in a schema and basic info, but not comments or properties.

upvoted 1 times

🗳️ **KadELbied** 9 months, 1 week ago

Selected Answer: A

suretly A

upvoted 1 times

🗳️ **Tedet** 11 months, 1 week ago

Selected Answer: A

C is wrong since Property value returned by this statement excludes some properties that are internal to spark and hive. The excluded properties are:

All the properties that start with prefix spark.sql

Property keys such as: EXTERNAL, comment

All the properties generated internally by hive to store statistics. Some of these properties are: numFiles, numPartitions, numRows.

upvoted 1 times

🗳️ **RandomForest** 1 year ago

Selected Answer: A

Answer A is correct as explained by lexaneon

upvoted 1 times

🗳️ **lexaneon** 1 year, 7 months ago

looks like A & C are correct.. <https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-aux-show-tblproperties.html#show-tblproperties>

upvoted 3 times

  **lexaneon** 1 year, 7 months ago

if we want to see also columns comments then A
upvoted 5 times

  **rok21** 1 year, 8 months ago

Selected Answer: A

correct A !

upvoted 4 times

The data governance team is reviewing code used for deleting records for compliance with GDPR. They note the following logic is used to delete records from the Delta Lake table named users.

```
DELETE FROM users
WHERE user_id IN
  (SELECT user_id FROM delete_requests)
```

Assuming that user_id is a unique identifying key and that delete_requests contains all users that have requested deletion, which statement describes whether successfully executing the above logic guarantees that the records to be deleted are no longer accessible and why?

- A. Yes; Delta Lake ACID guarantees provide assurance that the DELETE command succeeded fully and permanently purged these records.
- B. No; the Delta cache may return records from previous versions of the table until the cluster is restarted.
- C. Yes; the Delta cache immediately updates to reflect the latest data files recorded to disk.
- D. No; the Delta Lake DELETE command only provides ACID guarantees when combined with the MERGE INTO command.
- E. No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

Suggested Answer: E

Community vote distribution

E (100%)

🗳️ 👤 **kz_data** 1 year, 7 months ago

Selected Answer: E

E is correct

upvoted 4 times

🗳️ 👤 **ervinshang** 1 year, 7 months ago

Selected Answer: E

E is correct.

upvoted 3 times

🗳️ 👤 **alexvno** 1 year, 9 months ago

Selected Answer: E

Correct

upvoted 2 times

🗳️ 👤 **chokthewa** 1 year, 9 months ago

E is correct.

upvoted 2 times

An external object storage container has been mounted to the location /mnt/finance_eda_bucket.

The following logic was executed to create a database for the finance team:

```
CREATE DATABASE finance_eda_db
LOCATION '/mnt/finance_eda_bucket';
GRANT USAGE ON DATABASE finance_eda_db TO finance;
GRANT CREATE ON DATABASE finance_eda_db TO finance;
```

After the database was successfully created and permissions configured, a member of the finance team runs the following code:

```
CREATE TABLE finance_eda_db.tx_sales AS
SELECT *
FROM sales
WHERE state = "TX";
```

If all users on the finance team are members of the finance group, which statement describes how the tx_sales table will be created?

- A. A logical table will persist the query plan to the Hive Metastore in the Databricks control plane.
- B. An external table will be created in the storage container mounted to /mnt/finance_eda_bucket.
- C. A logical table will persist the physical plan to the Hive Metastore in the Databricks control plane.
- D. An managed table will be created in the storage container mounted to /mnt/finance_eda_bucket.
- E. A managed table will be created in the DBFS root storage container.

Suggested Answer: B

Community vote distribution

D (65%)

E (35%)

  **tkg13** Highly Voted 2 years, 5 months ago

Correct Answer D

<https://docs.databricks.com/en/data-governance/unity-catalog/create-schemas.html#language-SQL>

upvoted 11 times

  **cotardo2077** 2 years, 5 months ago

you are right, it is managed table

upvoted 2 times

  **CertPeople** 2 years, 5 months ago

Nope, you are talking about MANAGED LOCATION (from Unity). In the question states LOCATION (not Unity based), which is not managed

<https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-create-schema.html>

upvoted 2 times

  **CertPeople** 2 years, 5 months ago

Effectively doing a test on one of my clusters the table is MANAGED

upvoted 9 times

  **MarceloManhaes** Highly Voted 2 years, 4 months ago

Every unmanaged(external) table creation needs to put keyword LOCATION despite if database, that table resides, is put with LOCATION sententece. So B is incorrect. D is correct because the sentence to creates the table is a managed table.

<https://docs.databricks.com/en/lakehouse/data-objects.html>

upvoted 7 times

  **_Lukas_** Most Recent 2 months, 2 weeks ago

Selected Answer: D

Correct:

D - When a database is created with a 'LOCATION' clause, any table created in that database without its own 'LOCATION' is a managed table whose data is stored under the database's location.

Incorrect:

A, C - These describe views, which store only a logical query definition. 'CREATE TABLE ... AS SELECT' creates a physical table with data files, not just

a persisted plan.

B - A table becomes external only when a 'LOCATION' is specified at the table level. Pointing the database to a location still produces managed tables stored there by default.

E - The DBFS root is the default location only when no database or table location is specified. Since this database has an explicit 'LOCATION' the table data is not stored in the DBFS root but in '/mnt/finance_eda_bucket'.

upvoted 2 times

🗳️ 👤 **ASRCA** 1 year, 1 month ago

Selected Answer: B

This is because the storage container is mounted to /mnt/finance_eda_bucket, and the code executed by the finance team member would create an external table in that location.

upvoted 2 times

🗳️ 👤 **AlejandroU** 1 year, 1 month ago

Selected Answer: D

Answer D. The use of the LOCATION clause with a DBFS path (/mnt/finance_eda_bucket) suggests that Hive Metastore and DBFS location are being used. The answer is correct in the context of Hive Metastore and DBFS location, but if Unity Catalog (UC) is in use, the result would be an external table, not a managed one.

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: D

No USE DATABASE statement otherwise it would have been external

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: D

No USE DATABASE statement otherwise it would have been external

upvoted 1 times

🗳️ 👤 **coercion** 1 year, 8 months ago

Selected Answer: D

D as the word LOCATION is not specified. Although the data will be stored in an external location but the table will still be a managed table.

upvoted 2 times

🗳️ 👤 **Curious76** 1 year, 11 months ago

Selected Answer: E

E is correct coz this table is managed on top of the external source file. a managed tables are stored on DBFS.

upvoted 2 times

🗳️ 👤 **hal2401me** 1 year, 11 months ago

Selected Answer: D

Correct answer D.

just did a test. As with DBR12.2, UC databases are not supported with location on dbfs, but s3/abfss. However, Hive_metastore databases are supported with location on dbfs. Then, a table created in this database IS a managed table, as verified with describe extend command.

upvoted 5 times

🗳️ 👤 **s_villahermosa91** 2 years ago

Selected Answer: E

Correct Answer E

upvoted 2 times

🗳️ 👤 **kz_data** 2 years ago

Selected Answer: D

D is the correct answer, the table created is a managed table and not external, and it will be located under the location defined in the database's creation DDL.

upvoted 1 times

🗳️ 👤 **azurelearn2020** 2 years, 2 months ago

Selected Answer: D

It will be a managed table created under specified database. Location keyword used for database will make sure all the managed tables are stored in database location.

upvoted 2 times

🗳️ 👤 **Enduresoul** 2 years, 2 months ago

Selected Answer: D

D is correct. The table will be created as managed, because no LOCATION is specified on table creation. The table will be created in the location specified with database creation

upvoted 4 times

  **Dileepvikram** 2 years, 3 months ago

I think the answer id D

upvoted 1 times

  **PearApple** 2 years, 3 months ago

I followed the steps to create schema and table, the answer is D

upvoted 2 times

  **jerborder** 2 years, 3 months ago

Correct answer is D. "data for a managed table resides in the location of the database it is registered to

upvoted 1 times

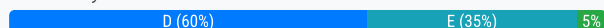
Although the Databricks Utilities Secrets module provides tools to store sensitive credentials and avoid accidentally displaying them in plain text users should still be careful with which credentials are stored here and which users have access to using these secrets.

Which statement describes a limitation of Databricks Secrets?

- A. Because the SHA256 hash is used to obfuscate stored secrets, reversing this hash will display the value in plain text.
- B. Account administrators can see all secrets in plain text by logging on to the Databricks Accounts console.
- C. Secrets are stored in an administrators-only table within the Hive Metastore; database administrators have permission to query this table by default.
- D. Iterating through a stored secret and printing each character will display secret contents in plain text.
- E. The Databricks REST API can be used to list secrets in plain text if the personal access token has proper credentials.

Suggested Answer: D

Community vote distribution



🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: E

suretly E

upvoted 2 times

🗳️ 👤 **Tedet** 11 months, 1 week ago

Selected Answer: D

Secret redaction

Storing credentials as Databricks secrets makes it easy to protect your credentials when you run notebooks and jobs. However, it is easy to accidentally print a secret to standard output buffers or display the value during variable assignment.

upvoted 1 times

🗳️ 👤 **AlejandroU** 1 year, 1 month ago

Selected Answer: D

Answer D. `dbutils.secrets.get(scope="myScope", key="myKey")` retrieves the plain text value of a secret, which is then available for use in code. Limitation: Once the secret is retrieved, if improperly handled (e.g., logged or iterated), its plain text value can be exposed. Option E: The REST API can list secrets in plain text if proper credentials (e.g., a personal access token) are provided. This is unrelated to `dbutils.secrets.get` but is a valid limitation of the overall secrets management framework in Databricks. Note that the difference between Option D or E is if it is a limitation related to Databricks Utilities Secret (`dbutils.secrets`), in this case option D is the correct option.

upvoted 1 times

🗳️ 👤 **Sriramiyer92** 1 year, 1 month ago

Selected Answer: D

Cannot be option E as it justs lists the Secret value. It does not print the content therein

upvoted 1 times

🗳️ 👤 **fe3b2fc** 1 year, 5 months ago

Selected Answer: D

`value = dbutils.secrets.get(scope="myScope", key="myKey")`

for char in value:

`print(char, end=" ")`

Out:

`y o u r _ v a l u e`

upvoted 4 times

🗳️ 👤 **coercion** 1 year, 8 months ago

Selected Answer: E

Only through REST API or CLI you can fetch the secret if you have valid token

upvoted 3 times

🗨️ 👤 **Er5** 1 year, 10 months ago

E: <https://docs.databricks.com/api/azure/workspace/secrets/listsecrets>

GET /api/2.0/secrets/list won't list secrets in plain text.

D: if print it without iterating it in a for loop the output is kind of encrypted where it is showing [REDACTED]. But, if I do it as shown in the screenshot, I'm able to see the value of the secret key.

<https://community.databricks.com/t5/data-engineering/how-to-avoid-databricks-secret-scope-from-exposing-the-value-of/td-p/12254>

<https://docs.databricks.com/en/security/secrets/redaction.html>

Secret redaction for notebook cell output applies only to literals. The secret redaction functionality does not prevent deliberate and arbitrary transformations of a secret literal.

upvoted 2 times

🗨️ 👤 **Lucario95** 1 year, 11 months ago

Selected Answer: E

Both D and E seems correct.

They are poorly written thought because for D just printing the characters (not separated by spaces, newlines or something) would not work, while E if launched inside databricks workspace would not work neither.

upvoted 2 times

🗨️ 👤 **PrashantTiwari** 1 year, 12 months ago

D is correct

upvoted 2 times

🗨️ 👤 **guillesd** 2 years ago

Selected Answer: D

D is for sure correct (tried it several times on a Databricks environment).

upvoted 2 times

🗨️ 👤 **guillesd** 2 years ago

Regarding E, it can list secrets (with scopes) but I am not sure it can list secret contents.

upvoted 1 times

🗨️ 👤 **DAN_H** 2 years ago

Selected Answer: D

D is correct

upvoted 3 times

🗨️ 👤 **spaceexplorer** 2 years ago

Selected Answer: D

D is correct

upvoted 2 times

🗨️ 👤 **Def21** 2 years ago

Selected Answer: E

At least E is a correct answer.

B: You can't see secrets in Admin console. Only via REST API, CLI etc.

C: Secrets are. not stored in Hive Metastore.

D: I am not sure if iterating through secret character by character would work?

E: This is at least correct. Using this.

upvoted 1 times

🗨️ 👤 **ranith** 2 years ago

B and E both seems to be correct:

<https://community.databricks.com/t5/data-engineering/how-to-avoid-databricks-secret-scope-from-exposing-the-value-of/td-p/12254/page/2>

upvoted 1 times

🗨️ 👤 **Jay_98_11** 2 years ago

Selected Answer: D

For sure it's D

upvoted 2 times

🗨️ 👤 **hkay** 2 years, 1 month ago

Answer is E:

```
/api/2.0/secrets/get
```

```
{  
  "key": "string",  
  "value": "string"  
}
```

The REST API can potentially expose secrets in plain text if a user with appropriate permissions (including access to both secrets/list and secrets/get) uses a personal access token.

upvoted 3 times

  **Patito** 2 years, 1 month ago

Selected Answer: D

Iterating through the secrets provides a way to see the secret's password.

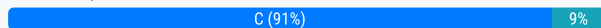
upvoted 2 times

What statement is true regarding the retention of job run history?

- A. It is retained until you export or delete job run logs
- B. It is retained for 30 days, during which time you can deliver job run logs to DBFS or S3
- C. It is retained for 60 days, during which you can export notebook run results to HTML
- D. It is retained for 60 days, after which logs are archived
- E. It is retained for 90 days or until the run-id is re-used through custom run configuration

Suggested Answer: B

Community vote distribution



stuart_gta1 1 year, 12 months ago

B is wrong, Should be C.
upvoted 9 times

Yogi05 1 year, 7 months ago

C is correct answer. <https://docs.databricks.com/en/workflows/jobs/monitor-job-runs.html>
upvoted 7 times

ealpuche 5 months, 3 weeks ago

Selected Answer: D
<https://docs.databricks.com/aws/en/jobs/monitor>
upvoted 1 times

ealpuche 5 months, 3 weeks ago

Selected Answer: D
<https://docs.databricks.com/aws/en/jobs/monitor>
upvoted 1 times

KadELbied 9 months ago

Selected Answer: C
suretly C
upvoted 1 times

Tedet 11 months, 1 week ago

Selected Answer: C
To export notebook run results for a job with a single task:

On the job detail page, click the View Details link for the run in the Run column of the Completed Runs (past 60 days) table.
Click Export to HTML.

To export notebook run results for a job with multiple tasks:

On the job detail page, click the View Details link for the run in the Run column of the Completed Runs (past 60 days) table.
Click the notebook task to export.

Click Export to HTML.
upvoted 1 times

Tedet 11 months, 1 week ago

Selected Answer: C
Databricks maintains a history of your job runs for up to 60 days. If you need to preserve job runs, Databricks recommends exporting results before they expire.
upvoted 1 times

janeZ 1 year, 1 month ago

Selected Answer: C
<https://learn.microsoft.com/en-us/azure/databricks/jobs/monitor>

upvoted 1 times

🗄️ 👤 **hal2401me** 1 year, 5 months ago

Selected Answer: C

<https://learn.microsoft.com/en-us/azure/databricks/workflows/jobs/monitor-job-runs>

Azure Databricks maintains a history of your job runs for up to 60 days. If you need to preserve job runs, Databricks recommends exporting results before they expire. For more information, see Export job run results.

upvoted 2 times

🗄️ 👤 **ATLTennis** 1 year, 7 months ago

Selected Answer: C

C is the correct answer

upvoted 3 times

🗄️ 👤 **Patito** 1 year, 7 months ago

Selected Answer: C

c is correct

upvoted 2 times

🗄️ 👤 **SwastikaM** 1 year, 7 months ago

Option C is correct

upvoted 2 times

🗄️ 👤 **f728f7f** 1 year, 7 months ago

Selected Answer: D

A secret CAN be printer character-by-character, so it's not really that secure.

upvoted 1 times

🗄️ 👤 **f728f7f** 1 year, 7 months ago

Whoops, answer meant for previous question in the bank. Admin, please delete or move.

upvoted 1 times

🗄️ 👤 **rok21** 1 year, 8 months ago

Selected Answer: C

C is correct

upvoted 3 times

🗄️ 👤 **azurelearn2020** 1 year, 8 months ago

Selected Answer: C

Correct Answer is C

upvoted 1 times

🗄️ 👤 **sturcu** 1 year, 9 months ago

Selected Answer: C

C is correct: retention is 60 days and export to html

upvoted 1 times

🗄️ 👤 **8605246** 2 years ago

this is incorrect databricks maintains a history of job runs for 60 days <https://docs.databricks.com/en/workflows/jobs/monitor-job-runs.html#:~:text=Databricks%20maintains%20a%20history%20of,see%20Export%20job%20run%20results>.

upvoted 3 times

A data engineer, User A, has promoted a new pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens.

Which statement describes the contents of the workspace audit logs concerning these events?

- A. Because the REST API was used for job creation and triggering runs, a Service Principal will be automatically used to identify these events.
- B. Because User B last configured the jobs, their identity will be associated with both the job creation events and the job run events.
- C. Because these events are managed separately, User A will have their identity associated with the job creation events and User B will have their identity associated with the job run events.
- D. Because the REST API was used for job creation and triggering runs, user identity will not be captured in the audit logs.
- E. Because User A created the jobs, their identity will be associated with both the job creation events and the job run events.

Suggested Answer: C

Community vote distribution

C (55%)

E (45%)

 **hal2401me** Highly Voted 1 year, 11 months ago

Selected Answer: E

<https://docs.databricks.com/api/azure/workspace/jobs/create>

API/jobs/create:run_as

object

Write-only setting, available only in Create/Update/Reset and Submit calls. Specifies the user or service principal that the job runs as. If not specified, the job runs as the user who created the job.

In the question, it's not stated that user A creates a service principal. So runas can only be himself.

upvoted 9 times

 **carlosmps** 1 year, 2 months ago

but the documentation says:

The REST API operation path, such as /api/2.0/clusters/get, to get information for the specified cluster.

Remember User B use his token to orchestration.

The answer should be C

upvoted 3 times

 **ecbeb68** Most Recent 2 weeks, 5 days ago

Selected Answer: E

Is there any of these questions are in real exam? please vote if you see some of them.

upvoted 1 times

 **363c4c5** 6 months, 3 weeks ago

Selected Answer: E

both C and E can be right

<https://www.examttopics.com/exams/databricks/certified-data-engineer-professional/view/10/#>

If User A specified the run_as parameter when creating the jobs, the identity specified in the run_as parameter would be associated with the job run events. Therefore, the correct statement would be E. If not - C.

upvoted 1 times

 **KadELbied** 9 months ago

Selected Answer: C

suretly C

upvoted 2 times

 **capt2101akash** 10 months, 2 weeks ago

Selected Answer: C

Both uses their own credential for specific tasks

upvoted 2 times

🗨️ 👤 **Nate_** 11 months, 1 week ago

Selected Answer: C

User A created the jobs via the REST API using their personal access token, so the workspace audit logs will record these job creation events with User A's identity. Conversely, when User B triggers job runs through the REST API (again, using their own personal access token) via an external orchestration tool, those events will be logged with User B's identity.

upvoted 2 times

🗨️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: C

Answer C - the run_as property is not said to be configured, so the job will run with the permissions of the creator - user A. However, still user B will be the one that triggered the run, which is what the question is about.

upvoted 3 times

🗨️ 👤 **LuminaBerry** 1 year, 1 month ago

Selected Answer: C

C should be the correct answer.

Although User A has its user associated to the creation, and by default the run as user if omitted on the creation of the job, the question specifies the Audit Logs (Run Event Logs) associated to the run.

I've tried it this out on a job and for a job which was created and has a run as user different from mine, if I go to the run event logs, there are logs which stated that my user triggered a "started" event type

upvoted 3 times

🗨️ 👤 **janeZ** 1 year, 1 month ago

Selected Answer: C

based on the standard understanding of how personal access tokens typically work, each user's actions should be logged separately with their respective identities. Therefore, "C" would be the standard answer unless there is a specific behavior or configuration in Databricks that causes the job run events to be attributed back to User A.

upvoted 2 times

🗨️ 👤 **AlejandroU** 1 year, 1 month ago

Selected Answer: C

Answer C. The audit logs distinguish between actions like job creation and job execution, so User A and User B will be identified separately for these actions.

upvoted 2 times

🗨️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: E

I tried myself and E seems correct

upvoted 1 times

🗨️ 👤 **JB90** 1 year, 2 months ago

Selected Answer: C

When you use the API to commit the jobs the creation is logged using the PAT info, the same happens when you start a run using a different PAT.

upvoted 2 times

🗨️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: E

Specifies the user, service principal or group that the job/pipeline runs as. If not specified, the job/pipeline runs as the user who created the job/pipeline.

Either user_name or service_principal_name should be specified. If not, an error is thrown.

upvoted 1 times

🗨️ 👤 **rsmf** 1 year, 3 months ago

Selected Answer: C

C is the right answer

upvoted 2 times

🗨️ 👤 **Carkeys** 1 year, 3 months ago

Selected Answer: C

In Databricks, audit logs capture the identity of the user associated with each distinct event, whether it's creating or running a job. Since User A used their personal access token to create the jobs and User B used theirs to trigger job runs, the audit logs will reflect User A's identity for job creation events and User B's identity for job run events.

upvoted 2 times

🗨️ 👤 **quaternion** 1 year, 5 months ago

Selected Answer: E

By default, jobs run as the identity of the job owner. This means that the job assumes the permissions of the job owner. You can change the identity that the job is running as to a service principal. Then, the job assumes the permissions of that service principal instead of the owner.

<https://docs.databricks.com/en/jobs/create-run-jobs.html#run-a-job-as-a-service-principal>

upvoted 2 times

🗨️ 👤 **spudteo** 1 year, 12 months ago

Selected Answer: E

When you create a job your role is IS OWNER and RUN AS. So when you trigger a job, it will run as the RUN AS entity. And it should be user A if someone doesn't have changed it

upvoted 1 times

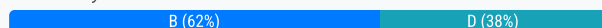
A user new to Databricks is trying to troubleshoot long execution times for some pipeline logic they are working on. Presently, the user is executing code cell-by-cell, using `display()` calls to confirm code is producing the logically correct results as new transformations are added to an operation. To get a measure of average time to execute, the user is running each cell multiple times interactively.

Which of the following adjustments will get a more accurate measure of how code is likely to perform in production?

- A. Scala is the only language that can be accurately tested using interactive notebooks; because the best performance is achieved by using Scala code compiled to JARs, all PySpark and Spark SQL logic should be refactored.
- B. The only way to meaningfully troubleshoot code execution times in development notebooks is to use production-sized data and production-sized clusters with Run All execution.
- C. Production code development should only be done using an IDE; executing code against a local build of open source Spark and Delta Lake will provide the most accurate benchmarks for how code will perform in production.
- D. Calling `display()` forces a job to trigger, while many transformations will only add to the logical query plan; because of caching, repeated execution of the same logic does not provide meaningful results.
- E. The Jobs UI should be leveraged to occasionally run the notebook as a job and track execution time during incremental code development because Photon can only be enabled on clusters launched for scheduled jobs.

Suggested Answer: D

Community vote distribution



guillesd Highly Voted 2 years ago

Selected Answer: B

Both B and D are correct statements. However, D is not an adjustment (see the question), it is just an affirmation which happens to be correct. B, however, is an adjustment, and it will definitely help with profiling.

upvoted 10 times

Emil03 5 months, 4 weeks ago

D tells the user what not to do in order to get actual run time results.

upvoted 1 times

Lukas Most Recent 2 months, 1 week ago

Selected Answer: D

The Main Challenge:

The goal is to recognize how Spark's lazy evaluation model and caching behavior affect performance measurements in development environments.

Correct:

D - The `display()` function is an action that forces Spark to execute the entire transformation chain, the rest of transformations are lazy operations that only add steps to the logical query plan.

Incorrect:

A - PySpark and Spark SQL can be accurately tested in notebooks and don't require conversion to Scala.

B - It's not the "only" way to meaningfully troubleshoot execution times.

C - A local environment provides significantly different performance characteristics than cloud-based Databricks environments.

E - Photon acceleration can be enabled on both all-purpose interactive clusters and job clusters.

upvoted 2 times

vrundapatel 4 months, 2 weeks ago

Selected Answer: D

Calling `display()` forces a job to trigger, while many transformations will only add to the logical query plan; because of caching, repeated execution of the same logic does not provide meaningful results.

upvoted 3 times

Emil03 5 months, 4 weeks ago

Selected Answer: D

because it's the only option that actually tells the user what they need to change to make the test more realistic.

upvoted 2 times

🗨️ 👤 **Billybob0604** 6 months, 1 week ago

Selected Answer: D

Running cells repeatedly might give inconsistent timing because Spark can cache results or optimize execution.

upvoted 2 times

🗨️ 👤 **363c4c5** 6 months, 3 weeks ago

Selected Answer: B

Option D explains why the current method is not providing meaningful results but does not offer a specific adjustment.

upvoted 1 times

🗨️ 👤 **79f0e18** 7 months, 1 week ago

Selected Answer: D

Transformations like `select()`, `filter()`, and `join()` are lazy – they build a logical execution plan but do not trigger actual computation.

Actions like `display()`, `collect()`, `count()`, or `write()` are what trigger execution of the plan.

When a user repeatedly executes the same code, Spark may cache intermediate results or re-optimize the plan, which makes performance inconsistent and not representative of real production runs.

Therefore, measuring execution time by re-running code interactively in a notebook (especially using `display()`) is not a reliable performance benchmark.

upvoted 2 times

🗨️ 👤 **sammy2025** 8 months, 2 weeks ago

Selected Answer: D

Lazy Evaluation in Spark: Many transformations in Spark only modify the logical query plan until an action (like `display()`, `collect()`, or `write()`) is triggered.

upvoted 2 times

🗨️ 👤 **KadELbied** 9 months ago

Selected Answer: B

suretly B

upvoted 1 times

🗨️ 👤 **Tedet** 11 months, 1 week ago

Selected Answer: D

Explanation:

Using `display()` in Databricks forces a job to trigger and display the output, which can lead to an inaccurate measure of performance when benchmarking code. This is because `display()` triggers the job and materializes the result, which does not accurately reflect how the code will perform in production when the job is run without the display output.

Additionally, repeated execution of the same logic (with caching) may not give you meaningful performance results since the results are cached in memory and not representative of fresh computations, as they would occur in a production environment.

To get a more accurate measure of execution time, the user should focus on using appropriate job execution techniques, such as running the notebook with "Run All" and avoiding reliance on `display()` calls, which are not representative of how the pipeline would behave in production.

upvoted 4 times

🗨️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: B

Answer B, see discussion under benni_ale.

upvoted 1 times

🗨️ 👤 **ultimomassimo** 10 months, 2 weeks ago

in any real life commercial project answer B is not feasible, sorry. You always use representative sample, but using same data volumes (especially when they are massive) is impractical and no one would sing off on the cost

upvoted 1 times

🗨️ 👤 **AlejandroU** 1 year, 1 month ago

Selected Answer: D

Answer D. While Option D doesn't directly provide an alternative adjustment, it points out a critical issue in the way interactive notebooks might give misleading results. It would be advisable to avoid using `display()` as a benchmark for performance in production-like environments.

upvoted 4 times

🗳️ 👤 **carlosmps** 1 year, 2 months ago

Selected Answer: B

Without much thought, I would vote for option B, but since it says 'the ONLY,' it makes me hesitate. While option D only points out the issues with the data engineer's executions, it doesn't really provide the adjustments that need to be made. On the other hand, option B at least gives you a way to simulate production behavior. I'll vote for B, but as I said, the word 'only' makes me doubt, because it's not the only way.

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: D

Answer: D.

Explanation:

Lazy Evaluation: Spark employs lazy evaluation, meaning transformations are not executed until an action (e.g., `display()`, `count()`, `collect()`) is called. Using `display()` triggers the execution of the transformations up to that point.

Caching Effects: Repeatedly executing the same cell can lead to caching, where Spark stores intermediate results. This caching can cause subsequent executions to be faster, not reflecting the true performance of the code.

Why not B:

Production-Sized Data and Clusters: While using production-sized data and clusters (as mentioned in option B) can provide insights into performance, it's not the only way to troubleshoot execution times. Proper testing can often be conducted on smaller datasets and clusters, especially during the development phase.

upvoted 2 times

🗳️ 👤 **af4a20a** 1 year, 2 months ago

Yep, what if your production size is 10 TB... But you have a 10GB sample. No idea what's actually right for the test, but D is correct.

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

D is correct. However, it does not show direction on what to do to troubleshoot the problem, which is the first statement in the question.

The only way to troubleshoot performance problems is to start with the data & processing platform of size that is representative of production. That is why I think B is a better choice.

upvoted 1 times

🗳️ 👤 **practitioner** 1 year, 5 months ago

Selected Answer: B

B and D are correct. The question says "which statements" which suggests us that this is a question with multiple choices

upvoted 2 times

🗳️ 👤 **HelixAbdu** 1 year, 6 months ago

Both D and B are correct. But in real life some times clients dose not accept to gave you there production data to test easily. Also it says in B it is "the only way" ans this is not true for me

So i will go with D

upvoted 4 times

🗳️ 👤 **RyanAck24** 1 year, 4 months ago

I would add to this and say that this *could* be a multi-choice question (possibly) as practitioner mentions above. But if it isn't, I would go with D as well.

upvoted 1 times

🗳️ 👤 **ffsdfdsfdfsdfdsf** 1 year, 11 months ago

Selected Answer: B

These people voting D have no reading comprehension.

upvoted 4 times

A production cluster has 3 executor nodes and uses the same virtual machine type for the driver and executor.

When evaluating the Ganglia Metrics for this cluster, which indicator would signal a bottleneck caused by code executing on the driver?

- A. The five Minute Load Average remains consistent/flat
- B. Bytes Received never exceeds 80 million bytes per second
- C. Total Disk Space remains constant
- D. Network I/O never spikes
- E. Overall cluster CPU utilization is around 25%

Suggested Answer: D

Community vote distribution

D (53%)

E (47%)

 **BrianNguyen95** Highly Voted 2 years, 5 months ago

Option E: In a Spark cluster, the driver node is responsible for managing the execution of the Spark application, including scheduling tasks, managing the execution plan, and interacting with the cluster manager. If the overall cluster CPU utilization is low (e.g., around 25%), it may indicate that the driver node is not utilizing the available resources effectively and might be a bottleneck.

upvoted 19 times

 **guillesd** 2 years ago

Overall CPU utilization can be misleading. The 25% utilization could be caused by the workload not requiring more than that rather than everything being executed in the driver node.

upvoted 2 times

 **fe3b2fc** 1 year, 5 months ago

A bottleneck occurs when resources are over utilized not underutilized, so that explanation doesn't make too much sense. CPU utilization would be at 100% and you wouldn't see spike in I/O if the driver was the issue. Conversely if the I/O was spiked and CPU utilization was at 25% , then network could be the issue. D is the only logical answer in this case.

upvoted 3 times

 **benni_ale** 1 year, 3 months ago

i like this more

upvoted 2 times

 **_Lukas_** Most Recent 2 months, 1 week ago

Selected Answer: E

$E - 100\% (\text{Driver}) + 0\% (\text{Exec 1}) + 0\% (\text{Exec 2}) + 0\% (\text{Exec 3}) / 4 \text{ Nodes } (1 \text{ Driver} + 3 \text{ Executors}) = 25\% \text{ overall Utilization}$

Incorrect:

A - A flat load average doesn't inherently indicate a problem. It just means the workload is stable.

B - This indicates a network or throughput limit, but doesn't specifically point to the driver being the bottleneck.

C - This is normal for processing jobs that don't write heavy data to disk (spill) or create new tables. It is not a performance bottleneck indicator.

D - While a driver might result in low network I/O (because data isn't moving to executors), low I/O is a symptom, not a precise indicator.

upvoted 1 times

 **ealpuche** 5 months, 3 weeks ago

Selected Answer: E

With 3 executors + 1 driver (all same VM size), if work is happening on the driver (e.g., collect(), toPandas(), heavy processing after an action), the driver's CPU will be near 100% while executors sit mostly idle. Averaged across 4 identical nodes, Ganglia would show ~25% overall CPU utilization— classic driver bottleneck.

upvoted 1 times

 **sammy2025** 8 months, 2 weeks ago

Selected Answer: E

If the driver is doing too much of the work (rather than distributing tasks to executors), cluster-wide CPU usage will appear low, which is a red flag for driver bottlenecks in a distributed Spark environment.

upvoted 1 times

🗨️ 👤 **KadELbied** 9 months ago

Selected Answer: E

Suretly E

upvoted 1 times

🗨️ 👤 **JoG1221** 9 months, 3 weeks ago

Selected Answer: A

Option E is valid and insightful, Option A is more targeted when you're specifically trying to detect a bottleneck on the driver.

upvoted 1 times

🗨️ 👤 **Tedet** 11 months, 1 week ago

Selected Answer: A

When you see the "Five Minute Load Average" remain consistent or flat, it could indicate that the driver is under heavy load and is struggling to keep up with the workload. In the case of a Spark cluster, if the driver is handling too much work, it can become a bottleneck and prevent the overall job from progressing efficiently.

upvoted 2 times

🗨️ 👤 **srinivasa** 1 year, 1 month ago

Selected Answer: A

Consistent/Flat Five Minute Load Average: If the load average on the driver node remains consistent and does not fluctuate, it suggests that the driver is under constant, significant load. This could be a sign that the driver is performing a lot of work, potentially leading to a bottleneck.

upvoted 3 times

🗨️ 👤 **AlejandroU** 1 year, 1 month ago

Selected Answer: E

Answer E. A low CPU usage could indicate that the driver isn't working as efficiently as expected, which can lead to underutilization of the cluster and slower processing times.

upvoted 2 times

🗨️ 👤 **JB90** 1 year, 2 months ago

Selected Answer: E

Only when the driver does all or most the work will the overall cluster CPU util be this low since the driver cpu is 25% of the overall cluster CPU amount

upvoted 1 times

🗨️ 👤 **nedlo** 1 year, 3 months ago

Selected Answer: E

bottleneck means data skew means one of the nodes is doing majority of work while other is idle, so E is correct

upvoted 2 times

🗨️ 👤 **m79590530** 1 year, 3 months ago

Selected Answer: E

D also means that Driver never send big data chunks to the Worker nodes but as it is not mentioned to be 0 then it has a constant flow of data going in & out between the Driver node and the Worker nodes. Therefore it is not a measure of Driver bottleneck. However Answer E means one of the 4 cluster nodes is always working at 100% which can not be other than the Driver node as it is always working and coordinating work across Executors.

upvoted 1 times

🗨️ 👤 **fe3b2fc** 1 year, 5 months ago

Selected Answer: D

Executors talk between each other and between nodes, if the code/driver is working as intended you would see a spike in I/O while transferring data. If the code/driver was the issue you would see a spike in CPU usage and little network traffic between nodes. The correct answer is D.

upvoted 2 times

🗨️ 👤 **lophonos** 1 year, 8 months ago

Selected Answer: E

E is correct

upvoted 1 times

🗨️ 👤 **guillesd** 2 years ago

Selected Answer: D

If there's no IO between driver and executor nodes then the executor nodes are not working

upvoted 1 times

🗨️ 👤 **Patito** 2 years, 1 month ago

Selected Answer: D

D seems to be right

upvoted 2 times

  **rok21** 2 years, 1 month ago

Selected Answer: E

E is correct

upvoted 1 times

Where in the Spark UI can one diagnose a performance problem induced by not leveraging predicate push-down?

- A. In the Executor's log file, by grepping for "predicate push-down"
- B. In the Stage's Detail screen, in the Completed Stages table, by noting the size of data read from the Input column
- C. In the Storage Detail screen, by noting which RDDs are not stored on disk
- D. In the Delta Lake transaction log, by noting the column statistics
- E. In the Query Detail screen, by interpreting the Physical Plan

Suggested Answer: E

Community vote distribution

E (100%)

 **KadELbied** 9 months ago

Selected Answer: E

suretly E

upvoted 1 times

 **Tedet** 11 months, 1 week ago

Selected Answer: E

Predicate push-down is an optimization where conditions (such as filters) are pushed as close to the data source as possible (often to the database or file system level), reducing the amount of data read and processed. If predicate push-down isn't being leveraged, it can result in reading unnecessary data, leading to performance degradation.

Execute a query --> Click View and go to Spark UI --> Navigate to SQL/DataFrame tab in SparkUI --> Click on any stage --> Navigate to details to find Physical Plan

upvoted 1 times

 **shaswat1404** 12 months ago

Selected Answer: B

when predicated pushdown is working properly, the amount of data read should be much lower because the data source is able to filter out the rows at read time based on the query predicates. if predicate pushdown is not leveraged, stages might read a much larger volume of data than necessary, which can be observed in the input column in the stage detail screen

therefore B is the correct option

not A : executor logs might contain some information, but they are not the most direct way to assess predicate push-down performance

not C : used to check RDD caching and persistence, not predicate push-down

not D : it holds meta data and statistics but is not viewed via the spark UI for diagnosing query performance

not E : while physical plan in the query detail screen might filter push-down, interpreting it requires more expertise, and the metric on the input data size(option B) is more straight forward indicator.

upvoted 2 times

 **benni_ale** 1 year, 3 months ago

Selected Answer: E

E

upvoted 1 times

 **dd1192d** 1 year, 4 months ago

Selected Answer: E

E is correct : <https://docs.datastax.com/en/dse/6.9/spark/predicate-push-down.html>

upvoted 2 times

 **P1314** 2 years ago

Selected Answer: E

Query plan. Correct is E

upvoted 1 times

Review the following error traceback:

```

AnalysisException                                Traceback (most recent call last)
<command-3293767849433948> in <module>
----> 1 display(df.select(3*"heartrate"))

/databricks/spark/python/pyspark/sql/dataframe.py in select(self, *cols)
1690         [Row(name='Alice', age=12), Row(name='Bob', age=15)]
1691         """
-> 1692         jdf = self._jdf.select(self._jcols(*cols))
1693         return DataFrame(jdf, self.sql_ctx)
1694

/databricks/spark/python/lib/py4j-0.10.9-src.zip/py4j/java_gateway.py in __call__(self, *args)
1302
1303         answer = self.gateway_client.send_command(command)
-> 1304         return_value = get_return_value(
1305             answer, self.gateway_client, self.target_id, self.name)
1306

/databricks/spark/python/pyspark/sql/utils.py in deco(*a, **kw)
121         # Hide where the exception came from that shows a non-Pythonic
122         # JVM exception message.
--> 123         raise converted from None
124     else:
125         raise

AnalysisException: cannot resolve ''heartrateheartrateheartrate'' given input columns:
[spark_catalog.database.table.device_id, spark_catalog.database.table.heartrate,
spark_catalog.database.table.mrn, spark_catalog.database.table.time];
'Project ['heartrateheartrateheartrate]
+- SubqueryAlias spark_catalog.database.table
   +- Relation[device_id#75L,heartrate#76,mrn#77L,time#78] parquet

```

Which statement describes the error being raised?

- A. The code executed was PySpark but was executed in a Scala notebook.
- B. There is no column in the table named heartrateheartrateheartrate
- C. There is a type error because a column object cannot be multiplied.
- D. There is a type error because a DataFrame object cannot be multiplied.
- E. There is a syntax error because the heartrate column is not correctly identified as a column.

Suggested Answer: E

Community vote distribution

B (74%)

E (26%)

 **CertPeople** Highly Voted 1 year, 11 months ago

Selected Answer: B

It's B, there is no column with that name
upvoted 8 times

 **rok21** Highly Voted 1 year, 8 months ago

Selected Answer: E

E is correct
upvoted 5 times

 **KadELbied** Most Recent 9 months ago

Selected Answer: B

Surely B
upvoted 1 times

 **Stalker200** 9 months, 3 weeks ago

Selected Answer: B

B is correct.

I created a simple df and ran this code : `display(df.select(3*"heartrate"))` and I got this error:

AnalysisException: [UNRESOLVED_COLUMN.WITH_SUGGESTION] A column or function parameter with name `heartrateheartrateheartrate` cannot be resolved. Did you mean one of the following? [`heartrate`, `device\_id`, `time`, `mrn`].;

'Project [heartrateheartrateheartrate]

+ LogicalRDD [device_id#2L, heartrate#3L, mrn#4L, time#5], false

upvoted 1 times

  **examtopicsms99** 10 months ago

Selected Answer: C

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql.functions import col
```

```
# Initialize Spark session
```

```
spark = SparkSession.builder.appName("ErrorReproduction").getOrCreate()
```

```
# Create a sample DataFrame with similar structure
```

```
data = [
```

```
(1, 72, 12345, "2023-01-01 10:00:00"),
```

```
(2, 68, 67890, "2023-01-01 10:01:00"),
```

```
(3, 75, 54321, "2023-01-01 10:02:00")
```

```
]
```

```
columns = ["device_id", "heartrate", "mrn", "time"]
```

```
df = spark.createDataFrame(data, columns)
```

```
# This will produce the AnalysisException
```

```
# The error occurs because you're trying to multiply a string "heartrate" by 3 literally
```

```
# instead of referencing the column and multiplying its values
```

```
display(df.select(3*"heartrate"))
```

```
# display(df.select(3*col("heartrate")))
```

upvoted 1 times

  **guillesd** 1 year, 6 months ago

Selected Answer: B

It's B. Regarding E, a syntax error would mean that the query is not valid due to a wrongfully written SQL statement. However, this is not the case. The column just does not exist.

upvoted 2 times

  **Jay_98_11** 1 year, 6 months ago

Selected Answer: B

<https://sparkbyexamples.com/spark/spark-cannot-resolve-given-input-columns/>

upvoted 1 times

  **Gulenur_GS** 1 year, 8 months ago

the answer is E, because

```
df.select(3*df['heartrate']).show()
```

 perfectly returns

upvoted 2 times

  **chokthewa** 1 year, 6 months ago

`3*"heartrate"` is triple of string "heartrate" ,isn't value of heartrate multiplied by 3.

upvoted 1 times

  **Gulenur** 1 year, 8 months ago

Answer is E

```
df.select(3*df['heartrate'])
```

 returns perfect result without error

upvoted 2 times

  **npc0001** 1 year, 9 months ago

Selected Answer: B

Answer B

upvoted 2 times

  **Dileepvikram** 1 year, 9 months ago

Answer is B

upvoted 2 times

  **sturcu** 1 year, 9 months ago

Selected Answer: B

No such column found

upvoted 2 times

Which distribution does Databricks support for installing custom Python code packages?

- A. sbt
- B. CRAN. npm
- D. Wheels
- E. jars

Suggested Answer: D

Community vote distribution

D (100%)

  **_Lukas_** 2 months, 2 weeks ago

Selected Answer: D

Correct:

D - Wheels are the standard build distribution format for Python packages.

Incorrect:

A - (sbt - Scala Build Tool) This is a build tool used primarily for Scala and Java projects, not for distributing Python packages.

B - (CRAN) This is the repository and distribution network for R programming language packages.

C - (npm) This is the package manager for JavaScript and Node.js, not Python.

E - (jar - Java Archive) JAR files are used to distribute Java and Scala code. While you can use JARs in Databricks, they do not contain Python code.
upvoted 1 times

  **KadELbied** 9 months ago

Selected Answer: D

suretly D

upvoted 1 times

  **benni_ale** 1 year, 3 months ago

Selected Answer: D

I think D is correct

upvoted 1 times

  **hal2401me** 1 year, 11 months ago

Selected Answer: D

<https://learn.microsoft.com/en-us/azure/databricks/workflows/jobs/how-to/use-python-wheels-in-workflows>

upvoted 4 times

  **sodere** 2 years, 1 month ago

Selected Answer: D

<https://learn.microsoft.com/en-us/azure/databricks/workflows/jobs/how-to/use-python-wheels-in-workflows>

upvoted 1 times

  **alexvno** 2 years, 1 month ago

Selected Answer: D

Wheels should be ok

upvoted 2 times

Which Python variable contains a list of directories to be searched when trying to locate required modules?

- A. `importlib.resource_path`
- B. `sys.path`
- C. `os.path`
- D. `pypi.path`
- E. `pylib.source`

Suggested Answer: B

Community vote distribution

B (100%)

  **alexvno** Highly Voted 2 years, 1 month ago

Selected Answer: B

`sys.path` is a built-in variable within the `sys` module. It contains a list of directories that the interpreter will search in for the required module
upvoted 7 times

  **KadELbied** Most Recent 9 months ago

Selected Answer: B

suretly B
upvoted 1 times

  **Sriramiyer92** 1 year, 1 month ago

Selected Answer: B

`sys.path` is a list in Python that contains the directories the interpreter searches for modules when importing them. It is initialized with the default paths when Python starts and can be modified during runtime if needed.
upvoted 1 times

  **benni_ale** 1 year, 3 months ago

Selected Answer: B

`sys.path` is a built-in variable within the `sys` module. It contains a list of directories that the interpreter will search in for the required module.
upvoted 1 times

Incorporating unit tests into a PySpark application requires upfront attention to the design of your jobs, or a potentially significant refactoring of existing code.

Which statement describes a main benefit that offset this additional effort?

- A. Improves the quality of your data
- B. Validates a complete use case of your application
- C. Troubleshooting is easier since all steps are isolated and tested individually
- D. Yields faster deployment and execution times
- E. Ensures that all steps interact correctly to achieve the desired end result

Suggested Answer: C

Community vote distribution

C (100%)

  **alexvno** Highly Voted 2 years, 1 month ago

Selected Answer: C

Unit tests are small, isolated tests that are used to check specific parts of the code, such as functions or classes
upvoted 5 times

  **_Lukas_** Most Recent 2 months, 2 weeks ago

Selected Answer: C

Correct:

C - The main benefit of unit testing is focused on isolating and validating individual components. This directly simplifies the process of finding and fixing bugs (troubleshooting).

Incorrect:

A - Unit tests validate the code logic that processes the data, not the real quality of the source data itself.

B - These benefits are achieved primarily through integration tests or end-to-end tests, not through unit tests alone.

D - Unit tests do not directly influence the speed of the production deployment or the execution time of the Spark job.

upvoted 1 times

  **KadELbied** 9 months ago

Selected Answer: C

Surety C

upvoted 1 times

  **nedlo** 1 year, 3 months ago

Selected Answer: C

D is integration tests (how they relate to each other how connect), E is E2E test, C is "testing individually" which is only one fitting definition of unittest

upvoted 2 times

  **nedlo** 1 year, 3 months ago

i mean E is integration test B is E2E test

upvoted 1 times

  **jmjm21** 1 year, 7 months ago

Selected Answer: C

Answer is C.

upvoted 1 times

Which statement describes integration testing?

- A. Validates interactions between subsystems of your application
- B. Requires an automated testing framework
- C. Requires manual intervention
- D. Validates an application use case
- E. Validates behavior of individual elements of your application

Suggested Answer: A

Community vote distribution

A (100%)

🗳️ 👤 **_Lukas_** 2 months, 1 week ago

Selected Answer: A

Correct:

A - Integration testing is a phase of software testing in which individual software modules (or units) are combined and tested as a group. The primary purpose is to expose faults in the interaction between integrated units.

Incorrect:

B - Integration testing can technically be performed manually, so this is not its defining characteristic.

C - Many integration tests are fully automated and run as part of a CI/CD pipeline, minimizing or eliminating the need for manual intervention.

D - This is a characteristic of system testing or end-to-end (E2E) testing.

E - This is the definition of unit testing.

upvoted 1 times

🗳️ 👤 **KadELbied** 8 months, 4 weeks ago

Selected Answer: A

Surely A

upvoted 1 times

🗳️ 👤 **robodog** 1 year, 5 months ago

Selected Answer: A

Answer is A

upvoted 2 times

🗳️ 👤 **alexvno** 2 years, 1 month ago

Selected Answer: A

Integration testing is a type of software testing where components of the software are gradually integrated and then tested as a unified group

upvoted 4 times

Which REST API call can be used to review the notebooks configured to run as tasks in a multi-task job?

- A. /jobs/runs/list
- B. /jobs/runs/get-output
- C. /jobs/runs/get
- D. /jobs/get
- E. /jobs/list

Suggested Answer: D

Community vote distribution

D (83%)

B (17%)

  **_Lukas_** 2 months, 2 weeks ago

Selected Answer: D

Correct:

D - This endpoint retrieves the details of a single job, including its settings and configuration. For a multi-task job, the response from this endpoint includes the 'setting' object, which contains a 'tasks' array listing all the tasks configured for that job.

Incorrect:

A - This endpoint lists the history of executions (runs) for jobs, not the static configuration of the job itself.

B - This retrieves the output (e.g. logs, notebook results) of a specific completed task run, not the configuration of the job.

C - This retrieves the metadata for a single specific run (execution) of a job, not the definition of the job itself.

E - This endpoint lists all jobs in the workspace (providing IDs and basic info), but it does not provide the detailed task configuration like which notebooks are attached) for each job in the response.

upvoted 1 times

  **KadELbied** 9 months ago

Selected Answer: D

suretly D

upvoted 2 times

  **JoG1221** 9 months, 3 weeks ago

Selected Answer: E

Provides a list of all jobs in the workspace along with their configurations

upvoted 1 times

  **AlejandroU** 1 year, 1 month ago

Selected Answer: E

The correct answer is E. /jobs/list, not C. /jobs/runs/get. Here's why:

/jobs/list: Provides a list of all jobs in the workspace along with their configurations, including task details like the notebooks assigned to each task. This makes it the best choice for reviewing notebooks configured as tasks in a multi-task job.

/jobs/get: Can also be used if the goal is to review the tasks (and notebooks) of a specific job. However, the question does not limit the scope to a single job.

upvoted 1 times

  **imatheushenrique** 1 year, 7 months ago

multi-task: /jobs/get

single-task: /jobs/runs/get

upvoted 2 times

  **hal2401me** 1 year, 11 months ago

Selected Answer: D

<https://docs.databricks.com/api/workspace/jobs/get>

responses/settings/tasks/notebook_task/notebook_path

upvoted 1 times

🗨️ 👤 **divingbell17** 2 years, 1 month ago

Selected Answer: D

The question asks for notebooks configured for a job, not a instance of a job run. D is correct.

upvoted 4 times

🗨️ 👤 **alexvno** 2 years, 1 month ago

Selected Answer: D

Get

Multi-task format jobs return an array of task data structures containing task settings.

upvoted 1 times

🗨️ 👤 **hamzaKhribi** 2 years, 2 months ago

Selected Answer: D

/jobs/get response under task array shows all the desired notebooks

upvoted 1 times

🗨️ 👤 **arye777** 2 years, 2 months ago

Selected Answer: B

should be B

upvoted 1 times

A Databricks job has been configured with 3 tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on task A.

If tasks A and B complete successfully but task C fails during a scheduled run, which statement describes the resulting state?

- A. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.
- B. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; any changes made in task C will be rolled back due to task failure.
- C. All logic expressed in the notebook associated with task A will have been successfully completed; tasks B and C will not commit any changes because of stage failure.
- D. Because all tasks are managed as a dependency graph, no changes will be committed to the Lakehouse until all tasks have successfully been completed.
- E. Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task C failed, all commits will be rolled back automatically.

Suggested Answer: A

Community vote distribution

A (100%)

 **IT3008** Highly Voted 2 years, 4 months ago

Should be 'A' only, as ACID compliance is applicable at operation level. For example if task C is having 3 target delta table writes (in independent Notebook cells) then it could have after 1 write the task fails during 2nd write. In that case 1st write will still be persisted. The ACID compliance will be applicable for only the 2nd write.

upvoted 9 times

 **alexvno** Highly Voted 2 years, 1 month ago

Selected Answer: A

A - for sure this is NOT ACID operations

upvoted 5 times

 **KadELbied** Most Recent 9 months ago

Selected Answer: A

Surely A

upvoted 1 times

 **dd1192d** 1 year, 4 months ago

Selected Answer: A

<https://community.databricks.com/t5/data-engineering/does-cancelling-a-job-run-rollback-any-actions-performed-by/td-p/8135>

upvoted 2 times

 **tkg13** 2 years, 5 months ago

Correct answer should be B as Databricks is ACID compliant

upvoted 2 times

 **eli91** 2 years, 5 months ago

What if an operation of C is to delete a file, will the file be created after a roll back?

upvoted 2 times

 **arekm** 1 year, 1 month ago

A single SQL command is ACID compliant, not a whole notebook.

upvoted 1 times

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_stor
USING DELTA
LOCATION "/mnt/prod/sales_by_store"
```

Realizing that the original query had a typographical error, the below code was executed:

```
ALTER TABLE prod.sales_by_stor RENAME TO prod.sales_by_store
```

Which result will occur after running the second command?

- A. The table reference in the metastore is updated and no data is changed.
- B. The table name change is recorded in the Delta transaction log.
- C. All related files and metadata are dropped and recreated in a single ACID transaction.
- D. The table reference in the metastore is updated and all data files are moved.
- E. A new Delta transaction log is created for the renamed table.

Suggested Answer: A

Community vote distribution

A (100%)

 **hal2401me** Highly Voted 1 year, 5 months ago

Selected Answer: A

did a test. No data is changed. dir & filename not changed. the rename is not recorded in transition log neither.

upvoted 9 times

 **KadELbied** Most Recent 9 months ago

Selected Answer: A

suretly A

upvoted 1 times

 **Tamele001** 1 year, 5 months ago

B is the correct answer. When you alter a table name in Delta Lake, the change is logged in the transaction log that Delta Lake uses to maintain a versioned history of all changes to the table. This is how Delta Lake maintains ACID properties and ensures a consistent view of the data. The transaction log is key to supporting features like time travel, auditing, and rollbacks in Delta Lake. The metadata and the actual data remain intact, and the reference to the table in the metastore is updated to reflect the new name.

upvoted 2 times

 **arekm** 1 year, 1 month ago

Transaction log captures operations on the data (including adding columns, renaming them). This operation is a change of the name of the external table - just a change in the metastore.

upvoted 2 times

 **adenis** 1 year, 6 months ago

Selected Answer: A

A is Correct

upvoted 4 times

The data engineering team maintains a table of aggregate statistics through batch nightly updates. This includes total sales for the previous day alongside totals and averages for a variety of time periods including the 7 previous days, year-to-date, and quarter-to-date. This table is named `store_sales_summary` and the schema is as follows:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT, total_sales_ytd FLOAT,
avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT, avg_daily_sales_7d
FLOAT, updated TIMESTAMP
```

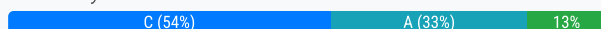
The table `daily_store_sales` contains all the information needed to update `store_sales_summary`. The schema for this table is: `store_id INT, sales_date DATE, total_sales FLOAT`

If `daily_store_sales` is implemented as a Type 1 table and the `total_sales` column might be adjusted after manual data auditing, which approach is the safest to generate accurate reports in the `store_sales_summary` table?

- A. Implement the appropriate aggregate logic as a batch read against the `daily_store_sales` table and overwrite the `store_sales_summary` table with each Update.
- B. Implement the appropriate aggregate logic as a batch read against the `daily_store_sales` table and append new rows nightly to the `store_sales_summary` table.
- C. Implement the appropriate aggregate logic as a batch read against the `daily_store_sales` table and use upsert logic to update results in the `store_sales_summary` table.
- D. Implement the appropriate aggregate logic as a Structured Streaming read against the `daily_store_sales` table and use upsert logic to update results in the `store_sales_summary` table.
- E. Use Structured Streaming to subscribe to the change data feed for `daily_store_sales` and apply changes to the aggregates in the `store_sales_summary` table with each update.

Suggested Answer: C -

Community vote distribution



hammer_1234_h Highly Voted 2 years, 4 months ago

The answer should be A. it is the safest to generate accurate report
upvoted 12 times

djohn_prasad 11 months, 2 weeks ago

But what if each batch did not have historical data but only updates? Then doing an overwrite will risk removing historical data in the target table so Option C is much preferable
upvoted 3 times

alexnvo 2 years, 1 month ago

Incorrect BATCH processing and OVERWRITE will give partial results
upvoted 2 times

Def21 2 years ago

This is confusing: "overwrite the `store_sales_summary` table with each Update." sounds like it is only doing updates, not inserting new possible stories.
upvoted 5 times

e671267 Most Recent 4 months, 3 weeks ago

Selected Answer: A
the question itself mention the answer, a type 1 overwrites. So the Answer is A
upvoted 1 times

kramit21 6 months ago

Selected Answer: A
This approach guarantees accuracy. By performing a full batch read of the entire `daily_store_sales` table and then overwriting `store_sales_summary`, any corrections made to historical `total_sales` values are automatically incorporated into the new aggregates. This simple, full-recalculation method eliminates the risk of missing a historical update, making it the safest option for generating accurate reports. Why I feel C (Upsert) is not correct - Using upsert logic (e.g., MERGE) would only work well if you were updating a single-day aggregate. However, the `store_sales_summary` table contains

rolling aggregates (7-day, QTD, YTD) that depend on a collection of past records. If a historical daily_store_sales record is corrected, a simple upsert would not automatically trigger the re-calculation of all the affected rolling aggregates. This makes the logic complex, fragile, and prone to errors.

upvoted 3 times

🗳️ 👤 **dalupus** 6 months, 3 weeks ago

Selected Answer: A

one of the columns is previous 7 days of sales. which means every single row in the table will change every day regardless of upstream changes. if you use a cdc type approach it will not update the records for stores which have not had additional sales.

upvoted 1 times

🗳️ 👤 **HarshaArora** 7 months ago

Selected Answer: A

A is the answer

upvoted 1 times

🗳️ 👤 **79f0e18** 7 months, 1 week ago

Selected Answer: A

The safest, simplest, and most reliable approach is ("A") Recompute the aggregates fully by batch reading the entire source and overwrite the summary table nightly. This guarantees the summary reflects all changes, including corrections from manual audits.

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: C

suretly C

upvoted 1 times

🗳️ 👤 **ultimomassimo** 10 months, 2 weeks ago

people that claim type 1 scd is not maintained by upsert need to make some more reading before posting anything here...

upvoted 1 times

🗳️ 👤 **Jamuro** 11 months, 1 week ago

Selected Answer: C

It makes no sense to recalculate all aggregations for all historical data and then overwrite. It would lead to a non scalable solution because all data should be recalculated to not "delete" rows in target table. Option C lets us filter by date ranges or even partitions to merge only target periods and get the same results in a much more performant, safer and cheaper way.

upvoted 1 times

🗳️ 👤 **AlejandroU** 1 year, 1 month ago

Selected Answer: C

The answer is C. Option A is correct in ensuring accuracy, as it recalculates the entire store_sales_summary table based on the full historical data in daily_store_sales. However, it is computationally expensive and may not scale well.

Option C (upsert logic) could be a better choice in most real-world scenarios, as it focuses only on the records that have changed, reducing computational costs and minimizing disruption for downstream systems.

upvoted 2 times

🗳️ 👤 **arekm** 1 year, 1 month ago

How do you know which records have changed? I think A is the safest answer.

upvoted 1 times

🗳️ 👤 **djohn_prasad** 11 months, 2 weeks ago

Option C tells us based on upsert logic where matching records (matching key assumed as storeid and sales date) are checked for differences in sales values and the value from batch is taken over that of the target table. C is computational much easier then

upvoted 1 times

🗳️ 👤 **Sriramiyer92** 1 year, 1 month ago

Selected Answer: A

A it is

SCD Type 1, so clearly Append and Upsert logic should not be used.

upvoted 3 times

🗳️ 👤 **ultimomassimo** 10 months, 2 weeks ago

you dont know what is scd type 1 , do you? you dont understand the difference between overwriting a column value and overwriting the whole table, do you?

upvoted 2 times

🗨️ **benni_ale** 1 year, 2 months ago

Selected Answer: A

A as the table does not require history
upvoted 1 times

🗨️ **shaojunni** 1 year, 3 months ago

Selected Answer: A

daily_store_sales is type1 table, no history is maintained. You have to treat every record as new record and do aggregation for every store. Overwrite is much efficient than upsert.
upvoted 1 times

🗨️ **Ati1362** 1 year, 7 months ago

Selected Answer: C

I will go with c. upsert
upvoted 3 times

🗨️ **MDWPartners** 1 year, 8 months ago

Selected Answer: C

A is not correct because the table is daily. If you overwrite you delete all history. You need to insert/update to keep history.
upvoted 4 times

🗨️ **fe3b2fc** 1 year, 5 months ago

Incorrect. The daily store sales table contains all of the history needed to update the table. The summary table holds no historical records. Seeing as this is a nightly job, any manual changes made to daily store sales will be captured. A is the correct answer.
upvoted 2 times

🗨️ **ThoBustos** 1 year, 9 months ago

Selected Answer: A

Not sure if that's right but I would go for A. What do you think?

Type1: Data is overwritten

Type 2: History is maintained, new data is inserted as new rows

Type 3: Stores two versions per record: a previous and a current value

A. batch + overwrite -> Match Type 1 requirements. YES

B: batch + append new rows -> Would be for type 2. NO

C. Batch + Upsert -> Data is not being overwritten (which is required for Type 1). NO

D. ReadStream + Upsert -> Data is not being overwritten (which is required for Type 1). NO

E. Change Data Feed to update -> Problem is manual edits + not overwriting (required for type 1). No

I have doubts around "which approach is the safest". Maybe because due to some manual changes it is hard to track changes or do upsert, so to make sure that the stats are right

overwriting is safer.

upvoted 4 times

🗨️ **vikram12apr** 1 year, 11 months ago

Selected Answer: C

Not A because overwriting will only provide a daily based data not the history of it.

Not B because it will not fix the issue of incorrect sales amount

As these data are fit for batch processing so neither D or E.

C will only upsert the changes while making sure we are updating the records based on sales_date & store_id

upvoted 2 times

A member of the data engineering team has submitted a short notebook that they wish to schedule as part of a larger data pipeline. Assume that the commands provided below produce the logically correct results when run as presented.

Cmd 1

```
rawDF = spark.table("raw_data")
```

Cmd 2

```
rawDF.printSchema()
```

Cmd 3

```
flattenedDF = rawDF.select("...", "values.*")
```

Cmd 4

```
finalDF = flattenedDF.drop("values")
```

Cmd 5

```
finalDF.explain()
```

Cmd 6

```
display(finalDF)
```

Cmd 7

```
finalDF.write.mode("append").saveAsTable("flat_data")
```

Which command should be removed from the notebook before scheduling it as a job?

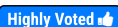
- A. Cmd 2
- B. Cmd 3
- C. Cmd 4
- D. Cmd 5
- E. Cmd 6

Suggested Answer: E

Community vote distribution

E (93%)

7%

 **petrv**  2 years, 2 months ago

Selected Answer: E

When scheduling a Databricks notebook as a job, it's generally recommended to remove or modify commands that involve displaying output, such as using the `display()` function. Displaying data using `display()` is an interactive feature designed for exploration and visualization within the notebook interface and may not work well in a production job context.

The `finalDF.explain()` command, which provides the execution plan of the DataFrame transformations and actions, is often useful for debugging and optimizing queries. While it doesn't display interactive visualizations like `display()`, it can still be informative for understanding how Spark is executing the operations on your DataFrame.

upvoted 10 times

 **KadELbied**  9 months ago

Selected Answer: E

Surely E

upvoted 1 times

🗨️ 👤 **Carkeys** 1 year, 3 months ago

Selected Answer: D

Cmd 5 (`finalDF.explain()`) is used for debugging and understanding the logical and physical plans of a DataFrame. It provides insights into how Spark plans to execute the query but does not produce output that is necessary for the scheduled job. Including this command in a scheduled job is unnecessary and could clutter the job logs without adding value to the final output.

upvoted 1 times

🗨️ 👤 **arekm** 1 year, 1 month ago

`display()` is more costly operation than `finalDF.explain()`. The DataFrame might contain millions of rows that you would be trying to print out each time the notebook is run.

upvoted 1 times

🗨️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: E

if i was multiple solutions than i would have gone for `.explain` method and print schema as well as they do not contribute in any sort of ETL operation but as a rule of thumb `display` should always be omitted first so -> E

upvoted 1 times

🗨️ 👤 **71dfab9** 1 year, 5 months ago

Selected Answer: E

I agree with petrV and KhoaLe, but I will add that not displaying the `finalDF` would be wise as it could display and log PII data and that to me is why I choose E. Like hal2401 said, commands 2, 5 & 6 can be removed as they don't manipulate the data.

upvoted 1 times

🗨️ 👤 **KhoaLe** 2 years ago

Selected Answer: E

Looking through at all steps, Cmd 2,5,6 can be eliminated without impacting to the whole process.

However, in terms of duration cost, Cmd 2 and 5 does not impact much as they only show the current results of logical query plan. In contrast, `display()` in Cmd6 is actually a transformation, which will take much time to run.

upvoted 2 times

🗨️ 👤 **alexvno** 2 years, 1 month ago

Selected Answer: E

No `display()`

upvoted 3 times

🗨️ 👤 **60ties** 2 years, 2 months ago

Selected Answer: D

No actions on production scripts. D is best

upvoted 1 times

🗨️ 👤 **ofed** 2 years, 2 months ago

in order to display a dataframe you also need to calculate it. So `display` also acts as an action.

upvoted 1 times

🗨️ 👤 **Karen1232123** 2 years, 3 months ago

Why not D?

upvoted 2 times

The business reporting team requires that data for their dashboards be updated every hour. The total processing time for the pipeline that extracts transforms, and loads the data for their pipeline runs in 10 minutes.

Assuming normal operating conditions, which configuration will meet their service-level agreement requirements with the lowest cost?

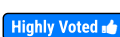
- A. Manually trigger a job anytime the business reporting team refreshes their dashboards
- B. Schedule a job to execute the pipeline once an hour on a new job cluster
- C. Schedule a Structured Streaming job with a trigger interval of 60 minutes
- D. Schedule a job to execute the pipeline once an hour on a dedicated interactive cluster
- E. Configure a job that executes every time new data lands in a given directory

Suggested Answer: C

Community vote distribution

B (83%)

C (17%)

  **divingbell17**  2 years, 1 month ago

Selected Answer: B

B is correct I think.

With option C, the cluster remains on 24/7 with trigger = 60 mins which is more costly

If there is an option with structure streaming with trigger = availablenow, and job scheduled per hour, that would be even more efficient.

<https://www.databricks.com/blog/2017/05/22/running-streaming-jobs-day-10x-cost-savings.html>

upvoted 10 times

  **arekm** 1 year, 1 month ago

Always a job cluster.

upvoted 1 times

  **KadELbied**  9 months ago

Selected Answer: B

Surely B

upvoted 1 times

  **robodog** 1 year, 5 months ago

Selected Answer: C

C. The lowest cost is obtained by using job cluster

upvoted 1 times

  **robodog** 1 year, 5 months ago

B answer i mean

upvoted 2 times

  **Curious76** 1 year, 11 months ago

Selected Answer: C

Databricks recommends using Structured Streaming with trigger AvailableNow for incremental workloads that do not have low latency requirements.

upvoted 2 times

  **spaceexplorer** 2 years ago

Selected Answer: B

B is correct

upvoted 4 times

  **alexvno** 2 years, 1 month ago

Selected Answer: B

B : Job cluster is cheap , hourly = 60 minutes

upvoted 4 times

🗨️ 👤 **aragorn_brego** 2 years, 2 months ago

Selected Answer: B

Scheduling a job to execute the pipeline on an hourly basis aligns with the requirement for data to be updated every hour. Using a job cluster (which is brought up for the job and torn down upon completion) rather than a dedicated interactive cluster will usually be more cost-effective. This is because you are only paying for the compute resources when the job is running, which is 10 minutes out of every hour, rather than paying for an interactive cluster that would be up and running (and incurring costs) continuously.

upvoted 2 times

🗨️ 👤 **ofed** 2 years, 2 months ago

It's either B or D. I think B, because we want the lowest cost.

upvoted 1 times

A Databricks SQL dashboard has been configured to monitor the total number of records present in a collection of Delta Lake tables using the following query pattern:

```
SELECT COUNT (*) FROM table -
```

Which of the following describes how results are generated each time the dashboard is updated?

- A. The total count of rows is calculated by scanning all data files
- B. The total count of rows will be returned from cached results unless REFRESH is run
- C. The total count of records is calculated from the Delta transaction logs
- D. The total count of records is calculated from the parquet file metadata
- E. The total count of records is calculated from the Hive metastore

Suggested Answer: A

Community vote distribution

C (100%)

 **aragorn_brego** Highly Voted 2 years, 2 months ago

Selected Answer: C

Delta Lake maintains a transaction log that records details about every change made to a table. When you execute a count operation on a Delta table, Delta Lake can use the information in the transaction log to calculate the total number of records without having to scan all the data files. This is because the transaction log includes information about the number of records in each file, allowing for an efficient aggregation of these counts to get the total number of records in the table.

upvoted 7 times

 **Syd** Highly Voted 2 years, 3 months ago

Answer C

<https://delta.io/blog/2023-04-19-faster-aggregations-metadata/#:~:text=You%20can%20get%20the%20number,a%20given%20Delta%20table%20version.>

upvoted 7 times

 **1deab72** 3 weeks, 4 days ago

Brilliant!

upvoted 1 times

 **Emil03** Most Recent 5 months, 4 weeks ago

Selected Answer: A

Yes, delta logs do have numRecords field. However, databricks does not just sum the numRecords from the transaction log, count(*) is implemented as a physical plan to count the rows from the files even if stats exists.

upvoted 3 times

 **c315d10** 8 months, 2 weeks ago

Selected Answer: A

Metadata could be outdated

upvoted 2 times

 **KadELbied** 9 months ago

Selected Answer: C

Surely C

upvoted 1 times

 **AlejandroU** 1 year, 1 month ago

Selected Answer: D

Answer D. Parquet Metadata Usage: Delta Lake does utilize Parquet file metadata for COUNT(*) operations. Parquet files store metadata, including row counts. Delta efficiently reads this metadata to get the total count without scanning the actual data within the files. This is a key optimization for

performance.

Why not always scan: Scanning all data files for every COUNT(*) would be extremely inefficient, especially for large tables. This defeats the purpose of using a columnar storage format like Parquet and the optimizations built into Delta Lake and Spark.

The transaction log tracks changes to the table (adds, deletes, updates) but doesn't store pre-computed row counts. It's used for time travel, ACID properties, and other Delta features.

upvoted 2 times

🗲️ 👤 **arekm** 1 year, 1 month ago

Definitely C - see link posted by Syd

upvoted 1 times

🗲️ 👤 **Sriramiyer92** 1 year, 1 month ago

Selected Answer: C

"stats": {"numRecords": 3, "minValues": {"x": 1}, "maxValues": {"x": 3}, "nullCount": {"x": 0}},

numRecords - In Delta tx logs will give you the value

upvoted 1 times

🗲️ 👤 **Ati1362** 1 year, 7 months ago

Selected Answer: C

Delta transaction log

upvoted 2 times

🗲️ 👤 **sodere** 2 years, 1 month ago

Selected Answer: C

Transaction log provides statistics about the delta table.

upvoted 4 times

🗲️ 👤 **alexvno** 2 years, 1 month ago

Selected Answer: C

C - transaction logs contains info about files rows count

upvoted 3 times

🗲️ 👤 **Dileepvikram** 2 years, 3 months ago

The answer is C

upvoted 2 times

🗲️ 👤 **PearApple** 2 years, 3 months ago

Selected Answer: C

The answer should be C

upvoted 2 times

🗲️ 👤 **sturcu** 2 years, 3 months ago

Selected Answer: C

total rows will be calculated from delta logs

upvoted 3 times

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_store
AS (
  SELECT *
  FROM prod.sales a
  INNER JOIN prod.store b
  ON a.store_id = b.store_id
)
```

Consider the following query:

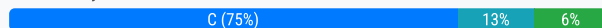
DROP TABLE prod.sales_by_store -

If this statement is executed by a workspace admin, which result will occur?

- A. Nothing will occur until a COMMIT command is executed.
- B. The table will be removed from the catalog but the data will remain in storage.
- C. The table will be removed from the catalog and the data will be deleted.
- D. An error will occur because Delta Lake prevents the deletion of production data.
- E. Data will be marked as deleted but still recoverable with Time Travel.

Suggested Answer: D

Community vote distribution



🗳️ 👤 **dalupus** 6 months, 3 weeks ago

Selected Answer: B

given how this was created this will be a managed table. Which mean on deletion the data is not actually deleted by default for 7 days. Given how this is referring to Delta Lake I can only assume it is referring to the current functionality
upvoted 2 times

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: C

suretly C
upvoted 1 times

🗳️ 👤 **kishanu** 10 months ago

Selected Answer: C

UNDROP can be used, within a 7 day retention period .
upvoted 1 times

🗳️ 👤 **Er5** 1 year, 10 months ago

C. is only correct statement. Though the table can be UNDROP in 7 days
<https://learn.microsoft.com/en-us/azure/databricks/sql/language-manual/sql-ref-syntax-ddl-undrop-table>
E. Time Travel can retrieve versioned records but not tables.
<https://www.databricks.com/blog/2019/02/04/introducing-delta-time-travel-for-large-scale-data-lakes.html>
upvoted 4 times

🗳️ 👤 **Curious76** 1 year, 11 months ago

Selected Answer: E

I think E is better answer
upvoted 2 times

🗨️ 👤 **Sriramiyer92** 1 year, 1 month ago

Feel this is incorrect.

Why?

Undrop Syntax: `UNDROP TABLE { table_name | WITH ID table_id }`

So no question if Time travel

upvoted 1 times

🗨️ 👤 **Luv4data** 2 years, 1 month ago

E. Since the table is still recoverable from transaction logs.

upvoted 1 times

🗨️ 👤 **alexvno** 2 years, 1 month ago

Selected Answer: C

C : AS SELECT - Managed table

Will remove table and data

upvoted 2 times

🗨️ 👤 **aragorn_brego** 2 years, 2 months ago

Selected Answer: C

In Delta Lake, when a DROP TABLE command is executed, it removes both the metadata entry for the table from the catalog and the data in storage associated with that table. Workspace administrators typically have the necessary permissions to drop tables, and unless there are additional protections or retention policies in place, the data is not recoverable through normal operations after the table is dropped.

upvoted 3 times

🗨️ 👤 **60ties** 2 years, 2 months ago

I meant C is correct, not D

upvoted 3 times

🗨️ 👤 **60ties** 2 years, 2 months ago

Selected Answer: D

D is most correct

upvoted 1 times

🗨️ 👤 **Dileepvikram** 2 years, 3 months ago

Answer is C as it is a managed table

upvoted 1 times

🗨️ 👤 **lokvamsi** 2 years, 3 months ago

Selected Answer: C

it is a managed table

upvoted 1 times

🗨️ 👤 **lokvamsi** 2 years, 3 months ago

Selected Answer: A

its a as it is managed table

upvoted 2 times

🗨️ 👤 **sturcu** 2 years, 3 months ago

Selected Answer: C

it is a managed table. So both table def and data will be deleted

upvoted 1 times

🗨️ 👤 **jyothsna12496** 2 years, 3 months ago

Selected Answer: C

Drop will usually delete the table structure and data if its managed, hence c

upvoted 1 times

Two of the most common data locations on Databricks are the DBFS root storage and external object storage mounted with `dbutils.fs.mount()`.

Which of the following statements is correct?

- A. DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.
- B. By default, both the DBFS root and mounted data sources are only accessible to workspace administrators.
- C. The DBFS root is the most secure location to store data, because mounted storage volumes must have full public read and write permissions.
- D. Neither the DBFS root nor mounted storage can be accessed when using `%sh` in a Databricks notebook.
- E. The DBFS root stores files in ephemeral block volumes attached to the driver, while mounted directories will always persist saved data to external storage between sessions.

Suggested Answer: E

Community vote distribution

A (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: A

Surely A

upvoted 1 times

🗳️ 👤 **Curious76** 1 year, 5 months ago

Selected Answer: A

A is correct . For E, This statement is partially incorrect. The DBFS root does use ephemeral storage, but not block volumes. Data saved there is lost when the cluster terminates unless explicitly persisted elsewhere. Mounted storage, however, can persist data between sessions depending on the underlying storage service and configuration.

upvoted 4 times

🗳️ 👤 **sodere** 1 year, 7 months ago

Selected Answer: A

DBFS is a layer on top of cloud storage providers.

upvoted 2 times

🗳️ 👤 **aragorn_brego** 1 year, 8 months ago

Selected Answer: A

Databricks File System (DBFS) is a layer over a cloud object storage (like AWS S3, Azure Blob Storage, or GCP Cloud Storage) that allows users to interact with data as if they were using a traditional file system. It provides familiar file system semantics and is designed to be consistent with POSIX-like file system behavior, which includes commands and actions similar to those used in Unix and Linux file systems.

upvoted 4 times

🗳️ 👤 **Dileepvikram** 1 year, 9 months ago

Answer is A

upvoted 1 times

🗳️ 👤 **sturcu** 1 year, 9 months ago

Selected Answer: A

it is not E.

The only one that would be plausible is A

upvoted 2 times

The following code has been migrated to a Databricks notebook from a legacy workload:

```
%sh
git clone https://github.com/foo/data_loader;
python ./data_loader/run.py;
mv ./output /dbfs/mnt/new_data
```

The code executes successfully and provides the logically correct results, however, it takes over 20 minutes to extract and load around 1 GB of data.

Which statement is a possible explanation for this behavior?

- A. %sh triggers a cluster restart to collect and install Git. Most of the latency is related to cluster startup time.
- B. Instead of cloning, the code should use %sh pip install so that the Python code can get executed in parallel across all nodes in a cluster.
- C. %sh does not distribute file moving operations; the final line of code should be updated to use %fs instead.
- D. Python will always execute slower than Scala on Databricks. The run.py script should be refactored to Scala.
- E. %sh executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

Suggested Answer: C

Community vote distribution

E (100%)

🗳️ 👤 **aragorn_brego** Highly Voted 2 years, 2 months ago

Selected Answer: E

When using %sh in a Databricks notebook, the commands are executed in a shell environment on the driver node. This means that only the resources of the driver node are used, and the execution does not leverage the distributed computing capabilities of the worker nodes in the Spark cluster. This can result in slower performance, especially for data-intensive tasks, compared to an approach that distributes the workload across all nodes in the cluster using Spark.

upvoted 9 times

🗳️ 👤 **KadELbied** Most Recent 9 months ago

Selected Answer: E

Surely E

upvoted 1 times

🗳️ 👤 **robodog** 1 year, 5 months ago

Selected Answer: E

Option E correct

upvoted 1 times

🗳️ 👤 **Freyr** 1 year, 8 months ago

Selected Answer: E

Option E: Correct. The %sh magic command in Databricks runs shell commands on the driver node only. This means the operations within %sh do not leverage the distributed nature of the Databricks cluster. Consequently, the Git clone, Python script execution, and file move operations are all performed on a single node (the driver), which explains why it takes a long time to process and move 1 GB of data. This approach does not utilize the parallel processing capabilities of the worker nodes or the optimization features of Databricks Spark.

Option C: Incorrect. %sh does not inherently distribute any operations, but the issue here is broader than just file moving operations. Using %fs for file operations is a best practice, but it does not resolve the inefficiency of running all commands on the driver node.

upvoted 2 times

🗳️ 👤 **Dileepvikram** 2 years, 3 months ago

E is the answer as the command is ran in the driver node and other nodes in the cluster are not used

upvoted 2 times

🗳️ 👤 **sturcu** 2 years, 3 months ago

Selected Answer: E

%sh run Bash commands on the driver node of the cluster.

<https://www.databricks.com/blog/2020/08/31/introducing-the-databricks-web-terminal.html>

upvoted 3 times

  **sturcu** 2 years, 3 months ago

you can use mv with %sh, but the syntax is not correct , it is missing the destination operand

upvoted 1 times

  **sturcu** 2 years, 3 months ago

I just noticed there is a space between the paths, so syntax is correct

upvoted 2 times

The data science team has requested assistance in accelerating queries on free form text from user reviews. The data is currently stored in Parquet with the below schema:

item_id INT, user_id INT, review_id INT, rating FLOAT, review STRING

The review column contains the full text of the review left by the user. Specifically, the data science team is looking to identify if any of 30 key words exist in this field.

A junior data engineer suggests converting this data to Delta Lake will improve query performance.

Which response to the junior data engineer's suggestion is correct?

- A. Delta Lake statistics are not optimized for free text fields with high cardinality.
- B. Text data cannot be stored with Delta Lake.
- C. ZORDER ON review will need to be run to see performance gains.
- D. The Delta log creates a term matrix for free text fields to support selective filtering.
- E. Delta Lake statistics are only collected on the first 4 columns in a table.

Suggested Answer: D

Community vote distribution

A (100%)

  **aragorn_brego** Highly Voted 1 year, 2 months ago

Selected Answer: A

Delta Lake uses statistics and data skipping to improve query performance, but these optimizations are most effective for columns with low to medium cardinality (i.e., columns with a limited set of distinct values). Free-form text fields like the review column typically have high cardinality, meaning each value in the column (each review text) is unique or nearly unique. Consequently, statistics on such columns do not significantly improve the performance of queries searching for specific keywords within the text.

upvoted 7 times

  **KadELbied** Most Recent 9 months ago

Selected Answer: A

Surely A

upvoted 1 times

  **bp_a_user** 10 months, 1 week ago

Selected Answer: A

Collecting statistics on a column containing long values such as string or binary is an expensive operation

<https://docs.delta.io/latest/optimizations-oss.html>

upvoted 1 times

  **Dileepvikram** 1 year, 3 months ago

answer is A

upvoted 2 times

  **mouad_attaqi** 1 year, 3 months ago

Selected Answer: A

A is correct

upvoted 2 times

  **sturcu** 1 year, 3 months ago

Selected Answer: A

Collecting statistics on long strings is an expensive operation

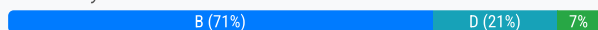
upvoted 2 times

Assuming that the Databricks CLI has been installed and configured correctly, which Databricks CLI command can be used to upload a custom Python Wheel to object storage mounted with the DBFS for use with a production job?

- A. configure
- B. fs
- C. jobs
- D. libraries
- E. workspace

Suggested Answer: C

Community vote distribution



arik90 Highly Voted 1 year, 4 months ago

Selected Answer: B

databricks fs cp dist/<...>.whl dbfs:/some/place/appropriate
upvoted 6 times

KadELbied Most Recent 9 months ago

Selected Answer: B

Suretly B
upvoted 1 times

AlejandroU 1 year, 1 month ago

Selected Answer: D

Answer D. The Databricks CLI libraries command is used to manage libraries, including installing custom Python wheels. Specifically, the install subcommand can be used to install a wheel. In contrast, Option B. fs: This command interacts with the Databricks File System (DBFS) to manage files, but it is primarily used for basic file operations (like cp, ls, rm), not specifically for uploading libraries.
upvoted 1 times

arekm 1 year, 1 month ago

The question is about upload - answer B.
upvoted 1 times

Curious76 1 year, 5 months ago

Selected Answer: D

Here's how you can use the libraries command to upload your wheel:
Bash

```
databricks libraries upload --file <path_to_wheel_file> --name <library_name>
```

upvoted 1 times

ojudz08 1 year, 5 months ago

Selected Answer: C

this is a bit tricky, question is asked to upload custom Python Wheel, you can use fs command, but since it'll be used in production job, job command might be needed to perform databricks jobs operations?
<https://docs.databricks.com/en/dev-tools/cli/commands.html>
upvoted 1 times

Somesh512 1 year, 6 months ago

Selected Answer: B

Its asking to upload to DBFS and not install on cluster
upvoted 2 times

petrv 1 year, 8 months ago

Selected Answer: B

the question is about copying the file not about installing.

upvoted 3 times

🗨️ 👤 **Enduresoul** 1 year, 8 months ago

Selected Answer: B

Answer B is correct:

"... which Databricks CLI command can be used to upload a custom Python Wheel to object storage mounted with the DBFS ..."

The question asks, how to upload the wheel. Not install it or configure it in a job.

<https://docs.databricks.com/en/archive/dev-tools/cli/dbfs-cli.html>

upvoted 4 times

🗨️ 👤 **aragorn_brego** 1 year, 8 months ago

Selected Answer: B

The Databricks CLI fs command is used for interacting with the Databricks File System (DBFS). You can use it to put files into DBFS, which includes uploading custom Python Wheels to a directory in DBFS. The fs command has subcommands like cp that can be used to copy files from your local file system to DBFS, which is backed by an object storage mounted with `dbutils.fs.mount()`.

```
databricks fs cp my_package.whl dbfs:/mnt/my-mount-point/my_package.whl
```

upvoted 2 times

🗨️ 👤 **mouad_attaqi** 1 year, 9 months ago

Selected Answer: D

It is done using the command: `databricks libraries install`

upvoted 1 times

🗨️ 👤 **sturcu** 1 year, 9 months ago

Selected Answer: D

you can add a library section to the jobs command, but you can install a wheel with the library command

upvoted 1 times

🗨️ 👤 **sturcu** 1 year, 9 months ago

```
/api/2.0/libraries/install
```

upvoted 1 times

The business intelligence team has a dashboard configured to track various summary metrics for retail stores. This includes total sales for the previous day alongside totals and averages for a variety of time periods. The fields required to populate this dashboard have the following schema:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT, total_sales_ytd
FLOAT, avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT,
avg_daily_sales_7d FLOAT, updated TIMESTAMP
```

For demand forecasting, the Lakehouse contains a validated table of all itemized sales updated incrementally in near real-time. This table, named `products_per_order`, includes the following fields:

```
store_id INT, order_id INT, product_id INT, quantity INT, price FLOAT,
order_timestamp TIMESTAMP
```

Because reporting on long-term sales trends is less volatile, analysts using the new dashboard only require data to be refreshed once daily. Because the dashboard will be queried interactively by many users throughout a normal business day, it should return results quickly and reduce total compute associated with each materialization.

Which solution meets the expectations of the end users while controlling and limiting possible costs?

- A. Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten with each update.
- B. Use Structured Streaming to configure a live dashboard against the `products_per_order` table within a Databricks notebook.
- C. Configure a webhook to execute an incremental read against `products_per_order` each time the dashboard is refreshed.
- D. Use the Delta Cache to persist the `products_per_order` table in memory to quickly update the dashboard with each query.
- E. Define a view against the `products_per_order` table and define the dashboard against this view.

Suggested Answer: A

Community vote distribution

A (100%)

  **dmov**  1 year, 7 months ago

Selected Answer: A

looks like A to me, as long as they only need the data for the aggregates based on the previous day only
upvoted 14 times

  **Def21** 1 year, 6 months ago

E - a view, could be an option but it would require computation every time used.
upvoted 1 times

  **42f87fd** 7 months, 3 weeks ago

A view would not be cost effective. Needs to be refreshed each time.
upvoted 1 times

  **KadELbied**  9 months ago

Selected Answer: A

Surely A
upvoted 1 times

  **srinivasa** 1 year, 1 month ago

Selected Answer: D

Delta cache avoids having to read data from the table every time it's queried during the day.
upvoted 1 times

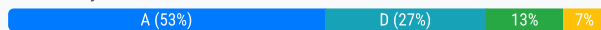
A data ingestion task requires a one-TB JSON dataset to be written out to Parquet with a target part-file size of 512 MB. Because Parquet is being used instead of Delta Lake, built-in file-sizing features such as Auto-Optimize & Auto-Compaction cannot be used.

Which strategy will yield the best performance without shuffling data?

- A. Set `spark.sql.files.maxPartitionBytes` to 512 MB, ingest the data, execute the narrow transformations, and then write to parquet.
- B. Set `spark.sql.shuffle.partitions` to 2,048 partitions ($1\text{TB} \times 1024 \times 1024 / 512$), ingest the data, execute the narrow transformations, optimize the data by sorting it (which automatically repartitions the data), and then write to parquet.
- C. Set `spark.sql.adaptive.advisoryPartitionSizeInBytes` to 512 MB bytes, ingest the data, execute the narrow transformations, coalesce to 2,048 partitions ($1\text{TB} \times 1024 \times 1024 / 512$), and then write to parquet.
- D. Ingest the data, execute the narrow transformations, repartition to 2,048 partitions ($1\text{TB} \times 1024 \times 1024 / 512$), and then write to parquet.
- E. Set `spark.sql.shuffle.partitions` to 512, ingest the data, execute the narrow transformations, and then write to parquet.

Suggested Answer: B

Community vote distribution



aragorn_brego Highly Voted 2 years, 2 months ago

Selected Answer: A

This strategy aims to control the size of the output Parquet files without shuffling the data. The `spark.sql.files.maxPartitionBytes` parameter sets the maximum size of a partition that Spark will read. By setting it to 512 MB, you are aligning the read partition size with the desired output file size.

Since the transformations are narrow (meaning they do not require shuffling), the number of partitions should roughly correspond to the number of output files when writing out to Parquet, assuming the data is evenly distributed and there is no data expansion during processing.

upvoted 11 times

Def21 Highly Voted 2 years ago

Selected Answer: D

D is the only one that does the trick.

Note, we can not do shuffling.

Wrong answers:

A: `spark.sql.files.maxPartitionBytes` is about reading, not writing. (The maximum number of bytes to pack into a single partition when reading files. This configuration is effective only when using file-based sources such as Parquet, JSON and ORC.)

B: `spark.sql.adaptive.advisoryPartitionSizeInBytes` takes effect while shuffling and sorting does not make sense (The advisory size in bytes of the shuffle partition during adaptive optimization (when `spark.sql.adaptive.enabled` is true). It takes effect when Spark coalesces small shuffle partitions or splits skewed shuffle partition.)

C: Would work but `spark.sql.adaptive.advisoryPartitionSizeInBytes` would need shuffling.

E. `spark.sql.shuffle.partitions` (Configures the number of partitions to use when shuffling data for joins or aggregations.) is not about writing.
upvoted 6 times

azurefan777 1 year, 3 months ago

Answer D is wrong -> repartition does perform shuffling in Spark. When you use repartition, Spark redistributes the data across the specified number of partitions, which requires moving data between nodes to achieve the new partitioning. Answer A should be correct

upvoted 4 times

carlosmps 1 year, 2 months ago

`spark.sql.files.maxPartitionBytes` is not just for reading files

upvoted 1 times

arekm 1 year, 1 month ago

D does repartition, which the question says we should try to avoid.

upvoted 1 times

🗉  **gizzamo** Most Recent 4 months, 3 weeks ago

Selected Answer: C

Why This Works Best:

1. No Shuffling:

Coalesce only combines adjacent partitions - no data movement across nodes

Preserves data locality during partition consolidation

Meets the absolute constraint of no shuffling

2. Achieves Target File Size:

AQE advisory setting guides Spark to optimize partition sizes toward 512MB during narrow transformations

Coalesce to 2,048 partitions directly targets the math: $1\text{TB} \div 2,048 = \sim 512\text{MB}$ per file

Direct control over output file count and sizing

3. Best Performance:

Leverages AQE optimization during the transformation pipeline

Minimal overhead from coalesce operation (just partition metadata changes)

Efficient resource utilization without shuffle overhead

upvoted 1 times

🗉  **gizzamo** 4 months, 3 weeks ago

Selected Answer: D

- repartition(N) ensures the output is split into $\sim N$ files.

- This directly controls output partitioning without relying on shuffle hints or adaptive query execution.

- Best way to hit target file size.

upvoted 1 times

🗉  **dalupus** 6 months, 3 weeks ago

Selected Answer: C

This is still only kinda correct.

A is wrong because there may be no correlation to the size of the data being read versus the size of the data being written and this only controls read.

The reason I say this is only partially correct is because it makes a false assumption that if you simply take a 1TB dataset and divide it up that the sum of the pieces will equal 1TB which is not true. The size of dataset will be dependent on the cardinality of the columns, but by definition any time you increase the number of parquet files you will increase total data size. If you don't believe me try it yourself. Take a 5GB dataframe and save it as 1 parquet vs 500 parquets. you will see the size is very different

upvoted 1 times

🗉  **happyhelppy** 6 months, 3 weeks ago

Selected Answer: C

You want target Parquet file sizes around 512 MB without causing a full shuffle.

Using coalesce instead of repartition ensures that no shuffle occurs — perfect for performance-sensitive jobs that don't need data redistribution.

The `spark.sql.adaptive.advisoryPartitionSizeInBytes` helps optimize physical plan execution under Adaptive Query Execution (AQE) without enforcing a hard shuffle.

Writing to 2,048 partitions ($1\text{TB} \div 512\text{MB}$) gives the best chance of achieving the desired file sizes while minimizing write latency.

upvoted 2 times

🗉  **KadELbied** 9 months, 1 week ago

Selected Answer: B

I found this question in another's Exam test and all of them it looks like B

upvoted 1 times

🗉  **AlejandroU** 1 year, 1 month ago

Selected Answer: D

Answer D. Explicitly repartitioning to 2,048 partitions ensures that the output files are close to the desired size of 512 MB, provided the data distribution is relatively even.

Repartitioning directly addresses the problem by controlling the number of partitions, which directly affects the output file size

Why not option A ? Misinterpretation of `spark.sql.files.maxPartitionBytes` in Option A:

The assessment incorrectly states that this configuration controls the maximum size of files when writing to Parquet. This setting controls the size of partitions when reading data, not during writing.

upvoted 1 times

  **AlejandroU** 1 year, 1 month ago

Given the requirement to avoid shuffling, Option A is the most suitable choice. By setting `spark.sql.files.maxPartitionBytes` to 512 MB, you influence the partitioning during the read phase, which can help in achieving the desired file sizes during the write operation. However, it's important to note that this approach may not guarantee exact file sizes, and some variability may occur. If achieving precise file sizes is critical and shuffling is permissible, Option D would be the preferred strategy.

upvoted 2 times

  **temple1305** 1 year, 2 months ago

Selected Answer: C

`spark.sql.adaptive.advisoryPartitionSizeInBytes` -

The advisory size in bytes of the shuffle partition during adaptive optimization (when `spark.sql.adaptive.enabled` is true). It takes effect when Spark coalesces small shuffle partitions or splits skewed shuffle partition.

And then we do coalesce - without shuffle - so have to work!

upvoted 1 times

  **nedlo** 1 year, 3 months ago

Selected Answer: A

I though D, but default num of partitions is 200, so you cant do coalesce (2048) (you cant increase numb of partitions through coalesce), so its not possible to do it without repartitioning and shuffle. Only A can be done without Shuffle

upvoted 2 times

  **sdas1** 1 year, 4 months ago

Option A

`spark.sql.files.maxPartitionBytes` controls the maximum size of partitions during reading on the Spark cluster, and that reducing this value could lead to more partitions and thus potentially more output files. The key point is that it works best when no shuffles occur, which aligns with the scenario of having narrow transformations only.

upvoted 2 times

  **sdas1** 1 year, 4 months ago

Given that no shuffle occurs and you're aiming to control the file sizes during output, adjusting `spark.sql.files.maxPartitionBytes` could help indirectly by determining the partition size for reading. Since the number of input partitions can influence the size of the output files when no shuffle occurs, the partition size may closely match the size of the files being written out.

upvoted 1 times

  **sdas1** 1 year, 4 months ago

If the transformations remain narrow, then Spark won't repartition the data unless explicitly instructed to do so (e.g., through a repartition or coalesce operation). In this case, using `spark.sql.files.maxPartitionBytes` to adjust the read partition size to 512 MB could indirectly control the number of output files and ensure they align with the target file size.

upvoted 1 times

  **sdas1** 1 year, 4 months ago

Thus, Option A is also a valid strategy:

Set `spark.sql.files.maxPartitionBytes` to 512 MB, process the data with narrow transformations, and write to Parquet.

By reducing the value of `spark.sql.files.maxPartitionBytes`, you ensure more partitions are created during the read phase, leading to output files closer to the desired size, assuming the transformations are narrow and no shuffling occurs.

upvoted 1 times

  **vikram12apr** 1 year, 11 months ago

Selected Answer: A

D is not correct as it will create 2048 target files of 0.5 MB each

Only A will do the job as it will read this file in 2 partition (1 TB = 512*2 MB) and as we are not doing any shuffling(not mentioned in option) it will create those many partition file i.e 2 part files

upvoted 1 times

🗨️ 👤 **hal2401me** 1 year, 11 months ago

hey, $1\text{TB} = 1000\text{GB} = 1^6\text{MB}$.

upvoted 4 times

🗨️ 👤 **hal2401me** 1 year, 11 months ago

Selected Answer: D

ChatGPT says D: This strategy directly addresses the desired part-file size by repartitioning the data. It avoids shuffling during narrow transformations.

Recommended for achieving the desired part-file size without unnecessary shuffling.

upvoted 1 times

🗨️ 👤 **Curious76** 1 year, 11 months ago

Selected Answer: D

D is not suitable.

upvoted 1 times

🗨️ 👤 **vctrhugo** 2 years ago

Selected Answer: A

This approach ensures that each partition will be approximately the target part-file size, which can improve the efficiency of the data write. It also avoids the need for a shuffle operation, which can be expensive in terms of performance.

upvoted 3 times

🗨️ 👤 **adenis** 2 years ago

Selected Answer: C

C is correct

upvoted 1 times

🗨️ 👤 **spaceexplorer** 2 years ago

Selected Answer: A

Rest of the answers trigger shuffles

upvoted 2 times

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Incremental state information should be maintained for 10 minutes for late-arriving data.

Streaming DataFrame df has the following schema:

"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"

Code block:

```
df._____  
    .groupBy(  
        window("event_time", "5 minutes").alias("time"),  
        "device_id"  
    )  
    .agg(  
        avg("temp").alias("avg_temp"),  
        avg("humidity").alias("avg_humidity")  
    )  
    .writeStream  
    .format("delta")  
    .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

- A. withWatermark("event_time", "10 minutes")
- B. awaitArrival("event_time", "10 minutes")
- C. await("event_time + '10 minutes'")
- D. slidingWindow("event_time", "10 minutes")
- E. delayWrite("event_time", "10 minutes")

Suggested Answer: D

Community vote distribution

A (100%)

 **sturcu** Highly Voted 2 years, 3 months ago

Selected Answer: A

withWatermark.

There sliding window is doe through the window function

upvoted 9 times

 **aragorn_brego** Highly Voted 2 years, 2 months ago

Selected Answer: A

To handle late-arriving data in a streaming aggregation, you need to specify a watermark, which tells the streaming query how long to wait for late data. The withWatermark method is used for this purpose in Spark Structured Streaming. It defines the threshold for how late the data can be relative to the latest data that has been seen in the same window.

upvoted 9 times

 **KadELbied** Most Recent 9 months ago

Selected Answer: A

suretly A

upvoted 1 times

  **71dfab9** 1 year, 5 months ago

Selected Answer: A

The `withWatermark` method is used in streaming DataFrames when processing real-time data streams. This method helps in managing stateful operations, such as aggregations, by specifying a time column to use for watermarking. Watermarking is a mechanism to handle late data (data that arrives later than expected) by defining a threshold time window beyond which late data is considered too late to be included in aggregations.

The `slidingWindow` function mentioned in D is not a standard function in Databricks or Apache Spark.

upvoted 1 times

  **Dileepvikram** 2 years, 3 months ago

Answer is A

upvoted 3 times

A data team's Structured Streaming job is configured to calculate running aggregates for item sales to update a downstream marketing dashboard. The marketing team has introduced a new promotion, and they would like to add a new field to track the number of times this promotion code is used for each item. A junior data engineer suggests updating the existing query as follows. Note that proposed changes are in bold.

Original query:

```
df.groupBy("item")
  .agg(count("item").alias("total_count"),
       mean("sale_price").alias("avg_price"))
  .writeStream
  .outputMode("complete")
  .option("checkpointLocation", "/item_agg/__checkpoint")
  .start("/item_agg")
```

Proposed query:

```
df.groupBy("item")
  .agg(count("item").alias("total_count"),
       mean("sale_price").alias("avg_price"))
  .writeStream
  .outputMode("complete")
  .option("checkpointLocation", "/item_agg/__checkpoint")
  .start("/item_agg")
```

Proposed query:

```
.start("/item_agg")
```

Which step must also be completed to put the proposed query into production?

- A. Specify a new checkpointLocation
- B. Increase the shuffle partitions to account for additional aggregates
- C. Run REFRESH TABLE delta.`/item\_agg`
- D. Register the data in the "/item_agg" directory to the Hive metastore
- E. Remove .option('mergeSchema', 'true') from the streaming write

Suggested Answer: A

 **f728f7f** Highly Voted 1 year, 7 months ago

This question is broken. Proposed query cannot be identified.
upvoted 26 times

 **AlejandroU** Highly Voted 1 year, 1 month ago

Selected Answer: A

Below is the proposed query:

```
df.groupBy("item") .agg(count("item").alias("total_count"), mean("sale_price").alias("avg_price"), count("promo_code = 'NEW MEMBER'" ) .alias("new member_promo")) writeStream .outputMode("complete") .option('mergeSchema', 'true') .option("checkpointLocation", "/item_agg/ checkpoint") .start("/item_agg")
```

Answer A. When updating the schema of a streaming job by adding new fields (like the new_member_promo field), it's important to use a new

checkpoint location. This is because the existing checkpoint location is tied to the old schema, and adding a new field could lead to schema mismatch issues.

upvoted 5 times

  **OnlyPraveen** 1 year, 1 month ago

Thank you! Also check Question #114 which has the Proposed Query image too.

upvoted 1 times

  **KadELbied** Most Recent 9 months ago

Selected Answer: A

suretly A

upvoted 1 times

  **kino_1994** 1 year, 1 month ago

Selected Answer: A

Since the new field is a count (an aggregation), it is non-nullable, making the change incompatible with the existing schema. This requires a new checkpointLocation to avoid schema mismatch issues. Additionally, the "mergeSchema=true" option must remain enabled to allow Spark to handle the schema evolution properly.

However, if the field were nullable and not an aggregation, it would be a backward-compatible change, allowing the checkpoint to remain unchanged, as happens with schema evolution in Kafka. In this case, the correct answer is A.

upvoted 2 times

  **Sriramiyer92** 1 year, 1 month ago

Selected Answer: A

The given answer is correct.

In case of addition of new cols (or changes) the checkpoint location also needs to change.

upvoted 1 times

A Structured Streaming job deployed to production has been resulting in higher than expected cloud storage costs. At present, during normal execution, each microbatch of data is processed in less than 3s; at least 12 times per minute, a microbatch is processed that contains 0 records. The streaming write was configured using the default trigger settings. The production job is currently scheduled alongside many other Databricks jobs in a workspace with instance pools provisioned to reduce start-up time for jobs with batch execution.

Holding all other variables constant and assuming records need to be processed in less than 10 minutes, which adjustment will meet the requirement?

- A. Set the trigger interval to 3 seconds; the default trigger interval is consuming too many records per batch, resulting in spill to disk that can increase volume costs.
- B. Increase the number of shuffle partitions to maximize parallelism, since the trigger interval cannot be modified without modifying the checkpoint directory.
- C. Set the trigger interval to 10 minutes; each batch calls APIs in the source storage account, so decreasing trigger frequency to maximum allowable threshold should minimize this cost.
- D. Set the trigger interval to 500 milliseconds; setting a small but non-zero trigger interval ensures that the source is not queried too frequently.
- E. Use the trigger once option and configure a Databricks job to execute the query every 10 minutes; this approach minimizes costs for both compute and storage.

Suggested Answer: C

Community vote distribution

C (50%)

E (50%)

 **aragorn_brego** Highly Voted 2 years, 2 months ago

Selected Answer: E

Given that there are frequent microbatches with 0 records being processed, it indicates that the job is polling the source too often. Using the "trigger once" option would allow each microbatch to process all available data and then stop. By scheduling the job to run every 10 minutes, you ensure that the system is not constantly checking for new data when there is none, thus reducing the number of read operations from the source storage and potentially reducing costs associated with those reads.

upvoted 5 times

 **Gulenur_GS** 2 years, 2 months ago

in this case why not C? Processing trigger in 10 min ensures the same I guess..

upvoted 1 times

 **gizzamo** Most Recent 4 months, 4 weeks ago

Selected Answer: E

E: Correct. The trigger once option runs a single microbatch and then stops. By scheduling a Databricks job to run this query every 10 minutes, you satisfy the processing requirement ("less than 10 minutes") while drastically reducing the number of times the source storage is queried. This approach minimizes the compute time (the job only runs when needed) and, more importantly, minimizes the number of API calls to the source storage, directly addressing the core problem of high storage costs. This is a common and recommended pattern for cost-effective data ingestion with Databricks.

upvoted 4 times

 **KadELbied** 9 months ago

Selected Answer: C

suretly C

upvoted 1 times

 **AlejandroU** 1 year, 1 month ago

Selected Answer: C

Answer C. Setting the trigger interval to 10 minutes (option C) directly aligns with the requirement to process records within a 10-minute window. It achieves the same reduction in processing frequency as option E but without the added complexity of job scheduling or reliance on trigger once. Using the trigger once option requires external orchestration (e.g., a scheduled Databricks job) to execute every 10 minutes. This adds operational overhead and potential delays due to job scheduling or startup times, especially in a shared workspace using instance pools.

upvoted 2 times

🗳️ 👤 **Urcolbz** 1 year, 1 month ago

Selected Answer: C

In my opinion, both C and E met the requirements. But the sentence says 'Holding all other variables constant'. This indicates me that E cannot be the solution, as new variables are introduced.

upvoted 3 times

🗳️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: E

The fact that the question mentions instance pools provisioned make me guess that we should go for trigger once option otherwise instance pools are useless.

upvoted 3 times

🗳️ 👤 **pk07** 1 year, 4 months ago

Selected Answer: C

E WRONG. Using trigger once would stop the stream after one execution, not meeting the requirement of continuous processing.

upvoted 2 times

🗳️ 👤 **practitioner** 1 year, 5 months ago

Selected Answer: E

E is correct for two reasons:

- 1) we have been using the connection pool that allows us to start our job instantly
- 2) the questions are about reducing costs. Triggering one per 10 minutes allows not to use running VM (as in option C) and to keep the same SLA (due to 1)) with lower cost for compute as well as for storage (fewer API calls which are not free)

upvoted 2 times

🗳️ 👤 **Er5** 1 year, 10 months ago

required "to be processed in less than 10 minutes".

C. "set the trigger interval to 10 minutes" means Process time + interval > 10 minutes

E. "trigger once", "execute the query every 10 minutes"

upvoted 3 times

🗳️ 👤 **vikram12apr** 1 year, 11 months ago

Selected Answer: E

default trigger time is 0.5 seconds

Hence in a minute there are 120 triggers happens

Each trigger consume 3 seconds to complete

now $120 * 3 = 360$ seconds = 6 minutes

Hence the job is completing in 6 minutes

Now there is buffer of 4 minutes which can be utilized in compute spin up

but as we are using the spot instances which will further decrease the start up time

I think E is correct option to decrease the cost.

upvoted 3 times

🗳️ 👤 **hidelux** 1 year, 11 months ago

Selected Answer: E

The question indicates that they are using instance pools for fast startup time. option C would block a VM permanently which is not intended. E will grab a VM, run the job, and return it to the pool to be available for other jobs mentioned in the question.

upvoted 4 times

🗳️ 👤 **practitioner** 1 year, 5 months ago

you are right. But we need to guarantee SLA and for this reason to block VM (with autoscaling) is a good practice

upvoted 1 times

🗳️ 👤 **spaceexplorer** 2 years ago

Selected Answer: C

C is more effective than E as E will incur startup time for spinning new job cluster

upvoted 3 times

🗳️ 👤 **ranith** 2 years ago

The default trigger interval is 500ms, but the question says it processes batches with 0 records and the avg time to process is 3s. If the requirement is to process under 10 minutes the best option here is to trigger every 3s.

upvoted 1 times

🗨️ 👤 **divingbell17** 2 years, 1 month ago

Selected Answer: C

Both C and E meet the requirement to reduce cloud storage cost. E further reduces compute cost however reducing compute cost is not a requirement in the question.

upvoted 2 times

🗨️ 👤 **alexvno** 2 years, 1 month ago

Selected Answer: C

For production -> records need to be processed in less than 10 minutes. So we need to schedule each 10 minutes

upvoted 3 times

Which statement describes the correct use of `pyspark.sql.functions.broadcast`?

- A. It marks a column as having low enough cardinality to properly map distinct values to available partitions, allowing a broadcast join.
- B. It marks a column as small enough to store in memory on all executors, allowing a broadcast join.
- C. It caches a copy of the indicated table on attached storage volumes for all active clusters within a Databricks workspace.
- D. It marks a DataFrame as small enough to store in memory on all executors, allowing a broadcast join.
- E. It caches a copy of the indicated table on all nodes in the cluster for use in all future queries during the cluster lifetime.

Suggested Answer: C

Community vote distribution

D (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: D

suretly D

upvoted 1 times

🗳️ 👤 **Freyr** 1 year, 2 months ago

Selected Answer: D

Correct Answer: D. It marks a DataFrame as small enough to store in memory on all executors, allowing a broadcast join.

Reference: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.broadcast.html>

upvoted 3 times

🗳️ 👤 **aragorn_brego** 1 year, 8 months ago

Selected Answer: D

The broadcast function in PySpark is used in the context of joins. When you mark a DataFrame with broadcast, Spark tries to send this DataFrame to all worker nodes so that it can be joined with another DataFrame without shuffling the larger DataFrame across the nodes. This is particularly beneficial when the DataFrame is small enough to fit into the memory of each node. It helps to optimize the join process by reducing the amount of data that needs to be shuffled across the cluster, which can be a very expensive operation in terms of computation and time.

upvoted 3 times

🗳️ 👤 **Dileepvikram** 1 year, 9 months ago

Answer is D

upvoted 1 times

🗳️ 👤 **PearApple** 1 year, 9 months ago

The answer is D

upvoted 1 times

🗳️ 👤 **hm358** 1 year, 9 months ago

Selected Answer: D

<https://spark.apache.org/docs/3.1.3/api/python/reference/api/pyspark.sql.functions.broadcast.html>

upvoted 2 times

🗳️ 👤 **sturcu** 1 year, 9 months ago

Selected Answer: D

Marks a DataFrame as small enough for use in broadcast joins.

upvoted 3 times

A data engineer is configuring a pipeline that will potentially see late-arriving, duplicate records.

In addition to de-duplicating records within the batch, which of the following approaches allows the data engineer to deduplicate data against previously processed records as it is inserted into a Delta table?

- A. Set the configuration `delta.deduplicate = true`.
- B. VACUUM the Delta table after each batch completes.
- C. Perform an insert-only merge with a matching condition on a unique key.
- D. Perform a full outer join on a unique key and overwrite existing data.
- E. Rely on Delta Lake schema enforcement to prevent duplicate records.

Suggested Answer: D

Community vote distribution

C (100%)

🗳️ **aragorn_brego** Highly Voted 2 years, 2 months ago

Selected Answer: C

To handle deduplication against previously processed records in a Delta table, the MERGE INTO command can be used to perform an upsert operation. This means that if the incoming data has a record that matches an existing record based on a unique key, the MERGE INTO operation can update the existing record (if needed) or simply ignore the duplicate. If there is no match (i.e., the record is new), then the record will be inserted
upvoted 6 times

🗳️ **sturcu** Highly Voted 2 years, 3 months ago

Selected Answer: C

Merge, when not match insert
upvoted 5 times

🗳️ **KadELbied** Most Recent 9 months ago

Selected Answer: C

suretly C
upvoted 1 times

🗳️ **_lene_** 1 year ago

Selected Answer: C

this question was in the Databricks DE Professional exam guide
upvoted 1 times

🗳️ **hebied** 1 year, 2 months ago

Selected Answer: D

Option C is tricky since it should be merge on on Not match condition rather than matching .. Since Option D is more suitable
upvoted 1 times

🗳️ **60ties** 2 years, 2 months ago

Selected Answer: C

answer is C
upvoted 2 times

🗳️ **Dileepvikram** 2 years, 3 months ago

Answer is C
upvoted 2 times

🗳️ **hm358** 2 years, 3 months ago

Selected Answer: C

merge will be more efficient
upvoted 2 times

🗳️ **Crocjun** 2 years, 3 months ago

C

Reference: file:///C:/Users/yuen1/Downloads/databricks-certified-data-engineer-professional-exam-guide.pdf

upvoted 1 times

  **mouad_attaqi** 2 years, 3 months ago

you are referencing a local pdf in your computer !!!

upvoted 9 times

A data pipeline uses Structured Streaming to ingest data from Apache Kafka to Delta Lake. Data is being stored in a bronze table, and includes the Kafka-generated timestamp, key, and value. Three months after the pipeline is deployed, the data engineering team has noticed some latency issues during certain times of the day.

A senior data engineer updates the Delta Table's schema and ingestion logic to include the current timestamp (as recorded by Apache Spark) as well as the Kafka topic and partition. The team plans to use these additional metadata fields to diagnose the transient processing delays.

Which limitation will the team face while diagnosing this problem?

- A. New fields will not be computed for historic records.
- B. Spark cannot capture the topic and partition fields from a Kafka source.
- C. New fields cannot be added to a production Delta table.
- D. Updating the table schema will invalidate the Delta transaction log metadata.
- E. Updating the table schema requires a default value provided for each field added.

Suggested Answer: A

Community vote distribution

A (100%)

  **dmov** Highly Voted 1 year, 7 months ago

Selected Answer: A

Looks like A to me. Does anyone think otherwise?
upvoted 8 times

  **vctrhugo** Highly Voted 1 year, 6 months ago

Selected Answer: A

When the schema of a Delta table is updated to include new fields, these fields will only be populated for new records ingested after the schema update. The new fields will not be retroactively computed for historic records already stored in the Delta table. Therefore, the additional metadata fields (current timestamp, Kafka topic, and partition) will not exist in the historic data, limiting the scope of the diagnosis to new data ingested after the schema update.
upvoted 7 times

  **KadELbied** Most Recent 9 months ago

Selected Answer: A

suretly A
upvoted 1 times

  **RandomForest** 1 year ago

Selected Answer: A

The correct answer is A.
upvoted 1 times

In order to facilitate near real-time workloads, a data engineer is creating a helper function to leverage the schema detection and evolution functionality of Databricks Auto Loader. The desired function will automatically detect the schema of the source directly, incrementally process JSON files as they arrive in a source directory, and automatically evolve the schema of the table when new fields are detected.

The function is displayed below with a blank:

```
def auto_load_json(source_path: str,
                   checkpoint_path: str,
                   target_table_path: str):
    (spark.readStream
     .format("cloudFiles")
     .option("cloudFiles.format", "json")
     .option("cloudFiles.schemaLocation", checkpoint_path)
     .load(source_path)
     _____
    )
```

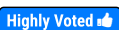
Which response correctly fills in the blank to meet the specified requirements?

- .writeStream
- A. .option("mergeSchema", True)
- .start(target_table_path)
- .writeStream
- .option("checkpointLocation", checkpoint_path)
- B. .option("mergeSchema", True)
- .trigger(once=True)
- .start(target_table_path)
- .write
- .option("checkpointLocation", checkpoint_path)
- C. .option("mergeSchema", True)
- .outputMode("append")
- .save(target_table_path)
- .write
- .option("mergeSchema", True)
- D. .mode("append")
- .save(target_table_path)
- .writeStream
- .option("checkpointLocation", checkpoint_path)
- E. .option("mergeSchema", True)
- .start(target_table_path)

Suggested Answer: C

Community vote distribution

E (100%)

 **Freyr**  1 year, 8 months ago

Selected Answer: E

Reference: <https://docs.databricks.com/en/ingestion/auto-loader/schema.html>

writeStream: Ensures real-time streaming write capabilities, which is essential for near real-time workloads.

checkpointLocation: Necessary for fault tolerance and tracking progress.

mergeSchema: Ensures automatic schema evolution, allowing new columns to be detected and added to the target table.

Why Option 'C' is incorrect?

Uses write instead of writeStream, which is for batch processing, making it inappropriate for real-time streaming.

Why Option 'B' is incorrect?

Although it includes checkpointLocation and mergeSchema, the addition of trigger(once=True) is not necessary in this context, and it is better suited for batch-like processing.

Reference: <https://docs.databricks.com/en/ingestion/auto-loader/schema.html>

upvoted 6 times

🗲️ 👤 **KadELbied** Most Recent 9 months ago

Selected Answer: E

surely E

upvoted 1 times

🗲️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: E

Evolve Schema = mergeSchema option is needed ; Incrementally = checkpointing is needed; Real-Time = WriteStream with default trigger . The only option that catches all of these is E

upvoted 2 times

🗲️ 👤 **35fd6dd** 1 year, 6 months ago

Selected Answer: E

write is not for spark streaming

upvoted 2 times

🗲️ 👤 **vikram12apr** 1 year, 11 months ago

Selected Answer: E

streamRead & StreamWrite shares the schema using checkpoint location

so cloudFiles.schemaLocation needs to be same for checkpointLocation so that we dont need to specify it manually

also mergeSchema True make sure if any new column detected , it will be added in the target table

<https://docs.databricks.com/en/ingestion/auto-loader/schema.html>

upvoted 2 times

🗲️ 👤 **hal2401me** 1 year, 11 months ago

Selected Answer: E

<https://notebooks.databricks.com/demos/auto-loader/01-Auto-loader-schema-evolution-Ingestion.html>

upvoted 2 times

🗲️ 👤 **aragorn_brego** 2 years, 2 months ago

Selected Answer: E

This response correctly fills in the blank to meet the specified requirements of using Databricks Auto Loader for automatic schema detection and evolution in a near real-time streaming context.

upvoted 1 times

🗲️ 👤 **AzureDE2522** 2 years, 2 months ago

Selected Answer: E

Please refer: <https://docs.databricks.com/en/ingestion/auto-loader/schema.html>

upvoted 3 times

🗲️ 👤 **Dileepvikram** 2 years, 3 months ago

It does not mention to write as stream, it mentions to write incrementally, so option C looks correct for me

upvoted 1 times

🗲️ 👤 **mouad_attaqi** 2 years, 3 months ago

Selected Answer: E

Correct answer is E, it is a streaming write, and the default outputMode is Append (so if it's optional in this case)

upvoted 2 times

🗲️ 👤 **sturcu** 2 years, 3 months ago

there is a type in the statement. Is it schema or checkpoint ?

Provided answer is not correct. It has to be a writestream, with mode append

upvoted 1 times

The data engineering team maintains the following code:

```
import pyspark.sql.functions as F

(spark.table("silver_customer_sales")
 .groupBy("customer_id")
 .agg(
     F.min("sale_date").alias("first_transaction_date"),
     F.max("sale_date").alias("last_transaction_date"),
     F.mean("sale_total").alias("average_sales"),
     F.countDistinct("order_id").alias("total_orders"),
     F.sum("sale_total").alias("lifetime_value")
 ).write
 .mode("overwrite")
 .table("gold_customer_lifetime_sales_summary")
 )
```

Assuming that this code produces logically correct results and the data in the source table has been de-duplicated and validated, which statement describes what will occur when this code is executed?

- A. The silver_customer_sales table will be overwritten by aggregated values calculated from all records in the gold_customer_lifetime_sales_summary table as a batch job.
- B. A batch job will update the gold_customer_lifetime_sales_summary table, replacing only those rows that have different values than the current version of the table, using customer_id as the primary key.
- C. The gold_customer_lifetime_sales_summary table will be overwritten by aggregated values calculated from all records in the silver_customer_sales table as a batch job.
- D. An incremental job will leverage running information in the state store to update aggregate values in the gold_customer_lifetime_sales_summary table.
- E. An incremental job will detect if new rows have been written to the silver_customer_sales table; if new rows are detected, all aggregates will be recalculated and used to overwrite the gold_customer_lifetime_sales_summary table.

Suggested Answer: E

Community vote distribution

C (100%)

 **aragorn_brego** Highly Voted 1 year, 8 months ago

Selected Answer: C

The code is performing a batch aggregation operation on the "silver_customer_sales" table grouped by "customer_id". It calculates the first and last transaction dates, the average sales, the total number of distinct orders, and the lifetime value of sales for each customer. The .mode("overwrite") operation specifies that the output table "gold_customer_lifetime_sales_summary" should be overwritten with the result of this aggregation. This means that every time this code runs, it will replace the existing "gold_customer_lifetime_sales_summary" table with a new version that reflects the current state of the "silver_customer_sales" table.

upvoted 8 times

 **sainandam** Most Recent 7 months, 2 weeks ago

Selected Answer: D

Writing data to a folder does not register an External table.

upvoted 1 times

 **KadELbied** 9 months ago

Selected Answer: C

Surety C

upvoted 1 times

🗨️ 👤 **hal2401me** 1 year, 5 months ago

Selected Answer: C

C. there's nowhere implicating streaming.

upvoted 1 times

🗨️ 👤 **Dileepvikram** 1 year, 9 months ago

C is the answer

upvoted 1 times

🗨️ 👤 **mouad_attaqi** 1 year, 9 months ago

Selected Answer: C

Correct Answer is C, it is an overwrite mode

upvoted 3 times

🗨️ 👤 **sturcu** 1 year, 9 months ago

Selected Answer: C

it does overwrite, so no incremental load

upvoted 4 times

The data architect has mandated that all tables in the Lakehouse should be configured as external (also known as "unmanaged") Delta Lake tables.

Which approach will ensure that this requirement is met?

- A. When a database is being created, make sure that the LOCATION keyword is used.
- B. When configuring an external data warehouse for all table storage, leverage Databricks for all ELT.
- C. When data is saved to a table, make sure that a full file path is specified alongside the Delta format.
- D. When tables are created, make sure that the EXTERNAL keyword is used in the CREATE TABLE statement.
- E. When the workspace is being configured, make sure that external cloud object storage has been mounted.

Suggested Answer: D

Community vote distribution

C (100%)

🗳️ **sturcu** Highly Voted 2 years, 3 months ago

Non of the provided.

It should be: When a table is created, make sure LOCATION is provided
upvoted 9 times

🗳️ **vctrhugo** Highly Voted 2 years ago

Selected Answer: C

In Delta Lake, an external (or unmanaged) table is a table created outside of the data lake but is still accessible from the data lake. The data for external tables is stored in a location specified by the user, not in the default directory of the data lake. When you save data to an external table, you need to specify the full file path where the data will be stored. This makes the table "external" because the data itself is not managed by Delta Lake, only the metadata is. This is why specifying a full file path alongside the Delta format when saving data to a table will ensure that the table is configured as an external Delta Lake table.

upvoted 5 times

🗳️ **KadELbied** Most Recent 9 months ago

Selected Answer: C

Suretly C

upvoted 1 times

🗳️ **Sriramiyer92** 1 year, 1 month ago

Selected Answer: C

Folks note:

While creating a table - Use of External keyword - Non Mandatory.

Mentioning Location and providing a path - Mandatory.

In option C, it is not mentioned explicitly that Location keyword is used. But since the path is provided.. implies the use of Location keyword indirectly. The devil is in the details. :)

upvoted 4 times

🗳️ **hvj** 1 year, 4 months ago

'create external table' statement is using in HIVE, so C is correct.

upvoted 1 times

🗳️ **jkhan2405** 2 years ago

Selected Answer: C

C is correct.

upvoted 2 times

🗳️ **JamesWright** 2 years, 1 month ago

C is correct

upvoted 1 times

🗳️ **aragorn_brego** 2 years, 2 months ago

Selected Answer: C

Here's why the other options may not ensure the requirement is met:

D. Delta Lake does not use the EXTERNAL keyword in the same way as some other SQL-based systems. In Delta Lake, whether a table is external is determined by where the data files are stored, not by a keyword in the CREATE TABLE statement.

%sql

```
CREATE TABLE f1_demo.results_external
USING DELTA
LOCATION '/mnt/formula1dl/demo/results_external'
```

upvoted 3 times

  **Dileepvikram** 2 years, 3 months ago

possible answer is C

upvoted 1 times

  **Laraujo2022** 2 years, 3 months ago

I think it should be A because when a database is created using a location all tables within this database are automatically assign as unmanaged tables.

upvoted 1 times

  **60ties** 2 years, 2 months ago

Not quite. Test & see. The tables are 'managed' though database creation has 'LOCATION' keyword. C is best.

upvoted 5 times

  **sturcu** 2 years, 3 months ago

Selected Answer: C

provide path (LOCATION)

upvoted 1 times

  **mouad_attaqi** 2 years, 3 months ago

Selected Answer: C

C is plausible answer, as in this case we are writing the data to an external location

upvoted 1 times

The marketing team is looking to share data in an aggregate table with the sales organization, but the field names used by the teams do not match, and a number of marketing-specific fields have not been approved for the sales org.

Which of the following solutions addresses the situation while emphasizing simplicity?

- A. Create a view on the marketing table selecting only those fields approved for the sales team; alias the names of any fields that should be standardized to the sales naming conventions.
- B. Create a new table with the required schema and use Delta Lake's DEEP CLONE functionality to sync up changes committed to one table to the corresponding table.
- C. Use a CTAS statement to create a derivative table from the marketing table; configure a production job to propagate changes.
- D. Add a parallel table write to the current production pipeline, updating a new sales table that varies as required from the marketing table.
- E. Instruct the marketing team to download results as a CSV and email them to the sales organization.

Suggested Answer: A

Community vote distribution

A (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: A

suretly A

upvoted 1 times

🗳️ 👤 **Hadiler** 1 year, 6 months ago

Selected Answer: A

A is the simplest one

upvoted 1 times

🗳️ 👤 **vctrhugo** 2 years ago

Selected Answer: A

Creating a view is a simple and efficient way to provide access to a subset of data from a table. In this case, the view can be configured to include only the fields that have been approved for the sales team. Additionally, any fields that need to be renamed to match the sales team's naming conventions can be aliased in the view. This approach does not require the creation of additional tables or the configuration of jobs to sync data, making it a relatively straightforward solution. However, it's important to note that views do not physically store data, so any changes to the underlying marketing table will be reflected in the view. This means that the sales team will always have access to the most up-to-date approved data.

upvoted 4 times

🗳️ 👤 **spaceexplorer** 2 years ago

Selected Answer: A

A is the simplest

upvoted 1 times

🗳️ 👤 **dmov** 2 years, 1 month ago

Selected Answer: A

Looks like A to me

upvoted 2 times

A CHECK constraint has been successfully added to the Delta table named activity_details using the following logic:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
    latitude >= -90 AND
    latitude <= 90 AND
    longitude >= -180 AND
    longitude <= 180);
```

A batch job is attempting to insert new records to the table, including a record where latitude = 45.50 and longitude = 212.67.

Which statement describes the outcome of this batch insert?

- A. The write will fail when the violating record is reached; any records previously processed will be recorded to the target table.
- B. The write will fail completely because of the constraint violation and no records will be inserted into the target table.
- C. The write will insert all records except those that violate the table constraints; the violating records will be recorded to a quarantine table.
- D. The write will include all records in the target table; any violations will be indicated in the boolean column named valid_coordinates.
- E. The write will insert all records except those that violate the table constraints; the violating records will be reported in a warning log.

Suggested Answer: D

Community vote distribution

B (100%)

 **aragorn_brego** Highly Voted 1 year, 8 months ago

Selected Answer: B

In systems that support atomic transactions, such as Delta Lake, when a batch operation encounters a record that violates a CHECK constraint, the entire operation fails, and no records are inserted, including those that do not violate the constraint. This is to ensure the atomicity of the transaction, meaning that either all the changes are committed, or none are, maintaining data integrity. The record with a longitude of 212.67 violates the constraint because longitude values must be between -180 and 180 degrees.

upvoted 5 times

 **vctrhugo** Highly Voted 1 year, 6 months ago

Selected Answer: B

In Delta Lake, when a batch job attempts to insert records into a table that has a CHECK constraint, if any record violates the constraint, the entire write operation fails. This is because Delta Lake enforces strong transactional guarantees, which means that either all changes in a transaction are saved, or none are.

upvoted 5 times

 **KadELbied** Most Recent 9 months ago

Selected Answer: B

Surely B

upvoted 1 times

 **spaceexplorer** 1 year, 6 months ago

Selected Answer: B

B is correct

upvoted 1 times

 **Dileepvikram** 1 year, 9 months ago

B is the answer

upvoted 4 times

 **sturcu** 1 year, 9 months ago

Selected Answer: B

B is the answer

upvoted 4 times

  **PearApple** 1 year, 9 months ago

B is the ans

upvoted 3 times

A junior data engineer has manually configured a series of jobs using the Databricks Jobs UI. Upon reviewing their work, the engineer realizes that they are listed as the "Owner" for each job. They attempt to transfer "Owner" privileges to the "DevOps" group, but cannot successfully accomplish this task.

Which statement explains what is preventing this privilege transfer?

- A. Databricks jobs must have exactly one owner; "Owner" privileges cannot be assigned to a group.
- B. The creator of a Databricks job will always have "Owner" privileges; this configuration cannot be changed.
- C. Other than the default "admins" group, only individual users can be granted privileges on jobs.
- D. A user can only transfer job ownership to a group if they are also a member of that group.
- E. Only workspace administrators can grant "Owner" privileges to a group.

Suggested Answer: A

Community vote distribution

A (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: A

Surely A

upvoted 1 times

🗳️ 👤 **hal2401me** 1 year, 5 months ago

Selected Answer: A

did a test. "group cannot be owner" is displayed.

upvoted 3 times

🗳️ 👤 **vctrhugo** 1 year, 6 months ago

Selected Answer: A

In Databricks, each job must have exactly one owner, which is typically the user who created the job. This "Owner" privilege allows the user to perform any action on the job, including modifying its settings or deleting it. However, this privilege cannot be assigned to a group. If you want to allow multiple users or a group of users to manage a job, you can use ACLs (Access Control Lists) to grant them the necessary permissions. But the "Owner" privilege will still remain with the individual user who created the job.

upvoted 1 times

🗳️ 👤 **sturcu** 1 year, 9 months ago

Selected Answer: A

Correct

A job cannot have more than one owner. A job cannot have a group as an owner

upvoted 4 times

All records from an Apache Kafka producer are being ingested into a single Delta Lake table with the following schema:

key BINARY, value BINARY, topic STRING, partition LONG, offset LONG, timestamp LONG

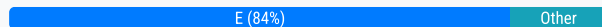
There are 5 unique topics being ingested. Only the "registration" topic contains Personal Identifiable Information (PII). The company wishes to restrict access to PII. The company also wishes to only retain records containing PII in this table for 14 days after initial ingestion. However, for non-PII information, it would like to retain these records indefinitely.

Which of the following solutions meets the requirements?

- A. All data should be deleted biweekly; Delta Lake's time travel functionality should be leveraged to maintain a history of non-PII information.
- B. Data should be partitioned by the registration field, allowing ACLs and delete statements to be set for the PII directory.
- C. Because the value field is stored as binary data, this information is not considered PII and no special precautions should be taken.
- D. Separate object storage containers should be specified based on the partition field, allowing isolation at the storage level.
- E. Data should be partitioned by the topic field, allowing ACLs and delete statements to leverage partition boundaries.

Suggested Answer: D

Community vote distribution



mouad_attaqi Highly Voted 2 years, 3 months ago

Selected Answer: E

I think answer E is correct, as by default partitioning by a column will create a separate folder for each subset data linked to the partition
upvoted 13 times

KadELbied Most Recent 9 months ago

Selected Answer: E

Surely E
upvoted 1 times

benni_ale 1 year, 2 months ago

Selected Answer: E

Partitioning by topic field let delete queries leverage partitioning boundaries
upvoted 2 times

benni_ale 1 year, 3 months ago

Selected Answer: E

E E E E
upvoted 1 times

ojudz08 1 year, 11 months ago

Selected Answer: D

i think it's best to isolate the storage to avoid mistakenly deleting tables in the same storage so I go with D
upvoted 1 times

spaceexplorer 2 years ago

Selected Answer: E

E is correct
upvoted 1 times

ervinshang 2 years, 1 month ago

Selected Answer: E

E is correct
upvoted 2 times

aragorn_brego 2 years, 2 months ago

Selected Answer: E

Partitioning data by the topic field would allow the data engineering team to apply access control lists (ACLs) to restrict access to the partition containing the "registration" topic, which holds PII. Furthermore, the team can set up automated deletion policies that specifically target the partition with PII data to delete records after 14 days, without affecting the data in other partitions. This approach meets both the privacy requirements for PII and the data retention goals for non-PII information.

upvoted 2 times

  **Dileepvikram** 2 years, 3 months ago

I think answer is E

upvoted 3 times

  **sturcu** 2 years, 3 months ago

Selected Answer: D

Correct

upvoted 1 times

  **sturcu** 2 years, 3 months ago

<https://docs.databricks.com/en/data-governance/table-acls/object-privileges.html#securable-objects>

upvoted 1 times

The data architect has decided that once data has been ingested from external sources into the Databricks Lakehouse, table access controls will be leveraged to manage permissions for all production tables and views.

The following logic was executed to grant privileges for interactive queries on a production database to the core engineering group.

```
GRANT USAGE ON DATABASE prod TO eng;
GRANT SELECT ON DATABASE prod TO eng;
```

Assuming these are the only privileges that have been granted to the eng group and that these users are not workspace administrators, which statement describes their privileges?

- A. Group members have full permissions on the prod database and can also assign permissions to other users or groups.
- B. Group members are able to list all tables in the prod database but are not able to see the results of any queries on those tables.
- C. Group members are able to query and modify all tables and views in the prod database, but cannot create new tables or views.
- D. Group members are able to query all tables and views in the prod database, but cannot create or edit anything in the database.
- E. Group members are able to create, query, and modify all tables and views in the prod database, but cannot define custom functions.

Suggested Answer: E

Community vote distribution

D (100%)

 **sturcu** Highly Voted 2 years, 3 months ago

Selected Answer: D

Usage and Selectsa abasically they can only select
upvoted 6 times

 **KadELbied** Most Recent 9 months ago

Selected Answer: D

Surety D
upvoted 1 times

 **benni_ale** 1 year, 3 months ago

Selected Answer: D

D is ok
upvoted 1 times

 **Curious76** 1 year, 11 months ago

Selected Answer: D

D is correct
upvoted 1 times

 **vctrhugo** 2 years ago

Selected Answer: D

The GRANT statements provided in the logic grant the USAGE privilege, allowing the group members to see the existence of the database, and the SELECT privilege, allowing them to query tables and views. However, they do not have permissions to create or edit anything in the database. Therefore, the correct description is that group members can query all tables and views in the prod database but cannot create or edit any objects in the database.
upvoted 1 times

 **divingbell17** 2 years, 1 month ago

Selected Answer: D

D is correct assuming unity catalog is not enabled
upvoted 1 times

 **aragorn_brego** 2 years, 2 months ago

Selected Answer: D

The GRANT USAGE ON DATABASE statement gives the eng group the ability to access the prod database. This means they can enter the database context and list the tables. The GRANT SELECT ON DATABASE statement additionally grants them permission to perform SELECT queries on all existing tables and views within the prod database. However, these privileges do not include creating new tables or views, modifying existing tables, or assigning permissions to other users or groups.

upvoted 4 times

  **Dileepvikram** 2 years, 3 months ago

D is answer

upvoted 4 times

A distributed team of data analysts share computing resources on an interactive cluster with autoscaling configured. In order to better manage costs and query throughput, the workspace administrator is hoping to evaluate whether cluster upscaling is caused by many concurrent users or resource-intensive queries.

In which location can one review the timeline for cluster resizing events?

- A. Workspace audit logs
- B. Driver's log file
- C. Ganglia
- D. Cluster Event Log
- E. Executor's log file

Suggested Answer: C

Community vote distribution

D (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: D

Surely D

upvoted 1 times

🗳️ 👤 **Curious76** 1 year, 5 months ago

Selected Answer: D

The Cluster Event Log provides detailed information about various events affecting the cluster throughout its lifecycle, including cluster creation, restarts, termination, and resizing events. It displays the timestamp, event type (e.g., "CLUSTER_RESIZED"), and relevant details for each event, allowing the administrator to review the timeline for cluster scaling behavior and identify potential patterns related to user activity or resource-intensive queries.

upvoted 3 times

🗳️ 👤 **vctrhugo** 1 year, 6 months ago

Selected Answer: D

The timeline for cluster resizing events can be reviewed in the Cluster Event Log. This log provides information about cluster scaling events, including when the cluster is scaled up or down. You can access this information to understand the reasons behind autoscaling events and whether they are triggered by many concurrent users or resource-intensive queries.

upvoted 1 times

🗳️ 👤 **alexvno** 1 year, 7 months ago

Selected Answer: D

Cluster event log

upvoted 2 times

🗳️ 👤 **aragorn_brego** 1 year, 8 months ago

Selected Answer: D

The Cluster Event Log in Databricks will show the timeline for cluster resizing events, including details about when and why a cluster was resized (scaled up or down). This log would help the workspace administrator determine the causes of cluster scaling, whether due to many concurrent users submitting jobs or a few users running resource-intensive queries.

less suitable:

C. Ganglia provides metrics on system-level performance, such as CPU and memory usage, but does not log specific cluster scaling events.

upvoted 2 times

🗳️ 👤 **PearApple** 1 year, 9 months ago

cluster event log. D

upvoted 2 times

🗳️ 👤 **sturcu** 1 year, 9 months ago

Selected Answer: D

Cluster Event Log

upvoted 3 times

When evaluating the Ganglia Metrics for a given cluster with 3 executor nodes, which indicator would signal proper utilization of the VM's resources?

- A. The five Minute Load Average remains consistent/flat
- B. Bytes Received never exceeds 80 million bytes per second
- C. Network I/O never spikes
- D. Total Disk Space remains constant
- E. CPU Utilization is around 75%

Suggested Answer: D

Community vote distribution

E (100%)

  **sturcu** Highly Voted 1 year, 9 months ago

Selected Answer: E

I would look at max CPU utilization and max Memory usage.
Having 75% CPU usage would signify we have a proper utilization of cpu resources
upvoted 7 times

  **KadELbied** Most Recent 9 months ago

Selected Answer: E

Surety E
upvoted 1 times

  **vctrhugo** 1 year, 6 months ago

Selected Answer: E

Proper utilization of VM resources, especially in a distributed computing environment like Spark, often involves efficient usage of CPU resources. A CPU utilization around 75% indicates that the CPU is being utilized without being fully saturated, allowing room for additional processing without causing excessive contention.
upvoted 2 times

  **alexvno** 1 year, 7 months ago

Selected Answer: E

75% good
upvoted 1 times

  **aragorn_brego** 1 year, 8 months ago

Selected Answer: E

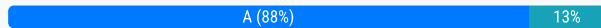
An average CPU utilization around 75% is a good indicator of proper utilization of the VM's resources in a distributed computing environment. It suggests that the CPUs are being actively used for computation without being maxed out, which could indicate a bottleneck. It leaves some headroom to handle additional load without causing excessive queuing or delays.
upvoted 3 times

Which of the following technologies can be used to identify key areas of text when parsing Spark Driver log4j output?

- A. Regex
- B. Julia
- C. pyspark.ml.feature
- D. Scala Datasets
- E. C++

Suggested Answer: E

Community vote distribution



🗳️ 👤 **vctrhugo** 1 year, 6 months ago

Selected Answer: A

It allows us to define patterns that match the structure of the log entries and capture relevant data.

upvoted 3 times

🗳️ 👤 **aragorn_brego** 1 year, 8 months ago

Selected Answer: A

Regular expressions (regex) can be used to identify and extract patterns from text data, which makes them very useful for parsing log files like the Spark Driver's log4j output. By defining specific regex patterns, you can search for error messages, timestamps, specific log levels, or any other text that follows a particular format within the log files.

upvoted 4 times

🗳️ 👤 **sturcu** 1 year, 9 months ago

Selected Answer: A

Regex to extract text

upvoted 3 times

🗳️ 👤 **sturcu** 1 year, 9 months ago

Selected Answer: E

Regex to extract text. C++ makes no sense in this context

upvoted 2 times

🗳️ 👤 **sturcu** 1 year, 9 months ago

I meant A

upvoted 2 times

🗳️ 👤 **hm358** 1 year, 9 months ago

Selected Answer: A

regex is for string identification

upvoted 2 times

🗳️ 👤 **mouad_attaqi** 1 year, 9 months ago

Selected Answer: A

Using regex, we can identify key ans values areas

upvoted 4 times

🗳️ 👤 **sturcu** 1 year, 9 months ago

Why C++, why not python or Java? Plus there are tools om parsing the log4j output like Chainsaw and xmlstarlet.

upvoted 1 times

You are testing a collection of mathematical functions, one of which calculates the area under a curve as described by another function.

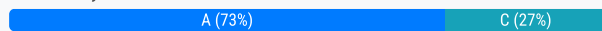
```
assert(myIntegrate(lambda x: x*x, 0, 3) [0] == 9)
```

Which kind of test would the above line exemplify?

- A. Unit
- B. Manual
- C. Functional
- D. Integration
- E. End-to-end

Suggested Answer: A

Community vote distribution



🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: A

Surely A

upvoted 1 times

🗳️ 👤 **Nickff** 1 year, 9 months ago

Selected Answer: A

Answer is A, unit test

upvoted 3 times

🗳️ 👤 **barnac1es** 1 year, 10 months ago

Selected Answer: C

I think it should be Functional Test

upvoted 3 times

🗳️ 👤 **HelixAbdu** 1 year, 6 months ago

There are 3 testing types:

Unit testing

Integration testing

And end to end testing

upvoted 5 times

🗳️ 👤 **vctrhugo** 2 years ago

Selected Answer: A

A. Unit

upvoted 3 times

🗳️ 👤 **divingbell17** 2 years, 1 month ago

Selected Answer: A

A is correct

upvoted 3 times

A Databricks job has been configured with 3 tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on Task A.

If task A fails during a scheduled run, which statement describes the results of this run?

- A. Because all tasks are managed as a dependency graph, no changes will be committed to the Lakehouse until all tasks have successfully been completed.
- B. Tasks B and C will attempt to run as configured; any changes made in task A will be rolled back due to task failure.
- C. Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task A failed, all commits will be rolled back automatically.
- D. Tasks B and C will be skipped; some logic expressed in task A may have been committed before task failure.
- E. Tasks B and C will be skipped; task A will not commit any changes because of stage failure.

Suggested Answer: C

Community vote distribution

D (100%)

🗳️ 👤 **mouad_attaqi** Highly Voted 1 year, 3 months ago

Selected Answer: D

D is correct, tasks B and C will definitely be skipped, since Task A is notebook, the ACID logic is at cell level, some logic might be executed before failing cell

upvoted 6 times

🗳️ 👤 **KadELbied** Most Recent 9 months ago

Selected Answer: D

Surely D

upvoted 1 times

🗳️ 👤 **aragorn_brego** 1 year, 2 months ago

Selected Answer: D

In Databricks job execution, if a task that other tasks depend on fails, the dependent tasks will not be executed. Since Tasks B and C depend on the successful completion of Task A, they will be skipped if Task A fails. However, if Task A performs any operations that commit changes before the failure occurs (such as writing to a Delta table), those changes remain and are not automatically rolled back unless the logic within Task A specifically includes rollback mechanisms for partial failures.

upvoted 4 times

🗳️ 👤 **Dileepvikram** 1 year, 3 months ago

D is the answer

upvoted 3 times

🗳️ 👤 **sturcu** 1 year, 3 months ago

Selected Answer: D

Some ops in task A may have finished before fail

upvoted 3 times

Which statement regarding Spark configuration on the Databricks platform is true?

- A. The Databricks REST API can be used to modify the Spark configuration properties for an interactive cluster without interrupting jobs currently running on the cluster.
- B. Spark configurations set within a notebook will affect all SparkSessions attached to the same interactive cluster.
- C. Spark configuration properties can only be set for an interactive cluster by creating a global init script.
- D. Spark configuration properties set for an interactive cluster with the Clusters UI will impact all notebooks attached to that cluster.
- E. When the same Spark configuration property is set for an interactive cluster and a notebook attached to that cluster, the notebook setting will always be ignored.

Suggested Answer: D

Community vote distribution

D (85%)

Other

  **hamzaKhribi** Highly Voted 2 years, 2 months ago

Selected Answer: D

I tried it myself, setting a spark conf on the cluster ui, will impact all notebooks attached to that cluster, for example i set the number of shuffle partitions to 4, and in every notebook when i inspect the number of partitions i find 4.
upvoted 8 times

  **KadELbied** Most Recent 9 months ago

Selected Answer: D

suretly D
upvoted 1 times

  **Sriramiyer92** 1 year, 1 month ago

Selected Answer: D

D is correct.
Point to Note:
Precedence - (Spark configurations set programmatically in a notebook or script) > (Cluster-wide configurations set in the Clusters UI)
upvoted 2 times

  **benni_ale** 1 year, 2 months ago

Selected Answer: D

Both via API or UI will trigger a restart of the cluster so A is wrong. D is correct given that "impact" does not mean the cluster can't be restarted.
upvoted 1 times

  **Curious76** 1 year, 11 months ago

Selected Answer: B

A. Incorrect: Modifying configurations through the Databricks REST API while jobs are running can lead to unexpected behavior or disruption. It's generally not recommended.
C. Incorrect: While global init scripts can be used, it's not the only way. Configurations can also be set within notebooks.
D. Incorrect: Configurations set through the Clusters UI apply to the entire cluster, but they might not necessarily override configurations set within notebooks attached to the cluster.
E. Incorrect: Notebook configurations can take precedence over cluster-level configurations for the same property, offering finer-grained control at the notebook level.
upvoted 1 times

  **vctrhugo** 2 years ago

Selected Answer: D

These settings are applied at the cluster level and affect all SparkSessions on the cluster.
upvoted 3 times

  **alexvno** 2 years, 1 month ago

Selected Answer: D

A wrong, cluster will restart -> D

upvoted 4 times

  **petrv** 2 years, 2 months ago

Selected Answer: A

In Databricks, you can use the Databricks REST API to modify Spark configuration properties for an interactive cluster without interrupting currently running jobs. This allows you to dynamically adjust Spark configurations to optimize performance or meet specific requirements without the need to restart the cluster.

upvoted 1 times

  **alexvno** 2 years, 1 month ago

wrong, cluster will restart

upvoted 1 times

  **petrv** 2 years, 2 months ago

If you update the configuration of a cluster using the Databricks REST API or the Clusters UI while the cluster is in a RUNNING state, the cluster will be restarted to apply the new configuration. However, Databricks typically handles this situation in a way that minimizes disruption to running jobs.

upvoted 2 times

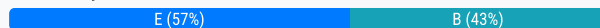
A developer has successfully configured their credentials for Databricks Repos and cloned a remote Git repository. They do not have privileges to make changes to the main branch, which is the only branch currently visible in their workspace.

Which approach allows this user to share their code updates without the risk of overwriting the work of their teammates?

- A. Use Repos to checkout all changes and send the git diff log to the team.
- B. Use Repos to create a fork of the remote repository, commit all changes, and make a pull request on the source repository.
- C. Use Repos to pull changes from the remote Git repository; commit and push changes to a branch that appeared as changes were pulled.
- D. Use Repos to merge all differences and make a pull request back to the remote repository.
- E. Use Repos to create a new branch, commit all changes, and push changes to the remote Git repository.

Suggested Answer: E

Community vote distribution



KadELbied 9 months ago

Selected Answer: E

Surely E

upvoted 1 times

AlejandroU 1 year, 1 month ago

Selected Answer: E

Answer E. Creating a new branch within the repository is the most effective and recommended approach for developers who lack write access to the main branch in Databricks Repos. While forking is a valid strategy in some situations, it's generally less common and potentially more complex within the context of Databricks Repos, especially when the primary goal is to contribute to the existing shared repository.

upvoted 1 times

benni_ale 1 year, 3 months ago

Selected Answer: E

I changed my mind i think is E

upvoted 1 times

benni_ale 1 year, 3 months ago

Selected Answer: B

Explanation:

The developer does not have privileges to make changes to the main branch of the remote repository, and it's the only branch visible in their workspace. To share their code updates without risking overwriting their teammates' work, the best approach is to:

Create a personal copy (fork) of the remote repository. This forked repository will be under the developer's own account or workspace, allowing full control over it.

Make Changes in the Fork:

Commit all code updates to the forked repository. Since the developer has full privileges on their fork, they can create branches, commit changes, and manage the repository as needed.

Option E: Creating a new branch and pushing changes to the remote repository requires write access to the repository, which the developer does not have.

By forking the repository, the developer avoids any permission issues and ensures that their work does not interfere with the main codebase until it is reviewed and approved by the team.

upvoted 1 times

benni_ale 1 year, 3 months ago

My bad i think E is correct

upvoted 1 times

🗨️ 👤 **hal2401me** 1 year, 11 months ago

Selected Answer: E

E is the regular collaboration approach.

B makes a fork so breaks away from the collaborating teammates. There's no way they can make a pull request on the source repository after making change to a fork.

upvoted 4 times

🗨️ 👤 **hal2401me** 1 year, 11 months ago

Sorry, my mistake, just tested B is a do-able way. Fork from github can create pull request against the original repository and contribute back. B seems to be a better answer.

upvoted 3 times

🗨️ 👤 **Curious76** 1 year, 11 months ago

Selected Answer: B

Databricks Repos itself does not currently support creating forks directly within the platform. However, you can achieve a similar workflow using the following steps:

Use the git clone command in a terminal or IDE to create a local copy of the remote repository. This effectively creates a local fork.

Make your changes in the local copy.

Use git push to push your changes to a new remote repository you create on a Git hosting service like GitHub.

Create a pull request from your new remote repository to the original repository on Databricks Repos.

This approach allows you to make changes to your own copy of the code, collaborate with others through code reviews, and propose your changes for integration into the main branch without directly modifying it

upvoted 2 times

🗨️ 👤 **carlosmps** 1 year, 6 months ago

The argument provided suggests using a traditional Git workflow that involves forking a repository on a Git hosting service like GitHub, making changes, and then creating a pull request. This is a valid and widely used approach in standard Git workflows. However, it is essential to distinguish between the capabilities and features provided directly by Databricks Repos and the general Git practices that can be applied outside of Databricks.

upvoted 3 times

🗨️ 👤 **Sriramiyer92** 1 year, 1 month ago

Exactly my thought as well!

upvoted 1 times

🗨️ 👤 **vctrhugo** 2 years ago

Selected Answer: E

This is a common workflow in collaborative development environments. The developer can create a new branch in their local repository, make changes, and then push the branch to the remote repository. This way, they can share their updates without modifying the main branch directly. After pushing the changes, they can create a pull request on the remote repository, allowing their teammates to review the changes before merging them into the main branch. This process ensures that the main branch remains stable and that all changes are reviewed and approved before they're incorporated. It also prevents any accidental overwrites of teammates' work.

upvoted 3 times

🗨️ 👤 **PrincipalJoe** 2 years ago

Databricks Repos cannot be used to fork a repository

upvoted 3 times

🗨️ 👤 **adenis** 2 years ago

Selected Answer: E

E is correct

upvoted 3 times

🗨️ 👤 **Rinscy** 2 years ago

E is correct

if you create a fork you create another repository so not B

upvoted 2 times

🗨️ 👤 **Crocjun** 2 years ago

Selected Answer: B

B is correct

upvoted 2 times

  **spaceexplorer** 2 years ago

Selected Answer: B

B is correct

upvoted 2 times

  **dmov** 2 years, 1 month ago

Isn't this B?

upvoted 1 times

In order to prevent accidental commits to production data, a senior data engineer has instituted a policy that all development work will reference clones of Delta Lake tables. After testing both DEEP and SHALLOW CLONE, development tables are created using SHALLOW CLONE.

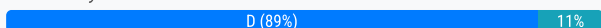
A few weeks after initial table creation, the cloned versions of several tables implemented as Type 1 Slowly Changing Dimension (SCD) stop working. The transaction logs for the source tables show that VACUUM was run the day before.

Which statement describes why the cloned tables are no longer working?

- A. Because Type 1 changes overwrite existing records, Delta Lake cannot guarantee data consistency for cloned tables.
- B. Running VACUUM automatically invalidates any shallow clones of a table; DEEP CLONE should always be used when a cloned table will be repeatedly queried.
- C. Tables created with SHALLOW CLONE are automatically deleted after their default retention threshold of 7 days.
- D. The metadata created by the CLONE operation is referencing data files that were purged as invalid by the VACUUM command.
- E. The data files compacted by VACUUM are not tracked by the cloned metadata; running REFRESH on the cloned table will pull in recent changes.

Suggested Answer: D

Community vote distribution



alexvno Highly Voted 2 years, 1 month ago

Selected Answer: D

Shallow clone: only duplicates the metadata of the table being cloned; the data files of the table itself are not copied. These clones are cheaper to create but are not self-contained and depend on the source from which they were cloned as the source of data. If the files in the source that the clone depends on are removed, for example with VACUUM, a shallow clone may become unusable. Therefore, shallow clones are typically used for short-lived use cases such as testing and experimentation.

upvoted 9 times

benni_ale Most Recent 1 year, 3 months ago

I was not sure whether B or D but somehow I think that running VACUUM comand does not invalidate SHALLOW CLONES . I mean its just that the data referenced by the clone is no longer present. It can still happen that a SHALLOW CLONE is working even after a VACUUM command run on the cloned table (origin) . So B is not completely correct.

upvoted 2 times

vctrhugo 2 years ago

Selected Answer: D

In Delta Lake, the VACUUM command deletes data files that are no longer referenced by a Delta table and are older than the retention threshold. When a table is cloned using SHALLOW CLONE, the clone references the same data files as the original table but creates a new transaction log. If VACUUM is run on the original table, it can delete data files that are still being referenced by the cloned table's metadata, causing the cloned table to stop working. This is because the VACUUM command doesn't know about the cloned table's references to the data files. Therefore, it's important to be cautious when running VACUUM on tables that have clones.

upvoted 3 times

spaceexplorer 2 years ago

Selected Answer: D

D is correct

upvoted 1 times

AzureDE2522 2 years, 2 months ago

Selected Answer: D

Please refer:

<https://docs.databricks.com/en/delta/clone.html#what-are-the-semantics-of-delta-clone-operations>

upvoted 2 times

60ties 2 years, 2 months ago

Selected Answer: B

B is best

upvoted 2 times

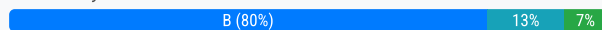
You are performing a join operation to combine values from a static userLookup table with a streaming DataFrame streamingDF.

Which code block attempts to perform an invalid stream-static join?

- A. `userLookup.join(streamingDF, ["userid"], how="inner")`
- B. `streamingDF.join(userLookup, ["user_id"], how="outer")`
- C. `streamingDF.join(userLookup, ["user_id"], how="left")`
- D. `streamingDF.join(userLookup, ["userid"], how="inner")`
- E. `userLookup.join(streamingDF, ["user_id"], how="right")`

Suggested Answer: E

Community vote distribution



Enduresoul Highly Voted 1 year, 8 months ago

Selected Answer: B

Answer B is correct:

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#support-matrix-for-joins-in-streaming-queries>

When we take a look in the supported join matrix between static and stream inputs, we can identify, that Stream-Static + outer is not supported.

Answer E is wrong, because the Static-Stream + right join is supported.

upvoted 11 times

KadELbied Most Recent 9 months, 1 week ago

Selected Answer: B

Surely B

upvoted 1 times

lene 1 year ago

Selected Answer: B

according to the Support matrix for joins in streaming queries

upvoted 1 times

AlejandroU 1 year, 1 month ago

Selected Answer: B

Answer B. We can directly discard options C and D since the streaming DataFrame (streamingDF) must be the left table in the join and the join type must be inner join or left outer join. Thus, the most directly invalid code block due to an unsupported join type is B.

upvoted 1 times

imatheushenrique 1 year, 2 months ago

B.

We match all the records from a static DataFrame on the left with a stream DataFrame on the right. If records do not match from the static DF (Left) to stream DF (Right), then the system cannot return null since the data changes on stream DF (Right), and we cannot guarantee if we will get matching records. That is why full_outer join is not supported.

upvoted 2 times

hal2401me 1 year, 4 months ago

Selected Answer: E

in my exam today, BCD are removed. i chose E, because I recall that stream-static right join are less supported.

upvoted 4 times

Curious76 1 year, 5 months ago

Selected Answer: B

b is correct

upvoted 1 times

vctrhugo 1 year, 6 months ago

Selected Answer: B

Specifically, outer joins are not supported with a static DataFrame on the right and a streaming DataFrame on the left. This is because it's not possible to guarantee all necessary rows will be available in the streaming DataFrame for every micro-batch.

upvoted 1 times

  **kz_data** 1 year, 7 months ago

Selected Answer: D

I think the correct answer is D.

upvoted 1 times

  **kz_data** 1 year, 7 months ago

Sorry I missread the question.

upvoted 1 times

  **lexaneon** 1 year, 7 months ago

Selected Answer: B

believe B is correct as provided below

upvoted 2 times

Spill occurs as a result of executing various wide transformations. However, diagnosing spill requires one to proactively look for key indicators.

Where in the Spark UI are two of the primary indicators that a partition is spilling to disk?

- A. Query's detail screen and Job's detail screen
- B. Stage's detail screen and Executor's log files
- C. Driver's and Executor's log files
- D. Executor's detail screen and Executor's log files
- E. Stage's detail screen and Query's detail screen

Suggested Answer: B

Community vote distribution

B (90%)

10%

60ties Highly Voted 2 years, 2 months ago

Selected Answer: B

B is correct

upvoted 7 times

vctrhugo Highly Voted 2 years ago

Selected Answer: B

In the Spark UI, the Stage's detail screen provides key metrics about each stage of a job, including the amount of data that has been spilled to disk. If you see a high number in the "Spill (Memory)" or "Spill (Disk)" columns, it's an indication that a partition is spilling to disk.

The Executor's log files can also provide valuable information about spill. If a task is spilling a lot of data, you'll see messages in the logs like "Spilling UnsafeExternalSorter to disk" or "Task memory spill". These messages indicate that the task ran out of memory and had to spill data to disk.

upvoted 5 times

KadELbied Most Recent 9 months ago

Selected Answer: B

Surely B

upvoted 1 times

dd1192d 1 year, 4 months ago

Selected Answer: B

b

upvoted 1 times

jin1991 2 years, 1 month ago

Selected Answer: E

E is correct

upvoted 2 times

jin1991 2 years, 1 month ago

My bad, looking again, B is correct.

upvoted 2 times

A task orchestrator has been configured to run two hourly tasks. First, an outside system writes Parquet data to a directory mounted at `/mnt/raw_orders/`. After this data is written, a Databricks job containing the following code is executed:

```
(spark.readStream
  .format("parquet")
  .load("/mnt/raw_orders/")
  .withWatermark("time", "2 hours")
  .dropDuplicates(["customer_id", "order_id"])
  .writeStream
  .trigger(once=True)
  .table("orders")
)
```

Assume that the fields `customer_id` and `order_id` serve as a composite key to uniquely identify each order, and that the time field indicates when the record was queued in the source system.

If the upstream system is known to occasionally enqueue duplicate entries for a single order hours apart, which statement is correct?

- A. Duplicate records enqueued more than 2 hours apart may be retained and the orders table may contain duplicate records with the same `customer_id` and `order_id`.
- B. All records will be held in the state store for 2 hours before being deduplicated and committed to the orders table.
- C. The orders table will contain only the most recent 2 hours of records and no duplicates will be present.
- D. Duplicate records arriving more than 2 hours apart will be dropped, but duplicates that arrive in the same batch may both be written to the orders table.
- E. The orders table will not contain duplicates, but records arriving more than 2 hours late will be ignored and missing from the table.

Suggested Answer: A

Community vote distribution

A (78%)

E (22%)

 alexvno Highly Voted 2 years, 1 month ago

Selected Answer: A

Only A seems logical
upvoted 8 times

 Isio05 Highly Voted 1 year, 8 months ago

Selected Answer: A

It's A, rows are deduplicated only in 2hrs window, therefore final table may eventually contain duplicates
upvoted 5 times

 KadELbied Most Recent 9 months ago

Selected Answer: A

Surely A
upvoted 1 times

 a85becd 10 months, 1 week ago

Selected Answer: E

- The code snippet uses `withWatermark("time", "2 hours")`, which means that Spark will only process records with a timestamp (`time`` field) that falls within a 2-hour window from the most recent watermark. Any records arriving `more than 2 hours late` will be ignored and won't be written to the `orders`` table.

- The `dropDuplicates(["customer_id", "order_id"])` ensures that duplicate records (based on the composite key of `customer_id`` and `order_id``) are removed `within the 2-hour window`. This guarantees no duplicates will be present in the `orders`` table for data falling within the watermark range.

upvoted 3 times

🗨️ **EZZALDIN** 10 months, 2 weeks ago

Selected Answer: E

E. The orders table will not contain duplicates, but records arriving more than 2 hours late will be ignored and missing from the table.

This matches Spark's actual behavior with watermark + dropDuplicates: the stateful deduplication ensures no duplicate (customer_id, order_id) rows are ever written to the Delta table, and the 2-hour watermark means any record that comes in more than 2 hours behind the stream's current event-time will be dropped (i.e. not processed/output)

upvoted 2 times

🗨️ **RandomForest** 1 year ago

Selected Answer: A

A is correct. The watermark ensures that data arriving up to 2h late will be deduplicated in the incoming batch.

upvoted 1 times

🗨️ **arekm** 1 year, 1 month ago

Selected Answer: A

A - since the same [customer_id, order_id] record can be emitted twice with the "time" value set apart by more than 2 hours. The target table will then contain duplicates (duplicates from the perspective of the composite key).

upvoted 1 times

🗨️ **benni_ale** 1 year, 2 months ago

Per me è la cipolla

upvoted 1 times

🗨️ **cf56faf** 1 year, 2 months ago

Selected Answer: E

Seems that E should be the correct answer.

As time is the time it was queued in the *source_system*.

And withWatermark ignores records that have a "time" more than 2 hours old.

upvoted 2 times

🗨️ **Ananth4Sap** 1 year, 3 months ago

When watermarking is set to 2 hours, the system will wait for up to 2 hours for late data to arrive. Any data that arrives within this 2-hour window will be considered for processing and deduplication. However, data that arrives later than 2 hours after the event time will be considered too late and will be discarded. This ensures that the state store does not grow indefinitely, but it also means that any records arriving more than 2 hours late will not be included in the orders table.

upvoted 2 times

🗨️ **m79590530** 1 year, 3 months ago

Selected Answer: E

Every Stream micro-batch is executed on all of the new data that arrived after the last run 2 hours ago by the .trigger(once=True) option.

Deduplication is done for it based on the combined key fields values but all records older than 2 hours based on the 'time' field will be ignored thanks to the .withWatermark() option/function. So target table will have deduplicated data withOUT the late records arriving more than 2 hours later based on the 2 hours watermark buffer set for the readStream.

upvoted 2 times

🗨️ **arekm** 1 year, 1 month ago

Deduplication happens each hour (when the job is run) and contains records from the last 2 hours based on the time column.

However, the source system might generate [customer_id, order_id] combination with a newer time value set apart more than 2 hours from the original record. It will be a new record, however from the key perspective a duplicate. Since deduplication happens in 2h windows, the new record will be appended (default for structured streaming) to the target table.

upvoted 2 times

🗨️ **shaojunni** 1 year, 3 months ago

Selected Answer: E

Data arrive outside of watermark will be dropped.

upvoted 2 times

🗨️ **quaternion** 1 year, 5 months ago

Selected Answer: E

Watermark("time", "2 hours") --> doesn't let records arriving more than 2 hours late to be written

dropDuplicates --> removes duplicate records from the records that are read

upvoted 3 times

  **QuangTrinh** 1 year, 8 months ago

Selected Answer: E

Should be E.

Watermarking (withWatermark("time", "2 hours")): This sets a 2-hour watermark on the time column. The watermark specifies the event time threshold for data completeness, meaning that data older than 2 hours will be considered late and may be dropped.

Deduplication (dropDuplicates(["customer_id", "order_id"])): This operation removes duplicates based on the composite key (customer_id and order_id). However, it only works within the window defined by the watermark.

upvoted 1 times

  **arekm** 1 year, 1 month ago

Deduplication happens each hour (when the job is run) and contains records from the last 2 hours based on the time column. However, the source system might generate [customer_id, order_id] combination with a newer time value set apart more than 2 hours from the original record. It will be a new record, however from the key perspective a duplicate. Since deduplication happens in 2h windows, the new record will be appended (default for structured streaming) to the target table.

upvoted 2 times

A junior data engineer is migrating a workload from a relational database system to the Databricks Lakehouse. The source system uses a star schema, leveraging foreign key constraints and multi-table inserts to validate records on write.

Which consideration will impact the decisions made by the engineer while migrating this workload?

- A. Databricks only allows foreign key constraints on hashed identifiers, which avoid collisions in highly-parallel writes.
- B. Databricks supports Spark SQL and JDBC; all logic can be directly migrated from the source system without refactoring.
- C. Committing to multiple tables simultaneously requires taking out multiple table locks and can lead to a state of deadlock.
- D. All Delta Lake transactions are ACID compliant against a single table, and Databricks does not enforce foreign key constraints.
- E. Foreign keys must reference a primary key field; multi-table inserts must leverage Delta Lake's upsert functionality.

Suggested Answer: D

Community vote distribution

D (100%)

  **vctrhugo**  1 year, 6 months ago

Selected Answer: D

In Databricks Delta Lake, transactions are ACID compliant at the table level, meaning that transactions apply to a single table. However, Delta Lake does not enforce foreign key constraints across tables. Therefore, the data engineer needs to be aware that Databricks does not automatically enforce referential integrity between tables through foreign key constraints, and it becomes the responsibility of the data engineer to manage these relationships appropriately.

upvoted 7 times

  **KadELbied**  9 months ago

Selected Answer: D

Surely D

upvoted 1 times

  **alexvno** 1 year, 7 months ago

Selected Answer: D

Primary and foreign keys are informational only and are not enforced.

upvoted 2 times

  **60ties** 1 year, 8 months ago

Selected Answer: D

D makes more sense.

Since there are no database-level transactions, locks, or guarantees, and since primary key & foreign key constraints are informational only, there is no guarantee of enforced relations (the start schema) in place will remain in place after migration. This means B cannot be right.

upvoted 1 times

A data architect has heard about Delta Lake's built-in versioning and time travel capabilities. For auditing purposes, they have a requirement to maintain a full record of all valid street addresses as they appear in the customers table.

The architect is interested in implementing a Type 1 table, overwriting existing records with new values and relying on Delta Lake time travel to support long-term auditing. A data engineer on the project feels that a Type 2 table will provide better performance and scalability.

Which piece of information is critical to this decision?

- A. Data corruption can occur if a query fails in a partially completed state because Type 2 tables require setting multiple fields in a single update.
- B. Shallow clones can be combined with Type 1 tables to accelerate historic queries for long-term versioning.
- C. Delta Lake time travel cannot be used to query previous versions of these tables because Type 1 changes modify data files in place.
- D. Delta Lake time travel does not scale well in cost or latency to provide a long-term versioning solution.
- E. Delta Lake only supports Type 0 tables; once records are inserted to a Delta Lake table, they cannot be modified.

Suggested Answer: D

Community vote distribution

D (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: D

Surely D

upvoted 1 times

🗳️ 👤 **_lene_** 1 year ago

Selected Answer: D

Vacuuming restricts time travel capability

upvoted 1 times

🗳️ 👤 **Sriramiyer92** 1 year, 1 month ago

Selected Answer: D

Also performing a Vacuum Op on Data will not suite well with the requirement in the long run.

upvoted 3 times

🗳️ 👤 **vish9** 1 year, 2 months ago

Selected Answer: D

D makes sense

upvoted 1 times

🗳️ 👤 **KB_Ai_Champ** 1 year, 4 months ago

its C time travel cant be performed on type 1

upvoted 1 times

🗳️ 👤 **vctrhugo** 2 years ago

Selected Answer: D

Delta Lake's time travel feature allows you to access previous versions of the data, which can be useful for auditing purposes. However, if you're planning to use time travel as a long-term versioning solution, it's important to know that it may not scale well in terms of cost or latency. This is because every time you perform a write operation, a new version of the data is created, which can consume significant storage over time.

Additionally, querying older versions of the data may require scanning through many files, which can increase query latency.

upvoted 4 times

🗳️ 👤 **60ties** 2 years, 2 months ago

Selected Answer: D

D makes more sense

upvoted 4 times

A table named user_ltv is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The user_ltv table has the following schema:

email STRING, age INT, ltv INT

The following view definition is executed:

```
CREATE VIEW user_ltv_no_minors AS
SELECT email, age, ltv
FROM user_ltv
WHERE
  CASE
    WHEN is_member("auditing") THEN TRUE
    ELSE age >= 18
  END
```

An analyst who is not a member of the auditing group executes the following query:

```
SELECT * FROM user_ltv_no_minors
```

Which statement describes the results returned by this query?

- A. All columns will be displayed normally for those records that have an age greater than 17; records not meeting this condition will be omitted.
- B. All age values less than 18 will be returned as null values, all other columns will be returned with the values in user_ltv.
- C. All values for the age column will be returned as null values, all other columns will be returned with the values in user_ltv.
- D. All records from all columns will be displayed with the values in user_ltv.
- E. All columns will be displayed normally for those records that have an age greater than 18; records not meeting this condition will be omitted.

Suggested Answer: A

Community vote distribution

A (92%)

8%

 **dmov** Highly Voted 2 years, 1 month ago

Selected Answer: A

Definitely A. It's greater than or equal to
upvoted 9 times

 **KadELbied** Most Recent 9 months ago

Selected Answer: A

Surely A
upvoted 1 times

 **91d511b** 1 year ago

Selected Answer: E

>=18 means that the analyst that is NOT in the auditing group will only see records greater than or equal to 18.
upvoted 1 times

 **arekm** 1 year, 1 month ago

Selected Answer: A

A - at first I thought that was a silly answer since 17.5 is still greater than 17, but not really what the condition says. However, if you look at the schema - the age is an integer - you can only have ..., 17, 18, 19, ...

upvoted 2 times

🗨️ 👤 **Sriramiyer92** 1 year, 1 month ago

Selected Answer: A

Simple - Play of Words in option A and E. Read the option carefully.

upvoted 1 times

🗨️ 👤 **c00ccb7** 1 year, 7 months ago

Selected Answer: A

Because greater than 18 doesnt include 18

All columns will be displayed normally for those records that have an age greater than 18; records not meeting this condition will be omitted.

upvoted 1 times

🗨️ 👤 **Tamele001** 1 year, 11 months ago

Option E

upvoted 1 times

🗨️ 👤 **Freyr** 1 year, 8 months ago

Incorrect because the condition specified is age ≥ 18 , not age > 18 . So, the answer is A.

upvoted 2 times

🗨️ 👤 **divingbell17** 2 years, 1 month ago

Selected Answer: A

A is correct. "greater than 17" is the equivalent to "equal or greater than 18"

upvoted 4 times

🗨️ 👤 **sodere** 2 years, 1 month ago

Selected Answer: E

18 not 17.

upvoted 2 times

🗨️ 👤 **sodere** 2 years, 1 month ago

A is right.

upvoted 3 times

🗨️ 👤 **vctrhugo** 2 years ago

≥ 18 (greather than 17)

upvoted 3 times

The data governance team is reviewing code used for deleting records for compliance with GDPR. The following logic has been implemented to propagate delete requests from the user_lookup table to the user_aggregates table.

```
(spark.read
  .format("delta")
  .option("readChangeData", True)
  .option("startingTimestamp", '2021-08-22 00:00:00')
  .option("endingTimestamp", '2021-08-29 00:00:00')
  .table("user_lookup")
  .createOrReplaceTempView("changes"))

spark.sql("""
DELETE FROM user_aggregates
WHERE user_id IN (
  SELECT user_id
  FROM changes
  WHERE _change_type='delete'
)
""")
```

Assuming that user_id is a unique identifying key and that all users that have requested deletion have been removed from the user_lookup table, which statement describes whether successfully executing the above logic guarantees that the records to be deleted from the user_aggregates table are no longer accessible and why?

- A. No; the Delta Lake DELETE command only provides ACID guarantees when combined with the MERGE INTO command.
- B. No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.
- C. Yes; the change data feed uses foreign keys to ensure delete consistency throughout the Lakehouse.
- D. Yes; Delta Lake ACID guarantees provide assurance that the DELETE command succeeded fully and permanently purged these records.
- E. No; the change data feed only tracks inserts and updates, not deleted records.

Suggested Answer: B

Community vote distribution

B (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: B

Suretly B

upvoted 1 times

🗳️ 👤 **_lene_** 1 year ago

Selected Answer: B

vacuum prevents delta travel

upvoted 1 times

🗳️ 👤 **Hadiler** 1 year, 6 months ago

Selected Answer: B

B records will be available in time travel until VACUUM will be executed

upvoted 2 times

🗳️ 👤 **alexvno** 2 years, 1 month ago

Selected Answer: B

Delta travel

upvoted 2 times

🗳️ 👤 **60ties** 2 years, 2 months ago

Selected Answer: B

B is best.

VACUUM command is needed to completely remove logs of the deleted files.

upvoted 2 times

The data engineering team has been tasked with configuring connections to an external database that does not have a supported native connector with Databricks. The external database already has data security configured by group membership. These groups map directly to user groups already created in Databricks that represent various teams within the company.

A new login credential has been created for each group in the external database. The Databricks Utilities Secrets module will be used to make these credentials available to Databricks users.

Assuming that all the credentials are configured correctly on the external database and group membership is properly configured on Databricks, which statement describes how teams can be granted the minimum necessary access to using these credentials?

- A. "Manage" permissions should be set on a secret key mapped to those credentials that will be used by a given team.
 - B. "Read" permissions should be set on a secret key mapped to those credentials that will be used by a given team.
 - C. "Read" permissions should be set on a secret scope containing only those credentials that will be used by a given team.
 - D. "Manage" permissions should be set on a secret scope containing only those credentials that will be used by a given team.
- No additional configuration is necessary as long as all users are configured as administrators in the workspace where secrets have been added.

Suggested Answer: B

Community vote distribution

C (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: C

Surely C

upvoted 1 times

🗳️ 👤 **vctrhugo** 1 year, 6 months ago

Selected Answer: C

In Databricks, secret scopes are used to manage and organize secrets. By setting "Read" permissions on a secret scope containing the credentials, you allow the team to access the necessary credentials without granting unnecessary privileges. This approach ensures that the teams have the minimum necessary access to the credentials required for connecting to the external database. "Manage" permissions would provide more access than needed for just using the credentials.

Option A and D suggest setting permissions on individual secret keys, which might work, but using a secret scope for organizational purposes is a cleaner and more scalable solution.

upvoted 4 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Answer C.

You cannot manage permissions on secret keys, only on secret scopes: <https://docs.databricks.com/en/security/auth/access-control/index.html#secret-acls>

upvoted 1 times

🗳️ 👤 **Somesh512** 1 year, 6 months ago

Selected Answer: C

Access is at scope level and not key level

upvoted 2 times

🗳️ 👤 **petrv** 1 year, 8 months ago

Selected Answer: C

In summary, while technically feasible, setting "Read" permissions on a secret key might not be the most efficient or scalable solution when dealing with multiple teams and their corresponding credentials. Using secret scopes provides a more organized and maintainable approach for managing secrets in Databricks.

upvoted 1 times

  **Enduresoul** 1 year, 8 months ago

Selected Answer: C

Answer C is correct:

<https://docs.databricks.com/en/security/auth-authz/access-control/secret-acl.html#secret-access-control>

"Access control for secrets is managed at the secret scope level"

upvoted 3 times

Which indicators would you look for in the Spark UI's Storage tab to signal that a cached table is not performing optimally? Assume you are using Spark's MEMORY_ONLY storage level.

- A. Size on Disk is < Size in Memory
- B. The RDD Block Name includes the "*" annotation signaling a failure to cache
- C. Size on Disk is > 0
- D. The number of Cached Partitions > the number of Spark Partitions
- E. On Heap Memory Usage is within 75% of Off Heap Memory Usage

Suggested Answer: C

Community vote distribution

C (100%)

  **vctrhugo** Highly Voted 2 years ago

Selected Answer: C

C. Size on Disk is > 0

When using Spark's MEMORY_ONLY storage level, the ideal scenario is that the data is fully cached in memory, and the Size on Disk should be 0 (indicating that the data is not spilled to disk). If the Size on Disk is greater than 0, it suggests that some data has been spilled to disk, which can lead to degraded performance as reading from disk is slower than reading from memory.

upvoted 7 times

  **Billybob0604** Most Recent 6 months, 1 week ago

Selected Answer: B

In the Spark UI's Storage tab, when you're using the MEMORY_ONLY storage level, Spark tries to cache the RDD/table completely in memory. If a partition of the RDD does not fit into memory, Spark does not cache that partition and recomputes it when needed.

Indicator: * in the Block Name

A * (asterisk) next to an RDD block name in the Storage tab indicates that Spark failed to cache that block.

upvoted 3 times

  **gloomy_marmot** 6 months, 2 weeks ago

Selected Answer: B

In the Spark UI's Storage tab, an indicator that a cached table is not performing optimally would be the presence of the _disk annotation in the RDD Block Name. This annotation indicates that some partitions of the cached data have been spilled to disk because there wasn't enough memory to hold them. This is suboptimal because accessing data from disk is much slower than from memory. The goal of caching is to keep data in memory for fast access, and a spill to disk means that this goal is not fully achieved.

upvoted 1 times

  **79f0e18** 7 months, 1 week ago

Selected Answer: A

Under MEMORY_ONLY, Spark does not write to disk, so Size on Disk should be 0.

Under MEMORY_ONLY, off-heap memory is not used

In the Storage tab, an asterisk (*) next to the RDD block name (e.g., rdd_42_3*) indicates the partition could not be cached due to memory constraints

upvoted 1 times

  **gloomy_marmot** 6 months, 2 weeks ago

But it should be B

upvoted 1 times

  **KadELbied** 9 months ago

Selected Answer: C

suretly C

upvoted 1 times

  **benni_ale** 1 year, 2 months ago

Selected Answer: C

I think is C

upvoted 1 times

  **Isio05** 1 year, 8 months ago

Selected Answer: C

In this case any data on disk means that cache is not performing optimally

upvoted 2 times

What is the first line of a Databricks Python notebook when viewed in a text editor?

- A. %python
- B. // Databricks notebook source
- C. # Databricks notebook source
- D. -- Databricks notebook source
- E. # MAGIC %python

Suggested Answer: B

Community vote distribution

C (100%)

 **bacckom**  2 years ago

Selected Answer: C

Python: # Databricks notebook source
SQL: -- Databricks notebook source
Scala: // Databricks notebook source
R: # Databricks notebook source
upvoted 9 times

 **KadELbied**  9 months ago

Selected Answer: C

Surely C
upvoted 1 times

 **Ati1362** 1 year, 7 months ago

Selected Answer: C

c is the answer
upvoted 2 times

 **divingbell17** 2 years, 1 month ago

Selected Answer: C

<https://docs.databricks.com/en/notebooks/notebook-export-import.html#import-a-file-and-convert-it-to-a-notebook>
upvoted 2 times

 **aragorn_brego** 2 years, 2 months ago

Selected Answer: C

This is the correct line that you would find at the top of a Databricks notebook when viewed in a text editor, especially for Python notebooks. The # symbol is used for comments in Python, and the comment # Databricks notebook source is used by Databricks to indicate the start of the notebook's source code in the plain text file.

These lines are comments in the respective languages (Scala uses // and SQL uses -- for single-line comments) and indicate the beginning of the Databricks notebook content in the text file.

upvoted 2 times

 **AWSMaster69** 2 years, 2 months ago

Selected Answer: C

The Answer is C, Just downloaded a notebook from Databricks and viewed it in a text editor.
upvoted 1 times

 **60ties** 2 years, 2 months ago

Selected Answer: C

Answer is C
upvoted 1 times

 **60ties** 2 years, 2 months ago

```
// Databricks notebook source - Scala  
# Databricks notebook source - Python  
-- Databricks notebook source - SQL
```

Answer is C

upvoted 1 times

Which statement describes a key benefit of an end-to-end test?

- A. Makes it easier to automate your test suite
- B. Pinpoints errors in the building blocks of your application
- C. Provides testing coverage for all code paths and branches
- D. Closely simulates real world usage of your application
- E. Ensures code is optimized for a real-life workflow

Suggested Answer: D

Community vote distribution

D (80%)

B (20%)

  **vctrhugo** Highly Voted 1 year, 6 months ago

Selected Answer: D

End-to-end testing is a methodology used to test whether the flow of an application is performing as designed from start to finish. The purpose of carrying out end-to-end tests is to identify system dependencies and to ensure that the right information is passed between various system components and systems. The entire application is tested in a real-world scenario such as communicating with the database, network, hardware, and other applications. Therefore, it closely simulates real-world usage of the application. Other options are benefits of different types of testing, not specifically end-to-end testing.

upvoted 5 times

  **KadELbied** Most Recent 9 months ago

Selected Answer: D

Surely D

upvoted 1 times

  **spaceexplorer** 1 year, 6 months ago

Selected Answer: D

D is correct

upvoted 2 times

  **Def21** 1 year, 6 months ago

Selected Answer: B

End-to-end tests use an example scenario, do not necessarily follow complex real world. Unit tests are component wise, end-to-end tests go over components.

upvoted 1 times

  **arekm** 1 year, 1 month ago

B suggests an integration tests

upvoted 1 times

The Databricks CLI is used to trigger a run of an existing job by passing the `job_id` parameter. The response that the job run request has been submitted successfully includes a field `run_id`.

Which statement describes what the number alongside this field represents?

- A. The `job_id` and number of times the job has been run are concatenated and returned.
- B. The total number of jobs that have been run in the workspace.
- C. The number of times the job definition has been run in this workspace.
- D. The `job_id` is returned in this field.
- E. The globally unique ID of the newly triggered run.

Suggested Answer: *E*

Community vote distribution

E (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: E

Surely E

upvoted 1 times

🗳️ 👤 **vctrhugo** 1 year, 6 months ago

Selected Answer: E

The number alongside the "run_id" field represents the globally unique identifier assigned to the newly triggered run of the job. Each run of a job in Databricks is assigned a unique run_id, allowing you to track and reference that specific execution of the job.

upvoted 4 times

🗳️ 👤 **Def21** 1 year, 6 months ago

Selected Answer: E

Verified from Databricks UI

upvoted 1 times

The data science team has created and logged a production model using MLflow. The model accepts a list of column names and returns a new column of type DOUBLE.

The following code correctly imports the production model, loads the customers table containing the customer_id key column into a DataFrame, and defines the feature columns needed for the model.

```
model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
```

Which code block will output a DataFrame with the schema "customer_id LONG, predictions DOUBLE"?

- A. `df.map(lambda x:model(x[columns])).select("customer_id, predictions")`
- B. `df.select("customer_id", model(*columns).alias("predictions"))`
- C. `model.predict(df, columns)`
- D. `df.select("customer_id", pandas_udf(model, columns).alias("predictions"))`
- E. `df.apply(model, columns).select("customer_id, predictions")`

Suggested Answer: D

Community vote distribution

B (100%)

  **aragorn_brego** Highly Voted 1 year, 8 months ago

Selected Answer: B

This code block applies the Spark UDF created from the MLflow model to the DataFrame df by selecting the existing customer_id column and the new column produced by the model, which is aliased to predictions. The model(*columns) part is where the UDF is applied to the columns specified in the columns list, and alias("predictions") is used to name the output column of the model's predictions. This will result in a DataFrame with the desired schema: "customer_id LONG, predictions DOUBLE".

upvoted 7 times

  **divingbell17** Highly Voted 1 year, 7 months ago

Selected Answer: B

B is correct. It's a spark udf not pandas

upvoted 6 times

  **KadELbied** Most Recent 9 months ago

Selected Answer: B

Surely B

upvoted 1 times

  **60ties** 1 year, 8 months ago

I think it is B

upvoted 2 times

A nightly batch job is configured to ingest all data files from a cloud object storage container where records are stored in a nested directory structure YYYY/MM/DD. The data for each date represents all records that were processed by the source system on that date, noting that some records may be delayed as they await moderator approval. Each entry represents a user review of a product and has the following schema:

user_id STRING, review_id BIGINT, product_id BIGINT, review_timestamp TIMESTAMP, review_text STRING

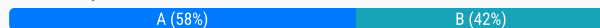
The ingestion job is configured to append all data for the previous date to a target table reviews_raw with an identical schema to the source system. The next step in the pipeline is a batch write to propagate all new records inserted into reviews_raw to a table where data is fully deduplicated, validated, and enriched.

Which solution minimizes the compute costs to propagate this batch of data?

- A. Perform a batch read on the reviews_raw table and perform an insert-only merge using the natural composite key user_id, review_id, product_id, review_timestamp.
- B. Configure a Structured Streaming read against the reviews_raw table using the trigger once execution mode to process new records as a batch job.
- C. Use Delta Lake version history to get the difference between the latest version of reviews_raw and one version prior, then write these records to the next table.
- D. Filter all records in the reviews_raw table based on the review_timestamp; batch append those records produced in the last 48 hours.
- E. Reprocess all records in reviews_raw and overwrite the next table in the pipeline.

Suggested Answer: C

Community vote distribution



alexvno Highly Voted 1 year, 11 months ago

Selected Answer: A

Deduplication , so insert-only merge
upvoted 6 times

bacckom Highly Voted 2 years ago

Selected Answer: A

Should we consider deduplicate? For Time travel, I don't think it can be used to duplicate the target table.
upvoted 5 times

gizzamo Most Recent 4 months, 3 weeks ago

Selected Answer: C

WHY: reading only the delta (the files/rows that were added in the most recent commit) minimizes I/O and compute — you avoid scanning or reprocessing the entire reviews_raw table. Delta's transaction log records which files were added in each commit, so you can load just those new files (or use the change data feed if enabled) and propagate only the new records into the downstream dedupe/validation/enrichment table.
upvoted 1 times

ealpuche 4 months, 4 weeks ago

Selected Answer: C

your ingest appends a new batch daily, so the cheapest next step is to read only the newly added rows (between version N-1 and N, e.g., via Delta Change Data Feed if enabled or by selecting files added in the latest commit). This avoids rescanning the full reviews_raw table and is more precise than timestamp windows, which can miss very late/approved records or read extra dat
upvoted 1 times

KadELbied 9 months ago

Selected Answer: A

suretly A
upvoted 1 times

aarora 1 year ago

Selected Answer: C

To minimize compute costs, the most efficient approach is to leverage Delta Lake's version history to identify only the new records added since the previous ingestion and process those. Here's why this solution works best:

- Delta Lake versioning: Delta Lake tracks changes to the data through its transaction log. By comparing the latest version of the table with the previous version, you can identify only the records that were appended (new data for the previous date).
- Efficient processing: By working only with the delta (new records), you avoid scanning the entire reviews_raw table, which reduces compute and storage I/O costs.
- Accurate and optimized: This approach ensures no unnecessary reprocessing of older data while still capturing any delayed records. It works well for use cases involving deduplication and validation.

upvoted 3 times

🗨️ **Hienlv1** 1 year, 1 month ago

Selected Answer: C

I think C is the correct answer, use the time travel feature to get the previous version and compare it to the current version to figure out which record needs to be inserted instead of a full scan during read like option A. The goal is to minimize compute costs while propagating only new records inserted into the reviews_raw table to the next table in the pipeline.

upvoted 2 times

🗨️ **Sriramiyer92** 1 year, 1 month ago

Selected Answer: A

In case of D. The 48 hrs point just added to confuse us folks.

A is enough.

upvoted 1 times

🗨️ **cales** 1 year, 3 months ago

Selected Answer: B

"The next step in the pipeline is a batch write to propagate all new records inserted into reviews_raw to a table where data is fully deduplicated, validated, and enriched." The deduplication will be performed in the following step. Answer B should fit better with cost minimization

upvoted 2 times

🗨️ **Sriramiyer92** 1 year, 1 month ago

Keyword - batch

Propagated = Movement. Does not necessarily mean "Stream". Also with Data streaming it would become an expensive affair.

upvoted 2 times

🗨️ **shaojunni** 1 year, 3 months ago

Selected Answer: A

Batch read load full table, but guarantee no duplication with merge. Trigger Once only load new data, you have to run merge to guarantee no duplication in the whole target file. But B does not indicate that.

upvoted 1 times

🗨️ **RyanAck24** 1 year, 4 months ago

Selected Answer: A

A is Correct

upvoted 1 times

🗨️ **shaojunni** 1 year, 4 months ago

Selected Answer: B

B is correct, trigger once is the option in structured streaming for batch style job, but much more efficient.

upvoted 2 times

🗨️ **shaojunni** 1 year, 4 months ago

B is correct, trigger once is the option in structured streaming for batch style job, but much more efficient.

upvoted 1 times

🗨️ **spaceexplorer** 2 years ago

Selected Answer: B

B is correct

upvoted 1 times

🗨️ **ranith** 2 years ago

B should be correct when looking at cost minimalization, a batch read would scan the whole reviews_raw table, this is unnecessary as historical data is not changed. If a review is delayed to be approved by the moderator still it is inserted as a new record. Capturing the new data is sufficient.

upvoted 3 times

  **divingbell17** 2 years, 1 month ago

Selected Answer: B

B should be correct.

<https://www.databricks.com/blog/2017/05/22/running-streaming-jobs-day-10x-cost-savings.html>

upvoted 4 times

  **Istiaque** 2 years, 1 month ago

It is a batch process.

upvoted 2 times

Which statement describes Delta Lake optimized writes?

- A. Before a Jobs cluster terminates, OPTIMIZE is executed on all tables modified during the most recent job.
- B. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 1 GB.
- C. Data is queued in a messaging bus instead of committing data directly to memory; all data is committed from the messaging bus in one batch once the job is complete.
- D. Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.
- E. A shuffle occurs prior to writing to try to group similar data together resulting in fewer files instead of each executor writing multiple files based on directory partitions.

Suggested Answer: E

Community vote distribution

E (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: E

Surely E

upvoted 1 times

🗳️ 👤 **vctrhugo** 1 year, 6 months ago

Selected Answer: E

Optimized writes improve file size as data is written and benefit subsequent reads on the table.

Optimized writes are most effective for partitioned tables, as they reduce the number of small files written to each partition. Writing fewer large files is more efficient than writing many small files, but you might still see an increase in write latency because data is shuffled before being written.

<https://learn.microsoft.com/en-us/azure/databricks/delta/tune-file-size#--optimized-writes-for-delta-lake-on-azure-databricks>

upvoted 1 times

🗳️ 👤 **lexaneon** 1 year, 7 months ago

Selected Answer: E

<https://docs.databricks.com/en/delta/tune-file-size.html#optimized-writes>

upvoted 3 times

🗳️ 👤 **alexvno** 1 year, 7 months ago

Selected Answer: E

Optimized writes are most effective for partitioned tables, as they reduce the number of small files written to each partition. Writing fewer large files is more efficient than writing many small files, but you might still see an increase in write latency because data is shuffled before being written.

upvoted 3 times

Which statement describes the default execution mode for Databricks Auto Loader?

- A. Cloud vendor-specific queue storage and notification services are configured to track newly arriving files; the target table is materialized by directly querying all valid files in the source directory.
- B. New files are identified by listing the input directory; the target table is materialized by directly querying all valid files in the source directory.
- C. Webhooks trigger a Databricks job to run anytime new data arrives in a source directory; new data are automatically merged into target tables using rules inferred from the data.
- D. New files are identified by listing the input directory; new files are incrementally and idempotently loaded into the target Delta Lake table.
- E. Cloud vendor-specific queue storage and notification services are configured to track newly arriving files; new files are incrementally and idempotently loaded into the target Delta Lake table.

Suggested Answer: D

Community vote distribution

D (83%)

E (17%)

 **vctrhugo**  2 years ago

Selected Answer: D

"Auto Loader uses directory listing mode by default. In directory listing mode, Auto Loader identifies new files by listing the input directory."

<https://learn.microsoft.com/en-us/azure/databricks/ingestion/auto-loader/directory-listing-mode>

upvoted 8 times

 **e671267**  4 months, 3 weeks ago

Selected Answer: D

The default execution mode for Databricks Auto Loader is directory listing mode, where new files are identified by listing the input directory. This mode works by performing an incremental listing of the input directory to discover new files, allowing for quick stream startup with basic cloud storage access.

How it works:

Auto Loader uses directory listing mode to identify new files by listing the input directory.

It then incrementally and idempotently loads these files into the target Delta Lake table.

upvoted 1 times

 **gizzamo** 4 months, 3 weeks ago

Selected Answer: E

E is Correct:

Databricks Auto Loader's default execution mode uses:

1. Cloud vendor-specific notification services:

- AWS: SQS (Simple Queue Service) + S3 event notifications

- Azure: Event Grid + Queue Storage

- GCP: Pub/Sub + Cloud Storage notifications

2. File tracking: The notification services automatically detect when new files arrive in the source directory, eliminating the need to constantly list directories

3. Incremental processing: Only new files are processed, not all files in the directory

4. Idempotent loading: Files are processed exactly once, even if the stream restarts or fails

This approach is much more efficient than directory listing because it uses event-driven notifications rather than polling.

upvoted 1 times

 **KadELbied** 9 months ago

Selected Answer: D

Surtetly D

upvoted 1 times

 **arekm** 1 year, 1 month ago

Selected Answer: D

D - Autoloader supports:

1. directory listing mode
2. file notification mode

The first option is the default. Answer E describes the second option.

upvoted 3 times

🗨️ 👤 **Rinscy** 2 years ago

D definitely !

Auto Loader is an optimized file source that overcomes all the above limitations and provides a seamless way for data teams to load the raw data at low cost and latency with minimal DevOps effort. You just need to provide a source directory path and start a streaming job. The new structured streaming source, called "cloudFiles", will automatically set up file notification services that subscribe file events from the input directory and process new files as they arrive, with the option of also processing existing files in that directory.

upvoted 2 times

🗨️ 👤 **csrazdan** 1 year, 5 months ago

Correct answer is D. However, listing the input directory is the default way of identifying new files for auto loader. Cloud Native Notification services can be used but this is not default setting for auto loader.

upvoted 1 times

🗨️ 👤 **ranith** 2 years ago

[https://docs.databricks.com/en/ingestion/auto-](https://docs.databricks.com/en/ingestion/auto-loader/options.html#:~:text=By%20default%2C%20Auto%20Loader%20makes,as%20true%20or%20false%20respectively.)

[loader/options.html#:~:text=By%20default%2C%20Auto%20Loader%20makes,as%20true%20or%20false%20respectively.](https://docs.databricks.com/en/ingestion/auto-loader/options.html#:~:text=By%20default%2C%20Auto%20Loader%20makes,as%20true%20or%20false%20respectively.)

Selected answer: D

upvoted 1 times

🗨️ 👤 **get_certified9** 2 years ago

D is the answer. The default execution mode for Databricks Auto Loader is the Directory Listing mode

upvoted 1 times

🗨️ 👤 **spaceexplorer** 2 years ago

Selected Answer: E

E is the answer

upvoted 1 times

🗨️ 👤 **spaceexplorer** 2 years ago

<https://www.databricks.com/blog/2020/02/24/introducing-databricks-ingest-easy-data-ingestion-into-delta-lake.html>

upvoted 3 times

🗨️ 👤 **Isio05** 1 year, 8 months ago

Surely it's not vendor specific solution

upvoted 1 times

A Delta Lake table representing metadata about content posts from users has the following schema:

user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude FLOAT, post_time TIMESTAMP, date DATE

Based on the above schema, which column is a good candidate for partitioning the Delta Table?

- A. post_time
- B. latitude
- C. post_id
- D. user_id
- E. date

Suggested Answer: E

Community vote distribution

E (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: E

Surely E

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: E

Date is usually best candidate for time series data without further specifications

upvoted 1 times

🗳️ 👤 **vctrhugo** 2 years ago

Selected Answer: E

Partitioning a Delta Lake table on the date column is a common practice. This is because partitioning by date can significantly improve query performance when dealing with time-series data. It allows for efficient filtering of data based on time periods, which is a common requirement in many analytics workloads. Partitioning by date also helps manage the size of your partitions, as each partition will contain only the data for a specific date. This can lead to more efficient reads and writes, and can also make it easier to manage and maintain your data.

upvoted 4 times

A large company seeks to implement a near real-time solution involving hundreds of pipelines with parallel updates of many tables with extremely high volume and high velocity data.

Which of the following solutions would you implement to achieve this requirement?

- A. Use Databricks High Concurrency clusters, which leverage optimized cloud storage connections to maximize data throughput.
- B. Partition ingestion tables by a small time duration to allow for many data files to be written in parallel.
- C. Configure Databricks to save all data to attached SSD volumes instead of object storage, increasing file I/O significantly.
- D. Isolate Delta Lake tables in their own storage containers to avoid API limits imposed by cloud vendors.
- E. Store all tables in a single database to ensure that the Databricks Catalyst Metastore can load balance overall throughput.

Suggested Answer: B

Community vote distribution

A (62%)

B (23%)

D (15%)

  **KadELbied** 9 months ago

Selected Answer: B

Surely B

upvoted 1 times

  **natadatabricksadf** 1 year, 2 months ago

Selected Answer: B

High Concurrency clusters are deprecated, so B <https://learn.microsoft.com/en-us/answers/questions/1688410/are-high-concurrency-clusters-deprecated-or-rename>

upvoted 3 times

  **gloomy_marmot** 6 months, 2 weeks ago

But High Concurrency Mode wasn't deprecated. Please read carefully your link - you should correct configure access mode to create cluster with High Concurrency Mode.

So I think the correct answer is A

upvoted 1 times

  **gloomy_marmot** 6 months, 2 weeks ago

I'm wrong - correct answer is B

upvoted 1 times

  **temple1305** 1 year, 2 months ago

Selected Answer: B

High Concurrency clusters are deprecated, so B?

upvoted 3 times

  **hesamh** 1 year ago

doesn't it led to write small files (less than 1 GB) in each partitions?

upvoted 1 times

  **shaojunni** 1 year, 4 months ago

Selected Answer: A

"hundreds of pipelines with parallel updates of many tables" indicates updating many tables concurrently via many pipelines. A is the best solution for that. B is the answer for updating a few large tables with few partitions.

upvoted 2 times

  **practitioner** 1 year, 5 months ago

Selected Answer: B

"Which of the following solutions"

I'm sure this is a question with multichoice. A and B options are correct together.

upvoted 1 times

🗨️ 👤 **BrianNguyen95** 1 year, 8 months ago

Selected Answer: B

High volume and high-velocity data ingestion often becomes a bottleneck due to limited write parallelism. By partitioning ingestion tables based on small time durations (e.g., hourly or even minutes), you create many smaller partitions. This allows concurrent writes to different partitions, significantly increasing the overall throughput of your data ingestion.

upvoted 2 times

🗨️ 👤 **svik** 1 year, 8 months ago

Selected Answer: A

Since multiple pipelines are being used high concurrency cluster would give maximum resource utilization.

upvoted 1 times

🗨️ 👤 **Er5** 1 year, 10 months ago

A.

B is only useful to improve performance of large tables ingestions.

upvoted 1 times

🗨️ 👤 **Curious76** 1 year, 11 months ago

Selected Answer: D

Why not D?

upvoted 2 times

🗨️ 👤 **vctrhugo** 2 years ago

Both options A and B could be relevant depending on the specific details of the use case. If the emphasis is on optimizing concurrent queries and overall data throughput, option A might be more appropriate. If the primary concern is parallel updates of tables with high-volume, high-velocity data, option B is a more targeted approach.

upvoted 1 times

🗨️ 👤 **PrincipalJoe** 2 years ago

Selected Answer: B

The best way to deal with high volume and high velocity data is to use partitioning

upvoted 1 times

🗨️ 👤 **bacckom** 2 years ago

Selected Answer: A

Databricks High Concurrency cluster

upvoted 2 times

🗨️ 👤 **petrv** 2 years, 2 months ago

Selected Answer: A

1) Partitioning by Time:

Partitioning tables by a small time duration allows for efficient parallelism in data writes. Each time partition can be processed independently, enabling parallel updates to multiple partitions concurrently.

2)Optimizing for Parallelism:

By partitioning the tables based on time, data can be ingested and processed in parallel, providing the ability to handle high volume and high velocity data effectively.

Regarding option A, Databricks High Concurrency clusters are more focused on supporting a large number of concurrent users, which might not directly address the requirement for parallel updates of many tables with extremely high volume and high velocity data

upvoted 1 times

🗨️ 👤 **petrv** 2 years, 2 months ago

sorry, the selected answer should have been B

upvoted 1 times

🗨️ 👤 **Isio05** 1 year, 8 months ago

Usage of high conc. clusters can be beneficial both for mulitple users and jobs/queries running on them

upvoted 1 times

🗨️ 👤 **Isio05** 1 year, 7 months ago

Sorry, after going through this question once more - I'll go with B also. It will allow utilize parallelism in an efficient way.

upvoted 1 times

🗨️ 👤 **aragorn_brego** 2 years, 2 months ago

Selected Answer: A

High Concurrency clusters in Databricks are designed for multiple concurrent users and workloads. They provide fine-grained sharing of cluster resources and are optimized for operations such as running multiple parallel queries and updates. This would be suitable for a solution that involves many pipelines with parallel updates, especially with high volume and high velocity data.

upvoted 4 times

Which describes a method of installing a Python package scoped at the notebook level to all nodes in the currently active cluster?

- A. Run `source env/bin/activate` in a notebook setup script
- B. Use `b` in a notebook cell
- C. Use `%pip install` in a notebook cell
- D. Use `%sh pip install` in a notebook cell
- E. Install libraries from PyPI using the cluster UI

Suggested Answer: D

Community vote distribution

C (100%)

🗳️ 👤 **_Lukas_** 2 months, 1 week ago

Selected Answer: C

Correct:

C - The `%pip` magic command is the recommended method for installing notebook-scoped libraries in Databricks. It ensures that the library is installed on both the driver and all worker nodes of the cluster, making it immediately available for distributed processing within that specific notebook session, without affecting other users sharing the cluster.

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: C

Suretly C

upvoted 1 times

🗳️ 👤 **JamesWright** 1 year, 1 month ago

C is correct

upvoted 3 times

🗳️ 👤 **aragorn_brego** 1 year, 2 months ago

Selected Answer: C

In Databricks notebooks, you can use the `%pip install` command in a notebook cell to install a Python package. This will install the package on all nodes in the currently active cluster at the notebook level. It is a feature provided by Databricks to facilitate the installation of Python libraries for the notebook environment specifically.

upvoted 3 times

🗳️ 👤 **60ties** 1 year, 2 months ago

Selected Answer: C

C is correct

upvoted 3 times

Each configuration below is identical to the extent that each cluster has 400 GB total of RAM 160 total cores and only one Executor per VM.

Given an extremely long-running job for which completion must be guaranteed, which cluster configuration will be able to guarantee completion of the job in light of one or more VM failures?

- A. • Total VMs: 8
 - 50 GB per Executor
 - 20 Cores / Executor
- B. • Total VMs: 16
 - 25 GB per Executor
 - 10 Cores / Executor
- C. • Total VMs: 1
 - 400 GB per Executor
 - 160 Cores/Executor
- D. • Total VMs: 4
 - 100 GB per Executor
 - 40 Cores / Executor
- E. • Total VMs: 2
 - 200 GB per Executor
 - 80 Cores / Executor

Suggested Answer: D

Community vote distribution

B (100%)

🗳️ 👤 **_Lukas_** 2 months, 1 week ago

Selected Answer: B

Correct:

B - This configuration offers the highest fault tolerance. With 16 nodes, the failure of a single VM results in only a ~6% loss of total cluster resources, minimizing the amount of data that must be recomputed and reducing the risk of job failure compared to configurations with fewer, larger nodes.

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: B

Surely B

upvoted 1 times

🗳️ 👤 **91d511b** 1 year ago

Selected Answer: B

Total VMs = 16

Resources per VM:

25 GB RAM and 10 cores per executor.

Impact of a VM Failure:

Losing one VM means losing only 6.25% of the cluster's resources.

Fault Tolerance:

Excellent fault tolerance. The cluster can handle multiple VM failures (up to ~3 VMs) and still function effectively.

Best Balance:

With smaller VMs, the job remains highly fault-tolerant while using resources efficiently.

upvoted 2 times

🗳️ 👤 **shaojunni** 1 year, 4 months ago

16 core provides more redundancy, fault tolerance and more parallelism. But if dataset is huge, 8VM maybe better. The question is missing some information.

upvoted 2 times

🗨️ 👤 **c00ccb7** 1 year, 7 months ago

Selected Answer: B

This setup ensures that the job can continue running and complete even if some VMs fail, as there are more VMs available to handle the workload

upvoted 2 times

🗨️ 👤 **ChayV** 1 year, 8 months ago

Selected Answer: B

If VM is down, performance is degraded, so opting for vm's which has distributed memory per executor and optimal cores per executor.

upvoted 3 times

🗨️ 👤 **hal2401me** 1 year, 11 months ago

Selected Answer: B

in my exam today, i chose B, 16VM, because the "extremely long-run".

upvoted 4 times

🗨️ 👤 **ThoBustos** 1 year, 9 months ago

do you have link to databricks doc?

upvoted 1 times

🗨️ 👤 **practicioner** 1 year, 5 months ago

I have no link for Databricks doc. It's just a logic. The more VMs we have, the more robust our pipeline is.

upvoted 1 times

🗨️ 👤 **arekm** 1 year, 1 month ago

So long as the data partition fits into a smaller VM. But we don't have that information. From the perspective of failures of multiple machines, the move of them the better :)

upvoted 1 times

A Delta Lake table in the Lakehouse named `customer_churn_params` is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

Immediately after each update succeeds, the data engineering team would like to determine the difference between the new version and the previous version of the table.

Given the current implementation, which method can be used?

- A. Execute a query to calculate the difference between the new version and the previous version using Delta Lake's built-in versioning and time travel functionality.
- B. Parse the Delta Lake transaction log to identify all newly written data files.
- C. Parse the Spark event logs to identify those rows that were updated, inserted, or deleted.
- D. Execute `DESCRIBE HISTORY customer_churn_params` to obtain the full operation metrics for the update, including a log of all records that have been added or modified.
- E. Use Delta Lake's change data feed to identify those records that have been updated, inserted, or deleted.

Suggested Answer: E

Community vote distribution

A (100%)

  **_Lukas_** 2 months, 1 week ago

Selected Answer: A

Correct:

A - Since Delta Lake automatically maintains a transaction log with version history, you can easily query the table as it existed before the update (using `VERSION AS OF` or `TIMESTAMP AS OF`) and compare it to the current version. This allows you to run standard SQL set operations (like `EXCEPT` or `ANTI JOIN`) to identify exactly which rows were added, removed, or changed between the two versions without needing to parse raw logs or enable additional features

upvoted 1 times

  **KadELbied** 9 months ago

Selected Answer: A

Surely A

upvoted 1 times

  **arekm** 1 year, 1 month ago

Selected Answer: A

D - see the discussion under Jugiboss comment.

upvoted 1 times

  **Sriramiyer92** 1 year, 1 month ago

Selected Answer: A

CDF is particularly useful in Incremental loads. In our case it is overwrite.

Hence A.

upvoted 2 times

  **Jugiboss** 1 year, 3 months ago

Selected Answer: E

he best method to determine the difference between the new version and the previous version of the `customer_churn_params` table in Delta Lake is:

E. Use Delta Lake's change data feed to identify those records that have been updated, inserted, or deleted.

This approach leverages Delta Lake's built-in functionality to track changes at the record level, providing a clear view of what has changed between versions.

upvoted 1 times

🗨️ 👤 **Jugiboss** 1 year, 3 months ago

Nevermind, it's A
upvoted 2 times

🗨️ 👤 **arekm** 1 year, 1 month ago

It is the best method so long as it is enabled, and we don't know. Moreover, CDF is designed for a small portion of the data being changed, not a full overwrite.
upvoted 1 times

🗨️ 👤 **cales** 1 year, 3 months ago

Selected Answer: E

Change data feed allows to check for changes between versions
upvoted 1 times

🗨️ 👤 **benni_ale** 1 year, 3 months ago

There is no info about cdf being enabled on the table
upvoted 1 times

🗨️ 👤 **robodog** 1 year, 5 months ago

Selected Answer: A

Answer is A. The easy way to get the difference between those tables is by travel time by version
upvoted 2 times

🗨️ 👤 **HelixAbdu** 1 year, 6 months ago

Answer is A. There is no clue that CDF is enabled for the table
upvoted 2 times

🗨️ 👤 **c00ccb7** 1 year, 7 months ago

Selected Answer: A

Answer A
upvoted 3 times

🗨️ 👤 **Deb9753** 1 year, 8 months ago

Answer : E
upvoted 2 times

A data team's Structured Streaming job is configured to calculate running aggregates for item sales to update a downstream marketing dashboard. The marketing team has introduced a new promotion, and they would like to add a new field to track the number of times this promotion code is used for each item. A junior data engineer suggests updating the existing query as follows. Note that proposed changes are in bold.

Original query:

```
df.groupBy("item")
  .agg(count("item").alias("total_count"),
       mean("sale_price").alias("avg_price"))
  .writeStream
  .outputMode("complete")
  .option("checkpointLocation", "/item_agg/__checkpoint")
  .start("/item_agg")
```

Proposed query:

```
df.groupBy("item")
  .agg(count("item").alias("total_count"),
       mean("sale_price").alias("avg_price"),
       count("promo_code = 'NEW_MEMBER'").alias("new_member_promo"))
  .writeStream
  .outputMode("complete")
  .option('mergeSchema', 'true')
  .option("checkpointLocation", "/item_agg/__checkpoint")
  .start("/item_agg")
```

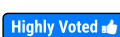
Which step must also be completed to put the proposed query into production?

- A. Specify a new checkpointLocation
- B. Remove `.option('mergeSchema', 'true')` from the streaming write
- C. Increase the shuffle partitions to account for additional aggregates
- D. Run `REFRESH TABLE delta.`/item_agg``

Suggested Answer: A

Community vote distribution

A (100%)

 **Deb9753**  1 year, 8 months ago

Answer: A

When updating the schema of a streaming job, specifying a new checkpoint location ensures that the streaming query starts fresh with the new schema. This avoids issues that might arise from schema mismatches between the previous state and the new schema. This is especially relevant when adding new fields because the existing state might not be compatible with the new schema.

upvoted 5 times

 **Ral17**  2 months, 1 week ago

Selected Answer: A

This is especially relevant when adding new fields because the existing state might not be compatible with the new schema.

upvoted 1 times

  **KadELbied** 9 months ago

Selected Answer: A

Suretly A

upvoted 1 times

  **SeRo42** 1 year ago

Selected Answer: A

Are filter conditions in count() allowed?

upvoted 1 times

  **robodog** 1 year, 5 months ago

Selected Answer: A

A answer

upvoted 1 times

  **MDWPartners** 1 year, 8 months ago

Selected Answer: A

This checkpoint location preserves all of the essential information that identifies a query. Each query must have a different checkpoint location. Multiple queries should never have the same location. For more information, see the Structured Streaming Programming Guide.

<https://docs.databricks.com/en/structured-streaming/query-recovery.html>

upvoted 2 times

When using CLI or REST API to get results from jobs with multiple tasks, which statement correctly describes the response structure?

- A. Each run of a job will have a unique job_id; all tasks within this job will have a unique job_id
- B. Each run of a job will have a unique job_id; all tasks within this job will have a unique task_id
- C. Each run of a job will have a unique orchestration_id; all tasks within this job will have a unique run_id
- D. Each run of a job will have a unique run_id; all tasks within this job will have a unique task_id
- E. Each run of a job will have a unique run_id; all tasks within this job will also have a unique run_id

Suggested Answer: D

Community vote distribution

D (100%)

  **cales**  1 year, 3 months ago

Selected Answer: E

The correct answer is E. <https://docs.databricks.com/api/workspace/jobs/getrun>

You can visit the link and observe the response sample of the API

```
{
  "job_id": 11223344,
  "run_id": 455644833,
```

job_id is the unique id of the job. run_id is the unique id of the run

Then each task will have its unique run id:

```
"tasks": [
  {
    ...
    "run_id": 2112892,
    ...
  }
]
```

upvoted 6 times

  **_Lukas_**  2 months, 1 week ago

Selected Answer: E

Correct:

E - In the Databricks Jobs API (specifically for multi-task jobs), the execution of the entire job is identified by a unique top-level run_id (often referred to as the job_run_id or parent run ID). Within that job run, each individual task execution is also assigned its own unique run_id (task run ID). This allows you to query the status and output of specific tasks independently using their specific task run_id.

upvoted 1 times

  **Billybob0604** 6 months, 1 week ago

Selected Answer: D

job_id → job definition

run_id → a specific execution of a job

task_id → a uniquely identified task within a job run

upvoted 2 times

  **KadELbied** 9 months ago

Selected Answer: E

Surely E

upvoted 1 times

  **mohadjhamad** 11 months ago

Selected Answer: D

Each job execution (run) gets a unique run_id

This run_id identifies a specific instance of a job run.
It allows tracking of job execution details, logs, and results.
Each task within that job run has a unique task_id

Multi-task jobs have multiple tasks, each assigned a distinct task_id.
The task_id helps in monitoring and retrieving individual task details.
upvoted 2 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: E

E - but between D & E is a play on the words. Still E seems a tiny bit more explicit. The bottom line the run_id attribute of each task within the job will be different.
upvoted 1 times

🗳️ 👤 **AlejandroU** 1 year, 1 month ago

Selected Answer: E

Answer E. Job Run (run_id): Each execution of a job has a unique run_id for the entire job.
Task Run (run_id): For jobs with multiple tasks, each task also gets its own run_id, which is distinct from the job's run_id. This run_id for tasks can be used to retrieve individual task outputs.

<https://docs.databricks.com/api/workspace/jobs/getrun>

"tasks: The list of tasks performed by the run. Each task has its own run_id which you can use to call JobsGetOutput to retrieve the run results. "
upvoted 2 times

🗳️ 👤 **Thameur01** 1 year, 2 months ago

Selected Answer: B

B if the question is about /api/2.1/jobs/get
D if the question /api/2.1/jobs/runs/get
upvoted 2 times

🗳️ 👤 **Thameur01** 1 year, 2 months ago

Selected Answer: D

The question should clearly specify, get jobs or get job run, there differences in response
upvoted 1 times

🗳️ 👤 **cf56faf** 1 year, 2 months ago

Selected Answer: D

Must be D.
upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: E

E
upvoted 2 times

🗳️ 👤 **Kreshu** 1 year, 3 months ago

Selected Answer: E

Correct answer is E
upvoted 2 times

🗳️ 👤 **thelio_team** 1 year, 4 months ago

tested answer is E :
'tasks': [{'run_id': '****',
'task_key': '#####',
upvoted 3 times

🗳️ 👤 **MDWPartners** 1 year, 8 months ago

Selected Answer: D

Seems right
upvoted 2 times

The data engineering team is configuring environments for development, testing, and production before beginning migration on a new data pipeline. The team requires extensive testing on both the code and data resulting from code execution, and the team wants to develop and test against data as similar to production data as possible.

A junior data engineer suggests that production data can be mounted to the development and testing environments, allowing pre-production code to execute against production data. Because all users have admin privileges in the development environment, the junior data engineer has offered to configure permissions and mount this data for the team.

Which statement captures best practices for this situation?

- A. All development, testing, and production code and data should exist in a single, unified workspace; creating separate environments for testing and development complicates administrative overhead.
- B. In environments where interactive code will be executed, production data should only be accessible with read permissions; creating isolated databases for each environment further reduces risks.
- C. As long as code in the development environment declares `USE dev_db` at the top of each notebook, there is no possibility of inadvertently committing changes back to production data sources.
- D. Because Delta Lake versions all data and supports time travel, it is not possible for user error or malicious actors to permanently delete production data; as such, it is generally safe to mount production data anywhere.
- E. Because access to production data will always be verified using passthrough credentials, it is safe to mount data to any Databricks development environment.

Suggested Answer: B

Community vote distribution

B (100%)

MDWPartners **Highly Voted** 1 year, 8 months ago

Selected Answer: B

Seems right

upvoted 5 times

Lukas **Most Recent** 2 months, 1 week ago

Selected Answer: B

Correct:

B - This statement adheres to the "Principle of Least Privilege" and environment isolation best practices. Development environments are often "sandboxes" where users have high privileges (like Admin) to experiment. If production data is mounted there with write access, a simple coding error (like `mode("overwrite")`) could wipe out production data. Therefore, if production data must be accessed, it must be strictly read-only, or better yet, production data should be isolated completely and only authorized subsets/snapshots (sanitized/masked) should be moved to lower environments.

upvoted 2 times

KadELbied 9 months ago

Selected Answer: B

Surely B

upvoted 1 times

lene 1 year ago

Selected Answer: B

Shallow clones of prod data can be created in dev for testing purposes

upvoted 1 times

Colje 1 year, 3 months ago

Selected Answer: B

The correct answer is B. In environments where interactive code will be executed, production data should only be accessible with read permissions; creating isolated databases for each environment further reduces risks.

Explanation:

Best practices for managing production, development, and testing environments involve minimizing the risk of unintended data modifications or deletions, especially when dealing with production data. The ideal setup includes:

Limiting permissions: Production data should only be accessible with read permissions in development or testing environments to prevent accidental changes.

Isolating environments: Creating separate databases for development, testing, and production environments ensures that there are clear boundaries and that development code cannot unintentionally affect production data.

upvoted 1 times

A data engineer, User A, has promoted a pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens.

A workspace admin, User C, inherits responsibility for managing this pipeline. User C uses the Databricks Jobs UI to take "Owner" privileges of each job. Jobs continue to be triggered using the credentials and tooling configured by User B.

An application has been configured to collect and parse run information returned by the REST API. Which statement describes the value returned in the `creator_user_name` field?

- A. Once User C takes "Owner" privileges, their email address will appear in this field; prior to this, User A's email address will appear in this field.
- B. User B's email address will always appear in this field, as their credentials are always used to trigger the run.
- C. User A's email address will always appear in this field, as they still own the underlying notebooks.
- D. Once User C takes "Owner" privileges, their email address will appear in this field; prior to this, User B's email address will appear in this field.
- E. User C will only ever appear in this field if they manually trigger the job, otherwise it will indicate User B.

Suggested Answer: C

Community vote distribution

B (75%)

A (25%)

  **_Lukas_** 2 months, 1 week ago

Selected Answer: C

Correct:

C - The `creator_user_name` field in the Databricks Jobs API response represents the user who originally created the job definition. This value is immutable and persists even if ownership is transferred to another user (User C) or if the job is triggered by a different user (User B). It is a historical record of creation.

upvoted 2 times

  **Billybob0604** 6 months, 1 week ago

Selected Answer: B

As long as User B's PAT is used to trigger the run, User B's email will appear in `creator_user_name`

upvoted 1 times

  **gloomy_marmot** 6 months, 2 weeks ago

Selected Answer: E

The correct answer is:

E. User C will only ever appear in this field if they manually trigger the job, otherwise it will indicate User B.

Explanation:

The `creator_user_name` field in Databricks job run metadata indicates the identity of the user or token that triggered the job run, not the job owner or notebook creator.

User A created the jobs but is no longer relevant once job ownership is transferred and jobs are triggered by another identity.

User B triggers the jobs via the REST API using their personal access token—this makes User B the creator of the runs.

User C now owns the jobs but does not trigger the runs unless they manually do so, in which case their email would appear.

So unless User C manually triggers a run, User B's identity (email) will consistently appear in the `creator_user_name` field because their token is being used for the API-triggered runs.

upvoted 1 times

🗨️ 👤 **KadELbied** 9 months ago

Selected Answer: B

Suretly B

upvoted 1 times

🗨️ 👤 **Stalker200** 9 months, 3 weeks ago

Selected Answer: E

It's E as hameur01 mentioned.

upvoted 2 times

🗨️ 👤 **kishanu** 10 months ago

Selected Answer: C

It should be User A.

Go to any existing workflow which has a few runs and see the JSON. It will have the creator_user_name as the creator of the workflow. For the user who has triggered the job, the field name is "run_as_user_name"

upvoted 2 times

🗨️ 👤 **kishanu** 10 months ago

Selected Answer: C

It should be User A.

Go to any existing workflow which has a few runs and see the JSON. It will have the creator_user_name as the creator of the workflow. For the user who has triggered the job, the field name is "run_as_user_name"

upvoted 1 times

🗨️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: B

B - the person that triggered the job. Since we are using User B personal access token, it is going to say User B's email address.

upvoted 1 times

🗨️ 👤 **temple1305** 1 year, 1 month ago

Selected Answer: A

User A was OWNER, B just run job, doesn't became OWNER, then User C became OWNER - so case "A"

upvoted 2 times

🗨️ 👤 **temple1305** 1 year, 1 month ago

Sorry - really B

The creator_user_name field in the Databricks REST API response for job runs reflects the user who triggered the job, not the owner of the job or the creator of the underlying notebooks. Since User B's external orchestration tool triggers the job runs using their personal access token, the field will always indicate User B's email address as the "creator" of those job runs.

upvoted 6 times

🗨️ 👤 **Thameur01** 1 year, 2 months ago

Selected Answer: E

creator_user_name reflects the user who triggered the job run, not the job owner or the notebook creator.

For programmatically triggered runs, the field will reflect the user whose personal access token was used (User B in this case).

If User C manually triggers a job, the field will show User C's email address for that specific run.

upvoted 3 times

🗨️ 👤 **arekm** 1 year, 1 month ago

The statement is correct, however the question does not mention manually triggering the job. Only the monitoring solution that retrieves the data from the REST API. Given the circumstances, it is going to be User B's email address.

upvoted 1 times

🗨️ 👤 **cf56faf** 1 year, 2 months ago

Selected Answer: B

It's B.

upvoted 2 times

🗨️ 👤 **smashit** 1 year, 3 months ago

ownership of job an be assigned to user not for a group. Option A is correct.

upvoted 1 times

🗨️ 👤 **Jugiboss** 1 year, 3 months ago

Selected Answer: A

Seems correct.

upvoted 2 times

🗨️ 👤 **Ananth4Sap** 1 year, 3 months ago

Selected Answer: C

The creator_user_name field in the run information returned by the REST API reflects the user who originally created the job. Since User A created the jobs using their personal access token, their email address will always appear in this field, regardless of who triggers the job runs or who takes ownership of the job later.

upvoted 3 times

🗨️ 👤 **pk07** 1 year, 4 months ago

Selected Answer: C

C based on previous comment

upvoted 2 times

🗨️ 👤 **shaojunni** 1 year, 4 months ago

Selected Answer: C

Job creator can't be changed. Owner can be changed. So creator_user_name field always return who created the job: User A. But UserA no longer owns the job. So Answer C is partially correct.

upvoted 1 times

🗨️ 👤 **shaojunni** 1 year, 4 months ago

Selected Answer: C

Job creator can't be changed. Owner can be changed. So creator_user_name field always return owner: User A. But UserA no longer owns the job. So Answer C is partially correct.

upvoted 1 times

A member of the data engineering team has submitted a short notebook that they wish to schedule as part of a larger data pipeline. Assume that the commands provided below produce the logically correct results when run as presented.

Cmd 1

```
rawDF = spark.table("raw_data")
```

Cmd 2

```
rawDF.printSchema()
```

Cmd 3

```
flattenedDF = rawDF.select("?", "values.*")
```

Cmd 4

```
finalDF = flattenedDF.drop("values")
```

Cmd 5

```
display(finalDF)
```

Cmd 6

```
finalDF.write.mode("append").saveAsTable("flat_data")
```

Which command should be removed from the notebook before scheduling it as a job?

- A. Cmd 2
- B. Cmd 3
- C. Cmd 4
- D. Cmd 5

Suggested Answer: D

Community vote distribution

D (100%)

🗨️ **KadELbied** 9 months ago

Selected Answer: D

CMD 6 E - display(finalDF)

upvoted 1 times

🗨️ **arekm** 1 year, 1 month ago

Selected Answer: D

D - Cmd 5 which is display(finalDF) is costly, produces a nicely formatted HTML table which is unnecessary during the production run. On top that there is a limit on how much output the notebook can generate. This causes me some problems in the past for legacy code.

upvoted 1 times

🗨️ **smashit** 1 year, 3 months ago

There is a limitation when you use display command inside a job. If the output exceeds 20MB it will throw an error. May be that was the reason.

upvoted 1 times

🗨️ **csrazdan** 1 year, 5 months ago

Selected Answer: D

The display function is built specifically for the notebook environment and will not work for Spark. Should you want to print contents of DF in Spark then replace it with df.show()

upvoted 2 times

Which statement regarding Spark configuration on the Databricks platform is true?

- A. The Databricks REST API can be used to modify the Spark configuration properties for an interactive cluster without interrupting jobs currently running on the cluster.
- B. Spark configurations set within a notebook will affect all SparkSessions attached to the same interactive cluster.
- C. When the same Spark configuration property is set for an interactive cluster and a notebook attached to that cluster, the notebook setting will always be ignored.
- D. Spark configuration properties set for an interactive cluster with the Clusters UI will impact all notebooks attached to that cluster.

Suggested Answer: D

Community vote distribution

D (100%)

🗲️ 👤 **KadELbied** 9 months ago

Selected Answer: D

Surety D

upvoted 1 times

🗲️ 👤 **nedlo** 1 year, 3 months ago

Selected Answer: D

this question repeats afaiK

upvoted 2 times

🗲️ 👤 **Melik3** 1 year, 5 months ago

Selected Answer: D

this looks correct

upvoted 2 times

🗲️ 👤 **vexor3** 1 year, 6 months ago

Selected Answer: D

Should be D

upvoted 3 times

The business reporting team requires that data for their dashboards be updated every hour. The total processing time for the pipeline that extracts, transforms, and loads the data for their pipeline runs in 10 minutes.

Assuming normal operating conditions, which configuration will meet their service-level agreement requirements with the lowest cost?

- A. Configure a job that executes every time new data lands in a given directory
- B. Schedule a job to execute the pipeline once an hour on a new job cluster
- C. Schedule a Structured Streaming job with a trigger interval of 60 minutes
- D. Schedule a job to execute the pipeline once an hour on a dedicated interactive cluster

Suggested Answer: B

Community vote distribution

B (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: B

suretly B

upvoted 1 times

🗳️ 👤 **Sriramiyer92** 1 year, 1 month ago

Selected Answer: B

Key words: updated every hour, pipeline runs in 10 minutes - Simple job cluster should do the job.

upvoted 1 times

🗳️ 👤 **Colje** 1 year, 3 months ago

Selected Answer: B

The correct answer is B. Schedule a job to execute the pipeline once an hour on a new job cluster.

Explanation:

In this scenario, the business reporting team needs the data to be updated every hour, and the processing time for the pipeline takes 10 minutes. To meet this requirement with the lowest cost, the best option is to schedule the job to run once per hour using a new job cluster.

A job cluster is created specifically for the duration of the job, and once the job finishes, the cluster is terminated. This is cost-efficient because resources are only consumed while the job is running, and the cluster does not stay active when it is not needed.

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

I agree with B. However, C would also work from the cost perspective provided you set the timeout to a low value. That would still be more resource consumption than B, which disposes of the cluster as soon as the job is done. Moreover, C does not mention "job cluster" which is kind of an implied best practice. However, always better to be explicit.

upvoted 1 times

A Databricks SQL dashboard has been configured to monitor the total number of records present in a collection of Delta Lake tables using the following query pattern:

```
SELECT COUNT (*) FROM table -
```

Which of the following describes how results are generated each time the dashboard is updated?

- A. The total count of rows is calculated by scanning all data files
- B. The total count of rows will be returned from cached results unless REFRESH is run
- C. The total count of records is calculated from the Delta transaction logs
- D. The total count of records is calculated from the parquet file metadata

Suggested Answer: A

Community vote distribution

C (100%)

🗳️ **ADFMDS** 1 month, 4 weeks ago

Selected Answer: A

COUNT(*) on a Delta table always results in a full data file scan, because Delta log does not store row counts.
upvoted 1 times

🗳️ **KadELbied** 9 months ago

Selected Answer: C

Surely C
upvoted 1 times

🗳️ **AlejandroU** 1 year, 1 month ago

Answer is D. This is same question #63.

Delta Lake stores its data in Parquet format. Parquet files include metadata (like row counts) that can be efficiently queried.

For a COUNT(*) query, Delta Lake does not need to scan all the data files; instead, it reads the row count information stored in the Parquet file metadata, making the operation faster.

upvoted 1 times

🗳️ **vexor3** 1 year, 6 months ago

Selected Answer: C

C is correct
upvoted 1 times

🗳️ **hpkr** 1 year, 7 months ago

Selected Answer: C

C is correct
upvoted 1 times

🗳️ **BrianNguyen95** 1 year, 8 months ago

Selected Answer: C

Delta Lake optimizes COUNT(*) queries by reading the row counts stored in the Delta transaction log. This eliminates the need for a full table scan, resulting in significantly faster query performance.

pen_spark

upvoted 2 times

🗳️ **Freyr** 1 year, 8 months ago

Selected Answer: C

Correct Answer: C
upvoted 1 times

  **MDWPartners** 1 year, 8 months ago

Selected Answer: C

I would've said C

upvoted 2 times

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_store
AS (
  SELECT *
  FROM prod.sales a
  INNER JOIN prod.store b
  ON a.store_id = b.store_id
)
```

Consider the following query:

DROP TABLE prod.sales_by_store -

If this statement is executed by a workspace admin, which result will occur?

- A. Data will be marked as deleted but still recoverable with Time Travel.
- B. The table will be removed from the catalog but the data will remain in storage.
- C. The table will be removed from the catalog and the data will be deleted.
- D. An error will occur because Delta Lake prevents the deletion of production data.

Suggested Answer: B

Community vote distribution

C (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: C

Surely C

upvoted 1 times

🗳️ 👤 **robodog** 1 year, 5 months ago

Selected Answer: C

no location keyword, so its a managed table

upvoted 1 times

🗳️ 👤 **Isio05** 1 year, 7 months ago

Selected Answer: C

It's managed table, so data will be also removed

upvoted 3 times

🗳️ 👤 **hpkr** 1 year, 7 months ago

Selected Answer: C

C is correct

upvoted 2 times

🗳️ 👤 **hpkr** 1 year, 8 months ago

Selected Answer: C

Option C

upvoted 1 times

🗳️ 👤 **imatheushenrique** 1 year, 8 months ago

C because its a managed table

upvoted 1 times

🗳️ 👤 **Freyr** 1 year, 8 months ago

Selected Answer: C

Correct Answer: C

No location provided in the table. So, it is a managed table. This will result in deleting the table meta data as well as table data.

upvoted 1 times

  **MDWPartners** 1 year, 8 months ago

Selected Answer: C

Seems C, it's a managed table

upvoted 1 times

A developer has successfully configured their credentials for Databricks Repos and cloned a remote Git repository. They do not have privileges to make changes to the main branch, which is the only branch currently visible in their workspace.

Which approach allows this user to share their code updates without the risk of overwriting the work of their teammates?

- A. Use Repos to create a new branch, commit all changes, and push changes to the remote Git repository.
- B. Use Repos to create a fork of the remote repository, commit all changes, and make a pull request on the source repository.
- C. Use Repos to pull changes from the remote Git repository; commit and push changes to a branch that appeared as changes were pulled.
- D. Use Repos to merge all differences and make a pull request back to the remote repository.

Suggested Answer: A

  **KadELbied** 9 months ago

Selected Answer: A

Suretly A

upvoted 1 times

  **RyanAck24** 1 year, 4 months ago

Selected Answer: A

A seems correct

upvoted 2 times

  **Ati1362** 1 year, 7 months ago

answer B

upvoted 3 times

The security team is exploring whether or not the Databricks secrets module can be leveraged for connecting to an external database.

After testing the code with all Python variables being defined with strings, they upload the password to the secrets module and configure the correct permissions for the currently active user. They then modify their code to the following (leaving all other variables unchanged).

```
password = dbutils.secrets.get(scope="db_creds", key="jdbc_password")

print(password)

df = (spark
      .read
      .format("jdbc")
      .option("url", connection)
      .option("dbtable", tablename)
      .option("user", username)
      .option("password", password)
      )
```

Which statement describes what will happen when the above code is executed?

- A. The connection to the external table will succeed; the string "REDACTED" will be printed.
- B. An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the encoded password will be saved to DBFS.
- C. An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the password will be printed in plain text.
- D. The connection to the external table will succeed; the string value of password will be printed in plain text.

Suggested Answer: A

Community vote distribution

A (100%)

 **KadELbied** 9 months ago

Selected Answer: A

Suretly A

upvoted 1 times

 **Hadiler** 1 year, 6 months ago

Selected Answer: A

A is the correct answer

upvoted 3 times

 **vexor3** 1 year, 6 months ago

Selected Answer: A

A is correct

upvoted 3 times

 **Deb9753** 1 year, 8 months ago

Answer A : When using Databricks secrets, the actual value of the secret is typically protected from being displayed in plain text. Databricks automatically redacts secret values when they are printed in the notebook. So, when you use the `print(password)` statement, the output will not show the actual password but will instead show [REDACTED].

upvoted 1 times

 **imatheushenrique** 1 year, 8 months ago

A. A. The connection to the external table will succeed; the string "REDACTED" will be printed.

upvoted 1 times

The data science team has created and logged a production model using MLflow. The model accepts a list of column names and returns a new column of type DOUBLE.

The following code correctly imports the production model, loads the customers table containing the customer_id key column into a DataFrame, and defines the feature columns needed for the model.

```
model = mlflow.pyfunc.spark_udf (spark,
model_uri="models:/churn/prod")

df = spark.table("customers")

columns = ["account_age", "time_since_last_seen", "app_rating"]
```

Which code block will output a DataFrame with the schema "customer_id LONG, predictions DOUBLE"?

- A. df.map(lambda x:model(x[columns])).select("customer_id, predictions")
- B. df.select("customer_id",
model(*columns).alias("predictions"))
- C. model.predict(df, columns)
- D. df.apply(model, columns).select("customer_id, predictions")

Suggested Answer: B

Community vote distribution

B (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: B

Surely B

upvoted 1 times

🗳️ 👤 **Hadiler** 1 year, 6 months ago

Selected Answer: B

B is the correct answer

upvoted 1 times

🗳️ 👤 **vexor3** 1 year, 6 months ago

Selected Answer: B

B is correct

upvoted 1 times

🗳️ 👤 **Freyr** 1 year, 8 months ago

Selected Answer: B

Correct Answer: B

This option uses select to specify columns from the DataFrame and applies the model to the specified columns (columns). The output of the model is aliased as "predictions", which ensures the output DataFrame will have the column names "customer_id" and "predictions" with appropriate data types assuming the model returns a double type. This syntax aligns with PySpark's DataFrame transformations and is a typical way to apply a machine learning model to specific columns in Databricks.

upvoted 3 times

A junior member of the data engineering team is exploring the language interoperability of Databricks notebooks. The intended outcome of the below code is to register a view of all sales that occurred in countries on the continent of Africa that appear in the geo_lookup table.

Before executing the code, running SHOW TABLES on the current database indicates the database contains only two tables: geo_lookup and sales.

Cmd 1

```
%python
countries_af = [x[0] for x in
spark.table("geo_lookup").filter("continent='AF'").select("country").collect()]
```

Cmd 2

```
%sql
CREATE VIEW sales_af AS
  SELECT *
  FROM sales
  WHERE city IN countries_af
  AND CONTINENT = "AF"
```

What will be the outcome of executing these command cells in order in an interactive notebook?

- A. Both commands will succeed. Executing SHOW TABLES will show that countries_af and sales_af have been registered as views.
- B. Cmd 1 will succeed. Cmd 2 will search all accessible databases for a table or view named countries_af: if this entity exists, Cmd 2 will succeed.
- C. Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable representing a PySpark DataFrame.
- D. Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable containing a list of strings.

Suggested Answer: D

Community vote distribution

D (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: D

Syuretyly D

upvoted 1 times

🗳️ 👤 **_lene_** 1 year ago

Selected Answer: D

D is correct

upvoted 1 times

🗳️ 👤 **nedlo** 1 year, 3 months ago

i wonder if not B? I guess name of view can be the same as name of Python variable?

upvoted 1 times

🗳️ 👤 **Hadiler** 1 year, 6 months ago

Selected Answer: D

D is correct. It will not be DataFrame since collect() will change it to list of strings

upvoted 4 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Still D, but collect() actually returns a DataFrame. It is however used in a comprehension that extracts first column of each row and gets the string.

upvoted 1 times

🗳️ 👤 **vexor3** 1 year, 6 months ago

Selected Answer: D

D is correct

upvoted 1 times

  **hpkr** 1 year, 8 months ago

Selected Answer: D

D is correct

upvoted 1 times

The data science team has requested assistance in accelerating queries on free-form text from user reviews. The data is currently stored in Parquet with the below schema:

item_id INT, user_id INT, review_id INT, rating FLOAT, review STRING

The review column contains the full text of the review left by the user. Specifically, the data science team is looking to identify if any of 30 key words exist in this field.

A junior data engineer suggests converting this data to Delta Lake will improve query performance.

Which response to the junior data engineer's suggestion is correct?

- A. Delta Lake statistics are not optimized for free text fields with high cardinality.
- B. Delta Lake statistics are only collected on the first 4 columns in a table.
- C. ZORDER ON review will need to be run to see performance gains.
- D. The Delta log creates a term matrix for free text fields to support selective filtering.

Suggested Answer: A

Community vote distribution

A (100%)

🗨️ 👤 **KadELbied** 9 months ago

Selected Answer: A

Surely A

upvoted 1 times

🗨️ 👤 **m79590530** 1 year, 3 months ago

Selected Answer: A

Delta Lake optimizations are not well suited for long TIMESTAMP or STRING fields and can not provide good indexing, data skipping or statistics logging for them.

upvoted 1 times

The data engineering team has configured a job to process customer requests to be forgotten (have their data deleted). All user data that needs to be deleted is stored in Delta Lake tables using default table settings.

The team has decided to process all deletions from the previous week as a batch job at 1am each Sunday. The total duration of this job is less than one hour. Every Monday at 3am, a batch job executes a series of VACUUM commands on all Delta Lake tables throughout the organization.

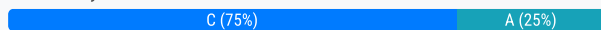
The compliance officer has recently learned about Delta Lake's time travel functionality. They are concerned that this might allow continued access to deleted data.

Assuming all delete logic is correctly implemented, which statement correctly addresses this concern?

- A. Because the VACUUM command permanently deletes all files containing deleted records, deleted records may be accessible with time travel for around 24 hours.
- B. Because the default data retention threshold is 24 hours, data files containing deleted records will be retained until the VACUUM job is run the following day.
- C. Because the default data retention threshold is 7 days, data files containing deleted records will be retained until the VACUUM job is run 8 days later.
- D. Because Delta Lake's delete statements have ACID guarantees, deleted records will be permanently purged from all storage systems as soon as a delete job completes.

Suggested Answer: C

Community vote distribution



KadELbied 9 months ago

Selected Answer: C

suretly C

upvoted 1 times

cales 1 year, 3 months ago

Selected Answer: C

Is C since by default Vacuum retains files no more referenced in the current table version for 7 days.

<https://docs.databricks.com/en/delta/history.html#configure-data-retention-for-time-travel-queries>

upvoted 1 times

Hadiler 1 year, 6 months ago

Selected Answer: C

C is the correct answer

upvoted 2 times

vexor3 1 year, 6 months ago

Selected Answer: C

C is correct

upvoted 3 times

03355a2 1 year, 7 months ago

Selected Answer: A

Since the team is expecting last week's data to be deleted on Sunday at 1am to 2am. The data will be available for approx 24hrs until the vacuum command is run on Monday at 3am.

upvoted 1 times

cales 1 year, 3 months ago

No! By default Vacuum does not remove rows deleted within the last 7 days. To do it you should modify the property `delta.deletedFileRetentionDuration`

<https://docs.databricks.com/en/delta/history.html#configure-data-retention-for-time-travel-queries>

upvoted 1 times

Assuming that the Databricks CLI has been installed and configured correctly, which Databricks CLI command can be used to upload a custom Python Wheel to object storage mounted with the DBFS for use with a production job?

- A. configure
- B. fs
- C. workspace
- D. libraries

Suggested Answer: *B*

Community vote distribution

B (100%)

🗨️ 👤 **KadELbied** 9 months ago

Selected Answer: B

suretly B

upvoted 1 times

🗨️ 👤 **vexor3** 1 year, 6 months ago

Selected Answer: B

B is correct

upvoted 1 times

The following table consists of items found in user carts within an e-commerce website.

```
Carts (id LONG, items ARRAY<STRUCT<id: LONG, count: INT>>)
id items email
1001[{id: "DESK65", count: 1}] "u1@domain.com"
1002[{id: "KYBD45", count: 1}, {id: "M27", count: 2}] "u2@domain.com"
1003[{id: "M27", count: 1}] "u3@domain.com"
```

The following MERGE statement is used to update this table using an updates view, with schema evolution enabled on this table.

```
MERGE INTO carts c
USING updates u
ON c.id = u.id
WHEN MATCHED
  THEN UPDATE SET *
```

How would the following update be handled?

```
(new nested field, missing existing column)
id items
1001[{id: "DESK65", count: 2, coupon: "BOGO50"}]
```

- A. The update throws an error because changes to existing columns in the target schema are not supported.
- B. The new nested Field is added to the target schema, and dynamically read as NULL for existing unmatched records.
- C. The update is moved to a separate "rescued" column because it is missing a column expected in the target schema.
- D. The new nested field is added to the target schema, and files underlying existing records are updated to include NULL values for the new field.

Suggested Answer: B

Community vote distribution

B (100%)

 **Freyr**  1 year, 8 months ago

Selected Answer: B

Correct Answer: B.

In the question it is mentioned that the schema evolution is enabled. This option states that the new nested field is added to the target schema, and existing records not matching the new schema format are populated with NULL for the newly added columns. This behavior aligns with how schema evolution functions in Delta Lake, dynamically adapting the schema to include new fields.

upvoted 5 times

 **_Lukas_**  2 months, 1 week ago

Selected Answer: B

Correct:

B - With schema evolution enabled (spark.databricks.delta.schema.autoMerge.enabled), Delta Lake detects the new coupon field inside the items array of the source data. It automatically updates the target table's schema to include this new nested field. For all existing records (like 1002 and 1003) that were not updated in this transaction and reside in old files, the new column simply doesn't exist physically. Delta Lake handles this at read time by returning NULL for the missing field, without needing to rewrite the underlying files.

upvoted 1 times

 **KadELbied** 9 months ago

Selected Answer: B

Surely B

upvoted 1 times

 **HairyTorso** 1 year, 1 month ago

Selected Answer: B

Seems like B is correct if below config is enabled:

```
SET spark.databricks.delta.schema.autoMerge.enabled = true;
```

upvoted 1 times

  **pppppppppie** 1 year, 5 months ago

because B says for unmatched records. but my new records has 1001 which is existing record. we should give ans what will happen with this new record. it will update the record in target with NULL as coupon

upvoted 1 times

  **pppppppppie** 1 year, 5 months ago

It has to be D

upvoted 1 times

  **MDWPartners** 1 year, 8 months ago

Selected Answer: B

schema evolution is enabled, so B.

upvoted 4 times

An upstream system is emitting change data capture (CDC) logs that are being written to a cloud object storage directory. Each record in the log indicates the change type (insert, update, or delete) and the values for each field after the change. The source table has a primary key identified by the field pk_id.

For auditing purposes, the data governance team wishes to maintain a full record of all values that have ever been valid in the source system. For analytical purposes, only the most recent value for each record needs to be recorded. The Databricks job to ingest these records occurs once per hour, but each individual record may have changed multiple times over the course of an hour.

Which solution meets these requirements?

- A. Iterate through an ordered set of changes to the table, applying each in turn to create the current state of the table, (insert, update, delete), timestamp of change, and the values.
- B. Use merge into to insert, update, or delete the most recent entry for each pk_id into a table, then propagate all changes throughout the system.
- C. Deduplicate records in each batch by pk_id and overwrite the target table.
- D. Use Delta Lake's change data feed to automatically process CDC data from an external system, propagating all changes to all dependent tables in the Lakehouse.

Suggested Answer: A

Community vote distribution

D (100%)

OnlyPraveen Highly Voted 1 year, 1 month ago

Selected Answer: D

If its exact duplicate of Question #13 then we are missing a good choice of :

"E. Ingest all log information into a bronze table; use MERGE INTO to insert, update, or delete the most recent entry for each pk_id into a silver table to recreate the current table state."

upvoted 7 times

tardigradum Most Recent 6 months ago

Selected Answer: A

You are discussing about an incomplete question. This question is the same as question number 13, where you can find an additional possible answer (E), which is the correct option for this question.

upvoted 2 times

Billybob0604 6 months, 1 week ago

Selected Answer: A

option A is the closest match. None of the answers fully capture both needs

upvoted 1 times

gloomy_marmot 6 months, 2 weeks ago

Selected Answer: A

A. Iterate through an ordered set of changes to the table...

* This approach preserves the order of events.

* Allows you to reconstruct the full history (needed for auditing).

* You can maintain a separate "current state" table as you apply changes.

* Handles multiple changes per pk_id in one batch.

* Works well with Delta Lake as an architecture.

upvoted 1 times

c315d10 8 months, 2 weeks ago

Selected Answer: A

Delta CDF is used to read changes from Delta tables, not ingest changes into them and history of changes are required

upvoted 3 times

KadELbied 9 months ago

Selected Answer: D

Sorry the correct one was E : Ingest all log information into a bronze table; use merge into to insert, update, or delete the most recent entry for each pk_id into a silver table to recreate the current table state

upvoted 1 times

🗨️ **KadELbied** 9 months, 1 week ago

Selected Answer: B

This is the correct answer because it meets the requirements of maintaining a full record of all values that have ever been valid in the source system and recreating the current table state with only the most recent value for each record. The code ingests all log information into a bronze table, which preserves the raw CDC data as it is. Then, it uses merge into to perform an upsert operation on a silver table, which means it will insert new records or update or delete existing records based on the change type and the pk_id columns. This way, the silver table will always reflect the current state of the source table, while the bronze table will keep the history of all changes. Verified References: [Databricks Certified Data Engineer Professional], under "Delta Lake" section; Databricks Documentation, under "Upsert into a table using merge" section.

upvoted 1 times

🗨️ **gloomy_marmot** 6 months, 2 weeks ago

What about

- ✗ This helps with maintaining the latest state only.
- ✗ Does not preserve full history (not sufficient for auditing).
- ✗ If multiple changes per pk_id occur in one batch, last-writer-wins logic may miss intermediate values.

upvoted 1 times

🗨️ **a85becd** 10 months, 1 week ago

Selected Answer: B

Delta Lake's Change Data Feed (CDF) only works on Delta tables, as it is a feature explicitly tied to Delta Lake's architecture. In the scenario described in the question, the source is an external system writing CDC logs to cloud object storage, and there is no mention of the data being stored in Delta tables initially.

upvoted 2 times

🗨️ **e904bf4** 1 year, 2 months ago

Selected Answer: B

The right answer is B because CDF only works on delta tables and here the source is external

upvoted 2 times

🗨️ **Huepig** 1 year, 3 months ago

There is no right answer. Closest is B only after the CDC logs are ingested to a Bronze table and then use merge into a silver table.

Why not D? Because CDF only works on delta tables and not on external CDC logs.

upvoted 3 times

🗨️ **HelixAbdu** 1 year, 6 months ago

The MERGE INTO statement in Delta Lake is a powerful feature designed to handle Change Data Capture (CDC) data efficiently. This approach meets both the auditing and analytical requirements.

CDF is not enabled by default. So these data is not generated by it to handel them.

upvoted 1 times

🗨️ **practicioner** 1 year, 5 months ago

I'd agree, but there is "a full record of all values that have ever been valid in the source system". After deleting records we can still use time-travel options... But after vacuuming audit team will be dissapinted

upvoted 1 times

🗨️ **Ati1362** 1 year, 7 months ago

Selected Answer: D

agree with D

upvoted 2 times

🗨️ **BrianNguyen95** 1 year, 8 months ago

Selected Answer: D

Delta Lake provides built-in change data feed functionality.

It captures changes (inserts, updates, deletes) and propagates them to dependent tables.

By using Delta Lake, you can maintain historical records and propagate changes efficiently.

upvoted 2 times

🔖 👤 **Freyr** 1 year, 8 months ago

Selected Answer: D

Correct Answer: D

Delta Lake's change data feed feature is specifically designed to handle CDC scenarios. It processes data from external systems, tracking all changes (inserts, updates, deletes) and maintaining a detailed history of these changes. This feature allows for keeping a comprehensive log while also ensuring the most recent state is correctly reflected in analytical tables.

upvoted 3 times

An hourly batch job is configured to ingest data files from a cloud object storage container where each batch represent all records produced by the source system in a given hour. The batch job to process these records into the Lakehouse is sufficiently delayed to ensure no late-arriving data is missed. The user_id field represents a unique key for the data, which has the following schema:

user_id BIGINT, username STRING, user_utc STRING, user_region STRING, last_login BIGINT, auto_pay BOOLEAN, last_updated BIGINT

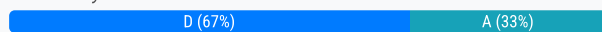
New records are all ingested into a table named account_history which maintains a full record of all data in the same schema as the source. The next table in the system is named account_current and is implemented as a Type 1 table representing the most recent value for each unique user_id.

Which implementation can be used to efficiently update the described account_current table as part of each hourly batch job assuming there are millions of user accounts and tens of thousands of records processed hourly?

- A. Filter records in account_history using the last_updated field and the most recent hour processed, making sure to deduplicate on username; write a merge statement to update or insert the most recent value for each username.
- B. Use Auto Loader to subscribe to new files in the account_history directory; configure a Structured Streaming trigger available job to batch update newly detected files into the account_current table.
- C. Overwrite the account_current table with each batch using the results of a query against the account_history table grouping by user_id and filtering for the max value of last_updated.
- D. Filter records in account_history using the last_updated field and the most recent hour processed, as well as the max last_login by user_id write a merge statement to update or insert the most recent value for each user_id.

Suggested Answer: D

Community vote distribution



Billybob0604 6 months, 1 week ago

Selected Answer: D

if efficiently update means as fast as possible, then the answer is C. if it means reflect the incremental requirement then the answer is D. i m not sure, but i think it s D.

upvoted 1 times

KadELbied 9 months ago

Selected Answer: D

Surety D

upvoted 1 times

benni_ale 1 year, 3 months ago

Selected Answer: D

B or D ... associate course tells us to use auto loader when bilions of rows...

upvoted 1 times

RyanAck24 1 year, 4 months ago

Selected Answer: D

D seems like the best option

upvoted 1 times

shaojunni 1 year, 4 months ago

Selected Answer: B

A, D both wrong. They only take data from the latest update. It is too narrow. Same user_id can have several updates within an hour to update different fields. So use auto loader to apply all the updates within an hour is the only correct answer.

upvoted 2 times

fe3b2fc 1 year, 5 months ago

Selected Answer: A

Answer is A. You're meeting all the requirements with less overhead. It's only updating on the most recent record, so duplicates are handled.

Answer D is too much overhead. They're doing a full table scan for all records, which as the question stated, is millions of records.

upvoted 1 times

  **Onobhas01** 1 year, 5 months ago

User Id would be a better column to merge into with, username might not be distinct

upvoted 1 times

  **Freyr** 1 year, 8 months ago

Selected Answer: D

Correct Answer: D

Similar to Option A, but specifically designed around the user_id, which is the primary key. This approach ensures that the account_current is always up-to-date with the latest information per user based on the primary key, reducing the risk of duplicate information and ensuring the integrity of the data with respect to the unique identifier.

upvoted 3 times

The business intelligence team has a dashboard configured to track various summary metrics for retail stores. This includes total sales for the previous day alongside totals and averages for a variety of time periods. The fields required to populate this dashboard have the following schema:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT, total_sales_ytd
FLOAT, avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT,
avg_daily_sales_7d FLOAT, updated TIMESTAMP
```

For demand forecasting, the Lakehouse contains a validated table of all itemized sales updated incrementally in near real-time. This table, named `products_per_order`, includes the following fields:

```
store_id INT, order_id INT, product_id INT, quantity INT, price FLOAT,
order_timestamp TIMESTAMP
```

Because reporting on long-term sales trends is less volatile, analysts using the new dashboard only require data to be refreshed once daily. Because the dashboard will be queried interactively by many users throughout a normal business day, it should return results quickly and reduce total compute associated with each materialization.

Which solution meets the expectations of the end users while controlling and limiting possible costs?

- A. Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten with each update.
- B. Use Structured Streaming to configure a live dashboard against the `products_per_order` table within a Databricks notebook.
- C. Define a view against the `products_per_order` table and define the dashboard against this view.
- D. Use the Delta Cache to persist the `products_per_order` table in memory to quickly update the dashboard with each query.

Suggested Answer: A

Community vote distribution

A (100%)

  **KadELbied** 9 months ago

Selected Answer: A

suretly A

upvoted 1 times

  **RandomForest** 1 year ago

Selected Answer: A

A is correct: The data should only be refreshed once a day so nightly load will do just fine.

upvoted 1 times

  **17kyu** 1 year, 3 months ago

E

#69 in professional one

upvoted 1 times

  **17kyu** 1 year, 3 months ago

C Define a viwe

upvoted 1 times

  **MDWPartners** 1 year, 8 months ago

Selected Answer: A

Seems correct

upvoted 2 times

A Delta lake table with CDF enabled table in the Lakehouse named customer_churn_params is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

The churn prediction model used by the ML team is fairly stable in production. The team is only interested in making predictions on records that have changed in the past 24 hours.

Which approach would simplify the identification of these changed records?

- A. Apply the churn model to all rows in the customer_churn_params table, but implement logic to perform an upsert into the predictions table that ignores rows where predictions have not changed.
- B. Convert the batch job to a Structured Streaming job using the complete output mode; configure a Structured Streaming job to read from the customer_churn_params table and incrementally predict against the churn model.
- C. Replace the current overwrite logic with a merge statement to modify only those records that have changed; write logic to make predictions on the changed records identified by the change data feed.
- D. Modify the overwrite logic to include a field populated by calling `spark.sql.functions.current_timestamp()` as data are being written; use this field to identify records written on a particular date.

Suggested Answer: C

Community vote distribution

C (100%)

🗳️ 👤 **KadELbied** 9 months ago

Selected Answer: C

Surely C

upvoted 1 times

🗳️ 👤 **RandomForest** 1 year ago

Selected Answer: C

C is correct: Use the fact that CDF is enabled to implement a SCD Type 1 UPSERT logic.

upvoted 1 times

🗳️ 👤 **m79590530** 1 year, 3 months ago

Selected Answer: C

Why use overwrite for a table that has CDF enabled? Best is to modify it in a SCD Type 1 style by using UPSERTs via the MERGE INTO logic and then/also use its CDF system fields as the means to filter which records are current and which not by selecting only the records that have 'insert' or 'update_postimage' in their _change_type CDF system column.

upvoted 3 times

A view is registered with the following code:

```
CREATE VIEW recent_orders AS (  
  SELECT a.user_id, a.email, b.order_id, b.order_date  
  FROM  
    (SELECT user_id, email  
     FROM users) a  
  INNER JOIN  
    (SELECT user_id, order_id, order_date  
     FROM orders  
     WHERE order_date >= (current_date() - 7)) b  
  ON a.user_id = b.user_id  
)
```

Both users and orders are Delta Lake tables.

Which statement describes the results of querying recent_orders?

- A. All logic will execute when the view is defined and store the result of joining tables to the DBFS; this stored data will be returned when the view is queried.
- B. Results will be computed and cached when the view is defined; these cached results will incrementally update as new records are inserted into source tables.
- C. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.
- D. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

Suggested Answer: D

Community vote distribution

D (67%)

B (33%)

🗳️ 👤 **Freyr** Highly Voted 1 year, 8 months ago

Selected Answer: D

Correct Answer: D

Correct because this option correctly describes the behavior of SQL views in Databricks. The view's query is executed against the current state of the data in the source tables at the moment the query begins. This means that any changes to the data that are committed while the query is running will not be reflected in the results of the query currently executing.

upvoted 6 times

🗳️ 👤 **benni_ale** Most Recent 1 year, 3 months ago

Selected Answer: D

its D u scums

upvoted 2 times

🗳️ 👤 **Isio05** 1 year, 8 months ago

Selected Answer: D

Also voting for D, such view results are recalculated each time when called

upvoted 1 times

🗳️ 👤 **MDWPartners** 1 year, 8 months ago

Selected Answer: B

It should be B

upvoted 1 times

A data ingestion task requires a one-TB JSON dataset to be written out to Parquet with a target part-file size of 512 MB. Because Parquet is being used instead of Delta Lake, built-in file-sizing features such as Auto-Optimize & Auto-Compaction cannot be used.

Which strategy will yield the best performance without shuffling data?

- A. Set `spark.sql.files.maxPartitionBytes` to 512 MB, ingest the data, execute the narrow transformations, and then write to parquet.
- B. Set `spark.sql.shuffle.partitions` to 2,048 partitions ($1\text{TB} \times 1024 \times 1024 / 512$), ingest the data, execute the narrow transformations, optimize the data by sorting it (which automatically repartitions the data), and then write to parquet.
- C. Set `spark.sql.adaptive.advisoryPartitionSizeInBytes` to 512 MB bytes, ingest the data, execute the narrow transformations, coalesce to 2,048 partitions ($1\text{TB} \times 1024 \times 1024 / 512$), and then write to parquet.
- D. Ingest the data, execute the narrow transformations, repartition to 2,048 partitions ($1\text{TB} \times 1024 \times 1024 / 512$), and then write to parquet.

Suggested Answer: A

Community vote distribution

A (60%)

D (40%)

 **arekm** Highly Voted 1 year, 1 month ago

Selected Answer: A

Definitely A - no repartitioning and subsequent shuffle (which the question is asking about). The parameter defines how many bytes per partition to read, tasks will read in those chunks, since only narrow operations performed (per definition - no shuffle), we just write what we read. The target files size is 512MBs and we did not shuffle.

upvoted 8 times

 **sainandam** Most Recent 7 months, 2 weeks ago

Selected Answer: C

AQE with coalesce is the only way to produce predictable file-sizes

upvoted 1 times

 **KadELbied** 9 months, 1 week ago

Selected Answer: B

The key to efficiently converting a large JSON dataset to Parquet files of a specific size without shuffling data lies in controlling the size of the output files directly. Setting `spark.sql.files.maxPartitionBytes` to 512 MB configures Spark to process data in chunks of 512 MB. This setting directly influences the size of the part-files in the output, aligning with the target file size.

Narrow transformations (which do not involve shuffling data across partitions) can then be applied to this data.

Writing the data out to Parquet will result in files that are approximately the size specified by `spark.sql.files.maxPartitionBytes`, in this case, 512 MB.

The other options involve unnecessary shuffles or repartitions (B, C, D) or an incorrect setting for this specific requirement (E).

upvoted 1 times

 **RandomForest** 1 year ago

Selected Answer: D

Correct answer is D:

Why Not Other Options?:

A. Set `spark.sql.files.maxPartitionBytes`: This configuration controls how many bytes Spark reads per input partition during a file scan, not the output file size. It does not help in controlling Parquet file sizes during writing.

B. Set `spark.sql.shuffle.partitions` and sort data: While sorting data can optimize performance in some cases, it introduces unnecessary overhead for this scenario. Additionally, `spark.sql.shuffle.partitions` controls the number of shuffle partitions, not directly the output partitioning of the data.

C. Use `spark.sql.adaptive.advisoryPartitionSizeInBytes`: Adaptive Query Execution (AQE) optimizes queries at runtime, but this configuration does not directly control Parquet file sizes. It dynamically adjusts partition sizes for shuffle stages, not for the write output.

upvoted 3 times

 **_lene_** 1 year ago

Selected Answer: A

arekm explanation

upvoted 3 times

 **temple1305** 1 year, 2 months ago

Selected Answer: C

I think, "execute the narrow transformations, coalesce to" is key words here - because coalesce is not cause shuffling.

upvoted 1 times

  **cf56faf** 1 year, 2 months ago

Selected Answer: D

It's D, because A primarily affects the reading of the data

upvoted 2 times

  **Jugiboss** 1 year, 3 months ago

Selected Answer: A

A does not shuffle while D shuffles

upvoted 3 times

  **m79590530** 1 year, 3 months ago

Selected Answer: A

Answer A as narrow transformations like union, filter and map do not cause shuffle across partitions.

upvoted 3 times

  **Colje** 1 year, 3 months ago

Selected Answer: D

The correct answer is D. Ingest the data, execute the narrow transformations, repartition to 2,048 partitions ($1\text{TB} * 1024 * 1024 / 512$), and then write to Parquet.

Explanation:

In this case, the goal is to write a 1 TB dataset to Parquet with a target file size of 512 MB without incurring the overhead of data shuffling. To achieve optimal performance, we must balance the number of partitions to match the file size requirements while avoiding expensive shuffle operations.

Narrow transformations: These transformations (such as map, filter) don't require shuffling the data, which keeps the operation efficient.

Repartition to 2,048 partitions: Given that the desired part-file size is 512 MB and the total dataset size is 1 TB, repartitioning the dataset into 2,048 partitions ensures that each partition will be approximately 512 MB in size, which matches the target file size. This avoids shuffle operations and allows for an efficient write.

upvoted 1 times

  **arekm** 1 year, 1 month ago

All true, but not a correct answer. We are looking for a solution without shuffle/repartition/coalesce.

upvoted 1 times

  **pk07** 1 year, 4 months ago

Selected Answer: D

Not A because spark.sql.files.maxPartitionBytes primarily affects the reading of data, not the writing. It determines the maximum size of a partition when reading files, not when writing them.

upvoted 2 times

  **shaojunni** 1 year, 4 months ago

Selected Answer: C

A, D will not prevent shuffling data. C using coalesce to reduce shuffling data.

upvoted 1 times

  **03355a2** 1 year, 7 months ago

Selected Answer: A

best performance without shuffling data

upvoted 4 times

  **hpkr** 1 year, 8 months ago

Selected Answer: D

option D

upvoted 1 times

  **Freyr** 1 year, 8 months ago

Selected Answer: D

Correct Answer D: Repartition to 2,048 partitions and write to Parquet

This option directly controls the number of output files by repartitioning the data into 2,048 partitions, assuming that 1TB/512MB per file roughly

translates to 2,048 files. Repartitioning the data involves shuffling, but it's a deliberate shuffle designed to achieve a specific partitioning beneficial for writing. After repartitioning, the data is written to Parquet files, each expected to be approximately 512 MB if the data is uniformly distributed across partitions.

upvoted 1 times

Which statement regarding stream-static joins and static Delta tables is correct?

- A. The checkpoint directory will be used to track updates to the static Delta table.
- B. Each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization.
- C. The checkpoint directory will be used to track state information for the unique keys present in the join.
- D. Stream-static joins cannot use static Delta tables because of consistency issues.

Suggested Answer: B

Community vote distribution

B (100%)

 **KadELbied** 9 months, 1 week ago

Selected Answer: D

need to add this response : E: Each microbatch of a stream-static join will use the most recent version of the static Delta table as of each microbatch is the right response

upvoted 1 times

 **arekm** 1 year, 1 month ago

Selected Answer: B

All answers are wrong:

A - checkpoint directory to track changes to Delta table?

B - microbatch uses the state of the table at the time the query is executed, not at initialization

C - unique keys? - stream-static joins are not stateful, so we are only looking at the current batch of records

D - you can totally have stream-static joins, see: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#support-matrix-for-joins-in-streaming-queries>

I believe they made a typo in the B, that seems to be the only logical explanation.

upvoted 2 times

 **benni_ale** 1 year, 3 months ago

Selected Answer: A

If you look at question 18 you find that the correct solution should be Each microbatch of a stream-static join will use the most recent version of the static Delta table as of each microbatch. This is not listed here meaning that B could not be correct leading to A being the only possible solution....

The wrong part about B is that the latest version of the static delta table is returned at each micro-batch rather than as of job initialisation...

upvoted 1 times

 **MDWPartners** 1 year, 8 months ago

Selected Answer: B

When Databricks processes a micro-batch of data in a stream-static join, the latest valid version of data from the static Delta table joins with the records present in the current micro-batch. Because the join is stateless, you do not need to configure watermarking and can process results with low latency. The data in the static Delta table used in the join should be slowly-changing.

<https://docs.databricks.com/en/transform/join.html#stream-static>

upvoted 3 times

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Events are recorded once per minute per device.

Streaming DataFrame df has the following schema:

"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"

Code block:

```
df.withWatermark("event_time", "10 minutes")
  .groupBy(
    _____,
    "device_id"
  )
  .agg(
    avg("temp").alias("avg_temp"),
    avg("humidity").alias("avg_humidity")
  )
  .writeStream
  .format("delta")
  .saveAsTable("sensor_avg")
```

Which line of code correctly fills in the blank within the code block to complete this task?

- A. to_interval("event_time", "5 minutes").alias("time")
- B. window("event_time", "5 minutes").alias("time")
- C. "event_time"
- D. lag("event_time", "10 minutes").alias("time")

Suggested Answer: B

Community vote distribution

B (100%)

 **happyhelppy** 6 months, 3 weeks ago

Selected Answer: B

Why are the questions repeated again and again? I did not pay for the copy-pasted questions!

upvoted 3 times

 **RandomForest** 1 year ago

Selected Answer: B

Correct answer is B: use the window-function

upvoted 1 times

 **m79590530** 1 year, 3 months ago

Selected Answer: B

This is the standard syntax to do non-overlapping time interval Window-ed grouping by the time field in a dataset in Structured Streaming.

.withWatermark() function defines the staging buffers after which delayed records will be dropped/ignored.

upvoted 2 times

A Structured Streaming job deployed to production has been resulting in higher than expected cloud storage costs. At present, during normal execution, each microbatch of data is processed in less than 3s; at least 12 times per minute, a microbatch is processed that contains 0 records. The streaming write was configured using the default trigger settings. The production job is currently scheduled alongside many other Databricks jobs in a workspace with instance pools provisioned to reduce start-up time for jobs with batch execution.

Holding all other variables constant and assuming records need to be processed in less than 10 minutes, which adjustment will meet the requirement?

- A. Set the trigger interval to 3 seconds; the default trigger interval is consuming too many records per batch, resulting in spill to disk that can increase volume costs.
- B. Use the trigger once option and configure a Databricks job to execute the query every 10 minutes; this approach minimizes costs for both compute and storage.
- C. Set the trigger interval to 10 minutes; each batch calls APIs in the source storage account, so decreasing trigger frequency to maximum allowable threshold should minimize this cost.
- D. Set the trigger interval to 500 milliseconds; setting a small but non-zero trigger interval ensures that the source is not queried too frequently.

Suggested Answer: B

Community vote distribution

C (100%)

Isio05 Highly Voted 1 year, 7 months ago

Selected Answer: C

C,

A - incorrect explanation

B - trigger once is not correct option here

D - 500 milliseconds is already used, it's default trigger interval
upvoted 5 times

8c1d06f Most Recent 4 months ago

Selected Answer: B

B. Use trigger once and schedule the job to run every 10 minutes.

This processes all available data then stops, cutting the frequent empty micro-batches (and their storage API calls) and also reducing compute by letting the cluster spin down between runs; using instance pools keeps start-up quick.

upvoted 3 times

happyhelpy 6 months, 3 weeks ago

Selected Answer: B

Trigger once + scheduled job (option B) is the most cost-effective way to minimize empty microbatch processing and storage overhead while meeting the latency requirement.

upvoted 2 times

c315d10 8 months, 2 weeks ago

Selected Answer: B

workspace with instance pools is best candidate for trigger once in batch

upvoted 2 times

hpkr 1 year, 8 months ago

Selected Answer: C

Option C

upvoted 1 times

Which statement describes Delta Lake optimized writes?

- A. Before a Jobs cluster terminates, OPTIMIZE is executed on all tables modified during the most recent job.
- B. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 1 GB.
- C. A shuffle occurs prior to writing to try to group similar data together resulting in fewer files instead of each executor writing multiple files based on directory partitions.
- D. Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.

Suggested Answer: C

Community vote distribution

C (100%)

🗳️ 👤 **sainandam** 7 months, 2 weeks ago

Selected Answer: B

C is the feature of Z-ordering. So it is B.

upvoted 1 times

🗳️ 👤 **KadELbied** 8 months, 4 weeks ago

Selected Answer: C

Surely C

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: C

C thanks

upvoted 1 times

🗳️ 👤 **Farid77** 1 year, 4 months ago

B is correct.

C is wrong OPTIMIZE is a separate process from write.

upvoted 1 times

🗳️ 👤 **vexor3** 1 year, 6 months ago

Selected Answer: C

C is correct

upvoted 2 times

🗳️ 👤 **only_vimal** 1 year, 6 months ago

Please provide your input to Questions 144,145,146,147,149 also. Thanks in advance

upvoted 1 times

Which statement characterizes the general programming model used by Spark Structured Streaming?

- A. Structured Streaming leverages the parallel processing of GPUs to achieve highly parallel data throughput.
- B. Structured Streaming is implemented as a messaging bus and is derived from Apache Kafka.
- C. Structured Streaming relies on a distributed network of nodes that hold incremental state values for cached stages.
- D. Structured Streaming models new data arriving in a data stream as new rows appended to an unbounded table.

Suggested Answer: D

Community vote distribution

D (100%)

🗨️ 👤 **KadELbied** 8 months, 4 weeks ago

Selected Answer: D

Surely D

upvoted 1 times

🗨️ 👤 **vexor3** 1 year, 6 months ago

Selected Answer: D

D is correct

upvoted 2 times

Which configuration parameter directly affects the size of a spark-partition upon ingestion of data into Spark?

- A. spark.sql.files.maxPartitionBytes
- B. spark.sql.autoBroadcastJoinThreshold
- C. spark.sql.adaptive.advisoryPartitionSizeInBytes
- D. spark.sql.adaptive.coalescePartitions.minPartitionNum

Suggested Answer: A

Community vote distribution

A (100%)

  **_lene_** 1 year ago

Selected Answer: A

Property Name Default Meaning

spark.sql.files.maxPartitionBytes 134217728 (128 MB) The maximum number of bytes to pack into a single partition when reading files. This configuration is effective only when using file-based sources such as Parquet, JSON and ORC.

upvoted 1 times

  **vexor3** 1 year, 6 months ago

Selected Answer: A

A is correct

upvoted 1 times

A Spark job is taking longer than expected. Using the Spark UI, a data engineer notes that the Min, Median, and Max Durations for tasks in a particular stage show the minimum and median time to complete a task as roughly the same, but the max duration for a task to be roughly 100 times as long as the minimum.

Which situation is causing increased duration of the overall job?

- A. Task queueing resulting from improper thread pool assignment.
- B. Spill resulting from attached volume storage being too small.
- C. Network latency due to some cluster nodes being in different regions from the source data
- D. Skew caused by more data being assigned to a subset of spark-partitions.

Suggested Answer: D

Community vote distribution

D (100%)

🗲️ 👤 **happyhelppy** 6 months, 3 weeks ago

Selected Answer: D

same as question #25

upvoted 1 times

🗲️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: D

D is ok

upvoted 1 times

🗲️ 👤 **vexor3** 1 year, 6 months ago

Selected Answer: D

D is correct

upvoted 1 times

Each configuration below is identical to the extent that each cluster has 400 GB total of RAM, 160 total cores and only one Executor per VM.

Given an extremely long-running job for which completion must be guaranteed, which cluster configuration will be able to guarantee completion of the job in light of one or more VM failures?

- A. • Total VMs: 8
 - 50 GB per Executor
 - 20 Cores / Executor
- B. • Total VMs: 16
 - 25 GB per Executor
 - 10 Cores / Executor
- C. • Total VMs: 1
 - 400 GB per Executor
 - 160 Cores/Executor
- D. • Total VMs: 4
 - 100 GB per Executor
 - 40 Cores / Executor

Suggested Answer: B

Community vote distribution

B (100%)

  **m79590530** 1 year, 3 months ago

Selected Answer: B

Distributing work across more Workers/Executors will have better guarantee in case of 1 or more of them fail
upvoted 1 times

A task orchestrator has been configured to run two hourly tasks. First, an outside system writes Parquet data to a directory mounted at /mnt/raw_orders/. After this data is written, a Databricks job containing the following code is executed:

```
(spark.readStream
  .format("parquet")
  .load("/mnt/raw_orders/")
  .withWatermark("time", "2 hours")
  .dropDuplicates(["customer_id", "order_id"])
  .writeStream
  .trigger(once=True)
  .table("orders")
)
```

Assume that the fields customer_id and order_id serve as a composite key to uniquely identify each order, and that the time field indicates when the record was queued in the source system.

If the upstream system is known to occasionally enqueue duplicate entries for a single order hours apart, which statement is correct?

- A. Duplicate records enqueued more than 2 hours apart may be retained and the orders table may contain duplicate records with the same customer_id and order_id.
- B. All records will be held in the state store for 2 hours before being deduplicated and committed to the orders table.
- C. The orders table will contain only the most recent 2 hours of records and no duplicates will be present.
- D. The orders table will not contain duplicates, but records arriving more than 2 hours late will be ignored and missing from the table.

Suggested Answer: A

Community vote distribution

A (100%)

 **arekm** 1 year, 1 month ago

Selected Answer: A

A - two orders [customer_id, order_id] might be emitted with time column value that is 2 or more hours apart. So the watermark will not drop the new record, since it will have a new time value. From the composite key perspective, it is going to be a duplicate.

upvoted 1 times

 **Urcolbz** 1 year, 1 month ago

Selected Answer: A

dropDuplicates only deletes the duplicates of the processed batch. If we have records with same key in different batches, we will have duplicates in the final table.

In addition, withWatermark, when is not used in a window, gets the MAX(eventTime) and uses the threshold to define the time range. As the time represent when the data has been queued in the source system, we can get records where the time we get older than 2 hours.

[pyspark.sql.DataFrame.dropDuplicates — PySpark 3.5.3 documentation](#)

[pyspark.sql.DataFrame.withWatermark — PySpark 3.5.3 documentation](#)

upvoted 2 times

 **73109a1** 11 months ago

Without any windowing or event-time-based aggregation, the watermark won't be used in the query.

upvoted 1 times

 **vish9** 1 year, 2 months ago

Selected Answer: D

The orders arriving 2 hours or later will be dropped. There is a chance that they can be processed, but still deduplication will happen.

upvoted 2 times

  **smashit** 1 year, 3 months ago

There might be chance that same record for example A1,O1 comes in Batch B1 also comes in B2. we need to implement merge logic inside our target table or perform insert-only merge.

upvoted 1 times

  **Jugiboss** 1 year, 3 months ago

Selected Answer: A

Watermark thresholds guarantee that records arriving within the specified threshold are processed according to the semantics of the defined query. Late-arriving records arriving outside the specified threshold might still be processed using query metrics, but this is not guaranteed.

upvoted 1 times

  **m79590530** 1 year, 3 months ago

Selected Answer: D

The default write mode is 'append'. Duplicate will be resolved for each 2 hr window and `.withWatermark()` will drop/ignore the records delayed more than 2 hours apart.

upvoted 1 times

  **csrazdan** 1 year, 5 months ago

Selected Answer: A

The default write mode is append. Duplicate will be resolved for only 2 hr window but may still exist because of previous execution.

upvoted 1 times

A data engineer is configuring a pipeline that will potentially see late-arriving, duplicate records.

In addition to de-duplicating records within the batch, which of the following approaches allows the data engineer to deduplicate data against previously processed records as it is inserted into a Delta table?

- A. Rely on Delta Lake schema enforcement to prevent duplicate records.
- B. VACUUM the Delta table after each batch completes.
- C. Perform an insert-only merge with a matching condition on a unique key.
- D. Perform a full outer join on a unique key and overwrite existing data.

Suggested Answer: C

Community vote distribution

C (100%)

🗳️ 👤 **KadELbied** 8 months, 3 weeks ago

Selected Answer: C

Surely C

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: C

C seems logical

upvoted 1 times

🗳️ 👤 **m79590530** 1 year, 3 months ago

Selected Answer: C

From all the provided options Answer C is the only meaningful and possible one. Also MERGE INTO ... WHEN NOT MATCHED INSERT *; is a standard solution for adding/appending non-existing records (by key) to the target table without duplicating.

upvoted 2 times

A junior data engineer seeks to leverage Delta Lake's Change Data Feed functionality to create a Type 1 table representing all of the values that have ever been valid for all rows in a bronze table created with the property `delta.enableChangeDataFeed = true`. They plan to execute the following code as a daily job:

```
from pyspark.sql.functions import col

(spark.read.format("delta")
 .option("readChangeFeed", "true")
 .option("startingVersion", 0)
 .table("bronze")
 .filter(col("_change_type").isin(["update_postimage", "insert"]))
 .write
 .mode("append")
 .table("bronze_history_type1")
)
```

Which statement describes the execution and results of running the above query multiple times?

- A. Each time the job is executed, newly updated records will be merged into the target table, overwriting previous values with the same primary keys.
- B. Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.
- C. Each time the job is executed, only those records that have been inserted or updated since the last execution will be appended to the target table, giving the desired result.
- D. Each time the job is executed, the differences between the original and current versions are calculated; this may result in duplicate entries for some records.

Suggested Answer: B

🗨️ **benni_ale** 1 year, 3 months ago

Selected Answer: B

B seems ok

upvoted 1 times

🗨️ **m79590530** 1 year, 3 months ago

Selected Answer: B

Since the code is using version 0 for the CDF-enabled table every time it is executed all the historical changes being insert or update for the table will be 'append'-ed to the target table since this is the write command option provided.

upvoted 2 times

🗨️ **Adrifersilva** 1 year, 4 months ago

B. This bad effect (many duplicates) happens because the code reads from the starting version 0, appending all changes since the beginning.

upvoted 3 times

A DLT pipeline includes the following streaming tables:

- raw_iot ingests raw device measurement data from a heart rate tracking device.
- bpm_stats incrementally computes user statistics based on BPM measurements from raw_iot.

How can the data engineer configure this pipeline to be able to retain manually deleted or updated records in the raw_iot table, while recomputing the downstream table bpm_stats table when a pipeline update is run?

- A. Set the pipelines.reset.allowed property to false on raw_iot
- B. Set the skipChangeCommits flag to true on raw_iot
- C. Set the pipelines.reset.allowed property to false on bpm_stats
- D. Set the skipChangeCommits flag to true on bpm_stats

Suggested Answer: B

Community vote distribution

A (100%)

  **c315d10** 8 months, 2 weeks ago

Selected Answer: A

<https://docs.databricks.com/aws/en/dlt/transform>

```
CREATE OR REFRESH STREAMING TABLE raw_user_table
TBLPROPERTIES(pipelines.reset.allowed = false)
AS SELECT * FROM STREAM read_files("/databricks-datasets/iot-stream/data-user", format => "csv");
```

```
CREATE OR REFRESH STREAMING TABLE bmi_table
AS SELECT userid, (weight/2.2) / pow(height*0.0254,2) AS bmi FROM STREAM(raw_user_table);
```

upvoted 2 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: A

In Databricks Lakehouse, to retain manually deleted or updated records in the raw_iot table while recomputing downstream tables when a pipeline update is run, the property pipelines.reset.allowed should be set to false. This property prevents the system from resetting the state of the table, which includes the removal of the history of changes, during a pipeline update. By keeping this property as false, any changes to the raw_iot table, including manual deletes or updates, are retained, and recomputation of downstream tables, such as bpm_stats, can occur with the full history of data changes intact.

upvoted 2 times

  **teowis** 10 months ago

Selected Answer: A

The purpose of it is to take into account changes made manually in the raw_iot table and propagate them to the downstream tables.

upvoted 1 times

  **Sriramiyer92** 1 year, 1 month ago

Selected Answer: B

Answer is B.

Ignore updates and deletes - <https://docs.databricks.com/en/structured-streaming/delta-lake.html#ignore-updates-and-deletes>

Structured Streaming does not handle input that is not an append and throws an exception if any modifications occur on the table being used as a source. There are two main strategies for dealing with changes that cannot be automatically propagated downstream:

You can delete the output and checkpoint and restart the stream from the beginning.

You can set either of these two options:

ignoreDeletes: ignore transactions that delete data at partition boundaries.

skipChangeCommits: ignore transactions that delete or modify existing records. skipChangeCommits subsumes ignoreDeletes.

upvoted 2 times

🗳️ 👤 **m79590530** 1 year, 3 months ago

Selected Answer: A

Setting pipelines.reset.allowed property to false on raw_iot prevents its full refresh/reset being done from its source, meaning it preserves all other modifications done for it additionally outside of its source streaming process. If we set skipChangeCommits for it or the target table transactions that delete or modify records on the table are ignored

upvoted 2 times

🗳️ 👤 **pk07** 1 year, 4 months ago

Selected Answer: B

B. Set the skipChangeCommits flag to true on raw_iot

Let's break down the requirements and explain why this is the best solution:

Retain manually deleted or updated records in raw_iot: The skipChangeCommits flag, when set to true, tells Delta Live Tables (DLT) to ignore any manual changes (updates or deletes) made to the table outside of the pipeline. This means that even if records are manually deleted or updated in the raw_iot table, these changes won't be reflected in the table when the pipeline runs again.

Recompute downstream bpm_stats table: By default, DLT will recompute downstream tables when their upstream dependencies change. Since bpm_stats is based on raw_iot, it will naturally be recomputed when the pipeline updates, without any special configuration.

Why the other options are not correct:

A. Setting pipelines.reset.allowed to false on raw_iot would prevent the table from being reset, but it wouldn't address the requirement to retain manually deleted or updated records.

upvoted 4 times

🗳️ 👤 **pk07** 1 year, 4 months ago

Selected Answer: B

B. Set the skipChangeCommits flag to true on raw_iot

Let's break down the requirements and explain why this is the best solution:

Retain manually deleted or updated records in raw_iot: The skipChangeCommits flag, when set to true, tells Delta Live Tables (DLT) to ignore any manual changes (updates or deletes) made to the table outside of the pipeline. This means that even if records are manually deleted or updated in the raw_iot table, these changes won't be reflected in the table when the pipeline runs again.

Recompute downstream bpm_stats table: By default, DLT will recompute downstream tables when their upstream dependencies change. Since bpm_stats is based on raw_iot, it will naturally be recomputed when the pipeline updates, without any special configuration.

Why the other options are not correct:

A. Setting pipelines.reset.allowed to false on raw_iot would prevent the table from being reset, but it wouldn't address the requirement to retain manually deleted or updated records.

upvoted 1 times

🗳️ 👤 **nikoliko** 1 year, 7 months ago

answer A

This property, when set to false, ensures that the table will not be reset during pipeline updates, thus preserving manually deleted or updated records. This is crucial for the raw_iot table to retain the manual modifications.

upvoted 4 times

🗳️ 👤 **c00ccb7** 1 year, 7 months ago

Selected Answer: A

Set the pipelines.reset.allowed property to false on raw_iot

upvoted 3 times

🗳️ 👤 **Deb9753** 1 year, 8 months ago

Answer: B

upvoted 1 times

A data pipeline uses Structured Streaming to ingest data from Apache Kafka to Delta Lake. Data is being stored in a bronze table, and includes the Kafka-generated timestamp, key, and value. Three months after the pipeline is deployed, the data engineering team has noticed some latency issues during certain times of the day.

A senior data engineer updates the Delta Table's schema and ingestion logic to include the current timestamp (as recorded by Apache Spark) as well as the Kafka topic and partition. The team plans to use these additional metadata fields to diagnose the transient processing delays.

Which limitation will the team face while diagnosing this problem?

- A. New fields will not be computed for historic records.
- B. Spark cannot capture the topic and partition fields from a Kafka source.
- C. Updating the table schema requires a default value provided for each field added.
- D. Updating the table schema will invalidate the Delta transaction log metadata.

Suggested Answer: A

Community vote distribution

A (100%)

🗨️ 👤 **RandomForest** 1 year ago

Selected Answer: A

A is correct: the old records are lost as no history was saved for the new topic.
upvoted 1 times

🗨️ 👤 **m79590530** 1 year, 3 months ago

Selected Answer: A

There is no way to reprocess history/old records to populate these values after 3 months as Kafka does not necessarily preserve them so long. This is the function of the Raw or Bronze table. Also, the other answers just don't make sense.
upvoted 2 times

A nightly job ingests data into a Delta Lake table using the following code:

```
from pyspark.sql.functions import current_timestamp, input_file_name, col
from pyspark.sql.column import Column

def ingest_daily_batch(time_col: Column, year:int, month:int, day:int):
    (spark.read
     .format("parquet")
     .load(f"/mnt/daily_batch/{year}/{month}/{day}")
     .select("time_col.alias('ingest_time'),
            input_file_name().alias('source_file')
            )
     .write
     .mode("append")
     .saveAsTable("bronze")
    )
```

The next step in the pipeline requires a function that returns an object that can be used to manipulate new records that have not yet been processed to the next table in the pipeline.

Which code snippet completes this function definition?

def new_records():

- A. return spark.readStream.table("bronze")
- B. return spark.read.option("readChangeFeed", "true").table("bronze")
- C.

```
return (spark.read
        .table("bronze")
        .filter(col("ingest_time") == current_timestamp())
        )
return (spark.read
        .table("bronze")
        .filter(col("source_file") == f"/mnt/daily_batch/{year}/{month}/{day}")
        )
```
- D.

```
return (spark.read
        .table("bronze")
        .filter(col("source_file") == f"/mnt/daily_batch/{year}/{month}/{day}")
        )
```

Suggested Answer: C

Community vote distribution

D (50%)

B (50%)

Freyr 1 year, 8 months ago

Selected Answer: B

Correct Answer: B

The Change Data Feed (CDF) feature in Delta Lake enables reading only the changes (inserts and updates) to a Delta table. This would allow the function to focus on new or modified data since the last trigger, making it ideal for processing only the new records that have not been processed yet. This directly meets the requirement for identifying and manipulating new records efficiently.

upvoted 7 times

practicioner 1 year, 5 months ago

We are ingesting data from the folder with a parquet in the bronze table. It doesn't make any sense to use the CDF feature for bronze table)

upvoted 1 times

practicioner 1 year, 5 months ago

I've changed my opinion. Yes, B looks as correct answer

upvoted 2 times

arekm 1 year, 1 month ago

It is missing the starting version. You either use readStream without the version or read (aka batch) with the version. In here in B the version is missing. Since the source table is append only, it is perfect for streaming, making A the right choice.

upvoted 3 times

🗨️ **gloomy_marmot** Most Recent 6 months, 2 weeks ago

Selected Answer: B

Looks like B is more suitable for batch processing with Delta (default for table) and CDF
upvoted 1 times

🗨️ **thierryb** 10 months, 2 weeks ago

Selected Answer: A

any tables can become a stream in Databricks, therefore answer A.
It cannot be answer B as the table named bronze does not have CDF enabled
upvoted 2 times

🗨️ **mohadjhamad** 10 months, 3 weeks ago

Selected Answer: B

The function `new_records()` is meant to retrieve new records that have not yet been processed in the pipeline.

Delta Lake provides a feature called the Change Data Feed (CDF), which allows you to track changes (inserts, updates, and deletes) in a Delta table.

By using `.option("readChangeFeed", "true")`, the function can read only the new changes from the Delta table named "bronze".

This ensures that only unprocessed records are returned, aligning with the pipeline's requirement.

upvoted 2 times

🗨️ **arekm** 1 year, 1 month ago

Selected Answer: A

A - append only table works for streaming, B is missing the starting version, C - timestamp during the insertion will be different from the one during the next step, D - that was a "like" type query (or maybe a substring on the file name == the directory), then yes - actual file names only start with the pattern presented, but are longer.

upvoted 1 times

🗨️ **Urcolbz** 1 year, 1 month ago

Selected Answer: A

Option A seems to be the correct answer.

Option B seems not to be the right one, because is missing the version. Based on the documentation 'Change data feed also supports batch execution, which requires specifying a starting version'.

<https://learn.microsoft.com/en-us/azure/databricks/delta/delta-change-data-feed#batch>

Option D, in my opinion, is not correct, as the function definition above is not having any input parameter.

upvoted 3 times

🗨️ **m79590530** 1 year, 3 months ago

Selected Answer: A

Correct answer is A as we have append-only mode writes which are ideal for simple Structured Streaming as a next step ;)

upvoted 3 times

🗨️ **shaojunni** 1 year, 3 months ago

Selected Answer: A

delta table returns new records in streaming read.

upvoted 3 times

🗨️ **pk07** 1 year, 4 months ago

Selected Answer: B

B. Set the `skipChangeCommits` flag to true on `raw_iot`

Let's break down the requirements and explain why this is the best solution:

Retain manually deleted or updated records in `raw_iot`: The `skipChangeCommits` flag, when set to true, tells Delta Live Tables (DLT) to ignore any manual changes (updates or deletes) made to the table outside of the pipeline. This means that even if records are manually deleted or updated in the `raw_iot` table, these changes won't be reflected in the table when the pipeline runs again.

Recompute downstream `bpm_stats` table: By default, DLT will recompute downstream tables when their upstream dependencies change. Since `bpm_stats` is based on `raw_iot`, it will naturally be recomputed when the pipeline updates, without any special configuration.

Why the other options are not correct:

A. Setting `pipelines.reset.allowed` to `false` on `raw_iot` would prevent the table from being reset, but it wouldn't address the requirement to retain manually deleted or updated records.

upvoted 1 times

  **shaojunni** 1 year, 4 months ago

Selected Answer: D

You have to know the CDF's current version and last processed the version in order to get not processed records. B does not provide those versions. It will just return content from the bronze table with CDF turned on. D is only possible solution.

upvoted 1 times

  **HelixAbdu** 1 year, 6 months ago

I did not test it. But i think D is wrong as it filtering agenst directory path using `==`

upvoted 2 times

  **MDWPartners** 1 year, 8 months ago

Selected Answer: D

Seems D

upvoted 4 times

A junior data engineer is working to implement logic for a Lakehouse table named `silver_device_recordings`. The source data contains 100 unique fields in a highly nested JSON structure.

The `silver_device_recordings` table will be used downstream to power several production monitoring dashboards and a production model. At present, 45 of the 100 fields are being used in at least one of these applications.

The data engineer is trying to determine the best approach for dealing with schema declaration given the highly-nested structure of the data and the numerous fields.

Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

- A. The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.
- B. Because Delta Lake uses Parquet for data storage, data types can be easily evolved by just modifying file footer information in place.
- C. Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.
- D. Because Databricks will infer schema using types that allow all observed data to be processed, setting types manually provides greater assurance of data quality enforcement.

Suggested Answer: D

Community vote distribution

D (100%)

Isio05 1 year, 7 months ago

Selected Answer: D

Agree with propopsed answer, D
upvoted 3 times

hpkr 1 year, 7 months ago

Selected Answer: D

D is correct
upvoted 2 times

The data engineering team maintains the following code:

```
accountDF = spark.table("accounts")
orderDF = spark.table("orders")
itemDF = spark.table("items")

orderWithItemDF = (orderDF.join(
    itemDF,
    orderDF.itemID == itemDF.itemID)
    .select(
        orderDF.accountID,
        orderDF.itemID,
        itemDF.itemName))

finalDF = (accountDF.join(
    orderWithItemDF,
    accountDF.accountID == orderWithItemDF.accountID)
    .select(
        orderWithItemDF["*"],
        accountDF.city))

(finalDF.write
    .mode("overwrite")
    .table("enriched_itemized_orders_by_account"))
```

Assuming that this code produces logically correct results and the data in the source tables has been de-duplicated and validated, which statement describes what will occur when this code is executed?

- A. A batch job will update the enriched_itemized_orders_by_account table, replacing only those rows that have different values than the current version of the table, using accountID as the primary key.
- B. The enriched_itemized_orders_by_account table will be overwritten using the current valid version of data in each of the three tables referenced in the join logic.
- C. No computation will occur until enriched_itemized_orders_by_account is queried; upon query materialization, results will be calculated using the current valid version of data in each of the three tables referenced in the join logic.
- D. An incremental job will detect if new rows have been written to any of the source tables; if new rows are detected, all results will be recalculated and used to overwrite the enriched_itemized_orders_by_account table.

Suggested Answer: B

Community vote distribution

B (100%)

 **c315d10** 8 months, 2 weeks ago

Selected Answer: B

correct

upvoted 1 times

 **KadELbied** 9 months, 1 week ago

Selected Answer: B

This is the correct answer because it describes what will occur when this code is executed. The code uses three Delta Lake tables as input sources: accounts, orders, and order_items. These tables are joined together using SQL queries to create a view called new_enriched_itemized_orders_by_account, which contains information about each order item and its associated account details. Then, the code uses write.format("delta").mode("overwrite") to overwrite a target table called enriched_itemized_orders_by_account using the data from the view. This means that every time this code is executed, it will replace all existing data in the target table with new data based on the current valid version of data in each of the three input tables. Verified References: [Databricks Certified Data Engineer Professional], under "Delta Lake" section; Databricks Documentation, under "Write to Delta tables" section.

upvoted 1 times

  **hpk** 1 year, 7 months ago

Selected Answer: B

B is correct

upvoted 3 times

The data engineering team is configuring environments for development, testing, and production before beginning migration on a new data pipeline. The team requires extensive testing on both the code and data resulting from code execution, and the team wants to develop and test against data as similar to production data as possible.

A junior data engineer suggests that production data can be mounted to the development and testing environments, allowing pre-production code to execute against production data. Because all users have admin privileges in the development environment, the junior data engineer has offered to configure permissions and mount this data for the team.

Which statement captures best practices for this situation?

- A. All development, testing, and production code and data should exist in a single, unified workspace; creating separate environments for testing and development complicates administrative overhead.
- B. In environments where interactive code will be executed, production data should only be accessible with read permissions; creating isolated databases for each environment further reduces risks.
- C. Because access to production data will always be verified using passthrough credentials, it is safe to mount data to any Databricks development environment.
- D. Because Delta Lake versions all data and supports time travel, it is not possible for user error or malicious actors to permanently delete production data; as such, it is generally safe to mount production data anywhere.

Suggested Answer: B

Community vote distribution

B (100%)

🗨️ 👤 **m79590530** 1 year, 3 months ago

Selected Answer: B

Production data should be maximum secured against intentional and unintentional modifications by developers or workspace/UC admins. So setting it up with read only access and in different catalog or schema/database per environment is best approach.

upvoted 2 times

The data architect has mandated that all tables in the Lakehouse should be configured as external Delta Lake tables.

Which approach will ensure that this requirement is met?

- A. Whenever a database is being created, make sure that the LOCATION keyword is used.
- B. When the workspace is being configured, make sure that external cloud object storage has been mounted.
- C. Whenever a table is being created, make sure that the LOCATION keyword is used.
- D. When tables are created, make sure that the UNMANAGED keyword is used in the CREATE TABLE statement.

Suggested Answer: C

Community vote distribution

C (100%)

🗳️ 👤 **benni_ale** 1 year, 3 months ago

Selected Answer: C

Location keyword in CTAS statment in only way to create External tables
upvoted 1 times

🗳️ 👤 **m79590530** 1 year, 3 months ago

Selected Answer: C

CREATE-ing a TABLE with LOCATION key word makes it EXTERNAL TABLE. By CREATE-ing the database/schema with the LOCATION key word we can have specific locations for the schemas/databases containing MANAGED tables when these tables inside these schemas/databases are CREATE-d withOUT the LOCATION key word.

This approach allows for configuring MANAGED TABLES at specific locations by fully leveraging Databricks Lakehouse automatic optimizations and performance tuning for them.

upvoted 2 times

🗳️ 👤 **benni_ale** 1 year, 3 months ago

U are a dragon

upvoted 1 times

The marketing team is looking to share data in an aggregate table with the sales organization, but the field names used by the teams do not match, and a number of marketing-specific fields have not been approved for the sales org.

Which of the following solutions addresses the situation while emphasizing simplicity?

- A. Create a view on the marketing table selecting only those fields approved for the sales team; alias the names of any fields that should be standardized to the sales naming conventions.
- B. Create a new table with the required schema and use Delta Lake's DEEP CLONE functionality to sync up changes committed to one table to the corresponding table.
- C. Use a CTAS statement to create a derivative table from the marketing table; configure a production job to propagate changes.
- D. Add a parallel table write to the current production pipeline, updating a new sales table that varies as required from the marketing table.

Suggested Answer: A

Community vote distribution

A (100%)

🗨️ 👤 **lakime** 10 months, 3 weeks ago

Selected Answer: A

Yup, no overhead, easy
upvoted 1 times

🗨️ 👤 **db22** 1 year, 5 months ago

Selected Answer: A

Creating a view is the simplest and most effective solution
upvoted 3 times

A Delta Lake table representing metadata about content posts from users has the following schema:

user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude FLOAT, post_time TIMESTAMP, date DATE

This table is partitioned by the date column. A query is run with the following filter:

longitude < 20 & longitude > -20

Which statement describes how data will be filtered?

- A. Statistics in the Delta Log will be used to identify partitions that might include files in the filtered range.
- B. No file skipping will occur because the optimizer does not know the relationship between the partition column and the longitude.
- C. The Delta Engine will scan the parquet file footers to identify each row that meets the filter criteria.
- D. Statistics in the Delta Log will be used to identify data files that might include records in the filtered range.

Suggested Answer: D

Community vote distribution

D (100%)

🗨️ 👤 **lakime** 11 months, 1 week ago

Selected Answer: D

Yup - data skipping D)

upvoted 1 times

🗨️ 👤 **vexor3** 1 year, 6 months ago

Selected Answer: D

D is correct

upvoted 2 times

A small company based in the United States has recently contracted a consulting firm in India to implement several new data engineering pipelines to power artificial intelligence applications. All the company's data is stored in regional cloud storage in the United States.

The workspace administrator at the company is uncertain about where the Databricks workspace used by the contractors should be deployed.

Assuming that all data governance considerations are accounted for, which statement accurately informs this decision?

- A. Databricks runs HDFS on cloud volume storage; as such, cloud virtual machines must be deployed in the region where the data is stored.
- B. Databricks workspaces do not rely on any regional infrastructure; as such, the decision should be made based upon what is most convenient for the workspace administrator.
- C. Cross-region reads and writes can incur significant costs and latency; whenever possible, compute should be deployed in the same region the data is stored.
- D. Databricks notebooks send all executable code from the user's browser to virtual machines over the open internet; whenever possible, choosing a workspace region near the end users is the most secure.

Suggested Answer: C

Community vote distribution

C (100%)

🗨️ **benni_ale** 1 year, 3 months ago

Selected Answer: C

C is correct

upvoted 1 times

🗨️ **hpkr** 1 year, 7 months ago

Selected Answer: C

option C

upvoted 2 times

A CHECK constraint has been successfully added to the Delta table named activity_details using the following logic:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
  latitude >= -90 AND
  latitude <= 90 AND
  longitude >= -180 AND
  longitude <= 180);
```

A batch job is attempting to insert new records to the table, including a record where latitude = 45.50 and longitude = 212.67.

Which statement describes the outcome of this batch insert?

- A. The write will insert all records except those that violate the table constraints; the violating records will be reported in a warning log.
- B. The write will fail completely because of the constraint violation and no records will be inserted into the target table.
- C. The write will insert all records except those that violate the table constraints; the violating records will be recorded to a quarantine table.
- D. The write will include all records in the target table; any violations will be indicated in the boolean column named valid_coordinates.

Suggested Answer: B

Community vote distribution

B (100%)

 **vexor3** 1 year, 6 months ago

Selected Answer: B

B is correct

upvoted 2 times

A junior data engineer is migrating a workload from a relational database system to the Databricks Lakehouse. The source system uses a star schema, leveraging foreign key constraints and multi-table inserts to validate records on write.

Which consideration will impact the decisions made by the engineer while migrating this workload?

- A. Databricks only allows foreign key constraints on hashed identifiers, which avoid collisions in highly-parallel writes.
- B. Foreign keys must reference a primary key field; multi-table inserts must leverage Delta Lake's upsert functionality.
- C. Committing to multiple tables simultaneously requires taking out multiple table locks and can lead to a state of deadlock.
- D. All Delta Lake transactions are ACID compliant against a single table, and Databricks does not enforce foreign key constraints.

Suggested Answer: D

Community vote distribution

D (100%)

🗳️ 👤 **lakime** 11 months, 1 week ago

Selected Answer: D

Yes, D is correct, not enforced
upvoted 1 times

🗳️ 👤 **RandomForest** 1 year ago

Selected Answer: D

D is correct as primary and foreign keys are informational only and are not enforced in databricks <https://learn.microsoft.com/en-us/azure/databricks/tables/constraints>
upvoted 1 times

🗳️ 👤 **hpkr** 1 year, 7 months ago

Selected Answer: D

D is correct
upvoted 3 times

A data architect has heard about Delta Lake's built-in versioning and time travel capabilities. For auditing purposes, they have a requirement to maintain a full record of all valid street addresses as they appear in the customers table.

The architect is interested in implementing a Type 1 table, overwriting existing records with new values and relying on Delta Lake time travel to support long-term auditing. A data engineer on the project feels that a Type 2 table will provide better performance and scalability.

Which piece of information is critical to this decision?

- A. Data corruption can occur if a query fails in a partially completed state because Type 2 tables require setting multiple fields in a single update.
- B. Shallow clones can be combined with Type 1 tables to accelerate historic queries for long-term versioning.
- C. Delta Lake time travel cannot be used to query previous versions of these tables because Type 1 changes modify data files in place.
- D. Delta Lake time travel does not scale well in cost or latency to provide a long-term versioning solution.

Suggested Answer: D

Community vote distribution

D (100%)

  **hpkr** 1 year, 7 months ago

Selected Answer: D

correct answer - D

upvoted 3 times

A data engineer wants to join a stream of advertisement impressions (when an ad was shown) with another stream of user clicks on advertisements to correlate when impressions led to monetizable clicks.

In the code below, Impressions is a streaming DataFrame with a watermark ("event_time", "10 minutes")

```
.groupBy(
  window("event_time", "5 minutes"),
  "id")
.count()
).      withWatermark("event_time", 2 hours)
impressions.join(clicks, expr("clickAdId = impressionAdId"), "inner")
```

The data engineer notices the query slowing down significantly.

Which solution would improve the performance?

- A. Joining on event time constraint: `clickTime >= impressionTime AND clickTime <= impressionTime interval 1 hour`
- B. Joining on event time constraint: `clickTime + 3 hours < impressionTime - 2 hours`
- C. Joining on event time constraint: `clickTime == impressionTime` using a `leftOuter` join
- D. Joining on event time constraint: `clickTime >= impressionTime - interval 3 hours` and removing watermarks

Suggested Answer: A

Community vote distribution

A (100%)

  **_Lukas_** 2 months, 1 week ago

Selected Answer: A

Correct:

A - To prevent the streaming state from growing indefinitely (which causes the slowdown), Spark needs to know when it can safely drop old data from memory. By adding an event time constraint (e.g., "a click can only occur within 1 hour of the impression"), combined with the watermarks, Spark can calculate a state expiration threshold. Once an impression is older than the watermark plus this interval, Spark knows no future click can match it and removes it from the state store, stabilizing performance and memory usage.

upvoted 1 times

  **tafilaj_alsi** 8 months, 1 week ago

Selected Answer: A

Streaming joins in Spark Structured Streaming require watermarks and time constraints to work efficiently and clean up state. And as well n streaming joins, bounding the join with a range condition will ensure that Spark can prune old data and limit state retention, which dramatically improves performance.

upvoted 1 times

  **m79590530** 1 year, 3 months ago

Selected Answer: A

Answer A is the only possible logically.

B configures `clickTime` to be earlier than `impressionTime`

C says that `clickTime` should be the same as `impressionTime` with all clicks left joined to impressions

D wants to remove Watermarks which will lead to memory leaks and depletion for both streams staging/aggregation purposes by Spark

upvoted 3 times

A junior data engineer has manually configured a series of jobs using the Databricks Jobs UI. Upon reviewing their work, the engineer realizes that they are listed as the "Owner" for each job. They attempt to transfer "Owner" privileges to the "DevOps" group, but cannot successfully accomplish this task.

Which statement explains what is preventing this privilege transfer?

- A. Databricks jobs must have exactly one owner; "Owner" privileges cannot be assigned to a group.
- B. The creator of a Databricks job will always have "Owner" privileges; this configuration cannot be changed.
- C. Only workspace administrators can grant "Owner" privileges to a group.
- D. A user can only transfer job ownership to a group if they are also a member of that group.

Suggested Answer: A

Community vote distribution

A (100%)

 **03355a2** Highly Voted 1 year, 7 months ago

Selected Answer: A

This is the correct answer for this question in a past Databricks version, however now you can indeed add a group as a owner to a job.
upvoted 5 times

 **imatheushenrique** Most Recent 1 year, 8 months ago

A. Databricks jobs must have exactly one owner; "Owner" privileges cannot be assigned to a group.
It's only possivel that a databricks JOB has an owner, not a group.
upvoted 1 times

A table named user_ltv is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The user_ltv table has the following schema:

email STRING, age INT, ltv INT

The following view definition is executed:

```
CREATE VIEW user_ltv_no_minors AS
SELECT email, age, ltv
FROM user_ltv
WHERE
  CASE
    WHEN is_member("auditing") THEN TRUE
    ELSE age >= 18
  END
```

An analyst who is not a member of the auditing group executes the following query:

```
SELECT * FROM user_ltv_no_minors
```

Which statement describes the results returned by this query?

- A. All columns will be displayed normally for those records that have an age greater than 17; records not meeting this condition will be omitted.
- B. All age values less than 18 will be returned as null values, all other columns will be returned with the values in user_ltv.
- C. All values for the age column will be returned as null values, all other columns will be returned with the values in user_ltv.
- D. All columns will be displayed normally for those records that have an age greater than 18; records not meeting this condition will be omitted.

Suggested Answer: D

Community vote distribution

A (100%)

086ce7d 6 months, 2 weeks ago

Selected Answer: A

Since in option D, it didn't mention greater than or equal to, so the answer is A
upvoted 1 times

Isio05 1 year, 7 months ago

Selected Answer: A

Surely, it's an A
upvoted 3 times

hpr 1 year, 7 months ago

Selected Answer: A

option A is correct
upvoted 2 times

BrianNguyen95 1 year, 8 months ago

Selected Answer: A

Greater than 17

upvoted 2 times

  **Freyr** 1 year, 8 months ago

Selected Answer: A

Correct Answer: A

(>17) qual to (>=18). So, all records above 17 years will get in result and other records will be omitted.

upvoted 1 times

  **imatheushenrique** 1 year, 8 months ago

A. All columns will be displayed normally for those records that have an age greater than 17; records not meeting this condition will be omitted.

Because the condition of age>=18 only is respected in option A.

upvoted 1 times

  **MDWPartners** 1 year, 8 months ago

Selected Answer: A

Nope, A greater than 18 is 19. D is incorrect.

upvoted 1 times

All records from an Apache Kafka producer are being ingested into a single Delta Lake table with the following schema:

key BINARY, value BINARY, topic STRING, partition LONG, offset LONG, timestamp LONG

There are 5 unique topics being ingested. Only the "registration" topic contains Personal Identifiable Information (PII). The company wishes to restrict access to PII. The company also wishes to only retain records containing PII in this table for 14 days after initial ingestion. However, for non-PII information, it would like to retain these records indefinitely.

Which solution meets the requirements?

- A. All data should be deleted biweekly; Delta Lake's time travel functionality should be leveraged to maintain a history of non-PII information.
- B. Data should be partitioned by the registration field, allowing ACLs and delete statements to be set for the PII directory.
- C. Data should be partitioned by the topic field, allowing ACLs and delete statements to leverage partition boundaries.
- D. Separate object storage containers should be specified based on the partition field, allowing isolation at the storage level.

Suggested Answer: C

Community vote distribution

C (100%)

🗨️ 👤 **hpkr** 1 year, 7 months ago

Selected Answer: C

C is correct

upvoted 2 times

🗨️ 👤 **imatheushenrique** 1 year, 8 months ago

C.

Partitioning the data by the topic field allows the company to apply different access control policies and retention policies for different topics. Although there is a performance optimization gain because of the read in the partition path.

upvoted 1 times

The data governance team is reviewing code used for deleting records for compliance with GDPR. The following logic has been implemented to propagate delete requests from the user_lookup table to the user_aggregates table.

```
(spark.read
  .format("delta")
  .option("readChangeData", True)
  .option("startingTimestamp", '2021-08-22 00:00:00')
  .option("endingTimestamp", '2021-08-29 00:00:00')
  .table("user_lookup")
  .createOrReplaceTempView("changes"))

spark.sql("""
DELETE FROM user_aggregates
WHERE user_id IN (
  SELECT user_id
  FROM changes
  WHERE _change_type='delete'
)
""")
```

Assuming that user_id is a unique identifying key and that all users that have requested deletion have been removed from the user_lookup table, which statement describes whether successfully executing the above logic guarantees that the records to be deleted from the user_aggregates table are no longer accessible and why?

- A. No; the Delta Lake DELETE command only provides ACID guarantees when combined with the MERGE INTO command.
- B. No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.
- C. No; the change data feed only tracks inserts and updates, not deleted records.
- D. Yes; Delta Lake ACID guarantees provide assurance that the DELETE command succeeded fully and permanently purged these records.

Suggested Answer: B

  **m79590530** 1 year, 3 months ago

Selected Answer: B

Default Delta Lake time travel retention is 7 days so records deleted are still accessible via previous table versions up to 7 days later unless somebody changes this default setting to less days and runs VACUUM on the table earlier.

upvoted 1 times

  **imatheushenrique** 1 year, 8 months ago

B. No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

upvoted 3 times

An external object storage container has been mounted to the location /mnt/finance_eda_bucket.

The following logic was executed to create a database for the finance team:

```
CREATE DATABASE finance_eda_db
LOCATION '/mnt/finance_eda_bucket';
GRANT USAGE ON DATABASE finance_eda_db TO finance;
GRANT CREATE ON DATABASE finance_eda_db TO finance;
```

After the database was successfully created and permissions configured, a member of the finance team runs the following code:

```
CREATE TABLE finance_eda_db.tx_sales AS
SELECT *
FROM sales
WHERE state = "TX";
```

If all users on the finance team are members of the finance group, which statement describes how the tx_sales table will be created?

- A. A logical table will persist the query plan to the Hive Metastore in the Databricks control plane.
- B. An external table will be created in the storage container mounted to /mnt/finance_eda_bucket.
- C. A managed table will be created in the DBFS root storage container.
- D. An managed table will be created in the storage container mounted to /mnt/finance_eda_bucket.

Suggested Answer: B

Community vote distribution

D (100%)

  **c315d10** 8 months, 2 weeks ago

Selected Answer: D

D is correct

upvoted 1 times

  **a85becd** 10 months, 1 week ago

Selected Answer: B

The table cannot be a ****managed table****:

- If a database specifies a `LOCATION`, all tables created inside it inherit that `LOCATION`, and they are ****external tables****.

- Managed tables cannot exist in a database where the storage location has been explicitly specified, as the managed table lifecycle is tied to Databricks' internal storage system.

-Even if you don't explicitly define the table as external, the ****database's external location forces all tables**** to behave as external tables.

upvoted 1 times

  **Sriramiyer92** 1 year, 1 month ago

Selected Answer: D

D. Managed tables - Since Location is not mentioned.

mounted to /mnt/finance_eda_bucket - This is where our database is created as per first query and in second query we are using it to create the table further.

upvoted 1 times

  **Isio05** 1 year, 7 months ago

Selected Answer: D

It will be created in database location, but it will be managed table (missing LOCATION keyword in CREATE TABLE).

upvoted 4 times

  **hpkr** 1 year, 7 months ago

Selected Answer: D

D is correct

upvoted 2 times

  **Freyr** 1 year, 8 months ago

Selected Answer: D

Correct Answer: D

The table is still managed by Spark SQL in terms of metadata, but the data files are stored in the specified path due to the database's location setting.

Given the inherited location from the database, if the CREATE TABLE statement had explicitly used USING EXTERNAL or specified a LOCATION, it would definitely be an external table. However, since it inherits the database's location without these specifications, it creates a managed table.

upvoted 3 times

  **MDWPartners** 1 year, 8 months ago

Selected Answer: D

Seems D

upvoted 1 times

The data engineering team has been tasked with configuring connections to an external database that does not have a supported native connector with Databricks. The external database already has data security configured by group membership. These groups map directly to user groups already created in Databricks that represent various teams within the company.

A new login credential has been created for each group in the external database. The Databricks Utilities Secrets module will be used to make these credentials available to Databricks users.

Assuming that all the credentials are configured correctly on the external database and group membership is properly configured on Databricks, which statement describes how teams can be granted the minimum necessary access to using these credentials?

- A. No additional configuration is necessary as long as all users are configured as administrators in the workspace where secrets have been added.
- B. "Read" permissions should be set on a secret key mapped to those credentials that will be used by a given team.
- C. "Read" permissions should be set on a secret scope containing only those credentials that will be used by a given team.
- D. "Manage" permissions should be set on a secret scope containing only those credentials that will be used by a given team.

Suggested Answer: B

Community vote distribution

C (100%)

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: C

surely C

upvoted 1 times

🗳️ 👤 **hpkr** 1 year, 7 months ago

Selected Answer: C

C is correct. Read permission on secret scope should work here.

upvoted 3 times

🗳️ 👤 **Freyr** 1 year, 8 months ago

Selected Answer: C

Correct Answer: C

This option is the best practice for managing access to sensitive data. By creating a secret scope dedicated to each team and setting "Read" permissions on the scope, you ensure that only the intended team members can access their respective credentials. This method aligns with security best practices by tightly controlling access based on group membership and reducing the risk of unauthorized access.

upvoted 3 times

🗳️ 👤 **MDWPartners** 1 year, 8 months ago

Selected Answer: C

Seems C

upvoted 2 times

What is the retention of job run history?

- A. It is retained until you export or delete job run logs
- B. It is retained for 30 days, during which time you can deliver job run logs to DBFS or S3
- C. It is retained for 60 days, during which you can export notebook run results to HTML
- D. It is retained for 60 days, after which logs are archived

Suggested Answer: C

Community vote distribution

C (100%)

🗨️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: C

Surely C

upvoted 1 times

🗨️ 👤 **Elotrovert** 1 year, 7 months ago

Selected Answer: C

For most Databricks workspaces, the job run history is retained for 60 days.

upvoted 3 times

🗨️ 👤 **hpkr** 1 year, 7 months ago

Selected Answer: C

C is correct

upvoted 2 times

A data engineer, User A, has promoted a new pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens.

Which statement describes the contents of the workspace audit logs concerning these events?

- A. Because the REST API was used for job creation and triggering runs, a Service Principal will be automatically used to identify these events.
- B. Because User A created the jobs, their identity will be associated with both the job creation events and the job run events.
- C. Because these events are managed separately, User A will have their identity associated with the job creation events and User B will have their identity associated with the job run events.
- D. Because the REST API was used for job creation and triggering runs, user identity will not be captured in the audit logs.

Suggested Answer: C

🗲️ 👤 **imatheushenrique** Highly Voted 1 year, 8 months ago

C, because the users has their own personal access tokens.
upvoted 5 times

🗲️ 👤 **KadELbied** Most Recent 9 months, 1 week ago

Selected Answer: C

suretly C
upvoted 1 times

🗲️ 👤 **Urcolbz** 1 year, 1 month ago

Selected Answer: C

There is some ambiguity in this question, as has not been explained how this orchestration tool works.

If the tool just call the job (run-now), this event is tracking the user who 'triggers an on-demand job run'.

But we have other options, like modify/edit a job configuration, and set the trigger. If this situation happens, and the 'run as' is not populated, when the job is triggered automatically, there is not tracked who run the job. Then, the job run is associated to the owner of the job.

I'm assuming the the orchestration tool is doing a run-now, so option C should be the right option.

<https://docs.databricks.com/api/azure/workspace/jobs/getrun>

<https://docs.databricks.com/en/admin/account-settings/audit-logs.html#jobs>

upvoted 1 times

🗲️ 👤 **73109a1** 10 months, 4 weeks ago

PATs for production not a good idea afaik.

upvoted 1 times

🗲️ 👤 **Jugiboss** 1 year, 3 months ago

Selected Answer: C

C is correct, same question earlier.

upvoted 1 times

🗲️ 👤 **m79590530** 1 year, 3 months ago

Selected Answer: C

There is creator_user_name property of a job as well as runs_as property and both can contain different identities assigned.

upvoted 1 times

A distributed team of data analysts share computing resources on an interactive cluster with autoscaling configured. In order to better manage costs and query throughput, the workspace administrator is hoping to evaluate whether cluster upscaling is caused by many concurrent users or resource-intensive queries.

In which location can one review the timeline for cluster resizing events?

- A. Workspace audit logs
- B. Driver's log file
- C. Ganglia
- D. Cluster Event Log

Suggested Answer: *D*

🗉 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

suretly D

upvoted 1 times

🗉 👤 **m79590530** 1 year, 3 months ago

Selected Answer: D

Cluster lifecycle events are visible in the Cluster Event Log

upvoted 1 times

🗉 👤 **imatheushenrique** 1 year, 8 months ago

Its possible to see the metricks of compute with Ganglia, but the question is about a timeline so D, Cluster Event Log seems correct.

upvoted 3 times

When evaluating the Ganglia Metrics for a given cluster with 3 executor nodes, which indicator would signal proper utilization of the VM's resources?

- A. The five Minute Load Average remains consistent/flat
- B. CPU Utilization is around 75%
- C. Network I/O never spikes
- D. Total Disk Space remains constant

Suggested Answer: *B*

  **imatheushenrique** Highly Voted 1 year, 2 months ago

B.

This level of CPU utilization indicates that the cluster is being used without being underutilized.

upvoted 5 times

  **gizzamo** Most Recent 4 months, 4 weeks ago

Selected Answer: B

B is the best answer. The other three options are consistent/flat, never spikes, and disk space remains constant.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

B suretly

upvoted 1 times

The data engineer is using Spark's MEMORY_ONLY storage level.

Which indicators should the data engineer look for in the Spark UI's Storage tab to signal that a cached table is not performing optimally?

- A. On Heap Memory Usage is within 75% of Off Heap Memory Usage
- B. The RDD Block Name includes the "x" annotation signaling a failure to cache
- C. Size on Disk is > 0
- D. The number of Cached Partitions > the number of Spark Partitions

Suggested Answer: B

Community vote distribution

C (75%)

B (25%)

  **gloomy_marmot** 6 months, 1 week ago

Selected Answer: B

MEMORY_ONLY:

Stores data in memory as deserialized objects.

Fastest access but uses the most memory.

If memory is insufficient, unpersisted partitions are recomputed.

Use Case: Small datasets with frequent access in memory-rich clusters.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

Correct Answer: B

In the Spark UI's Storage tab, an indicator that a cached table is not performing optimally would be the presence of the _disk annotation in the RDD Block Name. This annotation indicates that some partitions of the cached data have been spilled to disk because there wasn't enough memory to hold them. This is suboptimal because accessing data from disk is much slower than from memory. The goal of caching is to keep data in memory for fast access, and a spill to disk means that this goal is not fully achieved.

upvoted 2 times

  **KadELbied** 9 months, 1 week ago

sorry it's C

upvoted 1 times

  **RuiCarvalhoDEV** 1 year, 2 months ago

Selected Answer: C

is MEMORY_ONLY

upvoted 1 times

  **Hadiler** 1 year, 6 months ago

Selected Answer: C

C is correct

upvoted 2 times

  **03355a2** 1 year, 7 months ago

Selected Answer: C

It's simple, if MEMORY_ONLY is used, anything spilled to disk would indicate a problem.

upvoted 1 times

  **03355a2** 1 year, 7 months ago

The RDD answer is incorrect for this question due to the fact that while this indicates a failure to cache, it is more specific to identifying individual blocks that failed to cache rather than providing a general signal of a suboptimal performance for the entire cached table.

upvoted 1 times

  **hpkrr** 1 year, 7 months ago

Selected Answer: C

C is correct here

upvoted 2 times

  **Freyr** 1 year, 8 months ago

Selected Answer: B

Correct Answer: B

Option B, is the most correct and relevant choice for an indicator that a cached table is not performing optimally in a MEMORY_ONLY scenario. If an RDD block includes a "?" annotation, it strongly suggests issues with caching, which would directly impact the performance and expected behavior of MEMORY_ONLY caching. This indication points to a failure to cache the data entirely in memory, which is what MEMORY_ONLY intends to do.

Option C, could also be a relevant indicator in general caching scenarios (e.g., MEMORY_AND_DISK), but it contradicts the MEMORY_ONLY setting directly. Therefore, Option B is chosen based on the specific storage level described.

upvoted 2 times

  **Freyr** 1 year, 8 months ago

THE CORRECT ANSWER IS: C

PLEASE IGNORE MY PREVIOUS ANSWER.

Long story short, B is correct in the context of non-functional requirement, but the question is based in functional requirement, and sorry for the confusion.

upvoted 3 times

  **imatheushenrique** 1 year, 8 months ago

B.

This annotation says that some partitions of the cached data have been spilled to disk because there wasn't enough memory to keep them.

upvoted 1 times

  **MDWPartners** 1 year, 8 months ago

I would say C

upvoted 2 times

Review the following error traceback:

```
-----
AnalysisException                                Traceback (most recent call last)
<command-3293767849433948> in <module>
----> 1 display(df.select(3*"heartrate"))

/databricks/spark/python/pyspark/sql/dataframe.py in select(self, *cols)
   1690         [Row(name='Alice', age=12), Row(name='Bob', age=15)]
   1691         """
-> 1692         jdf = self._jdf.select(self._jcols(*cols))
   1693         return DataFrame(jdf, self.sql_ctx)
   1694

/databricks/spark/python/lib/py4j-0.10.9-src.zip/py4j/java_gateway.py in __call__(self, *args)
   1302
   1303         answer = self.gateway_client.send_command(command)
-> 1304         return_value = get_return_value(
   1305             answer, self.gateway_client, self.target_id, self.name)
   1306

/databricks/spark/python/pyspark/sql/utils.py in deco(*a, **kw)
   121             # Hide where the exception came from that shows a non-Pythonic
   122             # JVM exception message.
--> 123             raise converted from None
   124         else:
   125             raise

AnalysisException: cannot resolve 'heartrateheartrateheartrate' given input columns:
[spark_catalog.database.table.device_id, spark_catalog.database.table.heartrate,
spark_catalog.database.table.mrn, spark_catalog.database.table.time];
'Project ['heartrateheartrateheartrate]
+- SubqueryAlias spark_catalog.database.table
   +- Relation[device_id#75L,heartrate#76,mrn#77L,time#78] parquet
```

Which statement describes the error being raised?

- A. There is a syntax error because the heartrate column is not correctly identified as a column.
- B. There is no column in the table named heartrateheartrateheartrate
- C. There is a type error because a column object cannot be multiplied.
- D. There is a type error because a DataFrame object cannot be multiplied.

Suggested Answer: B

Community vote distribution

B (100%)

 **bd3d6f6** 5 months ago

Selected Answer: A

select(3*"heartrate") translates to select("heartrateheartrateheartrate") so it's a syntax error where the user probably wanted to say something like selectExpr("3*heartrate")

upvoted 1 times

 **KadELbied** 9 months, 1 week ago

Selected Answer: B

Surely B

upvoted 1 times

 **m79590530** 1 year, 3 months ago

Selected Answer: B

The final error clearly states that such column name can not be resolved in the source dataframe schema/structure

upvoted 1 times

What is a method of installing a Python package scoped at the notebook level to all nodes in the currently active cluster?

- A. Run `source env/bin/activate` in a notebook setup script
- B. Install libraries from PyPI using the cluster UI
- C. Use `%pip install` in a notebook cell
- D. Use `%sh pip install` in a notebook cell

Suggested Answer: *C*

  **m79590530** 1 year, 3 months ago

Selected Answer: C

C is correct as `%sh pip install ...` runs only on the driver node and the Cluster UI PyPi or other library installs are not scoped to a specific notebook only but to all spark sessions in all notebooks on all cluster nodes.

upvoted 2 times

  **imatheushenrique** 1 year, 8 months ago

Is necessary just run `%pip install some_library` inside a notebook cell

C.

OBS:

For the last update of a library can be executed `%pip install some_library -U`

upvoted 1 times

What is the first line of a Databricks Python notebook when viewed in a text editor?

- A. %python
- B. // Databricks notebook source
- C. # Databricks notebook source
- D. -- Databricks notebook source

Suggested Answer: C

Community vote distribution

C (100%)

  **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

  **minhnhh** 1 year, 5 months ago

Selected Answer: C

The correct answer is:

C. # Databricks notebook source

This is the comment line that appears at the beginning of a Databricks Python notebook when viewed in a text editor.

upvoted 2 times

  **imatheushenrique** 1 year, 8 months ago

C. # Databricks notebook source

The commentary in the first line will indicate a magic command for a notebook source.

upvoted 2 times

Incorporating unit tests into a PySpark application requires upfront attention to the design of your jobs, or a potentially significant refactoring of existing code.

Which benefit offsets this additional effort?

- A. Improves the quality of your data
- B. Validates a complete use case of your application
- C. Troubleshooting is easier since all steps are isolated and tested individually
- D. Ensures that all steps interact correctly to achieve the desired end result

Suggested Answer: C

  **m79590530** 1 year, 3 months ago

Selected Answer: C

C is testing each unit of the solution separately. It doesn't necessarily validate the data quality as mentioned in A.

B is more for Business Case scenario testing like end-to-end testing for real life, real data execution.

D is more related to Integration testing.

upvoted 2 times

  **imatheushenrique** 1 year, 8 months ago

C. Troubleshooting is easier since all steps are isolated and tested individually

The unit tests will ensure that specific functions and transformations will work as intended.

upvoted 2 times

What describes integration testing?

- A. It validates an application use case.
- B. It validates behavior of individual elements of an application,
- C. It requires an automated testing framework.
- D. It validates interactions between subsystems of your application.

Suggested Answer: *D*

🗉 👤 **m79590530** 1 year, 3 months ago

Selected Answer: D

Interactions and cooperation between the solution and/or interfacing systems/data consumers components, subsystems and interfaces is exactly Integration testing.

upvoted 1 times

🗉 👤 **imatheushenrique** 1 year, 8 months ago

D. It validates interactions between subsystems of your application.

An integration test is used for different softwares validation components, subsystems, or applications that has dependencies.

upvoted 2 times

The Databricks CLI is used to trigger a run of an existing job by passing the `job_id` parameter. The response that the job run request has been submitted successfully includes a field `run_id`.

Which statement describes what the number alongside this field represents?

- A. The `job_id` and number of times the job has been run are concatenated and returned.
- B. The globally unique ID of the newly triggered run.
- C. The number of times the job definition has been run in this workspace.
- D. The `job_id` is returned in this field.

Suggested Answer: B

Community vote distribution

B (100%)

  **m79590530** 1 year, 3 months ago

Selected Answer: B

Every job run creates and assigns a globally unique run ID to the job RUN as well as globally unique run ID's for the Tasks RUN's inside the Job.
upvoted 1 times

A Databricks job has been configured with three tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on task A.

What will be the resulting state if tasks A and B complete successfully but task C fails during a scheduled run?

- A. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.
- B. Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task C failed, all commits will be rolled back automatically.
- C. Because all tasks are managed as a dependency graph, no changes will be committed to the Lakehouse until all tasks have successfully been completed.
- D. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; any changes made in task C will be rolled back due to task failure.

Suggested Answer: A

  **m79590530** 1 year, 3 months ago

Selected Answer: A

Each Notebook or Task consists of multiple commands and actions performed by them. Each action may be on the data in the Delta Lake where ACID transactions take place and fully rollback certain data manipulations if some of them fail but the Notebooks/Tasks in the Job themselves will not completely fail or rollback. Therefore Answer A correctly describes the result considering the dependencies configured between the Notebooks/Tasks as described in the question.

upvoted 1 times

  **imatheushenrique** 1 year, 8 months ago

A: A. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.

Because this type of orchestration indicates a Fan-Out.

upvoted 1 times

When scheduling Structured Streaming jobs for production, which configuration automatically recovers from query failures and keeps costs low?

- A. Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
- B. Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: Unlimited
- C. Cluster: Existing All-Purpose Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
- D. Cluster: New Job Cluster;
Retries: None;
Maximum Concurrent Runs: 1

Correct Answer: A

  **RandomForest** 1 year ago

Selected Answer: A

The correct answer is A: The unlimited retries will take care of query failures while the max concurrent runs = 1 will keep the costs low
upvoted 1 times

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_stor  
USING DELTA  
LOCATION "/mnt/prod/sales_by_store"
```

Realizing that the original query had a typographical error, the below code was executed:

```
ALTER TABLE prod.sales_by_stor RENAME TO prod.sales_by_store
```

Which result will occur after running the second command?

- A. The table reference in the metastore is updated.
- B. All related files and metadata are dropped and recreated in a single ACID transaction.
- C. The table name change is recorded in the Delta transaction log.
- D. A new Delta transaction log is created for the renamed table.

Correct Answer: A

🗉  **d3567cc** 7 months, 4 weeks ago

Selected Answer: A

same question listed before with answer A which is corrected one
upvoted 3 times

🗉  **Stalker200** 9 months, 3 weeks ago

Selected Answer: A

The Hive Metastore (or Unity Catalog) updates the logical table name. The data and files stay untouched.
upvoted 3 times

🗉  **lakime** 11 months, 1 week ago

Selected Answer: C

while the metastore is updated, the key mechanism for tracking changes in Delta Lake is the transaction log.
upvoted 2 times

The data engineering team has configured a Databricks SQL query and alert to monitor the values in a Delta Lake table. The recent_sensor_recordings table contains an identifying sensor_id alongside the timestamp and temperature for the most recent 5 minutes of recordings.

The below query is used to create the alert:

```
SELECT MEAN(temperature), MAX(temperature), MIN(temperature)
FROM recent_sensor_recordings
GROUP BY sensor_id
```

The query is set to refresh each minute and always completes in less than 10 seconds. The alert is set to trigger when mean (temperature) > 120. Notifications are triggered to be sent at most every 1 minute.

If this alert raises notifications for 3 consecutive minutes and then stops, which statement must be true?

- A. The total average temperature across all sensors exceeded 120 on three consecutive executions of the query
- B. The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query
- C. The source query failed to update properly for three consecutive minutes and then restarted
- D. The maximum temperature recording for at least one sensor exceeded 120 on three consecutive executions of the query

Suggested Answer: B

Community vote distribution

B (100%)

🗨️ 👤 **Thameur01** 1 year, 2 months ago

Selected Answer: B

B, because avg temp is calculated by sensor_id and not total
upvoted 1 times

🗨️ 👤 **Ayomidetolu_A** 1 year, 2 months ago

Selected Answer: B

B is the correct answer
upvoted 1 times

🗨️ 👤 **e904bf4** 1 year, 2 months ago

Selected Answer: B

B is correct
upvoted 1 times

A junior developer complains that the code in their notebook isn't producing the correct results in the development environment. A shared screenshot reveals that while they're using a notebook versioned with Databricks Repos, they're using a personal branch that contains old logic. The desired branch named dev-2.3.9 is not available from the branch selection dropdown.

Which approach will allow this developer to review the current logic for this notebook?

- A. Use Repos to make a pull request use the Databricks REST API to update the current branch to dev-2.3.9
- B. Use Repos to pull changes from the remote Git repository and select the dev-2.3.9 branch.
- C. Use Repos to checkout the dev-2.3.9 branch and auto-resolve conflicts with the current branch
- D. Use Repos to merge the current branch and the dev-2.3.9 branch, then make a pull request to sync with the remote repository

Correct Answer: B

  **ealpuche** 5 months ago

Selected Answer: B

Pulling changes fetches all remote branches into the local workspace, after which the developer can select the dev-2.3.9 branch from the dropdown and review the current logic.

upvoted 1 times

Two of the most common data locations on Databricks are the DBFS root storage and external object storage mounted with `dbutils.fs.mount()`.

Which of the following statements is correct?

- A. DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.
- B. By default, both the DBFS root and mounted data sources are only accessible to workspace administrators.
- C. The DBFS root is the most secure location to store data, because mounted storage volumes must have full public read and write permissions.
- D. The DBFS root stores files in ephemeral block volumes attached to the driver, while mounted directories will always persist saved data to external storage between sessions.

Correct Answer: A

Currently there are no comments in this discussion, be the first to comment!

An upstream system has been configured to pass the date for a given batch of data to the Databricks Jobs API as a parameter. The notebook to be scheduled will use this parameter to load data with the following code:

```
df = spark.read.format("parquet").load(f"/mnt/source/(date)")
```

Which code block should be used to create the date Python variable used in the above code block?

- A. `date = spark.conf.get("date")`
- B. `import sys`
`date = sys.argv[1]`
- C. `date = dbutils.notebooks.getParam("date")`
- D. `dbutils.widgets.text("date", "null")`
`date = dbutils.widgets.get("date")`

Correct Answer: *D*

  **ealpuche** 5 months ago

Selected Answer: D

Surely, B

upvoted 1 times

  **ealpuche** 5 months ago

I meant D

upvoted 1 times

The Databricks workspace administrator has configured interactive clusters for each of the data engineering groups. To control costs, clusters are set to terminate after 30 minutes of inactivity. Each user should be able to execute workloads against their assigned clusters at any time of the day.

Assuming users have been added to a workspace but not granted any permissions, which of the following describes the minimal permissions a user would need to start and attach to an already configured cluster.

- A. "Can Manage" privileges on the required cluster
- B. Cluster creation allowed, "Can Restart" privileges on the required cluster
- C. Cluster creation allowed, "Can Attach To" privileges on the required cluster
- D. "Can Restart" privileges on the required cluster

Suggested Answer: C

Community vote distribution

D (100%)

🗳️ 👤 **Urcolbz** 1 year, 1 month ago

Selected Answer: D

'can restart' privileges as is needed start and attach a notebook.

'can attach to' is not having enough privileges to start a cluster.

<https://docs.databricks.com/en/security/auth/access-control/index.html#clusters>
upvoted 4 times

🗳️ 👤 **Thameur01** 1 year, 1 month ago

Selected Answer: C

"Can Attach To" Privileges:

The "Can Attach To" permission is sufficient for users to run workloads on an existing cluster. This permission allows users to attach notebooks or jobs to a cluster without needing additional management permissions.
Cluster Creation Allowed:

Allowing cluster creation ensures users can create clusters if needed. However, for this scenario, this isn't mandatory because clusters are already configured. The user needs only "Can Attach To" privileges.
upvoted 1 times

🗳️ 👤 **42f87fd** 7 months, 2 weeks ago

D - because "Can Attach To" privileges also includes unnecessary cluster creation. "Can Attach To" alone wouldn't allow starting a terminated cluster.
upvoted 1 times

🗳️ 👤 **temple1305** 1 year, 2 months ago

Selected Answer: D

Can restart privileges - it was even discussed
upvoted 2 times

The data science team has created and logged a production model using MLflow. The following code correctly imports and applies the production model to output the predictions as a new DataFrame named preds with the schema "customer_id LONG, predictions DOUBLE, date DATE".

```
from pyspark.sql.functions import current_date

model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
preds = (df.select(
    "customer_id",
    model(*columns).alias("predictions"),
    current_date().alias("date")
))
```

The data science team would like predictions saved to a Delta Lake table with the ability to compare all predictions across time. Churn predictions will be made at most once per day.

Which code block accomplishes this task while minimizing potential compute costs?

A. `preds.write.mode("append").saveAsTable("churn_preds")`

B. `preds.write.format("delta").save("/preds/churn_preds")`

C.

```
(preds.writeStream
    .outputMode("append")
    .option("checkpointPath", "/_checkpoints/churn_preds")
    .table("churn_preds")
)
```

D.

```
(preds.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("churn_preds")
)
```

Suggested Answer: C

Community vote distribution

A (100%)

🗳️ **kishanu** 10 months ago

Selected Answer: D

D seems to be the correct one.

Option A

Missing Delta format – without it, you lose ACID compliance and versioning.

Append mode is fine, but this is less efficient without Delta.

Option B

Saves as a Delta file, but not as a managed table. Harder to query in SQL unless you register it manually.

Not as convenient for analysts who use SQL-based workflows.

Option C:

Streaming is an overkill.

So D

upvoted 1 times

🗳️ **Retko** 1 week, 3 days ago

saveAsTable saves it as a delta table

upvoted 1 times

🗳️ **kishanu** 10 months ago

Selected Answer: D

D seems to be the correct one.

Option A

Missing Delta format – without it, you lose ACID compliance and versioning.

Append mode is fine, but this is less efficient without Delta.

Option B

Saves as a Delta file, but not as a managed table. Harder to query in SQL unless you register it manually.

Not as convenient for analysts who use SQL-based workflows.

Option C:

Streaming is an overkill.

So D

upvoted 1 times

🗨️ 👤 **_lene_** 1 year ago

Selected Answer: A

batch+append

upvoted 1 times

🗨️ 👤 **mouthwash** 1 year, 1 month ago

Selected Answer: A

A is the right answer.

upvoted 2 times

🗨️ 👤 **temple1305** 1 year, 2 months ago

Selected Answer: A

You need:

- Batch operation since it is at most once a day
- Append, since you need to keep track of past predictions

A is the correct answer. You don't need to specify "format" when you use saveAsTable.

upvoted 3 times

The following code has been migrated to a Databricks notebook from a legacy workload:

```
%sh
git clone https://github.com/foo/data_loader;
python ./data_loader/run.py;
mv ./output /dbfs/mnt/new_data
```

The code executes successfully and provides the logically correct results, however, it takes over 20 minutes to extract and load around 1 GB of data.

Which statement is a possible explanation for this behavior?

- A. %sh triggers a cluster restart to collect and install Git. Most of the latency is related to cluster startup time.
- B. Instead of cloning, the code should use %sh pip install so that the Python code can get executed in parallel across all nodes in a cluster.
- C. %sh does not distribute file moving operations; the final line of code should be updated to use %fs instead.
- D. %sh executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

Correct Answer: D

  **skggggg** 1 month, 1 week ago

Selected Answer: D

correct

upvoted 1 times

A Delta table of weather records is partitioned by date and has the below schema:

date DATE, device_id INT, temp FLOAT, latitude FLOAT, longitude FLOAT

To find all the records from within the Arctic Circle, you execute a query with the below filter:

latitude > 66.3

Which statement describes how the Delta engine identifies which files to load?

- A. All records are cached to an operational database and then the filter is applied
- B. The Parquet file footers are scanned for min and max statistics for the latitude column
- C. The Hive metastore is scanned for min and max statistics for the latitude column
- D. The Delta log is scanned for min and max statistics for the latitude column

Suggested Answer: B

Community vote distribution

D (100%)

  **Thameur01** 1 year, 2 months ago

Selected Answer: D

As per the documentation, I understand that the table statistics can be fetched through the delta log (eg min, max, count) in order to not read the underlying data of a delta table. This is the case for numerical types, and timestamp is supposed to be supported.

upvoted 3 times

  **temple1305** 1 year, 2 months ago

Selected Answer: D

Delta Table's log consist statistics for columns

upvoted 3 times

In order to prevent accidental commits to production data, a senior data engineer has instituted a policy that all development work will reference clones of Delta Lake tables. After testing both DEEP and SHALLOW CLONE, development tables are created using SHALLOW CLONE.

A few weeks after initial table creation, the cloned versions of several tables implemented as Type 1 Slowly Changing Dimension (SCD) stop working. The transaction logs for the source tables show that VACUUM was run the day before.

Which statement describes why the cloned tables are no longer working?

- A. Because Type 1 changes overwrite existing records, Delta Lake cannot guarantee data consistency for cloned tables.
- B. Running VACUUM automatically invalidates any shallow clones of a table; DEEP CLONE should always be used when a cloned table will be repeatedly queried.
- C. The data files compacted by VACUUM are not tracked by the cloned metadata; running REFRESH on the cloned table will pull in recent changes.
- D. The metadata created by the CLONE operation is referencing data files that were purged as invalid by the VACUUM command.

Correct Answer: D

  **RajeshMP2023** 6 months, 1 week ago

Selected Answer: D

shallow clone only copies metadata, unlike deep clone copies metadata and actual data, hence shallow clone tables will not work when vacuum on source table executed, which will delete old files and hence SCD which need old datafiles.

upvoted 1 times

A junior data engineer has configured a workload that posts the following JSON to the Databricks REST API endpoint 2.0/jobs/create.

```
{
  "name": "Ingest new data",
  "existing_cluster_id": "6015-954420-peace720",
  "notebook_task": {
    "notebook_path": "/Prod/ingest.py"
  }
}
```

Assuming that all configurations and referenced resources are available, which statement describes the result of executing this workload three times?

- A. The logic defined in the referenced notebook will be executed three times on the referenced existing all purpose cluster.
- B. The logic defined in the referenced notebook will be executed three times on new clusters with the configurations of the provided cluster ID.
- C. Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.
- D. One new job named "Ingest new data" will be defined in the workspace, but it will not be executed.

Suggested Answer: C

Community vote distribution

C (100%)

🗉 👤 **Ayomidetolu_A** 1 year, 2 months ago

Selected Answer: C

C is the correct answer
upvoted 1 times

🗉 👤 **divyapsingh** 1 year, 2 months ago

Selected Answer: C

C is the answer as 3 times call will create three jobs with same name but with different job id.
upvoted 2 times

🗉 👤 **temple1305** 1 year, 2 months ago

Selected Answer: C

C correct, 3 jobs created, no executions
upvoted 1 times

A Delta Lake table in the Lakehouse named `customer_churn_params` is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

Immediately after each update succeeds, the data engineering team would like to determine the difference between the new version and the previous version of the table.

Given the current implementation, which method can be used?

- A. Execute a query to calculate the difference between the new version and the previous version using Delta Lake's built-in versioning and time travel functionality.
- B. Parse the Delta Lake transaction log to identify all newly written data files.
- C. Parse the Spark event logs to identify those rows that were updated, inserted, or deleted.
- D. Execute `DESCRIBE HISTORY customer_churn_params` to obtain the full operation metrics for the update, including a log of all records that have been added or modified.

Correct Answer: A

stopthisnow 2 months, 1 week ago

Selected Answer: A

A - there lime is actually time.... D makes no sense
upvoted 2 times

RajeshMP2023 6 months, 1 week ago

Selected Answer: D

there is no lime history , D looks suitable
upvoted 1 times

A view is registered with the following code:

```
CREATE VIEW recent_orders AS (
  SELECT a.user_id, a.email, b.order_id, b.order_date
  FROM
    (SELECT user_id, email
     FROM users) a
  INNER JOIN
    (SELECT user_id, order_id, order_date
     FROM orders
     WHERE order_date >= (current_date() - 7)) b
  ON a.user_id = b.user_id
)
```

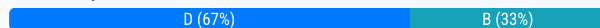
Both users and orders are Delta Lake tables.

Which statement describes the results of querying recent_orders?

- A. The versions of each source table will be stored in the table transaction log; query results will be saved to DBFS with each query.
- B. All logic will execute when the table is defined and store the result of joining tables to the DBFS; this stored data will be returned when the table is queried.
- C. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.
- D. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

Suggested Answer: C

Community vote distribution



🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

Surely D

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: D

D - with the view it is always at query time. Once it is run, the engine reads the data - meaning the data state is from when the query was run.

upvoted 1 times

🗳️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: D

View does not write to disk

upvoted 1 times

🗳️ 👤 **divyapsingh** 1 year, 2 months ago

Selected Answer: D

Here the view is getting created not the table. view is only save select query which will get data from the valid version of the underlying table at the beginning of the select statement made on view.

upvoted 2 times

🗳️ 👤 **temple1305** 1 year, 2 months ago

Selected Answer: B

B is correct, view is just query without materialization and return result for moment of execution

upvoted 1 times

🗳️ 👤 **carlosmps** 1 year, 1 month ago

but the option B is materialized stored the data on dbfs....

upvoted 1 times

A data engineer is performing a join operation to combine values from a static userLookup table with a streaming DataFrame streamingDF.

Which code block attempts to perform an invalid stream-static join?

- A. `userLookup.join(streamingDF, ["user_id"], how="right")`
- B. `streamingDF.join(userLookup, ["user_id"], how="inner")`
- C. `userLookup.join(streamingDF, ["user_id"], how="inner")`
- D. `userLookup.join(streamingDF, ["user_id"], how="left")`

Suggested Answer: D

Community vote distribution

D (100%)

🗳️ 👤 **temple1305** Highly Voted 👍 1 year, 2 months ago

Selected Answer: D

D correct

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#support-matrix-for-joins-in-streaming-queries>

Static join stream - left join not supported

upvoted 7 times

🗳️ 👤 **arekm** 1 year, 1 month ago

That is what the documentation says. We take all the records from the stream and join to the static table - left or right shows us where the stream in the join is. Static table-stream - right outer is supported, stream-static table - left outer is supported.

upvoted 2 times

🗳️ 👤 **Yahoo_Arun** Most Recent 🕒 2 weeks, 5 days ago

Selected Answer: A

It clearly violates Structured Streaming rules by attempting a right outer join with a streaming DataFrame, which Spark does not allow.

upvoted 1 times

🗳️ 👤 **stopthisnow** 2 months, 2 weeks ago

Selected Answer: A

"A" is not allowed.... The streaming df is on right and it is doing a right join meaning even if there is no match, streaming df will show all records as NULL

upvoted 1 times

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Incremental state information should be maintained for 10 minutes for late-arriving data.

Streaming DataFrame df has the following schema:

"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"

Code block:

```
df._____  
    .groupBy(  
        window("event_time", "5 minutes").alias("time"),  
        "device_id"  
    )  
    .agg(  
        avg("temp").alias("avg_temp"),  
        avg("humidity").alias("avg_humidity")  
    )  
    .writeStream  
    .format("delta")  
    .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

- A. withWatermark("event_time", "10 minutes")
- B. awaitArrival("event_time", "10 minutes")
- C. await("event_time + '10 minutes'")
- D. slidingWindow("event_time", "10 minutes")

Correct Answer: A

 **Ral17** 2 months, 1 week ago

Selected Answer: A

It's best to use withWatermark() for late available streaming data

upvoted 1 times

 **KadELbied** 9 months, 1 week ago

Selected Answer: A

Suretly A

upvoted 1 times

A data architect has designed a system in which two Structured Streaming jobs will concurrently write to a single bronze Delta table. Each job is subscribing to a different topic from an Apache Kafka source, but they will write data with the same schema. To keep the directory structure simple, a data engineer has decided to nest a checkpoint directory to be shared by both streams.

The proposed directory structure is displayed below:

```
./bronze
├── __checkpoint
├── __delta_log
├── year_week=2020_01
├── year_week=2020_02
└── ...
```

Which statement describes whether this checkpoint directory structure is valid for the given scenario and why?

- A. No; Delta Lake manages streaming checkpoints in the transaction log.
- B. Yes; both of the streams can share a single checkpoint directory.
- C. No; only one stream can write to a Delta Lake table.
- D. No; each of the streams needs to have its own checkpoint directory.

Suggested Answer: D

Community vote distribution

D (100%)

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

suretly D

upvoted 1 times

🗳️ 👤 **divyapsingh** 1 year, 2 months ago

Selected Answer: D

D is the answer. two steams can not have same checkpoint directory.

upvoted 1 times

A Structured Streaming job deployed to production has been experiencing delays during peak hours of the day. At present, during normal execution, each microbatch of data is processed in less than 3 seconds. During peak hours of the day, execution time for each microbatch becomes very inconsistent, sometimes exceeding 30 seconds. The streaming write is currently configured with a trigger interval of 10 seconds.

Holding all other variables constant and assuming records need to be processed in less than 10 seconds, which adjustment will meet the requirement?

- A. Decrease the trigger interval to 5 seconds; triggering batches more frequently allows idle executors to begin processing the next batch while longer running tasks from previous batches finish.
- B. Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.
- C. The trigger interval cannot be modified without modifying the checkpoint directory; to maintain the current stream state, increase the number of shuffle partitions to maximize parallelism.
- D. Use the trigger once option and configure a Databricks job to execute the query every 10 seconds; this ensures all backlogged records are processed with each batch.

Correct Answer: B

🗉 👤 **RajeshMP2023** 6 months, 1 week ago

Selected Answer: B

running them frequently will reduce amount of data to be processed which will enhance execution time.

upvoted 1 times

🗉 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

Surety B

upvoted 1 times

Which statement describes the default execution mode for Databricks Auto Loader?

- A. Cloud vendor-specific queue storage and notification services are configured to track newly arriving files; new files are incrementally and idempotently loaded into the target Delta Lake table.
- B. New files are identified by listing the input directory; the target table is materialized by directly querying all valid files in the source directory.
- C. Webhooks trigger a Databricks job to run anytime new data arrives in a source directory; new data are automatically merged into target tables using rules inferred from the data.
- D. New files are identified by listing the input directory; new files are incrementally and idempotently loaded into the target Delta Lake table.

Suggested Answer: A

Community vote distribution



🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

Surely D

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: D

D - it is between A & D, rest does not make sense. A describes the file notification mode, which is NOT the default.

upvoted 1 times

🗳️ 👤 **OnlyPraveen** 1 year, 1 month ago

Selected Answer: D

Also check answers to duplicate question 108.

upvoted 2 times

🗳️ 👤 **carlosmps** 1 year, 1 month ago

Selected Answer: D

"Auto Loader uses directory listing mode by default. In directory listing mode, Auto Loader identifies new files by listing the input directory."

<https://learn.microsoft.com/en-us/azure/databricks/ingestion/auto-loader/directory-listing-mode>

upvoted 2 times

🗳️ 👤 **temple1305** 1 year, 2 months ago

Selected Answer: B

Correct answer is B. However, listing the input directory is the default way of identifying new files for auto loader. Cloud Native Notification services can be used but this is not default setting for auto loader.

upvoted 1 times

🗳️ 👤 **divyapsingh** 1 year, 2 months ago

Selected Answer: A

solution a is better approach as it is more efficient but both vendor-specific notification service and directory file listing are used for tracking the new files.

check out below link :- <https://docs.databricks.com/en/ingestion/cloud-object-storage/auto-loader/file-detection-modes.html>

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

The question is not "what is the better approach", but "what is the default approach", hence D.

upvoted 1 times

Which statement describes the correct use of `pyspark.sql.functions.broadcast`?

- A. It marks a column as having low enough cardinality to properly map distinct values to available partitions, allowing a broadcast join.
- B. It marks a column as small enough to store in memory on all executors, allowing a broadcast join.
- C. It caches a copy of the indicated table on all nodes in the cluster for use in all future queries during the cluster lifetime.
- D. It marks a DataFrame as small enough to store in memory on all executors, allowing a broadcast join.

Correct Answer: D

🗉 👤 **RajeshMP2023** 6 months, 1 week ago

Selected Answer: D

In Broadcast join, spark copies smaller table to all worker nodes so join can happen locally at each executor without shuffling larger table. broadcast function forces the join.

upvoted 2 times

🗉 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

Surely D

upvoted 1 times

Spill occurs as a result of executing various wide transformations. However, diagnosing spill requires one to proactively look for key indicators.

Where in the Spark UI are two of the primary indicators that a partition is spilling to disk?

- A. Stage's detail screen and Query's detail screen
- B. Stage's detail screen and Executor's log files
- C. Driver's and Executor's log files
- D. Executor's detail screen and Executor's log files

Correct Answer: B

🗉 👤 **ADFMDS** 1 month, 3 weeks ago

Selected Answer: A

Spill diagnosis starts in the Spark UI, not logs
upvoted 1 times

🗉 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

Suretly B
upvoted 2 times

An upstream source writes Parquet data as hourly batches to directories named with the current date. A nightly batch job runs the following code to ingest all data from the previous day as indicated by the date variable:

```
(spark.read
  .format("parquet")
  .load(f"/mnt/raw_orders/{date}")
  .dropDuplicates(["customer_id", "order_id"])
  .write
  .mode("append")
  .saveAsTable("orders")
)
```

Assume that the fields `customer_id` and `order_id` serve as a composite key to uniquely identify each order.

If the upstream system is known to occasionally produce duplicate entries for a single order hours apart, which statement is correct?

- A. Each write to the orders table will only contain unique records, and only those records without duplicates in the target table will be written.
- B. Each write to the orders table will only contain unique records, but newly written records may have duplicates already present in the target table.
- C. Each write to the orders table will only contain unique records; if existing records with the same key are present in the target table, these records will be overwritten.
- D. Each write to the orders table will run deduplication over the union of new and existing records, ensuring no duplicate records are present.

Correct Answer: B

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 2 times

  **_lene_** 1 year ago

Selected Answer: B

No intra-batch duplicates, can be inter-batch duplicates

upvoted 1 times

A junior data engineer on your team has implemented the following code block.

```
MERGE INTO events
USING new_events
ON events.event_id = new_events.event_id
WHEN NOT MATCHED
  INSERT *
```

The view new_events contains a batch of records with the same schema as the events Delta table. The event_id field serves as a unique key for this table.

When this query is executed, what will happen with new records that have the same event_id as an existing record?

- A. They are merged.
- B. They are ignored.
- C. They are updated.
- D. They are inserted.

Correct Answer: B

🗲️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 2 times

🗲️ 👤 **billabro** 9 months, 1 week ago

Selected Answer: C

C is right

upvoted 1 times

🗲️ 👤 **7bad058** 7 months, 1 week ago

Please don't comment here if you are not sure, it's simple SQL code and you can mislead others.

upvoted 4 times

🗲️ 👤 **Kyries** 10 months, 3 weeks ago

Selected Answer: B

B is correct

upvoted 2 times

A new data engineer notices that a critical field was omitted from an application that writes its Kafka source to Delta Lake. This happened even though the critical field was in the Kafka source. That field was further missing from data written to dependent, long-term storage. The retention threshold on the Kafka service is seven days. The pipeline has been in production for three months.

Which describes how Delta Lake can help to avoid data loss of this nature in the future?

- A. The Delta log and Structured Streaming checkpoints record the full history of the Kafka producer.
- B. Delta Lake schema evolution can retroactively calculate the correct value for newly added fields, as long as the data was in the original source.
- C. Delta Lake automatically checks that all fields present in the source data are included in the ingestion layer.
- D. Ingesting all raw data and metadata from Kafka to a bronze Delta table creates a permanent, replayable history of the data state.

Correct Answer: *D*

  **KadELbied** 9 months, 1 week ago

Selected Answer: D

suretly D

upvoted 2 times

The data engineering team maintains the following code:

```
import pyspark.sql.functions as F

(spark.table("silver_customer_sales")
 .groupBy("customer_id")
 .agg(
     F.min("sale_date").alias("first_transaction_date"),
     F.max("sale_date").alias("last_transaction_date"),
     F.mean("sale_total").alias("average_sales"),
     F.countDistinct("order_id").alias("total_orders"),
     F.sum("sale_total").alias("lifetime_value")
 ).write
 .mode("overwrite")
 .table("gold_customer_lifetime_sales_summary")
 )
```

Assuming that this code produces logically correct results and the data in the source table has been de-duplicated and validated, which statement describes what will occur when this code is executed?

- A. The silver_customer_sales table will be overwritten by aggregated values calculated from all records in the gold_customer_lifetime_sales_summary table as a batch job.
- B. A batch job will update the gold_customer_lifetime_sales_summary table, replacing only those rows that have different values than the current version of the table, using customer_id as the primary key.
- C. The gold_customer_lifetime_sales_summary table will be overwritten by aggregated values calculated from all records in the silver_customer_sales table as a batch job.
- D. An incremental job will detect if new rows have been written to the silver_customer_sales table; if new rows are detected, all aggregates will be recalculated and used to overwrite the gold_customer_lifetime_sales_summary table.

Correct Answer: C

  **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

The data engineering team is migrating an enterprise system with thousands of tables and views into the Lakehouse. They plan to implement the target architecture using a series of bronze, silver, and gold tables. Bronze tables will almost exclusively be used by production data engineering workloads, while silver tables will be used to support both data engineering and machine learning workloads. Gold tables will largely serve business intelligence and reporting purposes. While personal identifying information (PII) exists in all tiers of data, pseudonymization and anonymization rules are in place for all data at the silver and gold levels.

The organization is interested in reducing security concerns while maximizing the ability to collaborate across diverse teams.

Which statement exemplifies best practices for implementing this system?

- A. Isolating tables in separate databases based on data quality tiers allows for easy permissions management through database ACLs and allows physical separation of default storage locations for managed tables.
- B. Because databases on Databricks are merely a logical construct, choices around database organization do not impact security or discoverability in the Lakehouse.
- C. Storing all production tables in a single database provides a unified view of all data assets available throughout the Lakehouse, simplifying discoverability by granting all users view privileges on this database.
- D. Working in the default Databricks database provides the greatest security when working with managed tables, as these will be created in the DBFS root.

Correct Answer: A

  **KadELbied** 9 months, 1 week ago

Selected Answer: A

Suretly A

upvoted 1 times

The data architect has mandated that all tables in the Lakehouse should be configured as external (also known as "unmanaged") Delta Lake tables.

Which approach will ensure that this requirement is met?

- A. When a database is being created, make sure that the LOCATION keyword is used.
- B. When the workspace is being configured, make sure that external cloud object storage has been mounted.
- C. When data is saved to a table, make sure that a full file path is specified alongside the USING DELTA clause.
- D. When tables are created, make sure that the UNMANAGED keyword is used in the CREATE TABLE statement.

Correct Answer: C

🗲️ 👤 **363c4c5** 6 months, 2 weeks ago

Selected Answer: A

An external table is a table that references an external storage path by using a LOCATION clause.

upvoted 1 times

🗲️ 👤 **363c4c5** 6 months, 2 weeks ago

I think C is the correct option

upvoted 1 times

🗲️ 👤 **c315d10** 8 months, 2 weeks ago

Selected Answer: C

path should be provided for external table

upvoted 4 times

🗲️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

the correct answer is E : When tables are created, make sure that the EXTERNAL keyword is used in the CREATE TABLE statement.

upvoted 1 times

To reduce storage and compute costs, the data engineering team has been tasked with curating a series of aggregate tables leveraged by business intelligence dashboards, customer-facing applications, production machine learning models, and ad hoc analytical queries.

The data engineering team has been made aware of new requirements from a customer-facing application, which is the only downstream workload they manage entirely. As a result, an aggregate table used by numerous teams across the organization will need to have a number of fields renamed, and additional fields will also be added.

Which of the solutions addresses the situation while minimally interrupting other teams in the organization without increasing the number of tables that need to be managed?

- A. Send all users notice that the schema for the table will be changing; include in the communication the logic necessary to revert the new table schema to match historic queries.
- B. Configure a new table with all the requisite fields and new names and use this as the source for the customer-facing application; create a view that maintains the original data schema and table name by aliasing select fields from the new table.
- C. Create a new table with the required schema and new fields and use Delta Lake's deep clone functionality to sync up changes committed to one table to the corresponding table.
- D. Replace the current table definition with a logical view defined with the query logic currently writing the aggregate table; create a new table to power the customer-facing application.

Correct Answer: B

  **RajeshMP2023** 6 months, 1 week ago

Selected Answer: B

Minimally disruptive: This approach ensures that the original schema and table name remain intact for other teams by creating a view that aliases the new table. This avoids breaking existing queries and dashboards that depend on the current schema.

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

A Delta Lake table representing metadata about content posts from users has the following schema:

user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude FLOAT, post_time TIMESTAMP, date DATE

Based on the above schema, which column is a good candidate for partitioning the Delta Table?

- A. post_time
- B. date
- C. post_id
- D. user_id

Correct Answer: B

  **RajeshMP2023** 6 months, 1 week ago

Selected Answer: B

Partitioning the Delta Lake table by the date column is the most efficient and practical choice, as it balances query performance, data distribution, and partition management

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

The downstream consumers of a Delta Lake table have been complaining about data quality issues impacting performance in their applications. Specifically, they have complained that invalid latitude and longitude values in the activity_details table have been breaking their ability to use other geolocation processes.

A junior engineer has written the following code to add CHECK constraints to the Delta Lake table:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
    latitude >= -90 AND
    latitude <= 90 AND
    longitude >= -180 AND
    longitude <= 180);
```

A senior engineer has confirmed the above logic is correct and the valid ranges for latitude and longitude are provided, but the code fails when executed.

Which statement explains the cause of this failure?

- A. The current table schema does not contain the field valid_coordinates; schema evolution will need to be enabled before altering the table to add a constraint.
- B. The activity_details table already exists; CHECK constraints can only be added during initial table creation.
- C. The activity_details table already contains records that violate the constraints; all existing data must pass CHECK constraints in order to add them to an existing table.
- D. The activity_details table already contains records; CHECK constraints can only be added prior to inserting values into a table.

Correct Answer: C

  **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

What is true for Delta Lake?

- A. Views in the Lakehouse maintain a valid cache of the most recent versions of source tables at all times.
- B. Primary and foreign key constraints can be leveraged to ensure duplicate values are never entered into a dimension table.
- C. Delta Lake automatically collects statistics on the first 32 columns of each table which are leveraged in data skipping based on query filters.
- D. Z-order can only be applied to numeric values stored in Delta Lake tables.

Correct Answer: C

  **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

The view updates represents an incremental batch of all newly ingested data to be inserted or updated in the customers table.

The following logic is used to process these records.

```
MERGE INTO customers
USING (
  SELECT updates.customer_id as merge_key, updates.*
  FROM updates

  UNION ALL

  SELECT NULL as merge_key, updates.*
  FROM updates JOIN customers
  ON updates.customer_id = customers.customer_id
  WHERE customers.current = true AND updates.address <> customers.address
) staged_updates
ON customers.customer_id = mergeKey
WHEN MATCHED AND customers.current = true AND customers.address <> staged_updates.address THEN
  UPDATE SET current = false, end_date = staged_updates.effective_date
WHEN NOT MATCHED THEN
  INSERT(customer_id, address, current, effective_date, end_date)
  VALUES(staged_updates.customer_id, staged_updates.address, true, staged_updates.effective_date,
  null)
```

Which statement describes this implementation?

- A. The customers table is implemented as a Type 2 table; old values are overwritten and new customers are appended.
- B. The customers table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.
- C. The customers table is implemented as a Type 0 table; all writes are append only with no changes to existing values.
- D. The customers table is implemented as a Type 1 table; old values are overwritten by new values and no history is maintained.

Correct Answer: B

🗨️ **Ral17** 2 months, 1 week ago

Selected Answer: B

The answer should be B. It's clear from the statements.

upvoted 1 times

🗨️ **RajeshMP2023** 6 months, 1 week ago

Selected Answer: B

A Type 2 Slowly Changing Dimension (SCD) table in Databricks (or any data platform) is a table designed to track historical changes to dimension data over time. This is achieved by maintaining multiple records for the same entity, with each record representing a specific version of the entity's data for a given time period. it doesn't override existing one.

upvoted 1 times

🗨️ **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

A team of data engineers are adding tables to a DLT pipeline that contain repetitive expectations for many of the same data quality checks. One member of the team suggests reusing these data quality rules across all tables defined for this pipeline.

What approach would allow them to do this?

- A. Add data quality constraints to tables in this pipeline using an external job with access to pipeline configuration files.
- B. Use global Python variables to make expectations visible across DLT notebooks included in the same pipeline.
- C. Maintain data quality rules in a separate Databricks notebook that each DLT notebook or file can import as a library.
- D. Maintain data quality rules in a Delta table outside of this pipeline's target schema, providing the schema name as a pipeline parameter.

Suggested Answer: C

Community vote distribution

D (100%)

🗳️ **benni_ale** Highly Voted 1 year, 2 months ago

Selected Answer: D

<https://docs.databricks.com/en/delta-live-tables/expectations.html>

"You can maintain data quality rules separately from your pipeline implementations. Databricks recommends storing the rules in a Delta table with each rule categorized by a tag."

upvoted 7 times

🗳️ **stopthisnow** Most Recent 2 months, 1 week ago

Selected Answer: D

Both C and D are correct. Databricks recommends D

upvoted 1 times

🗳️ **Ral17** 2 months, 1 week ago

Selected Answer: C

Why Option C is Correct

DLT native pattern for code reuse – Create a shared notebook with common expectation functions (e.g., `check_not_null()`, `check_valid_date()`), then import it into each DLT pipeline notebook using `%run` or Python imports – exactly how DLT is designed to work

upvoted 1 times

🗳️ **gizzamo** 4 months, 3 weeks ago

Selected Answer: C

Reasoning:

DLT expectations are often repeated across multiple tables (e.g., column not null, value ranges, valid enums).

To avoid duplication, the best practice is to factor them out into reusable functions or libraries, and then import them into multiple DLT notebooks.

Databricks supports importing shared Python modules or notebooks to centralize and reuse logic.

upvoted 2 times

🗳️ **ealpuche** 5 months ago

Selected Answer: C

C, For Sure

upvoted 1 times

🗳️ **Billybob0604** 6 months, 1 week ago

Selected Answer: C

The best practice for code reuse is write them once in a shared utility notebook

upvoted 2 times

🗳️ **RajeshMP2023** 6 months, 1 week ago

Selected Answer: C

Reusability of Data Quality Rules:

By maintaining the data quality rules in a separate notebook, the team can centralize the logic for expectations and reuse them across multiple tables and pipelines. This approach ensures consistency and reduces duplication of code.

Importing as a Library:

Databricks allows you to modularize code by creating reusable notebooks or Python files. These can be imported into other notebooks or DLT pipelines, making it easy to apply the same set of expectations across multiple tables.

upvoted 1 times

  **gloomy_marmot** 6 months, 1 week ago

Selected Answer: D

<https://docs.databricks.com/aws/en/dlt/expectation-patterns#portable-and-reusable-expectations>

Expectations should be stored in the table

upvoted 1 times

  **happyhelppy** 6 months, 2 weeks ago

Selected Answer: C

D answer is confusing when it comes to use parameter as schema.

Having expectations defined as python module and later imported is described in doc:

<https://docs.databricks.com/aws/en/dlt/expectation-patterns?language=Python%2%A0Module#portable-and-reusable-expectations>

upvoted 1 times

  **KadELbied** 9 months, 1 week ago

Selected Answer: D

Surely D

upvoted 1 times

  **lakime** 10 months, 2 weeks ago

Selected Answer: C

Initially C, currently D

upvoted 1 times

  **arekm** 1 year, 1 month ago

Selected Answer: D

D is what Databricks suggests as of now

upvoted 1 times

  **Thameur01** 1 year, 2 months ago

Selected Answer: C

To reuse repetitive data quality rules across multiple tables in a Delta Live Tables (DLT) pipeline, the most efficient approach is to maintain these rules in a separate notebook or Python module and import them where needed. This promotes code reusability, maintainability, and consistency

upvoted 2 times

The DevOps team has configured a production workload as a collection of notebooks scheduled to run daily using the Jobs UI. A new data engineering hire is onboarding to the team and has requested access to one of these notebooks to review the production logic.

What are the maximum notebook permissions that can be granted to the user without allowing accidental changes to production code or data?

- A. Can manage
- B. Can edit
- C. Can run
- D. Can read

Correct Answer: *D*

🗨️ 👤 **Ral17** 2 months, 1 week ago

Selected Answer: D

Anything other than read permissions can lead to interference in production data/production code.

upvoted 1 times

🗨️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

suretly D

upvoted 2 times

A table named `user_ltv` is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The `user_ltv` table has the following schema:

`email` STRING, `age` INT, `ltv` INT

The following view definition is executed:

```
CREATE VIEW email_ltv AS
SELECT
CASE WHEN
    is_member('marketing') THEN email
    ELSE 'REDACTED'
END AS email,
ltv
FROM user_ltv
```

An analyst who is not a member of the marketing group executes the following query:

```
SELECT * FROM email_ltv -
```

Which statement describes the results returned by this query?

- A. Three columns will be returned, but one column will be named "REDACTED" and contain only null values.
- B. Only the email and ltv columns will be returned; the email column will contain all null values.
- C. The email and ltv columns will be returned with the values in `user_ltv`.
- D. Only the email and ltv columns will be returned; the email column will contain the string "REDACTED" in each row.

Correct Answer: D

  **KadELbied** 9 months, 1 week ago

Selected Answer: D

suretly D

upvoted 1 times

The data governance team has instituted a requirement that all tables containing Personal Identifiable Information (PII) must be clearly annotated. This includes adding column comments, table comments, and setting the custom table property "contains_pii" = true.

The following SQL DDL statement is executed to create a new table:

```
CREATE TABLE dev.pii_test  
(id INT, name STRING COMMENT "PII")  
COMMENT "Contains PII"  
TBLPROPERTIES ('contains_pii' = True)
```

Which command allows manual confirmation that these three requirements have been met?

- A. DESCRIBE EXTENDED dev.pii_test
- B. DESCRIBE DETAIL dev.pii_test
- C. SHOW TBLPROPERTIES dev.pii_test
- D. DESCRIBE HISTORY dev.pii_test

Correct Answer: A

  **KadELbied** 9 months, 1 week ago

Selected Answer: A

suretly A

upvoted 1 times

The data governance team is reviewing code used for deleting records for compliance with GDPR. They note the following logic is used to delete records from the Delta Lake table named users.

```
DELETE FROM users
WHERE user_id IN
  (SELECT user_id FROM delete_requests)
```

Assuming that user_id is a unique identifying key and that delete_requests contains all users that have requested deletion, which statement describes whether successfully executing the above logic guarantees that the records to be deleted are no longer accessible and why?

- A. Yes; Delta Lake ACID guarantees provide assurance that the DELETE command succeeded fully and permanently purged these records.
- B. No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.
- C. Yes; the Delta cache immediately updates to reflect the latest data files recorded to disk.
- D. No; the Delta Lake DELETE command only provides ACID guarantees when combined with the MERGE INTO command.

Correct Answer: B

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

The data architect has decided that once data has been ingested from external sources into the Databricks Lakehouse, table access controls will be leveraged to manage permissions for all production tables and views.

The following logic was executed to grant privileges for interactive queries on a production database to the core engineering group.

```
GRANT USAGE ON DATABASE prod TO eng;  
GRANT SELECT ON DATABASE prod TO eng;
```

Assuming these are the only privileges that have been granted to the eng group and that these users are not workspace administrators, which statement describes their privileges?

- A. Group members are able to create, query, and modify all tables and views in the prod database, but cannot define custom functions.
- B. Group members are able to list all tables in the prod database but are not able to see the results of any queries on those tables.
- C. Group members are able to query and modify all tables and views in the prod database, but cannot create new tables or views.
- D. Group members are able to query all tables and views in the prod database, but cannot create or edit anything in the database.

Correct Answer: D

  **KadELbied** 9 months, 1 week ago

Selected Answer: D

suretly D

upvoted 1 times

A user wants to use DLT expectations to validate that a derived table report contains all records from the source, included in the table validation_copy.

The user attempts and fails to accomplish this by adding an expectation to the report table definition.

```
CREATE LIVE TABLE report (
  CONSTRAINT no_missing_records EXPECT (key IN validation_copy)
)
AS SELECT <...>
```

Which approach would allow using DLT expectations to validate all expected records are present in this table?

- A. Define a temporary table that performs a left outer join on validation_copy and report, and define an expectation that no report key values are null
- B. Define a SQL UDF that performs a left outer join on two tables, and check if this returns null values for report key values in a DLT expectation for the report table
- C. Define a view that performs a left outer join on validation_copy and report, and reference this view in DLT expectations for the report table
- D. Define a function that performs a left outer join on validation_copy and report, and check against the result in a DLT expectation for the report table

Suggested Answer: A

Community vote distribution

A (100%)

🗳️ **gizzamo** 4 months, 3 weeks ago

Selected Answer: C

How This Solves the Problem:

View encapsulates the join logic: Makes the complex validation logic reusable

Left outer join identifies missing records: NULL values in actual_key indicate missing records

Expectation checks completeness: Ensures no expected records are missing

upvoted 1 times

🗳️ **363c4c5** 6 months, 2 weeks ago

Selected Answer: A

in 'A' should be - define a view, but not a temp table.

in 'C' should be - set expectations reading from the view, but not referencing the view in the expectations

upvoted 1 times

🗳️ **KadELbied** 9 months, 1 week ago

Selected Answer: A

Surely A

upvoted 1 times

🗳️ **mohadjhamad** 10 months, 3 weeks ago

Selected Answer: C

Why a view works: By creating a view that performs a left outer join between validation_copy (the expected source of records) and report, missing records can be identified where the report.key is NULL.

Referencing the view in DLT: The view can then be used in an expectation rule within the report table to ensure all expected records are present.

upvoted 2 times

🗳️ **benni_ale** 1 year, 2 months ago

Selected Answer: A

Arguably a better solution would be Define a materialised view that performs a left outer join on validation_copy and report, and define an expectation that no report key values are null : https://docs.databricks.com/en/delta-live-tables/expectations.html?utm_source=chatgpt.com#perform-advanced-

validation-with-delta-live-tables-expectations

upvoted 2 times

A user new to Databricks is trying to troubleshoot long execution times for some pipeline logic they are working on. Presently, the user is executing code cell-by-cell, using `display()` calls to confirm code is producing the logically correct results as new transformations are added to an operation. To get a measure of average time to execute, the user is running each cell multiple times interactively.

Which of the following adjustments will get a more accurate measure of how code is likely to perform in production?

- A. The Jobs UI should be leveraged to occasionally run the notebook as a job and track execution time during incremental code development because Photon can only be enabled on clusters launched for scheduled jobs.
- B. The only way to meaningfully troubleshoot code execution times in development notebooks is to use production-sized data and production-sized clusters with Run All execution.
- C. Production code development should only be done using an IDE; executing code against a local build of open source Spark and Delta Lake will provide the most accurate benchmarks for how code will perform in production.
- D. Calling `display()` forces a job to trigger, while many transformations will only add to the logical query plan; because of caching, repeated execution of the same logic does not provide meaningful results.

Suggested Answer: D

Community vote distribution

B (100%)

🗳️ 👤 **c315d10** 8 months, 2 weeks ago

Selected Answer: D

production data in reality is not used for development testing

upvoted 2 times

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

🗳️ 👤 **kishanu** 10 months ago

Selected Answer: D

Using `display()` will force the job to trigger and hence won't be tested as per production

upvoted 1 times

🗳️ 👤 **kishanu** 10 months ago

Selected Answer: D

Using `display()` will force the job to trigger and hence won't be tested as per production

upvoted 1 times

🗳️ 👤 **suranga4** 10 months, 3 weeks ago

Selected Answer: B

B is the right answer

upvoted 1 times

🗳️ 👤 **mohadjhamad** 10 months, 3 weeks ago

Selected Answer: D

Calling `display()` forces a job to trigger,

upvoted 1 times

🗳️ 👤 **mouthwash** 1 year, 1 month ago

Selected Answer: B

B is the right answer.

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: B

B - this is a repeated question. You need production like data & environment to be able to troubleshoot the performance issues. `display()` does introduce overhead, but we use it here to investigate, where the problems start to surface. We still need the production like env and data first.

upvoted 1 times

Where in the Spark UI can one diagnose a performance problem induced by not leveraging predicate push-down?

- A. In the Executor's log file, by grepping for "predicate push-down"
- B. In the Stage's Detail screen, in the Completed Stages table, by noting the size of data read from the Input column
- C. In the Query Detail screen, by interpreting the Physical Plan
- D. In the Delta Lake transaction log, by noting the column statistics

Suggested Answer: C

Community vote distribution

C (100%)

  **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

  **lakime** 11 months ago

Selected Answer: C

Option C is correct because you can diagnose performance issues related to predicate push-down by examining the Physical Plan in the Query Detail screen of the Spark UI. Predicate push-down is a feature where filtering conditions (predicates) are applied as early as possible, typically in the data scan operation, to reduce the amount of data being read. If predicate push-down is not happening, the physical plan will show that the filter operation is being applied after the data is read, leading to inefficient queries.

upvoted 2 times

  **benni_ale** 1 year, 2 months ago

Selected Answer: C

Look at the Physical Plan

upvoted 1 times

A data engineer needs to capture pipeline settings from an existing setting in the workspace, and use them to create and version a JSON file to create a new pipeline.

Which command should the data engineer enter in a web terminal configured with the Databricks CLI?

- A. Use list pipelines to get the specs for all pipelines; get the pipeline spec from the returned results; parse and use this to create a pipeline
- B. Stop the existing pipeline; use the returned settings in a reset command
- C. Use the get command to capture the settings for the existing pipeline; remove the pipeline_id and rename the pipeline; use this in a create command
- D. Use the clone command to create a copy of an existing pipeline; use the get JSON command to get the pipeline definition; save this to git

Suggested Answer: C

Community vote distribution

C (100%)

🗨️ 👤 **_Lukas_** 2 months, 1 week ago

Selected Answer: C

Correct:

C - This describes the accurate CLI workflow for cloning a pipeline. You must first run databricks pipelines get --id <id> to retrieve the full JSON specification. Before this JSON can be used to create a new pipeline (via databricks pipelines create), you must manually remove the id field (which uniquely identifies the original pipeline) and typically change the name to avoid naming conflicts.

upvoted 1 times

🗨️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

🗨️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: C

I say C from common logical sense, however il have not properly tested it... I just don't see any problems with that

upvoted 2 times

Which Python variable contains a list of directories to be searched when trying to locate required modules?

- A. `importlib.resource_path`
- B. `sys.path`
- C. `os.path`
- D. `pypi.path`

Correct Answer: *B*

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

You are testing a collection of mathematical functions, one of which calculates the area under a curve as described by another function.

```
assert(myIntegrate(lambda x: x*x, 0, 3) [0] == 9)
```

Which kind of test would the above line exemplify?

- A. Unit
- B. Manual
- C. Functional
- D. Integration

Correct Answer: A

  **KadELbied** 9 months, 1 week ago

Selected Answer: A

Suretly A

upvoted 1 times

What is a key benefit of an end-to-end test?

- A. It makes it easier to automate your test suite
- B. It pinpoints errors in the building blocks of your application
- C. It provides testing coverage for all code paths and branches
- D. It closely simulates real world usage of your application

Correct Answer: *D*

  **KadELbied** 9 months, 1 week ago

Selected Answer: D

suretly D

upvoted 1 times

Which REST API call can be used to review the notebooks configured to run as tasks in a multi-task job?

- A. /jobs/runs/list
- B. /jobs/list
- C. /jobs/runs/get
- D. /jobs/get

Suggested Answer: *D*

Community vote distribution

D (100%)

🗨️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: D

Surely D

upvoted 1 times

🗨️ 👤 **temple1305** 1 year, 2 months ago

Selected Answer: D

multi-task: /jobs/get

single-task: /jobs/runs/get

upvoted 1 times

🗨️ 👤 **arekm** 1 year, 1 month ago

The answer is right, the explanation wrong. Both single & multi task jobs definitions are returned by /jobs/get. The /jobs/runs/get gets the executed/run jobs bases on the definitions you can get from /jobs/get.

upvoted 5 times

A Data Engineer wants to run unit tests using common Python testing frameworks on Python functions defined across several Databricks notebooks currently used in production.

How can the data engineer run unit tests against functions that work with data in production?

- A. Define and import unit test functions from a separate Databricks notebook
- B. Define and unit test functions using Files in Repos
- C. Run unit tests against non-production data that closely mirrors production
- D. Define unit tests and functions within the same notebook

Suggested Answer: B

Community vote distribution

C (100%)

🗳️ 👤 **_Lukas_** 2 months, 1 week ago

Selected Answer: B

Correct:

B - To use common Python testing frameworks (like pytest or unittest) effectively, code should be structured as standard Python modules (.py files) rather than notebooks. Databricks "Files in Repos" (now often called Databricks Git Folders or Workspace Files) allows you to create these .py files alongside your notebooks. You can then import the functions into your notebooks for execution and simultaneously import them into test files to run unit tests, enabling standard CI/CD workflows.

upvoted 1 times

🗳️ 👤 **KadELbied** 9 months, 1 week ago

Selected Answer: C

suretly C

upvoted 1 times

🗳️ 👤 **arekm** 1 year, 1 month ago

Selected Answer: B

B - to test functions we need to import them in our unit tests. This means that storing functions in notebooks is not a good idea. You store them separately - as "files" - and import in notebooks the same way you import them in unit tests.

upvoted 2 times

🗳️ 👤 **Thameur01** 1 year, 2 months ago

Selected Answer: B

Databricks Repos is the recommended way to organize and manage code, including functions and unit tests, in a scalable and maintainable way. By defining your functions and unit tests in Files in Repos, you can:

Modularize Your Code:

Functions can be organized into separate Python files or modules, making them reusable and easier to test.

Use Standard Testing Frameworks:

Frameworks like pytest or unittest can be used to write and execute unit tests against these functions.

Version Control Integration:

Files in Repos can be version-controlled using Git, ensuring traceability and collaboration.

Production Data Testing:

With proper safeguards, you can design unit tests to test production-like data while maintaining modularity and separation from production pipelines.

upvoted 2 times

🗳️ 👤 **benni_ale** 1 year, 2 months ago

Selected Answer: C

It is true that for Python and R notebooks, Databricks recommends storing functions and their unit tests outside of notebooks so given the question is in Python environment some could argue B being the correct solution. Nevertheless that is only an "advice" and Databricks more generally states that in general, it is a best practice to not run unit tests against functions that work with data in production. This is especially important for functions that add, remove, or otherwise change data. To protect your production data from being compromised by your unit tests in unexpected ways, you should run unit tests against non-production data. So I would go for option C. Alternatively I would have gone for B.

https://docs.databricks.com/en/notebooks/testing.html?utm_source=chatgpt.com#write-unit-tests

upvoted 1 times

  **arekm** 1 year, 1 month ago

This does not really answer the question.

upvoted 1 times

A data engineer wants to refactor the following DLT code, which includes multiple table definitions with very similar code.

```
@dlt.table(name=f"t1_dataset")
def t1_dataset():
    return spark.read.table(t1)

@dlt.table(name=f"t2_dataset")
def t2_dataset():
    return spark.read.table(t2)

@dlt.table(name=f"t3_dataset")
def t3_dataset():
    return spark.read.table(t3)

...
```

In an attempt to programmatically create these tables using a parameterized table definition, the data engineer writes the following code.

```
tables = ["t1", "t2", "t3"]

for t in tables:
    @dlt.table(name=f"{t}_dataset")
    def new_table():
        return spark.read.table(t)
```

The pipeline runs an update with this refactored code, but generates a different DAG showing incorrect configuration values for these tables.

How can the data engineer fix this?

- A. Wrap the for loop inside another table definition, using generalized names and properties to replace with those from the inner table definition.
- B. Convert the list of configuration values to a dictionary of table settings, using table names as keys.
- C. Move the table definition into a separate function, and make calls to this function using different input parameters inside the for loop.
- D. Load the configuration values for these tables from a separate file, located at a path provided by a pipeline parameter.

Suggested Answer: C

Community vote distribution

C (100%)

  **KadELbied** 9 months, 1 week ago

Selected Answer: B

suretly B

upvoted 1 times

  **Thameur01** 1 year, 1 month ago

Selected Answer: C

here is a correct implementation:

```
def create_table(t):
    @dlt.table(name=f"{t}_dataset")
    def table_definition():
        return spark.read.table(t)
```

```
tables = ["t1", "t2", "t3"]
```

```
for t in tables:
```

```
    create_table(t)
```

upvoted 3 times

  **benni_ale** 1 year, 2 months ago

Selected Answer: C

Problem seems to be the fact that the new_table function has no parameter and the t variable won't be recognized as a variable.. However i have not tested it as DLT is not available in free membership. :(

upvoted 2 times

A data engineer has created a 'transactions' Delta table on Databricks that should be used by the analytics team. The analytics team wants to use the table with another tool which requires Apache Iceberg format.

What should the data engineer do?

- A. Require the analytics team to use a tool which supports Delta table.
- B. Create an Iceberg copy of the 'transactions' Delta table which can be used by the analytics team.
- C. Convert the 'transactions' Delta to Iceberg and enable uniform so that the table can be read as a Delta table.
- D. Enable uniform on the transactions table to 'iceberg' so that the table can be read as an Iceberg table.

Suggested Answer: D

Community vote distribution

D (100%)

🗳️ 👤 **_Lukas_** 2 months, 1 week ago

Selected Answer: D

Correct:

D - This is the proper approach using Delta Lake UniForm. By setting the table property 'delta.universalFormat.enabledFormats' = 'iceberg' (along with 'delta.enableIcebergCompatV2' = 'true'), the Delta table automatically generates Iceberg metadata asynchronously. The table remains a fully functional Delta table for the data engineer, while Iceberg clients used by the analytics team can read it as if it were a native Iceberg table, all without duplicating data.

upvoted 1 times

🗳️ 👤 **Ral17** 2 months, 1 week ago

Selected Answer: D

UniForm is specifically designed for this exact scenario – enabling interoperability between Delta Lake and other formats (Iceberg, Hudi) without abandoning Delta's advantages.

upvoted 1 times

🗳️ 👤 **6aa83ae** 2 months, 2 weeks ago

Selected Answer: D

<https://docs.databricks.com/aws/en/delta/uniform>

You must set the following table properties to enable Iceberg reads:

'delta.enableIcebergCompatV2' = 'true'

'delta.universalFormat.enabledFormats' = 'iceberg'

upvoted 2 times

🗳️ 👤 **a68856c** 2 months, 2 weeks ago

Selected Answer: D

Enable Uniform in the Delta table allows to read it as an Iceberg table. A single copy of the data files serves multiple formats.

<https://docs.databricks.com/aws/en/delta/uniform>

upvoted 1 times

A junior data engineer is working to implement logic for a Lakehouse table named `silver_device_recordings`. The source data contains 100 unique fields in a highly nested JSON structure.

The `silver_device_recordings` table will be used downstream for highly selective joins on a number of fields, and will also be leveraged by the machine learning team to filter on a handful of relevant fields. In total, 15 fields have been identified that will often be used for filter and join logic.

The data engineer is trying to determine the best approach for dealing with these nested fields before declaring the table schema.

Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

- A. Because Delta Lake uses Parquet for data storage, Dremel encoding information for nesting can be directly referenced by the Delta transaction log.
- B. Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.
- C. The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.
- D. By default, Delta Lake collects statistics on the first 32 columns in a table; these statistics are leveraged for data skipping when executing selective queries.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A platform engineer is creating catalogs and schemas for the development team to use.

The engineer has created an initial catalog, Catalog_A, and initial schema, Schema_A. The engineer has also granted USE CATALOG, USE SCHEMA, and CREATE TABLE to the development team so that the engineer can begin populating the schema with new tables.

Despite being owner of the catalog and schema, the engineer noticed that they do not have access to the underlying tables in Schema_A.

What explains the engineer's lack of access to the underlying tables?

- A. The owner of the schema does not automatically have permission to tables within the schema, but can grant them to themselves at any point.
- B. Users granted with USE CATALOG can modify the owner's permissions to downstream tables.
- C. Permissions explicitly given by the table creator are the only way the Platform Engineer could access the underlying tables in their schema.
- D. The platform engineer needs to execute a REFRESH statement as the table permissions did not automatically update for owners.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineer has created a new cluster using shared access mode with default configurations. The data engineer needs to allow the development team access to view the driver logs if needed.

What are the minimal cluster permissions that allow the development team to accomplish this?

- A. CAN VIEW
- B. CAN RESTART
- C. CAN ATTACH TO
- D. CAN MANAGE

Suggested Answer: A

Community vote distribution

D (100%)

 **a68856c** Highly Voted 2 months, 2 weeks ago

Selected Answer: D

According with documentation, CAN VIEW does not exists in COMPUTE ACL, and the only way to view driver logs is with CAN MANAGE permissions.

<https://learn.microsoft.com/en-us/azure/databricks/security/auth/access-control/#log-access-note>

upvoted 5 times

A data engineer wants to create a cluster using the Databricks CLI for a big ETL pipeline. The cluster should have five workers and one driver of type i3.xlarge and should use the '14.3.x-scala2.12' runtime.

Which command should the data engineer use?

- A. databricks compute add 14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster
- B. databricks clusters create 14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster
- C. databricks compute create 14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster
- D. databricks clusters add 14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster

Suggested Answer: B

Community vote distribution

B (100%)

🗨️ 👤 **skggggg** 1 month, 1 week ago

Selected Answer: B

surely

upvoted 1 times

🗨️ 👤 **stopthisnow** 2 months, 1 week ago

Selected Answer: B

databricks clusters create SPARK_VERSION [flags]

<https://docs.databricks.com/aws/en/dev-tools/cli/reference/clusters-commands>

upvoted 1 times

A 'transactions' table has been liquid clustered on the columns 'product_id', 'user_id' and 'event_date'.

Which operation lacks support for cluster on write?

- A. CTAS and RTAS statements
- B. `spark.writeStream.format('delta').mode('append')`
- C. `spark.write.format('delta').mode('append')`
- D. INSERT INTO operations

Suggested Answer: B

Community vote distribution

B (100%)

🗳️ 👤 **_Lukas_** 2 months, 1 week ago

Selected Answer: B

Correct:

B - Structured Streaming operations using `spark.writeStream.format('delta').mode('append')` do NOT support cluster on write by default. To enable clustering on write for streaming workloads, you must explicitly set the Spark configuration `spark.databricks.delta.liquid.eagerClustering.streaming.enabled` to true. Without this configuration flag, streaming writes bypass the automatic clustering optimization that other operations receive.

upvoted 1 times

🗳️ 👤 **Ral17** 2 months, 1 week ago

Selected Answer: B

This should be B. Only writeStream lacks support for liquid cluster

upvoted 1 times

🗳️ 👤 **a68856c** 2 months, 2 weeks ago

Selected Answer: B

<https://docs.databricks.com/aws/en/delta/clustering#operations-that-support-clustering-on-write> . By default writeStream does not support liquid cluster.

upvoted 1 times

The data governance team has instituted a requirement that the "user" table containing Personal Identifiable Information (PII) must have the appropriate masking on the SSN column. This means that anyone outside of the HRAdminGroup should see masked social security numbers as ***-**-****.

The team created a masking function:

```
CREATE FUNCTION ssn_mask(ssn STRING)
  RETURN CASE WHEN is_member(HRAdminGroup) THEN ssn ELSE '***-**-****' END;
```

What does the data governance team need to do next to achieve this goal?

- A. CREATE TABLE users -
(name STRING);
ALTER TABLE users CREATE COLUMN ssn CREATE MASK ssn_mask;
- B. CREATE TABLE users -
(name STRING, int STRING);
ALTER TABLE users ALTER COLUMN ssn CREATE MASK if is_member('HRAdminGroup');
- C. CREATE TABLE users -
(name STRING, ssn INT MASKED ssn_mask);
- D. CREATE TABLE users -
(name STRING, ssn STRING);
ALTER TABLE users ALTER COLUMN ssn SET MASK ssn_mask;

Suggested Answer: D

 **stopthisnow** 2 months, 1 week ago

Selected Answer: D

D is the standard way of masking a column
upvoted 1 times

A data engineer needs to create an application that will collect information about the latest job run including the repair history.

How should the data engineer format the request?

- A. Call/api/2.1/jobs/runs/list with the run_id and include_history parameters
- B. Call/api/2.1/jobs/runs/get with the run_id and include_history parameters
- C. Call/api/2.1/jobs/runs/get with the job_id and include_history parameters
- D. Call/api/2.1/jobs/runs/list with the job_id and include_history parameters

Suggested Answer: D

Community vote distribution

B (100%)

🗲️ 👤 **Ral17** 2 months, 1 week ago

Selected Answer: B

B is the correct answer here, not D.

upvoted 1 times

🗲️ 👤 **a68856c** 2 months, 2 weeks ago

Selected Answer: B

<https://docs.databricks.com/api/workspace/jobs/getrun>

upvoted 2 times

A data engineer is working in an interactive notebook with many transformations before outputting the result from `display(df.collect())`. The notebook includes wide transformations and a cross join.

The data engineer is getting the following error: "The spark driver has stopped unexpectedly and is restarting. Your notebook will be automatically reattached."

Which action should the data engineer take?

- A. Run the notebook on a single node cluster to keep driver from falling.
- B. Rewrite their code to avoid putting memory pressure on the driver node.
- C. Check into the Spark UI to see how many jobs are assigned to each stage as they are employing fewer executors.
- D. Look at the compute metrics UI to see if the executors have higher than 90% memory utilization.

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

An analytics team wants run an experiment in the short term on the customer transaction Delta table (with 20 billions records) created by the data engineering team in Databricks SQL.

Which strategy should the data engineering team use to ensure minimal downtime and no impact on the ongoing ETL processes?

- A. Deep clone the table for the analytics team.
- B. Create a new table for the analytics team using a CTAS statement.
- C. Shallow clone the table for the analytics team.
- D. Give access to the table for the analytics team.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data team is working to optimize an existing large, fast-growing table 'orders' with high cardinality columns, which experiences significant data skew and requires frequent concurrent writes. The team notice that the columns 'user_id', 'event_timestamp' and 'product_id' are heavily used in analytical queries and filters, although those keys may be subject to change in the future due to different business requirements.

Which partitioning strategy should the team choose to optimize the table for immediate data skipping, incremental management over time, and flexibility?

- A. Partition the table with: ALTER TABLE orders PARTITION BY user_id, product_id, event_timestamp
- B. Use z-order after partiitiing the table: OPTIMIZE orders ZORDER BY (user_id, product_id) WHERE event_timestamp = current date () - 1 DAY
- C. Cluster the table with: ALTER TABLE orders CLUSTER BY user_id, product_id, event_timestamp
- D. Z-order the table with OPTIMIZE orders ZORDER BY (user_id, product_id, event_timestamp)

Suggested Answer: D

Community vote distribution

C (100%)

ef1715a 1 month ago

Selected Answer: C

Liquid Clustering

upvoted 1 times

Yahoo_Arun 1 month, 2 weeks ago

Selected Answer: C

Large table - Liquid Clustering

upvoted 1 times

balajir27 2 months ago

Selected Answer: C

Liquid clustering supports incremental processing

upvoted 1 times

a68856c 2 months, 2 weeks ago

Selected Answer: C

<https://docs.databricks.com/aws/en/delta/clustering> . With Liquid Clustering you can redefine cluster keys without rewriting data, this is appropriate for changing analytical needs. Also clustering is recommended when there is data skew, frequent concurrent writes and high cardinality.

upvoted 2 times

A faulty IoT sensor in a factory reports a temperature of -500, causing the LDP pipeline to fail the expectation, which only allows values between -100 and 200 degrees Celsius. The data engineer would like to further analyze the faulty data to better understand the reason behind this.

How should the data engineer resolve the faulty data while ensuring data quality standards are maintained?

- A. Remove all expectations from the pipeline to prevent any future failures, regardless of data quality.
- B. Ignore the error and simply re-run the pipeline, as Databricks will automatically skip the problematic record on the next run.
- C. Fix the pipeline code and implement a quarantine logic to isolate the faulty data before re-running the pipeline.
- D. Change the expectation action from fail to warn so that invalid records are included in the output and the pipeline does not fail.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer is optimizing a managed table that suffers from data skew and frequently changing query filter columns. The engineer needs to avoid costly data rewrites when query patterns evolve. The table size is under 1TB.

How should data engineer meet this requirement?

- A. Use Hive-style partitioning, as it provides efficient data skipping and is easy to change partition columns at any time.
- B. Combine partitioning and Z-ordering to maximize flexibility and minimize maintenance as query patterns change.
- C. Enable liquid clustering, as it efficiently handles data skew, allows clustering keys to be changed without rewriting existing data, and adapts to evolving query patterns.
- D. Apply Z-ordering, since it allows flexible reorganization of data layout without rewriting existing and adapts easily to new filter columns.

Suggested Answer: *C*

Currently there are no comments in this discussion, be the first to comment!

A security team wants to enforce data protection for a customer table containing customer PII data. To comply with local policies, sales team members should only see customers from their region, while non-admin users should have email addresses masked.

Which implementation approach should be used when using Unity Catalog row filters and column masks?

- A. Create SQL UDFs for row filtering based on user region and column masking based on group membership, then apply them using ALTER TABLE SET ROW FILTER and ALTER COLUMN SET MASK commands.
- B. Use table ACLs to restrict access using tags with GRANT SELECT ON table_name WITH TAG command, and rely on application-level filtering for sensitive data based on user region.
- C. Create a view with dynamic WHERE clauses for region filtering and use string replacement functions for email masking using ALTER COLUMN SET MASK command.
- D. Implement row filters with SQL UDFs based on user region only since column masks cannot be combined with row filters on the same table, then apply them by recreating the table with DROP TABLE and CREATE TABLE SET ROW FILTER commands.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineer is evaluating tools to build a production-grade data pipeline. The team must process change data from cloud object storage, filter out or isolate invalid records, and ensure the timely delivery of clean data to downstream consumers. The team is small, under tight deadlines, and wants to minimize operational overhead while keeping pipelines auditable and maintainable.

Which approach should the data engineer implement?

- A. Ingest data directly into Delta tables via Spark jobs, apply data quality filters using UDFs, and use LDP for creating Materialized Views.
- B. Use a hybrid approach: Ingest with Auto Loader into Bronze tables, then process using SQL queries in Databricks Workflows to generate cleaned Silver and Gold tables on a schedule.
- C. Implement ingestion using Auto Loader with Structured Streaming, and manage invalid data handling and table updates using checkpointing and merge logic.
- D. Use LDP to build declarative pipelines with Streaming Tables and Materialized Views, leveraging built-in support for data expectations and incremental processing.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is using Structured Streaming to read in transaction data from a bronze Delta table. It was discovered that the data has quality issues where sometimes the transaction value is negative, and when that occurs, the rows need to be routed to a separate quarantine table. They have low latency requirements for the good data since it is used by downstream systems, but the bad data will only be analyzed periodically and has no production dependencies. The quarantine job needs to be implemented so that it cannot affect the production processes that depend on the good data, and the cost of the job needs to be minimized.

How should the quarantine process be implemented in order to satisfy these requirements?

- A. The streaming job for the good data needs to be modified to filter out records with a transaction value less than 0 before writing. The streaming job for the quarantine data needs to filter out records with a transaction value greater than or equal to 0 before writing. Both should run as separate streams on the same cluster to minimize cost.
- B. The existing streaming job for the good data should be updated to incorporate the quarantining of the bad data. A new boolean column called "quarantine" should be added to the dataframe, and its value should be set to true if the transaction value is less than 0 and false if the transaction value is greater than or equal to 0. Processing and storing all the data together will save costs.
- C. The existing streaming job for the good data should be updated to incorporate the quarantining of the bad data. Inside a `foreachBatch` function, the dataframe should be filtered so that records with a transaction value greater than or equal to 0 are written to the good data table and records with a transaction value less than 0 are written to a quarantine table. `Try/Catch` can be added around the writes in the `foreachBatch` function so that the stream can't fail.
- D. The streaming job for the good data needs to be modified to filter out records with a transaction value less than 0 before writing, and should not share compute with other processes. The streaming job for the quarantine data needs to filter out records with a transaction value greater than or equal to 0 before writing, and should be implemented on a separate small cluster and only run once a day to minimize cost.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

When monitoring a complex workload, being able to see the query plan is critical to understanding what the workload is doing.

Where can the visualization of the query plan be found?

- A. In the Spart UI, under the Jobs tab
- B. In the Query Profiler, under Query Source
- C. In the Spark UI, under the SQL/DataFrame tab
- D. In the Query Profiler, under the Stages tab

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer is troubleshooting a slow-running Delta Lake query on Databricks SQL involves complex joins and large datasets. They need to identify whether the root cause is related to poor data skipping, inefficient join strategies, or excessive data shuffling.

Which approach should identify the specific bottlenecks using native Databricks tools?

- A. Analyze the Top Operators panel in the Query Profile to identify high-cost operations like BroadcastNestedLoopJoin
- B. Check the query's execution time in the Jobs UI and correlate it with cluster resource utilization metrics.
- C. Enable the EXPLAIN command to review the parsed logical plan and manually estimate shuffle sizes.
- D. Use the LIMIT clause to run a subset of the query and compare execution times with the full dataset.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineer is designing a secure data sharing strategy for their organization. The company needs to share sensitive customer analytics data with two different partners. Partner A uses Databricks with Unity Catalog enabled, while Partner B uses Apache Spark on AWS without Databricks.

How should the company implement secure data sharing for these scenarios?

- A. For Partner A, implement Databricks-to-Databricks sharing (D2D) with Unity Catalog integration and no-token exchange system. For Partner B, use open sharing protocol (D2O) with either bearer tokens or OIDC federation for authentication, ensuring both approaches maintain robust security and governance.
- B. Both partners should use the same Delta Sharing approach since security requirements are identical. You should create bearer tokens for both partners and use the open sharing protocol (D2O) for maximum compatibility.
- C. Open sharing protocol (D2O) should be used for both partners because it provides better security than D2D sharing. The bearer token approach is always more secure than Unity Catalog's native authentication.
- D. Databricks-to-Databricks sharing (D2D) can only be used within the same cloud provider, so you must use open sharing (D2O) for any cross-cloud scenarios. Unity Catalog governance is not available when sharing with external platforms.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

Which approach demonstrates a modular and testable way to use DataFrame transform for ETL code in PySpark?

- A.
- ```
class Pipeline:
 def transform(self, df):
 return df.withColumn("value_upper", upper(col("value")))

pipeline = Pipeline()
assertDataFrameEqual (pipeline.transform(test_input), expected)

def transform_data(input_df):
 # transformation logic here
 return output_df
```
- B.
- ```
test_input = spark.createDataFrame ([ (1, "a") ], ["id", "value"])
assertDataFrameEqual (transform_data (test_input), expected)

def upper_transform(df):
    return df.withColumn("value_upper", upper(col("value")))

C.
```
- ```
actual = test_input.transform(upper_transform)
assertDataFrameEqual (actual, expected)

def upper_value (df):
 return df.withColumn("value_upper", upper(col("value")))
```
- D.
- ```
def filter_positive(df):
    return df.filter (df["id"] > 0)

pipeline_df = df.transform(upper_value).transform
(filter_positive)
```

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A company stores account transactions in a Delta Lake table. The company needs to apply frequent account-level correlations (e.g., UPDATE statements) but wants to avoid rewriting entire Parquet files for each change to reduce file churn and improve write performance.

Which Delta Lake feature should they enable?

- A. Enable automatic file compaction on writes
- B. Enable change data feed on the Delta table
- C. Partition the Delta table by account_id
- D. Enable deletion vectors on the Delta table

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

In a Databricks Asset Bundle project, in the file resources/app.yml, the data engineer would like to deploy a Databricks Apps databricks_app_deployed and Volume volume_deployed and grant the Service Principal behind Databricks Apps permissions to READ and WRITE to the Volume.

How should the data engineer achieve the deployment?

- ```
resources:
 apps:
 databricks_app_deployed:
 name: "name-databricks_app"
 description: 'Databricks App'
 source_code_path: ../src/databricks_app_code

volumes:
```
- A.
- ```
  volume_deployed:
    name: name_volume
    catalog_name: name_catalog
    schema_name: name_schema
    grants:
      -principal: ${resources.app.databricks_app_deployed.id}
      privileges:
        -READ_VOLUME
        -WRITE_VOLUME

resources:
  apps:
    databricks_app_deployed:
      name: "name-databricks_app"
      description: 'Databricks App'
      source_code_path: ../src/databricks_app_code

volumes:
```
- B.
- ```
 volume_deployed:
 name: name_volume
 catalog_name: name_catalog
 schema_name: name_schema
 grants:
 -principal: ${resources.app.name-databricks_app.id}
 privileges:
 -READ_VOLUME
 -WRITE_VOLUME

resources:
 apps:
 databricks_app_deployed:
 name: "name-databricks_app"
 description: 'Databricks App'
 source_code_path: ../src/databricks_app_code

volumes:
```
- C.
- ```
  volume_deployed:
    name: name_volume
    catalog_name: name_catalog
    schema_name: name_scheama
    grants:
      -principal: ${resources.app.name-
databricks_app_deployed.id}
      privileges:
        -READ
        -WRITE
```

```
resources:
  apps:
    databricks_app_deployed:
      name: "name-databricks_app"
      description: 'Databricks App'
      source_code_path: ../src/databricks_app_code

  volumes:
D.   volume_deployed:
      name: name_volume
      catalog_name: name_catalog
      schema_name: name_schema
  grants:
    -principal: ${resources.app.name-name-databricks_app.id}
    privileges:
      -READ_VOLUME
      -WRITE_VOLUME
```

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineering teams needs to implement a tagging system for their tables as part of an automated ETL process, and needs to apply tags programmatically to tables in Unity Catalog.

Which SQL command adds tags to a table programmatically?

- A. ALTER TABLE table_name SET TAGS ('key1' = 'value1', 'key2' = 'value2');
- B. APPLY TAGS ON table_name VALUES ('key1' = 'value1', 'key2' = 'value2')
- C. COMMENT ON TABLE table_name TAGS ('key1' = 'value1', 'key2' = 'value2')
- D. SET TAGS FOR table_name AS ('key1' = 'value1', 'key2' = 'value2')

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A platform engineer needs to report the resource consumption, categorized by SKU tier, across all workspaces. The engineer decides to use the system.billing.usage system table to create a query.

Which SQL query will accurately return the daily usage by product?

- ```
SELECT
 DATE_TRUNC('day', usage_start_time) AS day,
 sku_name,
 SUM(dbus) AS daily_dbus
FROM
 system.billing.usage
WHERE
 sku_name IS NOT NULL
GROUP BY
 day, sku_name;
```
- A.
- ```
SELECT
    CAST(usage_start_time AS DATE) AS usage_day,
    sku_name,
    COUNT (usage_quantity) AS dbu_count
FROM
    system, billing.usage
GROUP BY
    usage_day, sku_name
ORDER BY
    usage_day;
```
- B.
- ```
SELECT
 date_trunc('day', usage_start_time) AS usage_day,
 sku_name,
 SUM(usage_quantity) AS total_daily_dbus
FROM
 system.billing.usage
WHERE
 usage_quantity IS NOT NULL
GROUP BY
 usage_day, sku_name
ORDER BY
 usage_day;
```
- C.
- ```
SELECT
    TO_DATE(usage_start_time)AS usage_day,
    sku_name,
    SUM(usage_quantity) AS total_dbus
FROM
    system.billing.usage
GROUP BY
    usage_day
ORDER BY
    usage_day
```
- D.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer is running a `groupBy` aggregation on a massive user activity log grouped by `user_id`. A few users have millions of records, causing task skew and long runtimes.

Which technique will fix the skew in this aggregation?

- A. Increase the Spark driver memory and retry.
- B. Filter out the skewed users before the aggregation
- C. Use salting by adding a random prefix to skewed keys before aggregation, then aggregate again after removing the prefix.
- D. Use `reduceByKey` instead of `groupBy` to avoid shuffles.

Suggested Answer: *C*

Currently there are no comments in this discussion, be the first to comment!

A job runs four independent tasks (X, Y, Z, W) in parallel to process regional sales data. The Data Engineering team recently updated its cluster policy to ban cost-prohibitive instance types. Task Y now fails due to the newly enforced cluster policy restricting the use of a specific instance type. A data engineer needs to resolve the failure quickly without disrupting the other tasks.

How should the data engineer resolve the failure of tasks?

- A. Delete the failed run, disable the cluster policy, and re-execute all tasks.
- B. Manually create a new cluster for Task Y, update the job configuration, and trigger a full re-run.
- C. Use "Repair run", override the cluster configuration for Task Y to use a permitted instance type, and let Databricks re-run only Task Y.
- D. Edit the global cluster policy to allow the restricted instance type, then re-run the entire job.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer is analyzing a large, partitioned retail dataset in Databricks, where each row represents a sale made by a salesperson. The dataset contains millions of records with the following schema:

```
sales_df: [salesperson_id: string, region: string, sale_amount: double, sale_date: date]
```

The data engineer needs to generate a DataFrame that ranks salespeople within each region based on their total cumulative sales, with the highest seller ranked as 1. If multiple salespeople have the same total sales, they should share the same rank.

The data engineer wants to implement this logic using a PySpark window function and the `dense_rank()` function.

Which code snippet will perform this ranking?

- A.
- ```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

window_spec = Window.partitionBy("region").orderBy(
 (F.desc("sale_amount"))

ranked_df= sales_df.withColumn("rank",
 F.dense_rank().over(window_spec))
```
- B.
- ```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

agg_df= sales_df.groupBY("salesperson_id", "region") \
    .agg(F.sum("sale_amount").alias("total_sales"))

window_spec = Window.partitionBy("region").orderBy(
    (F.desc("total_sales")))

ranked_df= agg_df.withColumn("rank", F.rank().over(window_spec))
```
- C.
- ```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

agg_df= sales_df.groupBY("salesperson_id", "region") \
 .agg(F.sum("sale_amount").alias("total_sales"))

window_spec = Window.orderBy (F.desc("total_sales"))

ranked_df= agg_df.withColumn("rank",
 F.dense_rank().over(window_spec))
```
- D.
- ```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

agg_df= sales_df.groupBY("salesperson_id", "region") \
    .agg(F.sum("sale_amount").alias("total_sales"))

window_spec =
    Window.parititionBy("region").orderBy(F.desc("total_sales"))

ranked_df=agg_df.withColumn("rank",
    F.dense_rank().over(window_spec))
```

Suggested Answer: D

What describes a primary technical challenge in ensuring consistent PII masking across all nodes in large-scale, distributed Databricks batch and streaming pipelines?

- A. PII masking is only required for direct identifiers.
- B. Dynamic data masking is applied only at rest, so it does not affect query performance.
- C. Masking functions must be standardized and managed through Unity Catalog, with enforcement applied across all relevant datasets to avoid any data inconsistency.
- D. Native masking in Databricks automatically synchronizes with all downstream external Databricks systems.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer is working on a Databricks notebook that requires several third-party Python libraries. Some of these are available on PyPI, while others are custom-developed and stored as local.wheel (.whl) and source (.tar.gz) files in an S3 bucket. The goal is to ensure all dependencies are installed and correctly available across multiple jobs running on any automated cluster in a Unity Catalog-enabled workspace. The engineer needs to install the required dependencies in a way that ensures a consistent environment setup across interactive notebooks and jobs and complies with workspace security policies (no internet access).

Which approach should the engineer use to install and manage these dependencies while also ensuring reproducibility and compliance?

- A. Use an init script on the cluster to install all dependencies using pip, referencing the local file system.
- B. Install all dependencies manually in the driver node of an interactive cluster, then export the environment and reimport on job clusters using %conda.
- C. Create a Python wheel file for the entire project, upload it to the Databricks Workspace Files or Volumes, and install it using a Cluster Library or pip install in a requirements.txt declared within a Databricks Asset Bundle.
- D. Use %pip install in every notebook and job to install packages directly from PyPI and custom S3 paths.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer is using Auto Loader to read in JSON data as it arrives. They have configured Auto Loader to quarantine invalid JSON records. They are noticing that over time, some records are being quarantined even though they are well-formed JSON.

The snippet of code is:

```
df = (spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "json")
      .option("badRecordsPath", "/tmp/somewhere/badRecordsPath")
      .schema("a int, b int")
      .load("/Volumes/catalog/schema/raw_data/"))
```

What is the cause of the missing data?

- A. The source data is valid JSON, but doesn't conform to their defined schema in some way
- B. The badRecordsPath location is accumulating many small files
- C. The engineer forgot to set the option "cloudFiles.quarantineMode", "rescue".
- D. At some point, the upstream data provider switched everything to multi-line JSON.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineer wants to enforce the principle of least privilege when configuring ACLs for Databricks jobs in a collaborative workspace.

Which approach should the data engineer use?

- A. Assign users only the minimum permission level (e.g., CAN RUN or CAN VIEW) required for their role on each job.
- B. Use only folder-level permissions and avoid setting permissions on individual jobs.
- C. Grant CAN RUN permission to everyone and CAN MANAGE to a single admin group.
- D. Grant all users CAN MANAGE permission on all jobs to avoid access issues.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineering workspace was automatically enabled for Unity Catalog, creating a workspace catalog. New team members report they can create tables in the default schema but cannot access table in other schemas within the same workspace catalog.

Why are the new team members unable to access tables in other schemas?

- A. Workspace catalog permissions are not subject to inheritance rules.
- B. Workspace users receive USE CATALOG and specific privileges on default schema only.
- C. Tables in other schemas require additional BROWSE privileges that new users don't receive automatically
- D. New users only receive CREATE TABLE privileges on the default schema.

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is implementing a job to download multiple PDF files from a third-party provided REST API endpoint by specifying different report types. The REST API is time-consuming and encounters intermittent errors, so the engineer wants to track each download activity to know when it fails and to retry partially, while providing scalable throughput. The engineer needs to download ten report types, and the list can be changed over time.

How should the data engineer achieve this?

- A. Use a foreach task with a list of report types as its inputs.
- B. Define ten Notebook tasks to clearly track which report download failed.
- C. Use a Delta Lake table to track each report download status as 10 rows, and use it as a source table to execute the download function as a Pandas UDF.
- D. Define a list variable within a Notebook to loop through the report types to download them, and print the download results. Execute it as a Notebook tasks.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineer and a platform engineer are working together to automate their system tasks. A script needs to be executed outside of Databricks only if a particular daily Databricks job finishes successfully for the day. Databricks CLI command was used to check the last execution of the job.

What are the required command options for that task?

- A. databricks jobs list-runs --job-id JOB_ID --start-time-to TODAY_MIDNIGHT_EPOCH_MS --completed-only
- B. databricks jobs list-runs --job-id JOB_ID --start-time-from TODAY_MIDNIGHT_EPOCH_MS --active-only
- C. databricks jobs list-runs --job-id JOB_ID --start-time-to TODAY_MIDNIGHT_EPOCH_MS --active-only
- D. databricks jobs list-runs --job-id JOB_ID --start-time-from TODAY_MIDNIGHT_EPOCH_MS --completed-only

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineering team needs to create a SQL Alert that monitors data quality across multiple columns in their customer table. They want to trigger an alert when both the percentage of customers with missing email addresses exceeds 15% AND the percentage of customers with invalid phone number formats exceeds 10%.

Which SQL query pattern is appropriate for implementing this multi-column alert condition?

- A. `SELECT COUNT (*) FROM customers WHERE email IS NULL OR phone_format_invalid = true`
- B. `SELECT email, phone FROM customers WHERE email IS NULL AND phone NOT RLIKE '^ [0-9-+()\\s]+ $'`
- C. `SELECT email_null_pct, phone_invalid_pct FROM (SELECT (COUNT(CASE WHEN email IS NULL THEN 1 END) * 100.0 / COUNT (*)) as email_null_pct, (COUNT(CASE WHEN phone NOT RLIKE '^ [0-9-+()\\s]+ $' THEN 1 END) * 100.0 / COUNT (*)) as phone_invalid_pct FROM customers)`
- D. `SELECT CASE WHEN email_null_pct > 15 AND phone_invalid_pct > 10 THEN 1 ELSE 0 END FROM (SELECT (COUNT (CASE WHEN email IS NULL THEN 1 END) * 100.0 / COUNT (*)) as phone_invalid_pct FROM customers) metrics`

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer is bringing an existing production Databricks job under asset bundle management and wants to ensure that:

- The job's current configuration is captured as YAML, and all referenced files are included in their bundle project.
- Future changes to the bundle's YAML will update the existing job in-place (not create a new job)

How should the data engineer successfully move the production job under asset bundle management?

- A. Run Databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deploy to deploy the bundle, which will always update the existing job automatically.
- B. Export the job definition as JSON, convert it to YAML, and place it in your bundle. Then, run Databricks bundle deploy to update the existing job.
- C. Manually create the YAML configuration for the job in your bundle project, ensuring all settings match the existing job. Then, run Databricks bundle deploy the bundle, which will update the existing job in your workspace.
- D. Run databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deployment, bind to link the bundle's job resource to the existing job in Databricks.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data architect is implementing Delta Sharing as part of their data governance strategy to enable secure data collaboration with external partners and internal business units. The architect must establish a permission framework that allows designated data stewards to create shares for their respective domains while maintaining security boundaries and audit compliance.

Which specific permissions and roles must be assigned to enable users to create, configure, and manage Delta Shares while maintaining proper security governance and access controls?

- A. Only workspace admins can create and manage shares
- B. Users need the MANAGE SHARES permission on the workspace
- C. Users need to be metastore admins or have CREATE SHARE privilege for the metastore
- D. Any user with USE_CATALOG privilege can create shares

Suggested Answer: *C*

Currently there are no comments in this discussion, be the first to comment!

A data engineer manages a production Lakeflow Spark Declarative Pipeline that processes customer transaction data. The pipeline includes several data quality expectations, such as: `transaction_amount > 0` and `customer_id IS NOT NULL`.

These expectations are defined using the `EXPECT` clause in SQL. The engineer aims to monitor the pipeline's data quality by analyzing the number of records that passed or failed each expectation during the latest pipeline update. The Lakeflow Spark Declarative Pipelines event logs are stored in a Delta table named `event_log_table`.

For the most recent pipeline update, determine a programmatically approach to extract information like the name of each expectation, associated dataset, count of records that passed the expectation, and count of records that failed the expectation.

Which method retrieves the desired data quality metrics from the Lakeflow Spark Declarative Pipelines event log?

- A. Access the `event_log_table`, filter for events where `event_type = 'flow progress'`, and parse the `details.flow_progress.data_quality.expectations` field to extract the required metrics.
- B. Use the Lakeflow Spark Declarative Pipelines UI to navigate to the specific pipeline, select the dataset, and view the Data Quality tab to manually retrieve the expectation metrics.
- C. Query the `event_log_table` for events with `event_type = 'data_quality'` and directly select the `passed_records` and `failed_records` fields.
- D. Access the `event_log_table`, filter for events where `event_type = 'expectatation_result'`, and extract the expectation metrics from the `details` field.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

Predictive Optimization is an automated Databricks service enabled by default for Unity Catalog Managed tables. It helps maintain Delta tables by continuously optimizing them to ensure optimal performance and costs.

Which two operations does Predictive Optimization run to maintain the Delta tables? (Choose two.)

- A. PARTITION BY
- B. COMPACT
- C. ANALYZE
- D. OPTIMIZE
- E. BUCKETING

Suggested Answer: *CD*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is building a customer data pipeline in Lakeflow Spark Declarative Pipelines. The source is a cloud-based event stream with limited retention containing inserts, updates, and deletes for customer records. These changes are being applied using the AUTO CDC INTO syntax to maintain an SCD Type 1 table as the target table, customer_dim.

How should the data engineer build a downstream job that streams from the customer_dim table to only act on updates and delete events, processing data incrementally?

- A. Use ignoreChanges flag while streaming from customer_dim to avoid breaking the pipeline during updates and deletes.
- B. Read change data feed from customer_dim table and apply filters to incrementally act on the change events.
- C. Streaming from customer_dim table would only be possible in the case of SCD 2 retention.
- D. When stored as SCD 1, the target of AUTO CDC INTO includes updates and deletes. Streaming from customer_dim can fail due to these operations. Instead, build another stream from the original source.

Suggested Answer: B

Currently there are no comments in this discussion, be the first to comment!

A data engineer is designing a system leveraging Lakeflow Declarative Pipeline technology to process real-time truck telemetry data ingested from JSON files in S3 using Auto Loader. The data includes truck_id, timestamp, location, speed, and fuel_level. The system must support two use cases:

1. Near-real-time monitoring of the latest location, speed, and fuel_level per truck_id for the operations team.
2. Daily aggregated reports of total distance traveled and average fuel efficiency per truck_id for the management team.

Which approach should the data engineer use for streaming tables and materialized views in the Lakeflow Declarative Pipeline to meet these requirements?

- A. Define a streaming table to ingest and store the raw telemetry data, and create a streaming table to compute the daily aggregated distance and fuel efficiency per truck_id reporting. Create a materialized view to compute the latest location, speed, and fuel_level per truck_id for real-time monitoring.
- B. Define a streaming table to ingest and store the raw telemetry data, and create a materialized view to compute the latest location, speed, and fuel_level per truck_id for real-time monitoring. Create another materialized view to compute the daily aggregated distance and fuel efficiency per truck_id for reporting.
- C. Define a streaming table to ingest and store the raw telemetry data, and create a streaming table to incrementally compute the latest location, speed, and fuel_level per truck_id for real-time monitoring. Create a materialized view to compute the daily aggregated distance and fuel efficiency per truck_id for reporting.
- D. Define a materialized view to ingest and store the raw telemetry data, and create a streaming table to compute the latest location, speed, and fuel_level per truck_id for real-time monitoring. Create another materialized view to compute the daily aggregated distance and fuel efficiency per truck_id for reporting.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A platform team lead is responsible for automating the individual teams attribution towards SQL Warehouse usage. The requirement is to identify the SQL warehouse usage at the individual user's level and generate a daily report to be shared with an executive team that includes leaders from all business units.

How should the platform lead generate an automated report that can be shared daily?

- A. Use the system tables to capture the audit and billing usage data and share the queries with the executive team. This enables the executives to execute the query and see the latest results any time.
- B. Use the system tables to capture the audit and billing usage data and create a dashboard with daily refresh schedules and shared with the executive team.
- C. Restrict users from running any SQL query unless they provide all the query details so that the attribution can be calculated and shared with the executive team.
- D. Let the users run the SQL query and then directly report the usage to the executives. The ownership of the SQL warehouse usage will be with the individual teams.

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

A data engineering team is collaborating on a Databricks project where each team member needs to develop and test code independently before merging changes into the main branch. They want to avoid accidental overwrites or branch switching issues while ensuring that all work is version- controlled and can be integrated into their CI/CD pipeline.

How should the data engineer achieve collaboration?

- A. Each team member creates their own Databricks Git folder, mapped to the same remote Git repository, and works in their own development branch within their personal folder.
- B. All team members work in the same Databricks Git folder and perform Git operations (pull, push, commit, branch switching) directly in that shared folder.
- C. Team members edit notebooks directly in the workspace's shared folder and periodically copy changes into a Git folder for version control.
- D. Team members use the Databricks CLI to clone the Git repository and perform Git operations from a cluster's web terminal.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineer is using the AUTO CDC API in Lakeflow Spark Declarative Pipeline to propagate deletions from a source table (orders_source) to a target table (orders_target). The source has Change Data Feed (CDF) enabled, but some delete events arrive out of order due to upstream delays.

How does the AUTO CDC API internally ensure deletions are applied correctly despite out-of-order events?

- A. It ignores deletions if they arrive after updates for the same key.
- B. It manually sorts incoming events by timestamp before applying changes.
- C. It runs VACUUM on the target table to purge conflicting records.
- D. It uses sequence_by to order events and retains tombstones for deleted rows until older sequences are processed.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data governance team at a large enterprise is improving data discoverability across its organization. The team has hundreds of tables in their Databricks Lakehouse with thousands of columns that lack proper documentation. Many of these tables were created by different teams over several years, with missing context about column meanings and business logic. The data governance team needs to quickly generate comprehensive column descriptions for all existing tables to meet compliance requirements and improve data literacy across the organization. They want to leverage modern capabilities to automatically generate meaningful descriptions rather than manually documenting each column, which would take months to complete. The team is looking for a solution that can understand data patterns, column names, and sample values to create intelligent descriptions.

Which approach should the team use in Databricks to automatically generate column comments and descriptions for existing tables?

- A. Write custom PySpark code using `df.describe()` and `df.schema` to programmatically generate basic statistical descriptions for each column.
- B. Navigate to the table in Databricks Catalog Explorer, select the table schema view, and use the "AI Generate" option which leverages artificial intelligence to automatically create meaningful column descriptions based on column names, data types, sample values, and data patterns.
- C. Use the `DESCRIBE TABLE` command to extract existing schema information and manually write descriptions based on column names and data types.
- D. Use Delta Lake's `DESCRIBE HISTORY` command to analyze table evolution and infer column purposes from historical changes.

Suggested Answer: B

Currently there are no comments in this discussion, be the first to comment!

A company processes semi-structured JSON files from an external source using Auto Loader in a classic Databricks job. Occasionally, records arrive with null critical fields, invalid types, or unexpected nested schema variations. The engineer must ensure that malformed or non-conforming records are not dropped silently and are captured in a separate quarantine table. The pipeline should continue processing good records into the Bronze layer without failing the job, and the approach must support both batch and streaming ingestion.

The data engineer needs to build a robust ingestion pattern that automatically routes bad records to a quarantine Delta table, while still ingesting good records into the Bronze layer for further processing.

Which approach fulfills the quarantine mechanism in this ingestion architecture?

- A. Create a notebook job with `inferSchema= True`, write a streaming query with `.foreachBatch()` and catch exceptions using `try/except` to redirect failed batches to quarantine.
- B. Use Auto Loader with `failFast` mode to set to `false`, and enable schema evolution; invalid records will be silently ignored during ingestion.
- C. Use Lakeflow Spark Declarative Pipelines with a SQL pipeline; configure it to drop rows with nulls using `where critical_fields is not null`, and rely on audit logs for malformed data.
- D. Use Auto Loader with LDP and implement an `EXPECT ()` constraint with a record audit logic to route bad records.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is analyzing transactional data in a PySpark DataFrame `df` containing `customer_id`, `transaction_timestamp` (precise to milliseconds), and `amount_spent`. The objective is to compute a cumulative sum of `amount_spent` per customer, strictly ordered by `transaction_timestamp`. The cumulative sum must include all transactions from the earliest timestamp up to and including the current row, respecting temporal ordering within each customer partition.

Which PySpark code snippet most accurately constructs the appropriate window specification and applies the aggregation to yield the correct cumulative expenditure per customer?

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col
```

A.

```
window_spec = Window.partitionBy("customer_id")\
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("cumulative_spent", sum("amount_spent").over(window_spec))

from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col
```

B.

```
window_spec = Window.orderBy("transaction_timestamp")\
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("cumulative_spent", sum("amount_spent").over(window_spec))

from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col
```

C.

```
window_spec = Window.partitionBy("customer_id").orderBy("transaction_timestamp")\
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("cumulative_spent", sum("amount_spent").over(window_spec))

from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col
```

D.

```
window_spec = Window.partitionBy("customer_id").orderBy("transaction_timestamp")
df.withColumn("cumulative_spent", sum("amount_spent").over(window_spec))
```

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data team is implementing an append-only Delta Lake pipeline that needs to handle both batch and streaming data. They want to ensure that schema changes in the source data can be automatically incorporated without breaking the pipeline.

Which configuration should the team use when writing data to the Delta table?

- A. ignoreChanges = false
- B. validateSchema= false
- C. overwriteSchema=true
- D. mergeSchema=true

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A senior data engineer is planning large-scale data workflows. The current task is to identify the considerations that form a foundation for creating scalable data models that are essential for effective management of large datasets. The data engineering team has identified the core capabilities as part of a scalable data model to build a modern data platform and provided their reasoning for considering Delta Lake for review. The senior data engineer is responsible for identifying the recommendations that are not valid.

Which key features can be ignored while evaluating Delta Lake?

- A. Delta Lake works with various data formats (e.g., Parquet, JSON, CSV) and integrates well with Spark and Databricks tools:
- B. Delta Lake optimizes metadata handling, efficiently managing billions of files and facilitating scalability to petabyte-scale datasets.
- C. Delta Lake's capability to process data in both batch and streaming modes seamlessly, providing flexibility in data ingestion and processing.
- D. Delta Lake provides limited support for monitoring and troubleshooting data pipelines, so relevant partner tools have to be identified and set up for enhanced operational efficiency.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineering team is implementing an append-only data pipeline using Delta Lake, and wants to ensure that data is never modified or deleted once written.

Which Delta Lake feature should the data engineer enable to prevent modifications to existing data?

- A. Delta APPEND_ONLY
- B. Delta VACUUM
- C. Delta OPTIMIZE
- D. Delta Time Travel

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineering team is setting up a Git project to automate integration tests using Databricks Asset Bundles and the Git provider's CI/CD functionalities. When a pull containing changes to their pipeline is sent, they need to run a Job to test their data pipeline.

What is the correct databricks bundle command sequence to be executed from the Git provider's CI/CD automation for this task?

- A. init, deploy, run, validate
- B. validate, deploy, run
- C. init, validate, deploy, run
- D. deploy, run, validate

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is attempting to execute the following PySpark code:

```
df=spark.read.table("sales")  
result=df.groupBy("region").agg(sum("revenue"))
```

However, upon inspecting the execution plan and profiling the Spark job, they observe excessive data shuffling during the aggregation phase.

Which technique should be applied to reduce shuffling during the groupBy aggregation operation?

- A. Use `coalesce(1)` after the aggregation
- B. Caching the DataFrame `df`
- C. Use broadcast join
- D. Repartition by region before aggregation

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

While reviewing a query's execution in the Deatricks Query Profile, a data engineer observes that the "Top operators" panel shows a sort operator with high "Time spent" and "Memory peak" metrics, and the Spark UI reports frequent data spilling.

How should the data engineering address this issue?

- A. Repartition the DataFrame to a single partition before sorting.
- B. Convert the sort operation to a filter operation.
- C. Switch to a broadcast join to reduce memory usage.
- D. Increase the number of shuffle partitions to better distribute data.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer manages a Unity Catalog table `customer_data` in schema `finance` that includes sensitive fields like `ssn` and `credit_score`. Intern Group should only see masked values, while Analyst Group should only access rows for their assigned region. The data engineer needs to restrict access based on user role and region without duplicating data.

How should the data engineer enforce this security policy?

- A. Use Unity Catalog's row filters based on the region and column masks based on user roles.
- B. Create views using `current_user()` and `is_account_group_member()` functions, and apply masking logic inside the SQL `SELECT` clause for each sensitive column.
- C. Create dynamic views for each user role and manage access with ACLs.
- D. Use Unity Catalog's row filters based on the user roles and column masks based on the region.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineer is ingesting JSON files from cloud object storage using Databricks Auto Loader. The source folder may occasionally receive large files of data, which risks overwhelming the stream. To ensure predictable micro-batch sizes, the team wants to throttle ingestion based on the volume of data scanned at 1 GB, regardless of the number of files.

Which Auto Loader configuration should the data engineer use to achieve this?

- A. Configure `cloudFiles.maxBytesPerTrigger` with 1 GB to place a limit.
- B. Configure `cloudFiles.maxSizePerTrigger` with 1 GB to place a limit.
- C. Configure `cloudFiles.maxFilesPerTrigger` and estimate the average file size to approximate a size-based throttle of 1 GB.
- D. Configure `cloudFiles.maxPartitionBytes` with 1GB to limit data in each partition.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data company uses Databricks Unity Catalog and has multiple enterprise data sources, including PostgreSQL, Snowflake, and SQL Server. The central data platform team wants to configure Lakehouse Federation so analysts can query external tables directly in Databricks using Databricks SQL, without duplicating data.

Which steps are necessary to configure Lakehouse Federation in a secure and governed manner?

- A. Mirror the external datasets into Delta Lake using Auto Loader, and govern them using Data Lineage and System Tables.
- B. Configure connections and foreign catalog in Unity Catalog, then grant access to foreign catalogs, schemas, and tables using Unity Catalog permissions.
- C. Use Partner Connect to create linked datasets, and apply table ACLs at the source system to govern access through Databricks.
- D. Create external locations and storage credentials to connect to each database, then register foreign tables in Unity Catalog.

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

Why are Pandas UDFs often preferred over traditional PySpark UDFs in performance-critical applications involving large datasets?

- A. They minimize memory usage by streaming each row individually through a lightweight Python wrapper, avoiding batch processing overhead.
- B. They leverage Apache Arrow to enable vectorized operations between the JVM and Python runtimes, reducing serialization costs and improving computational efficiency.
- C. They allow row-level execution of functions in Python with native Spark optimization, removing the need for columnar execution.
- D. They eliminate the JVM-Python boundary by bypassing serialization entirely, thereby avoiding data conversion overhead.

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is setting up a pipeline to ingest data from a message bus system that occasionally delivers duplicate messages. The duplicate messages can be a week apart. The target is a Databricks Delta Lake table where each record should appear exactly once.

Which Databricks ingestion pattern should be implemented to handle potential duplicates where events can arrive outside of the configured watermark?

- A. Use Delta Lake's change data feed to filter duplicate records
- B. Use Delta Lake time travel to identify and remove duplicates
- C. Configure Structured Streaming with dropDuplicates transformation
- D. Implement a write operation using MERGE INTO with a unique key

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer needs to design an efficient pipeline that automatically processes new CSV files as they arrive in S3 storage.

Which Databricks feature should the data engineer use to meet these requirements?

- A. Streaming from cloud storage using standard Spark readStream with format ("csv") and format ("json")
- B. COPY INTO SQL command with parameters to track processed files
- C. Traditional batch processing with scheduled Databricks Jobs
- D. Auto Loader with schema inference and evolution enabled

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is implementing liquid clustering on a Delta Lake table and needs to understand how it affects data management operations. The table will be updated frequently with new data. The table is an external table and not managed by Unity Catalog.

How does liquid clustering in Delta Lake handle new data that is inserted after the initial table creation?

- A. New data is rejected if it doesn't match the clustering pattern.
- B. New data is automatically clustered during write operations.
- C. New data is written to a staging area and clustered during scheduled maintenance.
- D. New data remains unclustered until the next OPTIMIZE operation.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is optimizing a MERGE operation on an 800GB UC-managed table that experiences frequent updates and deletions.

Which two actions should the engineer prioritize to improve MERGE performance? (Choose two.)

- A. Apply liquid clustering using the merge join keys.
- B. Enable deletion vectors on the table if not already enabled.
- C. Partition the table by date.
- D. Use ZORDER on high-cardinality columns.
- E. Overwrite the table instead of Merge.

Suggested Answer: AB

Currently there are no comments in this discussion, be the first to comment!

A data engineer is building a streaming data pipeline to ingest JSON files from cloud storage into a Delta Lake table. The pipeline must process files incrementally, handle schema evolution automatically, ensure exactly-once processing, and minimize manual infrastructure management.

How should the data engineer fulfill these requirements?

- A. Use Lakeflow Spark Declarative Pipelines with a static DataFrame read, merge schema with `spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")`
- B. Use Auto Loader in batch mode with a daily job to overwrite the Delta table.
- C. Use Lakeflow Spark Declarative Pipelines with Auto Loader and enabling schema inference with `"cloudFiles.schemaEvolutionMode"="addNewColumns"`
- D. Use traditional Spark Structured Streaming with Auto Loader, manually configuring checkpoints location and enabling schema inference with `"mergeSchema"="true"`

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

Given the following PySpark code snippet in a Databricks notebook:

```
filtered_df=spark.read.format("delta").load("/mnt/daya/large_table") \
.filter ("event_date> '2024-01-01'")
filtered_df.count ()
```

The data engineer notices from the Query Profile that the scan operator for filtered_df is reading almost all files, despite a filter being applied.

What is the probable reason for poor data skipping?

- A. The Delta table lacks optimization that enables dynamic file pruning.
- B. The filter condition involves a data type that is excluded from data skipping support.
- C. The filter is executed only after the full data scan, which prevents data skipping from taking place.
- D. The event_date column is outside the table's partitioning and Z-ordering scheme.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

When a new Databricks project starts, the central IP team provisions the required infrastructure using Terraform and a Service Principal. This includes creating a Databricks workspace, a Unity Catalog linked to an External Location, and a Databricks group containing all project team members. Project teams must store all assets – e.g., tables and volumes, as Managed assets in Unity Catalog. This model hides infrastructure complexity while giving teams autonomy within their catalog. They can create and manage schemas, tables, volumes, and related objects but cannot rename, delete, or change catalog permissions, those remain under IT's control.

Which rights should the project group be granted to enable this model?

- A. The group needs to have USE CATALOG and USE SCHEMA on the catalog.
- B. The group needs to have ALL PRIVILEGES and the MANAGE on the catalog.
- C. The group needs to have ALL PRIVILEGES on the catalog.
- D. The group should be made OWNER of the catalog.

Suggested Answer: A

  **max_manfred** 4 days, 1 hour ago

Selected Answer: C

I think C is correct because the question mentions grants for editing data objects
upvoted 1 times

A data engineer needs to productionize a new Spark application written by teammate. This application has numerous external dependencies, including libraries, and requires custom environment variables and Spark configuration parameters to be set.

Which two methods will help the data engineer accomplish the task? (Choose two.)

- A. Install libraries on DBFS
- B. Add libraries to compute policies
- C. Use secrets in init scripts to store configuration data
- D. Use compute policies to set system properties, environment variables, and Spark configuration parameters.
- E. Create init scripts on DBFS.

Suggested Answer: *DE*

Currently there are no comments in this discussion, be the first to comment!

A healthcare analytics team is implementing a dimensional model in Delta Lake for patient care analysis. They have a date dimension table and are evaluating design options to ensure it supports a wide range of time-based analyses.

Which design approach for the date dimension will support efficient time-based querying and aggregation?

- A. Store the date as string in ISO format (YYYY-MM-DD) for readability.
- B. Pre-calculate attributes like fiscal_period, quarter, month_name, day_of_week, and holiday.
- C. Store only the date value and calculate all time attributes in queries.
- D. Create separate dimension tables for different calendar systems (fiscal, academic, etc.)

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is tasked with ensuring that a Delta table in Databricks continuously retains deleted files for 15 days (instead of the default 7 days), in order to permanently comply with the organization's data retention policy.

Which code snippet correctly sets this retention period for deleted files?

A. `spark.sql("ALTER TABLE my_table SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = 'interval 15 days')")`

`from delta.tables import *`

B. `deltaTable = DeltaTable.forPath(spark, "/mnt/data/my_table")`
`deltaTable.deletedFileRetentionDuration = "interval 15 days"`

C. `spark.sql("VACUUM my_table RETAIN HOURS")`

D. `spark.conf.set("spark.databricks.delta.deletedFileRetemtionDuration", "15 days")`

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A Data Engineer is building a fraud detection pipeline that calls out to Open AI, via a Python library, and needs to include an access token when using the API.

Which Databricks CLI command should the Data Engineer use to create the secret?

- A. databricks secrets put-secret KEY SCOPE; dbutils.secrets.get (KEY, SCOPE)
- B. databricks tokens put-token SCOPE KEY; dbutils.tokens.get (SCOPE, KEY)
- C. databricks secrets put-secret SCOPE KEY; dbutils.secrets.get (SCOPE, KEY)
- D. databricks tokens put-token KEY SCOPE; dbutils.secrets.get (KEY, SCOPE)

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A company has a task management system that tracks the most recent status of tasks. The system takes task events as input and processes events in near real-time using Lakeflow Spark Declarative Pipelines. A new task event is ingested into the system when a task is created or the status is changed. Lakeflow Spark Declarative Pipelines provides a streaming table (table name: tasks_status) for the BI users to query. The table represents the latest status of all tasks and includes 5 columns: task_id(unique for each task), task_name, task_owner, task_status, task_event_time. The table enables 3 properties: deletion vectors, row tracking, and change data feed.

A data engineer is asked to create a new Lakeflow Spark Declarative Pipelines to enrich the "task_status" table in near real-time by adding one additional column representing task_owner's department, which can be looked up from a static dimension table (table name: employee).

How should this enrichment be implemented?

- A. Create a new Lakeflow Spark Declarative Pipelines: use readStream() function with option readChangeFeed to read tasks_status table CDF; enrich with the employee table; create a new streaming table as the result table and use apply_changes() function to process the changes from the enriched CDF.
- B. Create a new Lakeflow Spark Declarative pipeline: use the readStream()function to read tasks_status table, enrich with the employee table; store the result in a new streaming table.
- C. Create a new Lakeflow Spark Declarative Pipeline: use the readStream() function with the option skipChangeCommits to read the tasks_status table; enrich with the employee table; store the result in a new streaming table.
- D. Create a new Lakeflow Spark Declarative Pipeline: use the read () function to read tasks_status table; enrich with employee table; store the result in a materialized view.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineer is reviewing the PySpark code to copy a part of the production dataset to the sandbox environment, and needs to be sure that no PII(Personally Identifiable Information) data is being copied. After checking the sales table, the data engineer notices that it has user emails as the only PII data included as well as being the only column to identify the user.

from pyspark.sql import functions as F

```
# Load data from a table in the production environment
df = spark.read.table("production_catalog.production_schema.sales").sample(fraction=0.1)

# Anonymize the data before saving it to the sandbox
sandbox_df = << fill in >>

# Write the anonymized data to a table in the sandbox environment
sandbox_df.write.format("delta").mode("overwrite").saveAsTable("sandbox_catalog.sandbox_schema.sales")
```

Which anonymised code should be used to achieve the required outcome?

- A. df.withColumn ("user_email", F.expr("uuid()"))
- B. df.withColumn ("user_email", F.sha2 ("user_email"))
- C. df.withColumn ("hashed_email", sha2 ("user_email"))
- D. df.withColumn ("user_email", F.regexp_replace ("user_email", "@*", "@anonymized.com"))

Suggested Answer: B

Currently there are no comments in this discussion, be the first to comment!

A data engineer wants to ingest a large collection of image files (JPEG and PNG) from cloud object storage into Unity Catalog-managed table for further analysis and visualization.

Which two configurations and practices are recommended to incrementally ingest these images into the table? (Choose two.)

- A. Use Auto loader and set `cloudFiles.format` to "TEXT"
- B. Use Auto loader and set `cloudFiles.format` to "BINARYFILE"
- C. Use Auto loader and set `cloudFiles.format` to "IMAGE"
- D. Use the `pathGlobFilter` option to select only image files (e.g.. "*.jpg, *.png")
- E. Move files to a volume and read with SQL editor

Suggested Answer: *BD*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is configuring Delta Sharing for a Databricks-to-Databricks scenario to optimize read performance. The recipient needs to perform time travel queries and streaming reads on shared sales data.

Which configuration will provide the optimal performance while enabling these capabilities?

- A. Use the open sharing protocol instead of Databricks-to Databricks sharing for better performance.
- B. Share tables WITHOUT HISTORY and enable partitioning for better query performance.
- C. Share tables WITH HISTORY, ensure tables don't have partitioning enabled. and enable CDF before sharing.
- D. Share the entire schema WITHOUT HISTORY and rely on recipient-side caching for performance.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data organization has adopted Delta Sharing to securely distribute curated datasets from a Unity Catalog-enabled workspace. The data engineering team is sharing large Delta tables with an internal partner via Databricks-to-Databricks and aggregated reports with an external client via Open Sharing. While testing new sharing workflows, the data engineering team encounters challenges related to access control, data update visibility, and shareable object types.

What is a limitation of the Delta Sharing protocol or implementation when used with Databricks-to-Databricks or Open Sharing?

- A. Delta Sharing does not support Unity Catalog enabled tables; only legacy Hive Metastore tables are shareable.
- B. With Databricks-to-Databricks sharing, Unity Catalog recipients must re-ingest data manually using COPY INTO or REST APIs.
- C. Delta Sharing (both Databricks-to-Databricks and Open sharing) allows recipients to modify the source data if they have SELECT privileges.
- D. With Open sharing, recipients cannot access Volumes, Models, or notebooks – only static Delta tables are supported.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is creating a daily reporting Job. There are two reporting notebooks one for weekdays and the other for weekends. An "If/else condition" task is configured as "{{job.start_time.is_weekday}} == true" to route the job to either weekday or weekend notebook tasks. The same job would be used in multiple time zones.

Which action should a senior data engineer take upon reviewing the job to merge or reject the pull request?

- A. Reject. As they should use {{job.trigger_time.is_weekday}} instead.
- B. Reject. As the {{job.start_time.is_weekday}} is not a valid value reference.
- C. Reject. As the {{job.start_time.is_weekday}} is for the UTC timezone.
- D. Merge. As the job configuration looks good.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

How are the operational aspects of Lakeflow Spark Declarative Pipelines from Spark Structured Streaming different?

- A. Lakeflow Spark Declarative Pipelines automatically handle schema evolution, while Structured Streaming always requires manual schema management.
- B. Structured Streaming can process continuous data streams, while Lakeflow Spark Declarative Pipelines cannot.
- C. Lakeflow Spark Declarative Pipelines manage the orchestration of multi-stage pipelines automatically, while Structured Streaming requires external orchestration for complex dependencies.
- D. Lakeflow Spark Declarative Pipelines can write to Delta Lake format while structured streaming cannot.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A workspace admin has created a new catalog called 'finance_data' and wants to delegate permission management to a finance team lead without giving them full admin rights.

Which privilege should be granted to the finance team lead?

- A. GRANT_OPTION privilege on the finance_data catalog.
- B. Make the finance team lead a metastore admin.
- C. MANAGE privilege on the finance_data catalog.
- D. ALL PRIVILEGES on the finance_data catalog.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer is configuring a Lakeflow Spark Declarative Pipelines to process CDC data from a source. The source events sometimes arrive out of order, and multiple updates may occur with the same update_timestamp but with different update_sequence_id.

What should the data engineer do to ensure events are sequenced correctly?

- A. Use `SEQUENCE BY STRUCT(event_timestamp, update_sequence_id)` in AUTO CDC APIs.
- B. Set track history column list to [event timestamp, event id] in AUTO CDC APIs.
- C. Use `dropduplicate()` to remove out-of-order and duplicate records in LDP.
- D. Use a window function to sort update_sequence_id within the same partition, i.e., update_timestamp in the LDP pipeline.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A streaming video analytics team ingests billions of events daily into a Unity Catalog-managed Delta table `video_events`. Analysts run ad-hoc point-lookup queries on columns like `user_id`, `campaign_id`, and `region`. The team manually runs `OPTIMIZE video_events Z-ORDER user_id, campaign_id, region`, but still sees poor performance on recent data, and dislikes the operational overhead. The team wants a hands-off way to keep hot columns co-located as query patterns evolve.

Which Delta capability should the team leverage on `video_events`?

- A. Enable auto-compaction (`optimizeWrite` & `autoCompact`).
- B. Schedule `OPTIMIZE/ZORDER` to run after each job to improve recent file performance.
- C. Utilize Liquid Clustering `CLUSTER BY AUTO` and Predictive Optimization.
- D. Enable Delta caching.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer is designing a data pipeline in Databricks that needs to process records from a Kafka stream where late-arriving data common.

Which approach should the data engineer use?

- A. Use a watermark to specify the allowed lateness to accommodate records that arrive after their expected window, ensuring correct aggregation and state management.
- B. Use an Auto CDC pipeline with batch tables to simplify late data handling.
- C. Implement a custom solution using Databricks Jobs to periodically reprocess all historical data
- D. Use batch processing and overwrite the entire output table each time to ensure late data is incorporated correctly.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineering team is configuring access controls in Databricks Unity Catalog. They grant the SELECT privilege on the sales catalog to the analyst_group, expecting that members of this group will automatically have SELECT access to all current and future schemas, tables, and views within the sales catalog.

What describes the privilege inheritance behavior in Unity Catalog?

- A. Privileges in Unity Catalog do not cascade; SELECT must be explicitly granted on each schema and table, even if granted at the catalog level.
- B. Granting SELECT on a catalog automatically applies SELECT to all current and future schemas, tables, and views within that catalog.
- C. Granting SELECT at the catalog level applies to existing schemas and tables but not to those created in the future.
- D. Privileges granted at the schema level override any catalog-level privileges and prevent access unless explicitly revoked.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

An organization processes customer data from web and mobile applications. Data includes names, emails phone numbers, and location history. Data arrives both as Batch files from an SFTP drop (daily) and Streaming JSON events from Kafka (real-time).

To comply with internal data privacy policies, the following requirements must be met:

- Personally identifiable information (PII) like email, phone_number, and ip_address must be masked or anonymized before storage
- Both batch and streaming pipelines must apply consistent PII handling
- Masking logic must be auditable and reproducible
- The masked data must still be usable for downstream analytics

How should the data engineer design a compliant data pipeline on Databricks that supports both batch and streaming modes, applies data masking to PII, and maintains traceability of transformations for audits?

- A. Ingest both batch and streaming data using Lakeflow Spark Declarative Pipelines, and apply masking via Unity Catalog column masks at read time to avoid modifying the data during ingestion.
- B. Load batch data with notebooks and ingest streaming data with SQL Warehouses; use Unity Catalog column masks on Silver tables to redact fields after storage.
- C. Use Lakeflow Spark Declarative Pipelines for batch and streaming ingestion define a PII masking function, and apply it during Bronze ingestion before writing to Delta Lake.
- D. Allow PII to be stored unmasked in Bronze for lineage tracking, then apply masking logic in Gold tables used for reporting.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

Which method can be used to determine the total wall-clock time it took to execute a query?

- A. In the Spark UI, take the job duration of the longest-running job associated with that query.
- B. In the Spark UI, take the sum of all task durations that ran across all stages (or all jobs associated with that query).
- C. Open the Query Profiler associated with that query and use the Total wall-clock duration metric.
- D. Open the Query Profiler associated with that query and use the Aggregated task time metric.

Suggested Answer: *C*

Currently there are no comments in this discussion, be the first to comment!

A data architect is designing a Databricks solution to efficiently process data for different business requirements.

In which scenario should a data engineer use a materialized view compared to a streaming table?

- A. Precomputing complex aggregations and joins from multiple large tables to accelerate BI dashboard performance.
- B. Processing high-volume, continuous clickstream data from a website to monitor user behavior in real-time.
- C. Ingesting data from Apache Kafka topics with sub second processing requirements for immediate alerting,
- D. Implementing a CDC (Change Data Capture) pipeline that needs to detect and respond to database changes within seconds.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A query is taking too long to run. After investigating the Spark UI, the data engineer discovered a significant amount of disk spill. The type of compute instance the data engineer used only provides a core-to-memory ratio of 1:2.

What are the two steps the data engineer should take to minimize spillage? (Choose two.)

- A. Increase `spark.sql.files.maxPartitionBytes`.
- B. Choose a compute instance with more disk space.
- C. Choose a compute instance with a higher core-to-memory ratio
- D. Reduce `spark.sql.files.maxPartitionBytes`.
- E. Choose a compute instance with more network bandwidth.

Suggested Answer: *CD*

Currently there are no comments in this discussion, be the first to comment!

A data engineering team has a time-consuming data ingestion job. It has three data sources, and each data ingestion notebook takes about one hour to load new data. The job had been working fine, loading data every midnight. However, one day they received an alert message stating the job had failed. By investigating the cause, they figured out that the failed notebook code was updated, and a new required configuration was introduced to parameterize a hardcoded value without notice. Now, the team has to fix the issue as quickly as possible to load the latest data from the failing data source.

Which action should the team take in this scenario?

- A. Update the task by adding the missing task parameter, and manually run the job.
- B. Share the analysis with the failing notebook owner so that they can fix it quickly.
- C. Repair the run with the new parameter.
- D. Repair the run with the new parameter, and update the task by adding the missing task parameter.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is masking the provided data column containing the email address. The goal is to have an output of the same length for all rows, while keeping different outputs for different values.

Which SQL function should be used to achieve this?

- A. hash(email)
- B. mask(email, '?')
- C. sha1('email')
- D. sha2(email,0)

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A company wants to implement Lakehouse Federation across multiple data sources, but is worried about data consistency and ensuring all teams access the same authoritative version of their data.

Which statement is applicable for Lakehouse Federations to maintain data consistency?

- A. Federation creates local copies that must be manually refreshed.
- B. Federation implements change data capture from all sources.
- C. A separate data synchronization service must be deployed.
- D. Federation provides read-only access that reflects the current state of source systems.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer needs to implement column masking for a sensitive column in a Unity Catalog-managed table. The masking logic must dynamically check if users belong to specific groups defined in a separate table (group_access) that maps groups to allowed departments.

Which approach should the engineer use to efficiently enforce this requirement?

- A. Create a view without selecting the sensitive column
- B. Apply a column mask that references the group_access mapping table in its UDF
- C. Create a UDF that hardcodes allowed groups and apply it as a column mask.
- D. Use a row filter to restrict access based on the user's group.

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

Two data engineers are working on the same Databricks notebook in separate branches. Both have edited the same section of code. When one tries to merge the other's branch into their own using the Databricks Git folders UI, a merge conflict occurs on that notebook file. The UI highlights the conflict and presents options for resolution.

How should the data engineers resolve this merge conflict using Databricks Git folders?

- A. Use the Git CLI in the cluster's web terminal to force-push the conflicted merge (`git push --force`), overriding the remote branch with local version and discarding changes.
- B. Use the Git folders UI to manually edit the notebook file, selecting the desired lines from both versions and removing the conflict markers, then mark the conflict as resolved.
- C. Abort the merge, discard all local changes, and try the merge operation again without reviewing the conflicting code.
- D. Delete the conflicted notebook file via the Databricks workspace UI, commit the deletion, and recreate the notebook from scratch in a new commit to bypass the conflict entirely.

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

A data engineer wants to automate job monitoring and recovery in Databricks using the Jobs API. They need to list all jobs, identify a failed job, and rerun it.

Which sequence of API actions should the data engineer perform?

- A. Use the jobs list endpoint to list jobs, check job run statuses with jobs runs list, and rerun a failed job using jobs run-now.
- B. Use the jobs get endpoint to retrieve job details, then use jobs update to rerun failed jobs.
- C. Use the jobs list endpoint to list jobs, then use the jobs create endpoint to create a new job, and run the new job using jobs run-now.
- D. Use the jobs cancel endpoint to remove failed jobs, then recreate them with jobs create endpoint and run the new ones.

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineering team is setting up deployment automation. To deploy workspace assets remotely using the Databricks CLI command, they must configure it with proper authentication.

Which authentication approach will provide the highest level of security?

- A. Use a service principal and its Personal Access Token
- B. Use a service principal with OAuth token federation
- C. Use a service principal ID and its OAuth client secret
- D. Use a shared user account and its OAuth client secret

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

A data engineer treated a daily batch ingestion pipeline using a cluster with the latest DBR version to store banking transaction data, and persisted it in a MANAGED DELTA table called `prod.gold.all_banking_transactions_daily`. The data engineer is constantly receiving complaints, from business units that constantly read this table in an ad hoc way using a SQL Serverless Warehouse, regarding the poor performance of their queries. Analyzing the queries, the data engineer slated that these areas use high cardinality columns as filters, to perform their analysis. The data engineer is studying possibilities to implement a data layout optimization technique for that table, which is incremental, easy to maintain, and can easily evolve over time.

Which command should the data engineer implement?

- A. Alter the table to use Hive Style Partitions + Z-ORDER and implement a periodic OPTIMIZE command.
- B. Alter the table to use Hive Style Partitions and implement a periodic OPTIMIZE command.
- C. Alter the table to use Z-ORDER and implement a periodic OPTIMIZE command.
- D. Alter the table to use Liquid Clustering and implement a periodic OPTIMIZE command

Suggested Answer: D

Currently there are no comments in this discussion, be the first to comment!

A data engineer is designing an append-only pipeline that needs to handle both batch and streaming data in Delta Lake. The team wants to ensure that the streaming component can efficiently track which data has already been processed.

Which configuration should be set to enable this?

- A. checkpointLocation
- B. overwriteSchema
- C. mergeSchema
- D. partitionBy

Suggested Answer: A

Currently there are no comments in this discussion, be the first to comment!

A data engineer is implementing Unity Catalog governance for a multi-team environment. Data scientists need interactive clusters for basic data exploration tasks, while automated ETL jobs require dedicated processing.

How should the data engineer configure cluster isolation policies to enforce least privilege and ensure Unity Catalog compliance?

- A. Configure all dusters with NO_ISOLATION_SHARED access modes since Unity Catalog works with any cluster configuration
- B. Allow all users to create any duster type and rely on manual configuration to enable Unity Catalog access modes.
- C. Create compute policies with STANDARD access mode for interactive 'workloads and dedicated access mode for automated jobs.
- D. Use only DEDICATED access mode for both interactive workloads and automated jobs to maximize security isolation.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineer, while designing a Pandas UDF to process financial time series data with complex calculations that require maintaining state across rows within each stock symbol group must ensure the function is efficient and scalable.

Which approach will solve the problem with minimum overhead while preserving data integrity?

- A. Use a SCALAR_ITER Pandas UDF with iterator-based processing, implementing state management through persistent storage (Delta tables) that gets updated after each batch to maintain continuity across iterator chunks.
- B. Use a GROUPED_AGG UPF that processes each stock symbol group independently, maintaining state through intermediate aggregation results that get passed between successive UDF calls via broadcast variables.
- C. Use a SCALAR Pandas UDF that processes the entire dataset at once, implementing custom partitioning logic within the UDF to group by stock symbol and maintain state using global variables shared across all executor processes.
- D. Use ApplyInPandas method on spark dataframe that receives all rows for each stock symbol as a pandas DataFrame, allowing processing within each group while maintaining state variables local to each group's processing function.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer is developing an LDP pipeline using a Databricks notebook that is directly connected to their pipeline. After adding new table definitions and transformation logic in their notebook, they want to check for any syntax errors in the pipeline code without actually processing any data or running the pipeline.

How should the data engineer perform this syntax check?

- A. Open the web terminal from the notebook and run a shell command to validate the pipeline code.
- B. Disconnect the notebook from the pipeline and reconnect it to a compute cluster to access code validation features
- C. Use the "Validate" option in the notebook to check for syntax errors.
- D. Switch to a workspace file instead of a notebook to access the validation and diagnostics tools.

Suggested Answer: C

Currently there are no comments in this discussion, be the first to comment!

A data engineering team is in the process of migrating off its legacy Hadoop platform. As part of the process, they are evaluating the different storage formats to see which offers the best query performance. Their legacy platform utilizes ORC and RCFile formats. They converted a subset of their data to Delta Lake and ran a side-by-side benchmark, and found that their queries performed significantly better. Upon investigation, they discovered that the query plans were different, and the queries that read from the Delta Lake tables leveraged a Shuffle Hash Join, whereas the legacy formats used Sort Merge Joins. The queries that read from the Delta Lake tables also read less data.

Which reason could be attributed to the difference in query performance?

- A. The queries against the ORC tables leveraged the dynamic data skipping optimization but not the dynamic file pruning optimization.
- B. The queries against the Delta Lake tables were able to leverage the dynamic file pruning optimization.
- C. Delta Lake enables data skipping and file pruning using a vectorized Parquet reader.
- D. Shuffle Hash Joins are always more efficient than Sort Merge Joins.

Suggested Answer: *B*

Currently there are no comments in this discussion, be the first to comment!

A data team is automating a daily multitask ETL pipeline in Databricks. The pipeline includes a notebook for ingesting raw data, a Python wheel task for data transformation, and a SQL query to update aggregates. They want to trigger the pipeline programmatically and see previous runs in the GUI. They need to ensure tasks are retried on failure and stakeholders are notified by email if any task fails.

Which two approaches will meet these requirements? (Choose two.)

- A. Trigger the job programmatically using the Databricks Jobs REST API (/jobs/run-now), the CLI (databricks jobs run-now), or one of the Databricks SDKs.
- B. Use Databricks Asset Bundles (DABs) to deploy the workflow, then trigger individual tasks directly by referencing each task's notebook or script path in the workspace.
- C. Create a multi-task job using the UI, DABs, or the Jobs REST API (/jobs/create) with notebook, Python wheel, and SQL tasks. Configure task-level retries and email notifications in the job definition.
- D. Use the REST API endpoint /jobs/runs/submit to trigger each task individually as separate job runs and implement retries using custom logic in the orchestrator.
- E. Create a single notebook that uses dbutils.notebook.run() to call each step in sequence. Define a job on this orchestrator notebook and configure retries and notifications at the notebook level.

Suggested Answer: AC

Currently there are no comments in this discussion, be the first to comment!

A data engineer is designing a system to process batch patient encounter data stored in an S3 bucket, creating a Delta table (`patient_encounters`) with columns `encounter_id`, `patient_id`, `encounter_date`, `diagnostic_code`, and `treatment_cost`. The table is queried frequently by `patient_id` and `encounter_date`, requiring fast performance. Fine grained access controls must be enforced. The engineer wants to minimize maintenance and boost performance.

How should the data engineer create the `patient_encounters` table?

- A. Create an external table in Unity Catalog, specifying an S3 location for the data files. Enable predictive optimization through table properties, and configure Unity Catalog permissions for access controls.
- B. Create a managed table in Unity Catalog. Configure Unity Catalog permissions for access controls, schedule jobs to run `OPTIMIZE` and `vacuum` command daily to achieve best performance.
- C. Create a managed table in Hive metastore. Configure Hive metastore permissions for access controls, and rely on predictive optimization to enhance query performance and simplify maintenance.
- D. Create a managed table in Unity Catalog. Configure Unity Catalog permissions for access controls, and rely on predictive optimization to enhance query performance and simplify maintenance.

Suggested Answer: *D*

Currently there are no comments in this discussion, be the first to comment!

A data engineer has a delta table order with deletion vectors enabled for it. The engineer is attempting to execute the below code:

```
DELETE FROM orders WHERE status = 'cancelled'
```

What should be the behaviour of deletion vectors when the command is executed?

- A. Files are physically rewritten without the deleted row.
- B. Rows are marked as deleted both in metadata and in files.
- C. Rows are marked as deleted in metadata, not in files
- D. Delta automatically removes all cancelled orders permanently.

Suggested Answer: *C*

Currently there are no comments in this discussion, be the first to comment!

In case the questions count in this PDF doesn't match the number on the website, its likely that the file is being cached to an old version. Please contact through our email at support@certificationtest.net and we'll reply within 24 hours.